

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Facultatea de Matematica si Informatica – An II

Curs Introductiv

Sistemul de Notare

“

*Don't limit your
challenges.
Challenge your limits.*

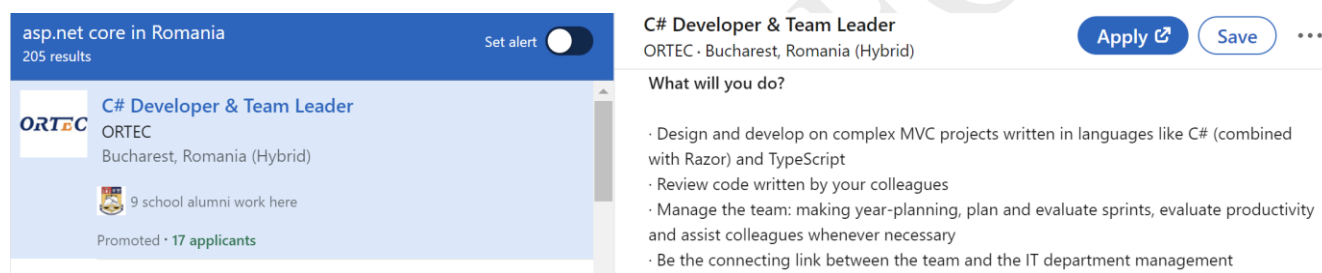
Lect. Univ. Dr. Cezara Benegui – cezara.benegui@fmi.unibuc.ro

Cursuri si Laboratoare - <https://www.cezarabenegui.com/>

Ce vom studia si de ce:

- ASP.NET CORE MVC (.NET 6)
- C#
- SQL Server
- Razor / Blazor
- HTML / CSS / Bootstrap

De ce?



asp.net core in Romania
205 results

Set alert ☐

C# Developer & Team Leader
ORTEC · Bucharest, Romania (Hybrid)

Apply Save ...

What will you do?

- Design and develop on complex MVC projects written in languages like C# (combined with Razor) and TypeScript
- Review code written by your colleagues
- Manage the team: making year-planning, plan and evaluate sprints, evaluate productivity and assist colleagues whenever necessary
- Be the connecting link between the team and the IT department management

Who you are:

- Experienced C# / .NET developer
- Experienced with SQL and relational databases
- Well versed in the common software development architecture patterns
- You have worked with Azure DevOps (with GIT) previously
- You've done your fair share of code reviews
- Experience with TypeScript is an advantage
- At least 5 years of relevant experience
- Bachelor or master's degree in Information Technology or comparable studies
- Proficient in English in speaking, listening, reading and writing
- Passionate about IT and always looking to improve the user experience
- You enjoy working with your team members, mentoring them and helping them whenever necessary
- You can spot ways to improve the productivity of the team, and you're not afraid to follow up on them
- Good in planning, allowing you to combine all responsibilities whilst also delivering on your own development projects



18 alumni work here

Promoted • 6 applicants

Software Developer .Net
Suvoda
Bucharest, Bucharest, Romania
38 alumni work here
2 hours ago

.Net Developer
Suvoda
Bucharest, Bucharest, Romania
38 alumni work here
6 days ago • 16 applicants

Requirements

- Bachelor's degree or equivalent experience
- 3+ years of experience in ASP.NET MVC or ASP.NET Core with focus on backend development
- Good knowledge of C#, .Net and SQL
- Good understanding of programming principles
- Ability to design, develop, and support software components with low complexity
- Excellent problem-solving skills and learning capability
- Excellent time management and organizational skills
- Strong oral and written communication skills
- Bonus: .NET Core/.NET 5+, distributed systems, Docker, Kubernetes, Jenkins

Benefits found in job post

- Medical insurance



SCHWARZ IT Hub Romania
Bucharest, Romania (Hybrid)
14 alumni work here
Promoted • 16 applicants

.Net Software Developer
Enghouse Transportation
Bucharest, Bucharest, Romania (Remote)
2 hours ago

.NET Software Developer, Services
Suvoda
Bucharest, Bucharest, Romania (Remote)
Your profile matches this job
5 days ago • 23 applicants

Essential Requirements:

- At least 3 years of experience with C# .Net, SQL Server
- Demonstrated expertise in the web technologies with proficiency in C#, .Net, SQL Server, Entity Framework & Model View Controller (MVC)
- Familiarity with Web-Services, Enterprise Service Bus, Kendo UI, GIT, Jenkins
- Programming fundamentals - knowledge of basic principles and concepts of programming languages, as well as conventional development methodologies, frameworks and databases
- Bachelor's degree in computer science preferred (or college degree in progress)
- Proficiency in English

Benefits found in job post

- Medical insurance



.Net Software Developer
Enghouse Transportation
Bucharest, Bucharest, Romania (Remote)
2 hours ago

.NET Software Developer, Services
Suvoda
Bucharest, Bucharest, Romania (Remote)
Your profile matches this job
5 days ago • 23 applicants

Senior Game Designer

What would you need to know?

You would likely be successful in this role if you:

- Proven experience working within scrum teams consisting of 5-10 members, working and planning in an agile environment
- C# development
- .NET Framework
- HTML5/CSS, Javascript, AngularJS and JQuery SQL
- DevOps tools Strong English and communication skills, both written and spoken
- Previous experience working in transportation & logistics applications will be a plus.

Ce este necesar pentru a incepe cursul:

- Visual Studio 2022
- Laptop
- Acces la internet
- Cunostinte minimale de Programare Orientata pe Obiecte
- Cunostinte minimale de C#
- Cunostinte minimale de SQL / Baze de Date
- Cunostinte minimale de HTML/CSS
- Dorinta de a invata si a progresa constant
- Entuziasm

Cuprinsul cursului (capitolele de mai jos nu sunt neaparat in ordinea in care vom lucra):

- Introducere in C#
- ASP.NET Core. Sistemul de fisiere
- Arhitectura MVC
- Sistemul de rutare
- MVC – Controller
- MVC – Model
- MVC – View
- Entity Framework Core
- Identity – Inregistrare si Autentificare
- Utilizatori
- Roluri
- Razor
- Bootstrap
- Designul intr-o aplicatie Web
- User Experience
- Motor de cautare
- Integrare componente 3rd party

Structura cursului:

- Saptamanile 1-12 **predare** (pana in saptamana 18-22 Decembrie inclusiv)
- Saptamanile 13-14 **evaluare** (dupa vacanta de Craciun: saptamanile 8-12 Ianuarie si 15-19 Ianuarie)

Sistemul de notare:

- **Examen pe calculator** – **60%** (nota trebuie sa fie ≥ 5) – [Vezi detalii](#);
- **Proiect** realizat in echipa (echipa = 2 persoane) – **40%** (nota trebuie sa fie ≥ 5) impreuna cu un planning realizat in platforma Trello – [Vezi detalii](#);
- **6 prezente obligatorii in laborator pentru a intra in examen** (nu se iau in considerare prezentele din cadrul orelor de examinare);
- **1 punct BONUS** – obtinut din quiz-uri – [Vezi detalii](#);

⚠ Cele doua probe, test si proiect, se pot promova individual, nedepinzand una de cealalta. Nota si bonusul se pastreaza pe parcursul intregului an universitar in curs.

Restanta:

- In restanta se reface proba nepromovata;
- **Bonusul** acumulat in timpul semestrului **ramane valabil** pe parcursul anului universitar curent;
- In cazul in care materia nu se promoveaza in anul universitar 2023-2024, se va reface in intregime anul urmator;

Examen (pe calculator):

- Are loc in **Saptamana 13**, penultima saptamana de laborator;
- Fiecare grupa va sustine examenul in timpul orei de laborator. De asemenea, se poate planifica examenul si sambata, **13 Ianuarie 2024**, cu cate 2 grupe (ex: 2 grupe in intervalul 10:00-13:00 si alte doua grupe in intervalul 14:00-17:00);
- Examenul se da pe calculator si este compus din:
 - O serie de intrebari de tip interviu, sub forma unui quiz. In cadrul acestei probe nu aveti voie cu materiale.
 - Operatii de tip CRUD asupra bazei de date impreuna cu elemente parcurse in timpul cursului/laboratorului. Examenul nu o sa cuprinda toata materia. Pentru partea de implementare din cadrul examenului, o sa primiti un template de la care sa porniti. Trebuie doar sa implementati anumite sectiuni peste acesta. Examenul este evaluat folosind teste automate, astfel este necesara respectarea tuturor regulilor ce vor fi mentionate in ziua examenului. In caz contrar, testarea automata nu poate fi executata corect, fapt ce duce la nepromovarea examenului. In cadrul acestei probe aveti voie cu materiale – cursuri si laboratoare.
- **⚠** Conditia este ca ceea ce rezolvati sa ruleze in browser. Daca exista sectiuni care nu ruleaza, acestea se vor comenta;
- **⚠** De asemenea, sa nu existe cod care nu are legatura cu subiectul pe care il primiti. Orice linie/secventa de cod care provine dintr-o alta sursa, atrage dupa sine nepromovarea examenului;
- **⚠** Denumirea variabilelor trebuie sa aiba legatura cu subiectul pe care il primiti. Denumirile care nu sunt justificabile atrag dupa sine nepromovarea examenului;

Proiectul:

- Realizat in ASP.NET MVC CORE, in echipa de doua persoane, impreuna cu un raport descris mai jos;
- Proiectul se alege dintr-o lista de proiecte pana pe data de **20 Octombrie 2023** (lista se afla pe site). **Atentie la deadline!**
- Fiecare proiect poate fi ales de un anumit numar de echipe. Fiecare echipa trebuie sa se noteze in tabel, in functie de proiectul ales (link-ul se afla pe site);
- Proiectul se implementeaza utilizand tehnologiile studiate in timpul cursului;
- **⚠** Fiecare membru al echipei trebuie sa implementeze jumatate din cerinte. Cerintele se impart echilibrat, fiecare echipa facand individual management-ul task-urilor. Un membru trebuie sa implementeze atat partea de backend, cat si pe cea de frontend. Acest lucru inseamna ca fiecare membru trebuie sa implementeze integral o componenta a aplicatiei. De asemenea, pot exista si parti din aplicatie care sa fie implementate de intreaga echipa. In final, fiecare membru trebuie sa fie capabil sa explice detaliat atat partile implementate impreuna, cat si pe cele pe care le-a implementat individual;
- Pentru management-ul task-urilor se va utiliza Trello - <https://trello.com/>

Raportul – planificare

- **⚠** Pe tot parcursul dezvoltării proiectului, fiecare echipă o să ataseze într-un document câteva print screen-uri din Trello, care să arate planificarea fiecărei etape de implementare, împartirea echilibrată a task-urilor, lucrul eficient în echipă, problemele aparute pe parcurs și rezolvarea lor, cât și metodologia de lucru. Fiecare echipă redactează acest raport în stilul propriu, explicând pe scurt procesul. Pe lângă imaginile atasate, care pot ocupa mai multe pagini, documentul trebuie să conțină maximum 2 pagini de text;
- Documentul se convertește în .pdf, se denumește după formatul de mai jos și se trimite împreună cu proiectul;

NumePrenumeGrupa1_NumePrenume_Grupa2.pdf – informațiile fiind pentru fiecare membru al echipei în parte

Ce doresc să observ în acest raport?

- Cum ați reușit să planificați implementarea;
- Cum ați comunicat și ați împartit eficient task-urile;
- Cât de productivi ați fost;
- Ce impedimente ați întâlnit;
- Cum ați ales să lucrați – metodologia de lucru;
- Cât ați dezvoltat de la un sprint la altul;
- Cum ați reușit să vă ajutați și să lucrați eficient în echipă;

Prezentare intermediara

- **⚠ In saptamana 12 → 18 – 22 Decembrie** (inainte de vacanta de Craciun) fiecare echipa prezinta o varianta intermediara a proiectului (cel putin jumatate din functionalitatile cerute in lista cerintelor);
- Echipele prezinta in cadrul laboratorului la care au participat tot semestrul;

Prezentare finala

- **⚠ Deadline-ul final de prezentare este in saptamana 14** (ultima saptamana din semestru) → **15 – 19 Ianuarie** in cadrul laboratorului, dar si in alte intervale stabilite impreuna - **Atentie la deadline!**
- Proiectele finale impreuna cu rapoartele se trimit pana pe data **12 Ianuarie 2024**, inclusiv - **Atentie la deadline!**
- **⚠ NU** se pot trimite proiectele dupa deadline;
- **⚠ NU** se pot prezenta proiectele dupa deadline;
- Proiectele se vor realiza gradual, dezvoltarea lor mergand in paralel cu informatia invatata la curs si laborator;

Bonusul:

- **0.5p** obtinute din unul sau doua quiz-uri din cadrul cursurilor;
- **0.5p** obtinute din unul sau doua quiz-uri din cadrul laboratoarelor;
- Punctajul de la curs se adauga notei din examen;
- Punctajul de la laborator se adauga notei din cadrul proiectului;

! Nota se rotunjeste o singura data la final!

Formula nota finala:

$$\text{NotaFinala} = 60\% * (\text{nota_examen} + \text{bonus_curs}) + 40\% * (\text{nota_proiect} + \text{bonus_laborator})$$

Reguli generale:

1. Prezenta in cadrul cursurilor nu este obligatorie, dar materia din laborator este implementarea practica a notiunilor de la curs. **NU** reiau cursul in timpul laboratorului. Asadar, ori veniti la curs, ori il parcurgeti acasa **INAINTE DE LABORATOR**.
2. Fiecare laborator reprezinta un progres, fiind o imbunatatire a laboratorului anterior. In cazul in care lipsiti, este **OBLIGATORIU** sa lucrati laboratorul acasa si sa veniti cu el rezolvat.
3. Intotdeauna trebuie sa aveti exercitiile din laboratorul anterior implementate.
4. Va ajut si raspund la intrebari ori de cate ori aveti nevoie, dar in momentul in care ceva nu functioneaza, trebuie sa imi aratati si variantele pe care le-ati incercat si nu au functionat.
5. Pentru raspunsurile la quiz-uri se foloseste intotdeauna adresa institutionala.
6. Pentru trimiterea diferitelor surse se utilizeaza adresa institutionala.
7. Nu se pot trimite surse (proiecte, rapoarte, teme) dupa deadline.
8. Prezentarea proiectelor se face **OBLIGATORIU** de catre intreaga echipa, atat in cadrul prezentarii intermediare, cat si in timpul prezentarii finale.
9. Orice problema se poate rezolva daca o aflui din timp.
10. Aveti grija la cerinte (reguli de redactare, implementare, trimitere).
11. Sa nu existe cod care nu are legatura cu subiectul pe care il primiti. Orice linie/secventa de cod care provine dintr-o alta sursa, atrage dupa sine nepromovarea examenului.

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 1

Cuprins

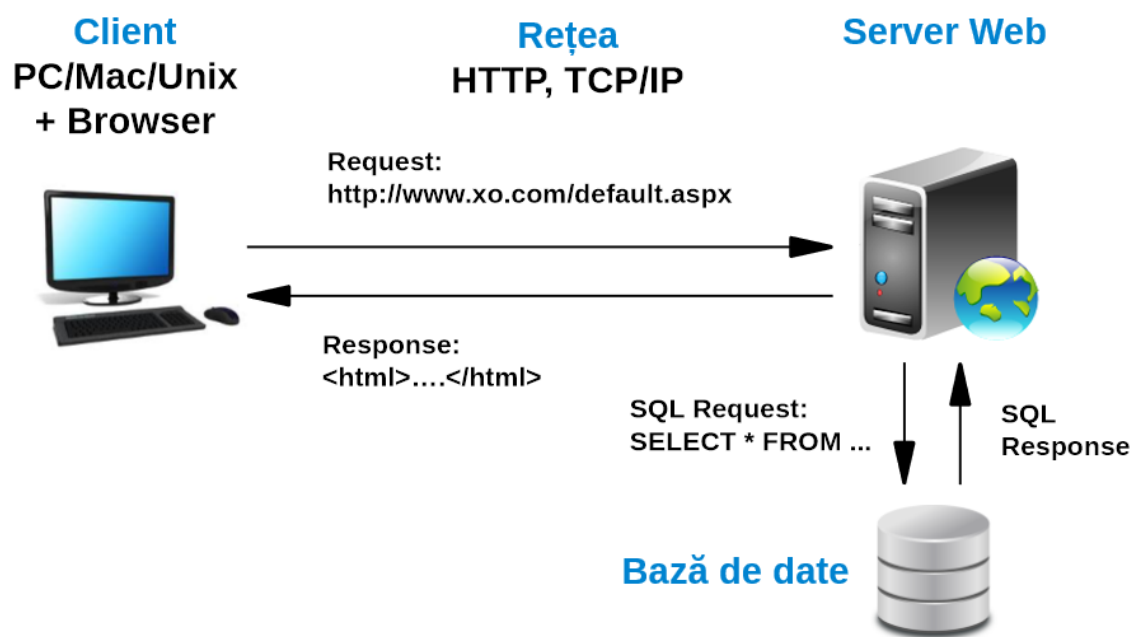
Ce este o aplicatie Web	2
Arhitectura Web	2
Avantajele aplicatiilor Web	3
Introducere in ASP.NET	3
Framework-ul .NET	4
ASP.NET Core	5
Introducere in C#	5
Limbaj compilat vs limbaj interpretat	6
Ciclul de viata al unei pagini Web	6

Ce este o aplicatie Web

O **aplicatie web** este o aplicatie care ruleaza intr-o arhitectura Client-Server bazata pe: **protocolul HTTP, TCP/IP; browser Web; Server Web.**

Aplicatiile web sunt executate intr-un browser web si implementate folosind tehnologii precum: PHP, ASP, PYTHON, HTML, CSS, JAVASCRIPT, etc.

Arhitectura Web



Avantajele aplicatiilor Web

- Sunt independente de sistemul de operare
- Nu necesita instalare
- Actualizari foarte usor de facut deoarece modificarile se fac intr-un singur loc – pe server, ele propagandu-se pentru toti utilizatorii (in cazul aplicatiilor client-server clasice, interfata cu utilizatorul este asigurata prin intermediul unui program client instalat pe calculatorul fiecarui utilizator, orice modificare necesitand reinstalarea aplicatiei pentru fiecare utilizator in parte)

Introducere in ASP.NET

- **ASP.NET** este un framework Web, open source, conceput si dezvoltat de Microsoft, care face parte din framework-ul .NET
- Este utilizat pentru a dezvolta aplicatii si servicii web
- Oferă o integrare foarte buna a codului HTML, CSS, JavaScript
- Oferă posibilitatea crearii paginilor dinamice, prin intermediul sintaxei Razor, utilizand C#, HTML, CSS si JavaScript
- ASP.NET furnizeaza librării specifice dezvoltării aplicațiilor web, cum sunt cele pentru sistemul de autentificare (autentificare în mai mulți pași, autentificare utilizand componente 3rd party, etc), cele pentru crearea si prelucrarea bazei de date, cele pentru lucrul cu fisiere, etc
- Este construit pe baza **CLR (Common Language Runtime)** – **ruleaza cod compilat** si permite utilizatorilor sa scrie cod folosind orice limbaj acceptat de framework-ul .NET

Ce este CLR – Common Language Runtime?

Se ocupa de executia programelor C#. Atunci cand este compilat un program C# rezultatul compilarii **nu este un cod executabil**. In locul acestuia se produce un fisier care contine un tip de cod apropiat de codul masinii, numit limbaj intermediar sau pe scurt **IL – Intermediate Language**.

Prin intermediul unui compilator denumit **JIT – Just in Time**, CLR transforma codul intermediar in cod executabil.

Framework-ul .NET

- Este compatibil cu peste 20 de limbaje diferite, cele mai populare fiind C#, C++, Visual Basic, F#. In prezent, limbajul cel mai des folosit ramane C# deoarece restul limbajelor de programare au dezvoltat librarii similare cu .NET framework
- Pune la dispozitie o colectie impresionanta de clase, organizate in biblioteci
- Este construit din doua entitati importante:

1. Common Language Runtime (CLR)

- mediul de executie al programelor fiind cel care se ocupa cu managementul si executia codului scris in limbaje specifice .NET

2. Base Class Library

- Este biblioteca de clase .NET
- Acopera o arie larga a necesitatilor de programare, incluzand **interfata cu utilizatorul, protocoale de conectare cu baza de date, accesarea datelor**

ASP.NET Core

- Este noul framework creat de Microsoft
- A fost conceput pentru a functiona indiferent de sistemul de operare (Windows, MAC, Linux), fiind astfel mult mai flexibil si rapid
- ASP.NET Core este un framework nou, nefiind o continuare a framework-ului ASP.NET 4.6
- ASP.NET Core aduce in plus securitate, scade costurile si imbunatateste performanta
- Se pot dezvolta foarte multe tipuri de aplicatii: desktop, web, mobile, machine learning, micro-servicii, jocuri
- Hostare mult mai rapida in cloud

Introducere in C#

- Este un limbaj compilat
- Este un limbaj orientat pe obiecte
- Permite dezvoltarea de aplicatii industriale, durabile
- A fost conceput ca un concurent pentru limbajul Java
- Este derivat al limbajului C++

Limbaj compilat vs limbaj interpretat

Limbaj compilat – codul scris, numit cod sursa, este translatat de catre compilator intr-un cod apropiat de nivelul masinii, numit **cod executabil**. Atunci cand aplicatia trece de compilare fara erori de sintaxa se va produce codul executabil, iar aplicatia va putea fi rulata. (exemplu limbaje compilate: C, C++, Java, C#).

Limbaj interpretat (la rulare) – cu ajutorul unui interpretor specific limbajului, fiecare linie de cod este interpretata chiar in momentul rularii, fiind preschimbata imediat in cod masina si executata (exemplu limbaje interpretate: PHP, Ruby, Python).

Ciclul de viata al unei pagini Web

Paginile ASP.NET ruleaza pe server-ul Web Microsoft IIS (Internet Information Server). In urma prelucrarii pe server rezulta o pagina web HTML, care este trimisa catre browser.

Ciclul de viata al unei pagini Web ASP.NET are urmatoorii pasi:

- **Page request (accesarea paginii)** – acest pas se intampla inaintea ciclului de viata, atunci cand o pagina este ceruta serverului
- **Start** – in acest stadiu se incarca proprietatile paginii, cum ar fi request-ul si raspunsul, dupa care se identifica tipul acestora (**GET – cerere resurse, POST – trimiterea de informatii catre server**)
- **Initialization (initializare)** – in acest pas se initializeaza directivele si controalele si se aplica codul din Master Page

- **Load (incarcarea)** – in aceasta faza daca cererea este de tip **postback**, controalele sunt incarcate cu informatii
- **Evenimentele Postback** – daca cererea este de tip postback se executa codul aferent. Dupa executia codului se aplica sistemele de validare
- **Rendering (ex: afisarea paginii)** – in acest pas se construiesc pagina finala pe server, care va fi afisata in browser
- **Unload (eliberarea memoriei)** – dupa ce pagina a fost trimisa utilizatorului, resursele alocate pentru aceasta sunt eliberate

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 2

Cuprins

Introducere in C#	2
Structura unui program	2
Variabile si Tipuri de date.....	7
Conversii de tip	8
Nullable.....	11
Instructiuni de control	11
Array-uri.....	13
Clase si Obiecte.....	14
Constructor.....	15
This.....	15
Conventii de nume - Naming conventions	16

Introducere in C#

C# este unul dintre cele mai populare limbaje de programare, dezvoltat de Microsoft, care ruleaza pe .NET Framework.

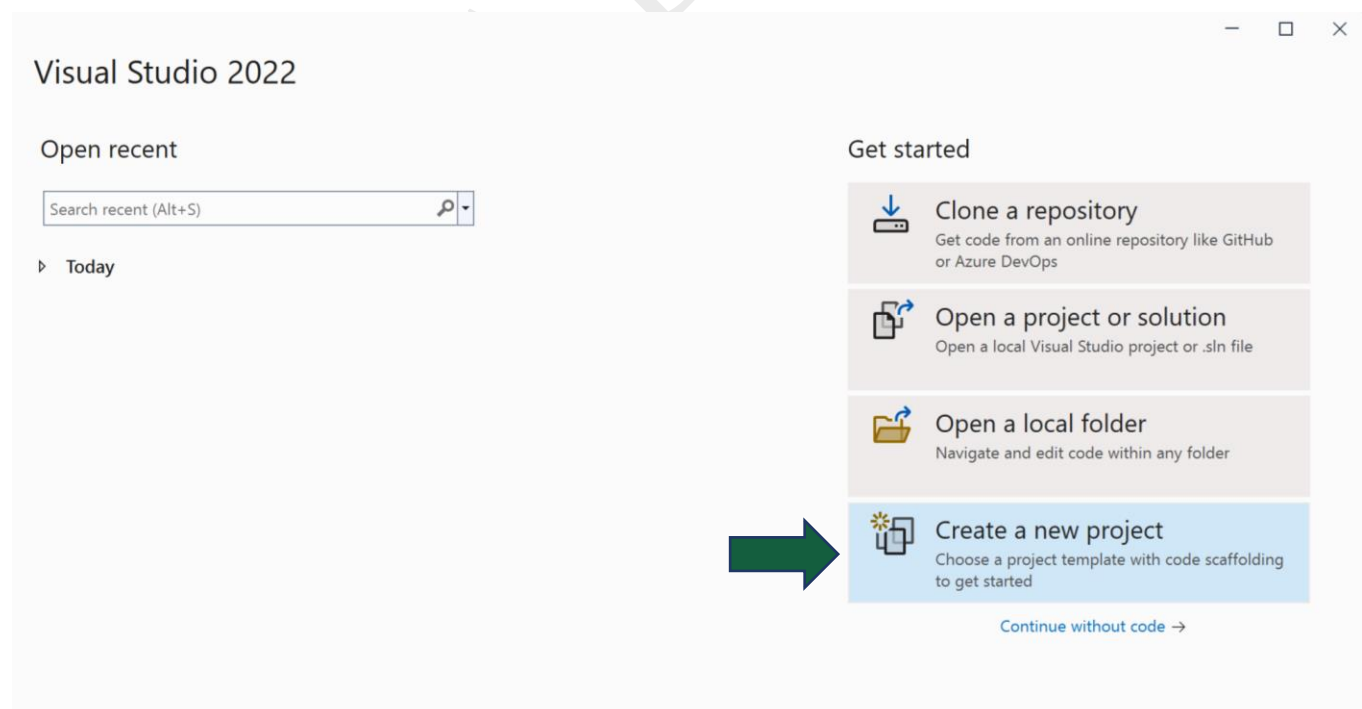
Este folosit pentru a dezvolta o gama variata de aplicatii, de la aplicatii web, mobile, desktop, pana la jocuri, servicii web, VR, etc.

Structura unui program

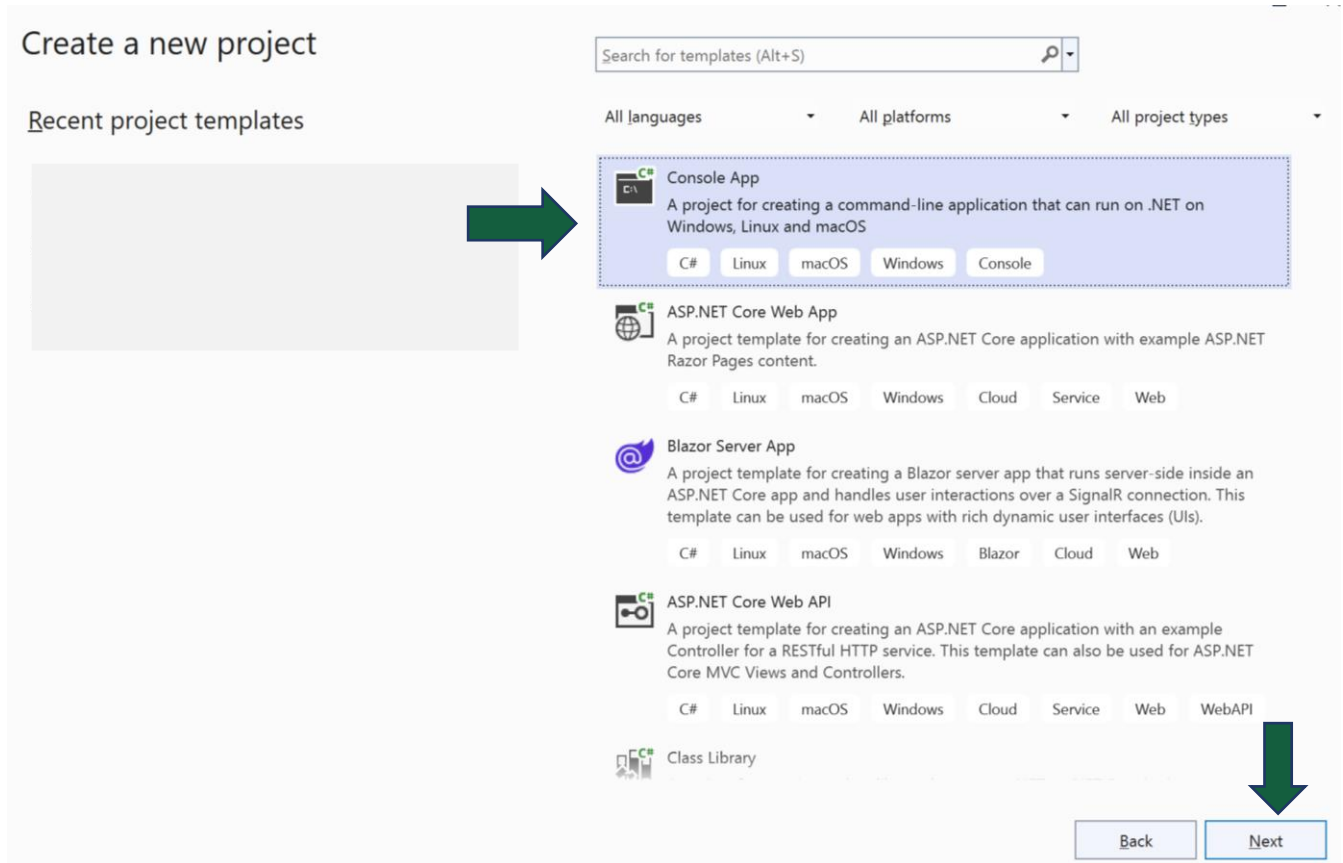
Pentru a dezvolta un program scris in C#, vom utiliza Visual Studio 2022 (**VEZI** Laborator 1 – instalare VS 2022).

Pentru inceput se creeaza un proiect folosind urmatoarele proprietati:

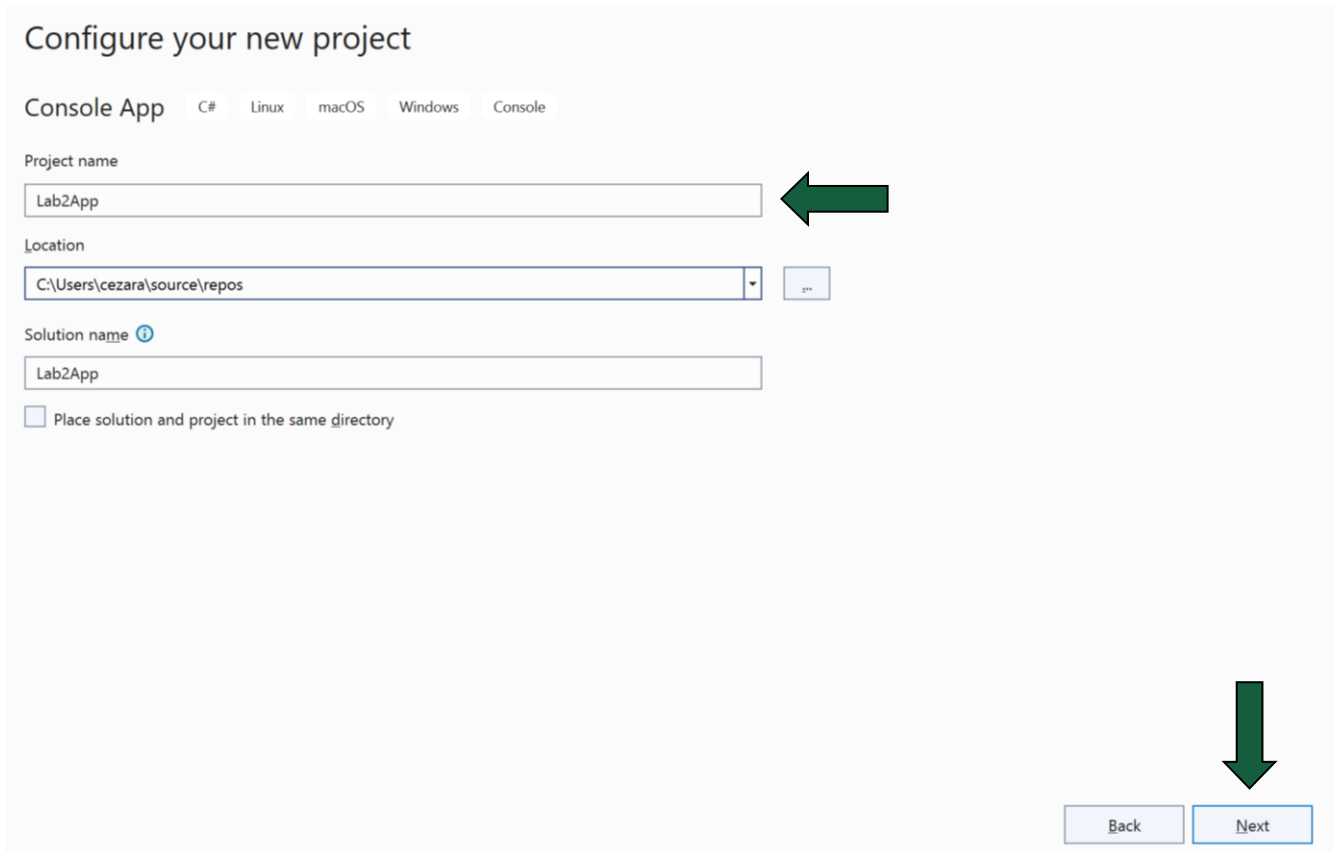
PASUL 1:



PASUL 2:



PASUL 3:



Configure your new project

Console App C# Linux macOS Windows Console

Project name
Lab2App

Location
C:\Users\cezara\source\repos

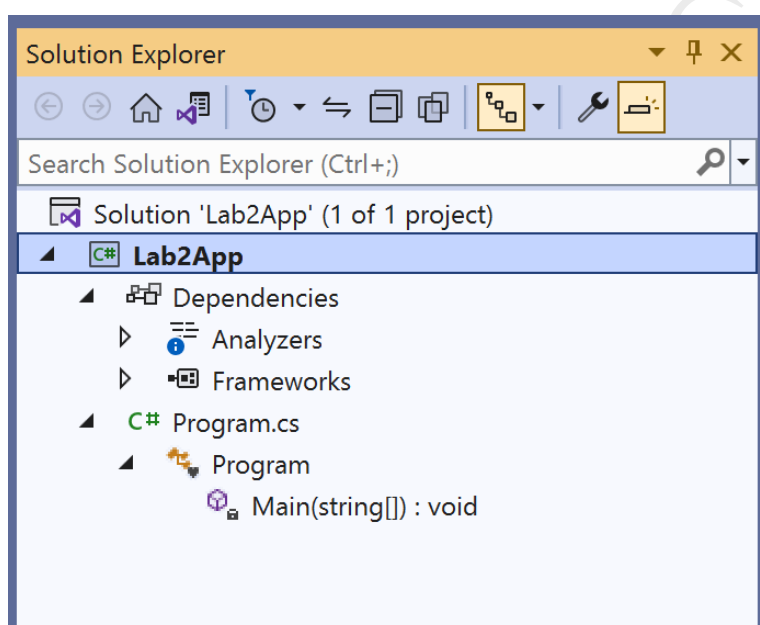
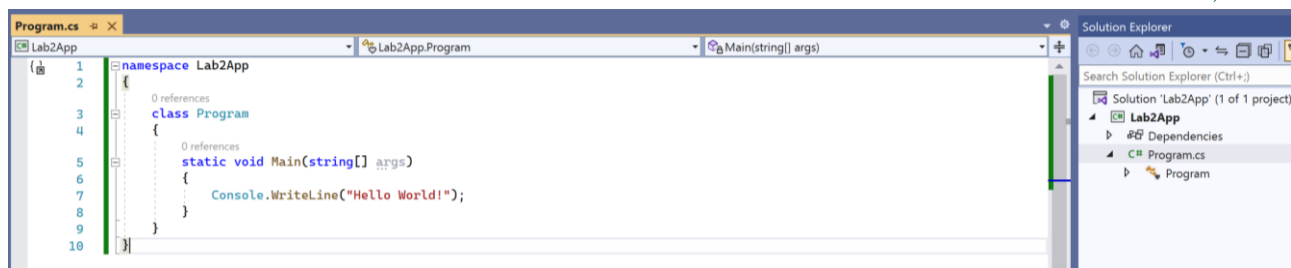
Solution name ⓘ
Lab2App

☐ Place solution and project in the same directory

Back Next

PASUL 4:

In fisierul **Program.cs** vom scrie cel mai simplu program in C#. Se va afisa in consola mesajul “Hello World!”

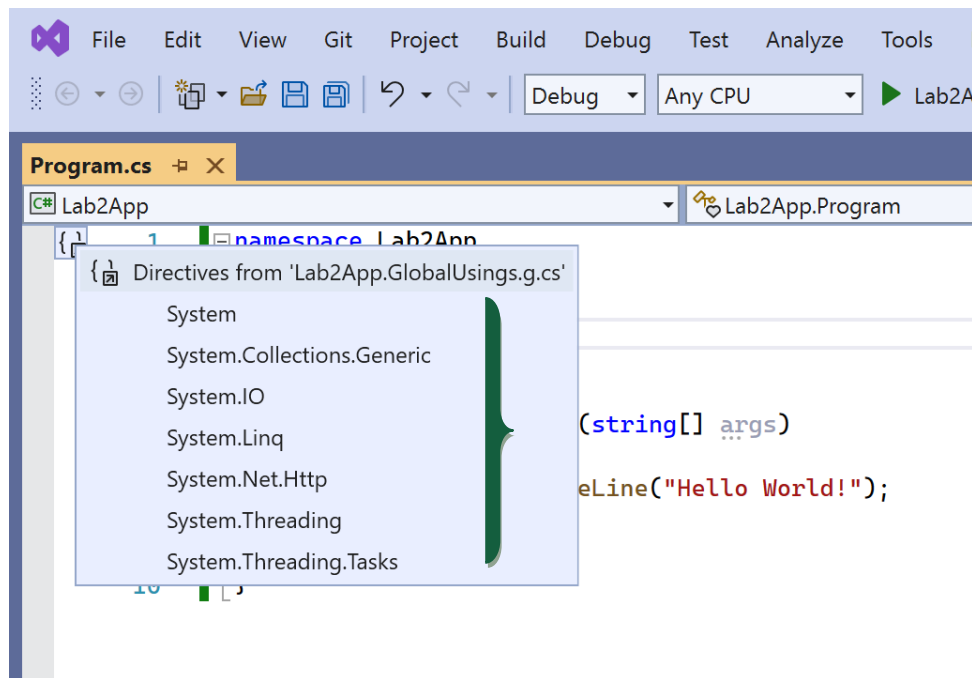


In momentul crearii proiectului, acesta a fost numit **Lab2App**. Denumirea proiectului devine automat **namespace**.

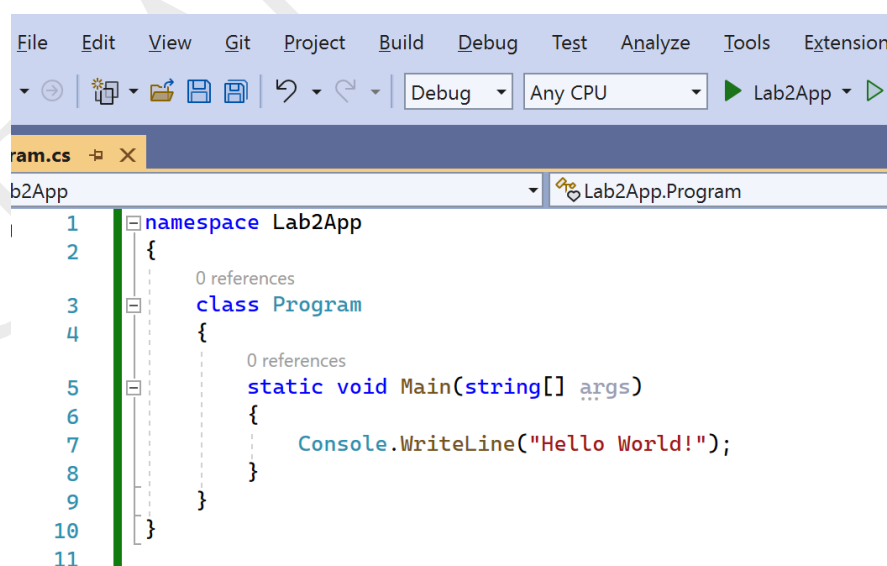
Un **namespace** – este o **colectie de clase**. Namespace-ul Lab2App contine clasa **Program**. Atunci cand scriem cod intr-un namespace, avem acces la toate clasele definite in el.

Daca se doreste utilizarea unei clase dintr-un alt namespace, atunci acel namespace trebuie importat.

Pentru import se utilizeaza **using**. Visual Studio are incluse cateva astfel de directive.



Directiva **System** – **ofera acces la toate clasele de baza**. In cazul de fata, System ofera acces la clasa Console care la randul ei contine metoda **WriteLine**.



Clasa **Program** – **defineste metoda Main**. Comparativ cu Java, unde clasa care contine metoda Main trebuie sa se numeasca tot Main, in cazul lui C# clasa poate avea orice denumire.

static void Main (string[] args) – este metoda principala a programului, metoda care se apeleaza prima.

! OBS:

- C# este case sensitive (**VEZI** mai jos – [naming conventions](#))
- Orice declaratie sau expresie se incheie cu ;
- Executia programelor incepe intotdeauna cu metoda Main
- Comentariul pe o singura linie se include folosind //
- Comentariile pe mai multe linii se includ folosind /**/

Variabile si Tipuri de date

O **variabila** – este un nume pe care il ia o zona de memorie. Ulterior acel spatiu de memorie poate fi manipulat prin intermediul denumirii variabilei.

Fiecare variabila are un anumit **tip de date**, care determina dimensiunea zonei de memorie.

Tipuri de date in C#

Cele mai utilizate tipuri de date sunt:

- int – numere intregi fara virgula
- double – numere intregi cu virgula
- char – stocheaza cate un caracter
- string – stocheaza text
- bool – stocheaza valori true sau false
- object – tipul obiect este clasa de baza pentru toate tipurile de date in C#. Tipurilor obiecte li se pot atribui valori de orice tip.

```
// TIPURILE DE DATE
```

```
int nr = 100;
string str = "Acesta este un text";
double d = 12.35;
char c = 'a';
bool b = true;
object obj = 100;

Console.WriteLine("Numarul este " + nr);
Console.WriteLine("Stringul este: " + str);
Console.WriteLine("Numarul in virgula mobila este: " + d);
Console.WriteLine("Caracterul este: " + c);
Console.WriteLine("Valoarea de adevar este: " + b);
Console.WriteLine("Obiectul este: " + obj);
```

Conversii de tip

Conversiile de tip se impart in doua categorii:

- **implicite** – compilatorul C# convertește automat un tip de date în alt tip de date. În general tipuri de date care ocupa o zona mai mica de memorie, cum este tipul int, sunt convertite automat în tipuri de date care ocupa o zona mai mare de memorie.

```
// CONVERSII IMPLICITE
```

```
int nrInt = 10;
```

```
// Metoda GetType() preia tipul de date
Type tipNrInt = nrInt.GetType();
```

```
// Conversie implicita
double nrDouble = nrInt;
```

```
// Se preia tipul
Type tipNrDouble = nrDouble.GetType();
```

```
// Afisare valori inainte de conversie
Console.WriteLine("nrInt value: " + nrInt);
Console.WriteLine("nrInt Type: " + tipNrInt);
```

```
// Afisare valori dupa conversia implicita
Console.WriteLine("nrDouble value: " + nrDouble);
Console.WriteLine("nrDouble Type: " + tipNrDouble);
```

Compilatorul a convertit implicit tipul int in double, fara pierdere de informatie.

- **explicite** – in cazul in care se doreste conversia unui tip care ocupa o zona mai mare de memorie, intr-un tip care are o dimensiune mai mica a memoriei

```
// CONVERSII EXPLICITE

double nDouble = 25.123;

// Conversie explicita
int nInt = (int)nDouble;

// Afisarea valorii inainte de conversie
Console.WriteLine("Valoarea inainte de conversie a fost: "
+ nDouble);

// Afisarea valorii dupa conversie
Console.WriteLine("Valoarea dupa conversie este: " +
nInt);
```

- Se poate utiliza si conversia folosind **Parse()**. Aceasta se utilizeaza cu precadere in cazurile in care se convertesc tipuri de date care nu sunt compatibile. De exemplu, int si string

Sintaxa: int.Parse(parametru);

```
// CONVERSIE UTILIZAND PARSE()
```

```
string st = "100";
```

```
// tipul de date
```

```
Type tip1 = st.GetType();
```

```
// Se convertește tipul string în int
```

```
int x = int.Parse(st);
```

```
Type tip2 = x.GetType();
```

```
Console.WriteLine("Valoarea initiala a fost: " + st);
```

```
Console.WriteLine("A avut tipul: " + tip1);
```

```
Console.WriteLine("Noua valoare dupa conversie este: " + x);
```

```
Console.WriteLine("Valoarea dupa conversie are tipul: " + tip2);
```

- **Conversii folosind clasa Convert** – clasa pune la dispozitie numeroase metode pentru a converti orice tip de date într-un alt tip de date -> **ToBoolean(), ToChar(), ToDouble(), ToInt16(), ToString()**

```
// CONVERSII FOLOSIND CLASA CONVERT
```

```
int num = 25;
```

```
Console.WriteLine("Valoare de tip int: " + num);
```

```
// Se convertește valoarea int în stringul "25"
```

```
string strConvert = Convert.ToString(num);
```

```
Console.WriteLine("Valoarea dupa conversie " + strConvert);
```

```
Console.WriteLine("Tipul dupa conversie: " + strConvert.GetType());
```

```
// Conversie în Double
```

```
Double doubleConvert = Convert.ToDouble(num);
```

```
Console.WriteLine("Valoarea dupa conversie " + doubleConvert);
```

```
Console.WriteLine("Tipul dupa conversie: " + doubleConvert.GetType());
```

Nullable

Nullable reprezintă în C# mai multe tipuri de date. Aceste tipuri de date pot lua valori dintr-un interval de valori sau pot fi null.

Sintaxa pentru declararea unui tip de date nullable este următoarea:

<tipul_de_date> ? <nume_variabila> = null;

Exemplu:

```
// NULLABLE
int? num1 = null;
int? num2 = 45;

Console.WriteLine("Valorile sunt: {0}, {1}", num1, num2);
```

Instructiuni de control

```
// INSTRUCȚIUNI DE CONTROL

// if
if (Conditie)
{
    // se executa daca conditia este true
}

// if else
if (Conditie)
{
    // se executa daca conditia este true
}
else
{
    // se executa daca conditia este false
}
```

```

// if else --- else
if (Conditie1)
{
    // se executa daca conditia1 este true
}
else if (Conditie2)
{
    // se executa daca conditia2 este true
}
else if (Conditie3)
{
    // se executa daca conditia3 este true
}
else
{
    // se executa daca nicio conditie din cele anterioare
    nu se indeplineste
}

// short-hand if...else
// se numeste si operator ternar (ternary operator)
// deoarece este format din 3 operanzi

// var = (condition) ? expressionTrue : expressionFalse;

int nr = 10;
string result = (nr % 2 == 0) ? "Numarul este par" :
"Numarul este impar";
Console.WriteLine(result);

// if imbricat (nested)
if (Conditie1)
{
    // se executa daca conditia1 este true
    if (Conditie2)
    {
        // se executa daca conditia2 este true
    }
}

// while loop
while (Conditie)
{
    statement(s);
}

```

```
// for loop
for (init; conditie; increment)
{
    statement(s);
}
```

```
// do...while loop
do
{
    statement(s);
} while(Conditie);
```

Array-uri

Un **array** este utilizat pentru a stoca o colectie de date de acelasi tip. Fiecare element din array este accesat prin indexul sau.

Sintaxa de declarare a unui array:

datatype[] numeArray;

```
// ARRAY
```

```
int[] n = new int[10];
```

```
for (int i = 0; i < 10; i++)
{
    n[i] = i + 1;
}
```

```
for (int i = 0; i < 10; i++)
{
```

```
    Console.WriteLine("Element[{0}] = {1}", i, n[i]);
}
```

Clase si Obiecte

Clasele si obiectele stau la baza Programarii Orientate pe Obiecte.

O **clasa** reprezinta o structura de date care contine proprietati si metode care se vor aplica asupra datelor, comportandu-se ca un tip de date. In ASP.NET, in momentul crearii bazei de date, clasele vor reprezenta tabelele din baza de date, fiecare clasa avand proprietati specifice.

Un **obiect** reprezinta o instanta a unei clase. In momentul in care se creeaza un obiect, se alocă si o zona de memorie.

Exemplu:

```
public class Employee
{
    // Definirea atributelor(proprietatilor)

    public int EmployeeId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public decimal Salary { get; set; }

    // Constructor pentru initializarea obiectului Employee

    public Employee(int employeeId, string firstName, string
lastName, decimal salary)
    {
        this.EmployeeId = employeeId;
        this.FirstName = firstName;
        LastName = lastName;
        Salary = salary;
    }

    // Metoda pentru afisarea detaliilor angajatului
    public void Index()
    {
        Console.WriteLine("Employee ID: " + EmployeeId);
        Console.WriteLine("First Name: " + FirstName);
        Console.WriteLine("Last Name: " + LastName);
        Console.WriteLine("Salary: " + Salary);
    }
}
```

```

class Program
{
    static void Main(string[] args)
    {
        // Crearea unui obiect al clasei Employee

        Employee emp = new Employee(1, "Pop", "Lucian", 2500);

        // Apelarea metodei Index prin intermediul obiectului emp,
        // obiectul fiind o instanta a clasei Employee
        // Astfel se afiseaza detaliile angajatului

        emp.Index();
    }
}

```

Constructor

- Trebuie sa aiba acelasi nume ca si clasa;
- Poate fi public, protected sau private;
- O clasa poate avea mai multi constructori, cu un numar diferit de parametri, dar nu poate avea decat un singur constructor fara parametri;
- Nu poate returna nicio valoare, deci nu are un tip pe care sa il returneze;
- Daca nu exista constructor, compilatorul C# o sa creeze unul in mod automat;

This

This este referinta la instanta curenta a clasei, dar doar in interiorul clasei in care este utilizat. Se utilizeaza in momentul in care se doreste accesarea membrilor clasei (proprietati, metode). In exemplul de mai jos sunt corecte ambele implementari.

```

this.EmployeeId = employeeId;
this.FirstName = firstName;
LastName = lastName;
Salary = salary;

```

Conventii de nume - Naming conventions

- Se utilizeaza **PascalCase** pentru:
 - Clase
 - Constructor
 - Metode

- Se utilizeaza **camelCase** pentru:
 - Variabile
 - Argumentele metodelor

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

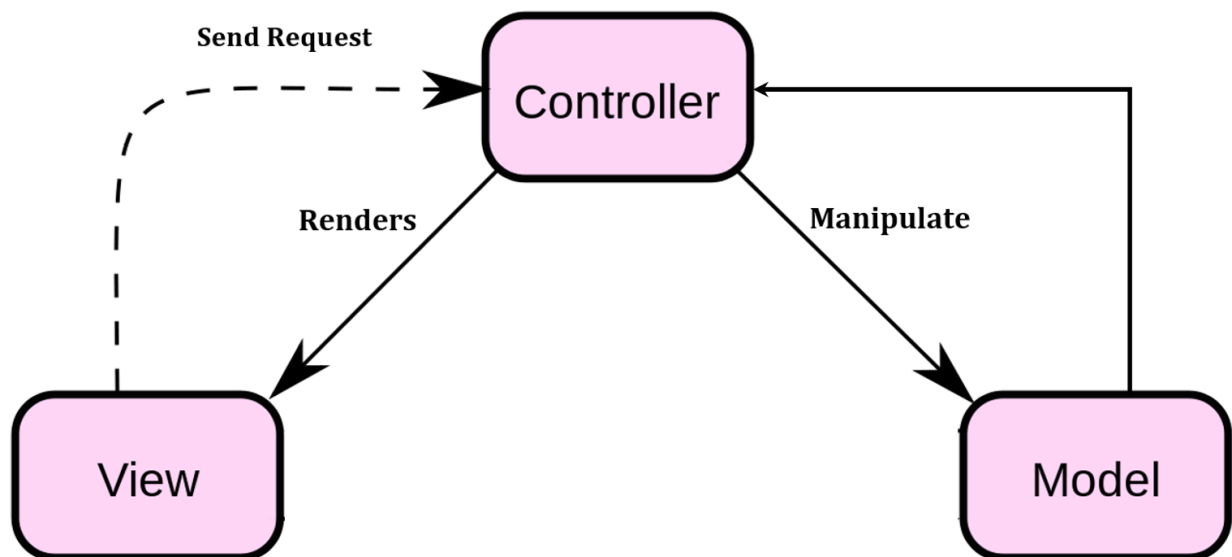
Curs 3

Cuprins:

Arhitectura MVC	2
Model (Stratul business – prelucrarea datelor)	2
Controller	3
View (interfata cu utilizatorul).....	3
Crearea unei aplicatii in ASP.NET Core 6.0 (Visual Studio 2022)	4
Structura unui proiect MVC – Sistemul de fisiere.....	7
Sistemul de rutare.....	10
Exemple de implementare a rutelor:	18
Configurarea rutelor:.....	18
Definirea rutelor custom	24
Constrangerile parametrilor	26

Arhitectura MVC

Model-View-Controller (MVC) este un model arhitectural utilizat în dezvoltarea aplicațiilor. Succesul modelului se datorează izolării logicii aplicației (business logic) față de interfața cu utilizatorul, rezultând o aplicație unde aspectul vizual și nivelele inferioare ale regulilor de business sunt mai ușor de modificat, fără a afecta alte nivele.



Model (Stratul business – prelucrarea datelor)

Modelul este responsabil cu gestionarea datelor din aplicație și manipularea acestora. Acesta răspunde cererilor care vin din View prin intermediul Controller-ului, Modelul comunicând doar cu Controller-ul. Este cel mai de jos nivel care se ocupă cu **procesarea** și **manipularea** datelor, reprezentând nucleul aplicației, fiind cel care realizează legătura cu baza de date.

Controller

Controller-ul este reprezentat de clase, fiind componenta care controleaza accesul la aplicatie.

In Controller:

- sunt procesate requesturile HTTP
- se citesc datele introduse de utilizator
- are loc procesarea prin trimiterea datelor catre Model - unde se executa operatiile
- se trimite raspunsul catre View

Asadar, Controller-ul comunica cu Modelul si cu View-ul.

Fiecare Controller contine asa numitele **Action-uri (Actions)**, reprezentate de metode. Aceste metode sunt publice, se definesc in interiorul unui Controller si sunt accesate in momentul in care are loc un request prin intermediul unei rute.

View (interfata cu utilizatorul)

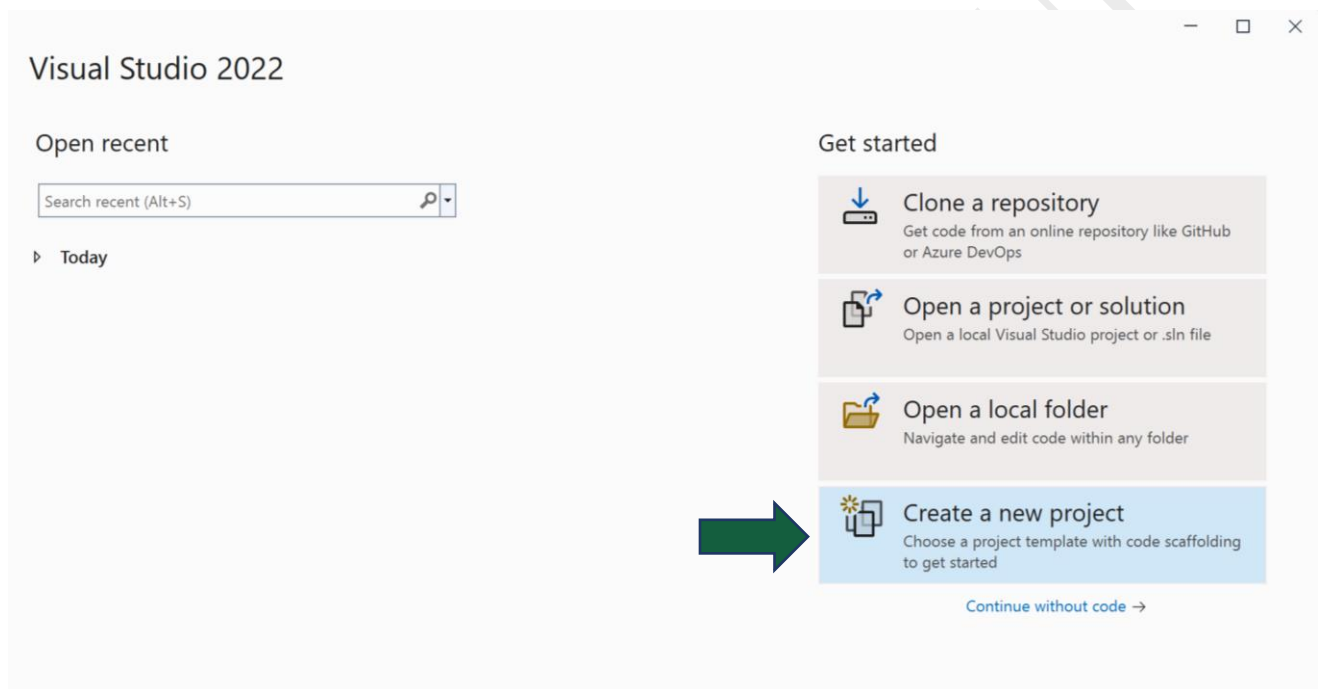
View-ul reprezinta interfata cu utilizatorul, fiind componenta arhitecturii MVC cu care utilizatorii interactioneaza prin intermediul browser-ului.

In View se afiseaza datele, adica inregistrarile din baza de date si informatiile generate de aplicatie.

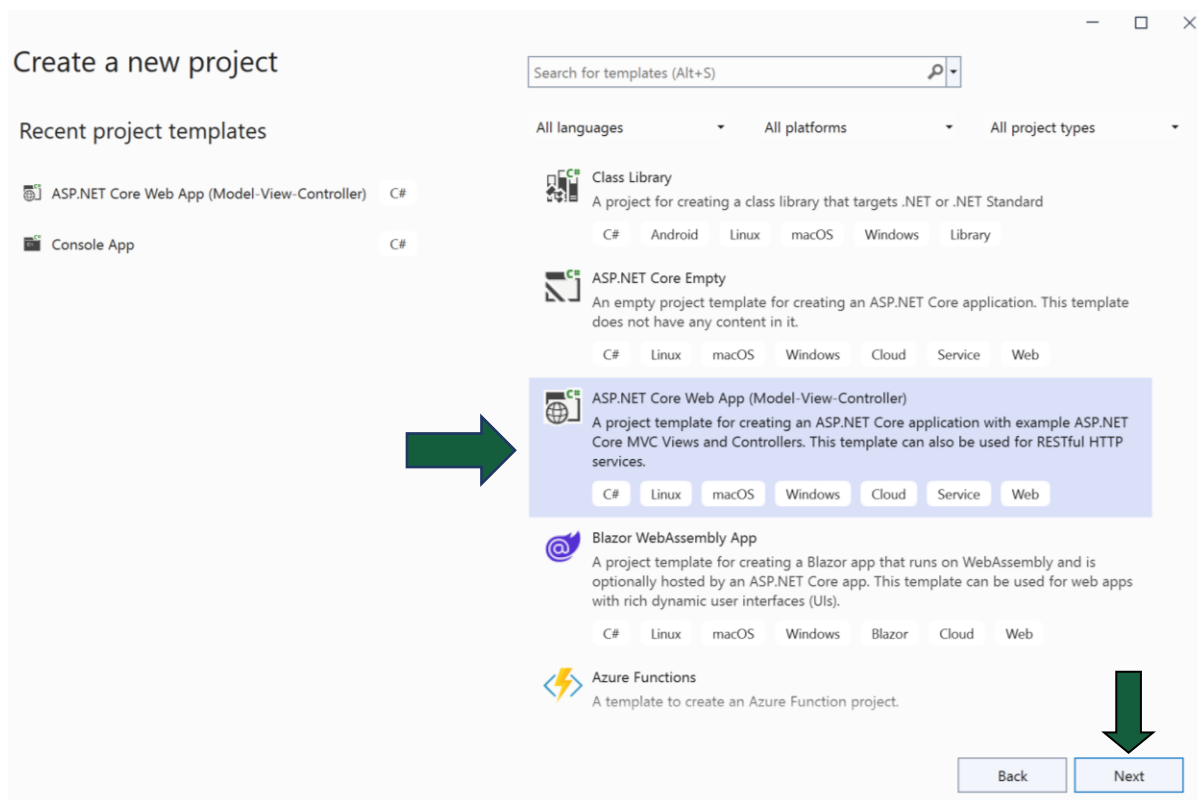
View-ul poate contine HTML static sau chiar HTML trimis din Controller (HTML dinamic). In cadrul arhitecturii MVC, View-ul comunica doar cu Controller-ul, iar cu Modelul indirect tot prin intermediul Controller-ului.

Crearea unei aplicatii in ASP.NET Core 6.0 (Visual Studio 2022)

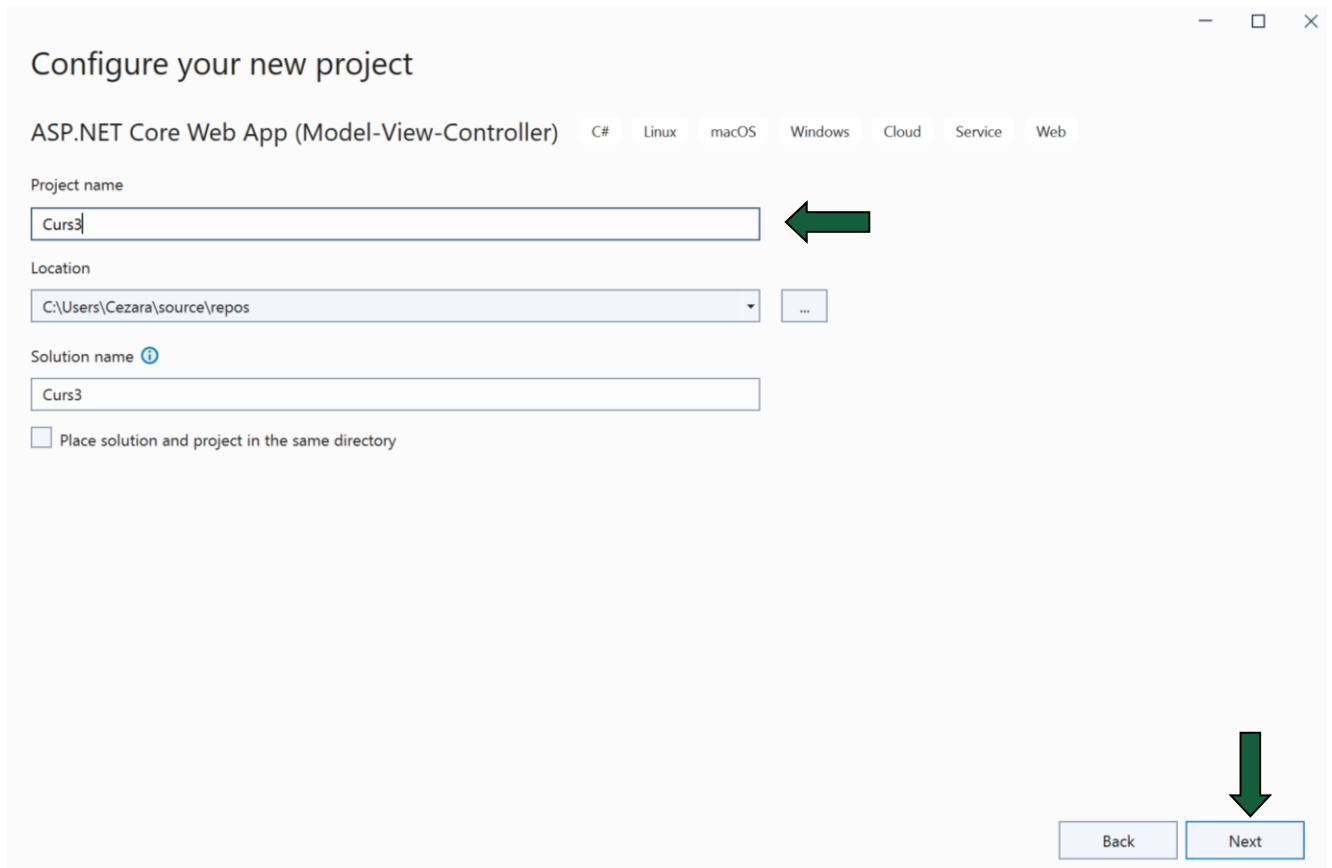
PASUL 1:



PASUL 2:



PASUL 3:



Configure your new project

ASP.NET Core Web App (Model-View-Controller) C# Linux macOS Windows Cloud Service Web

Project name

Curs3

Location

C:\Users\Cezara\source\repos

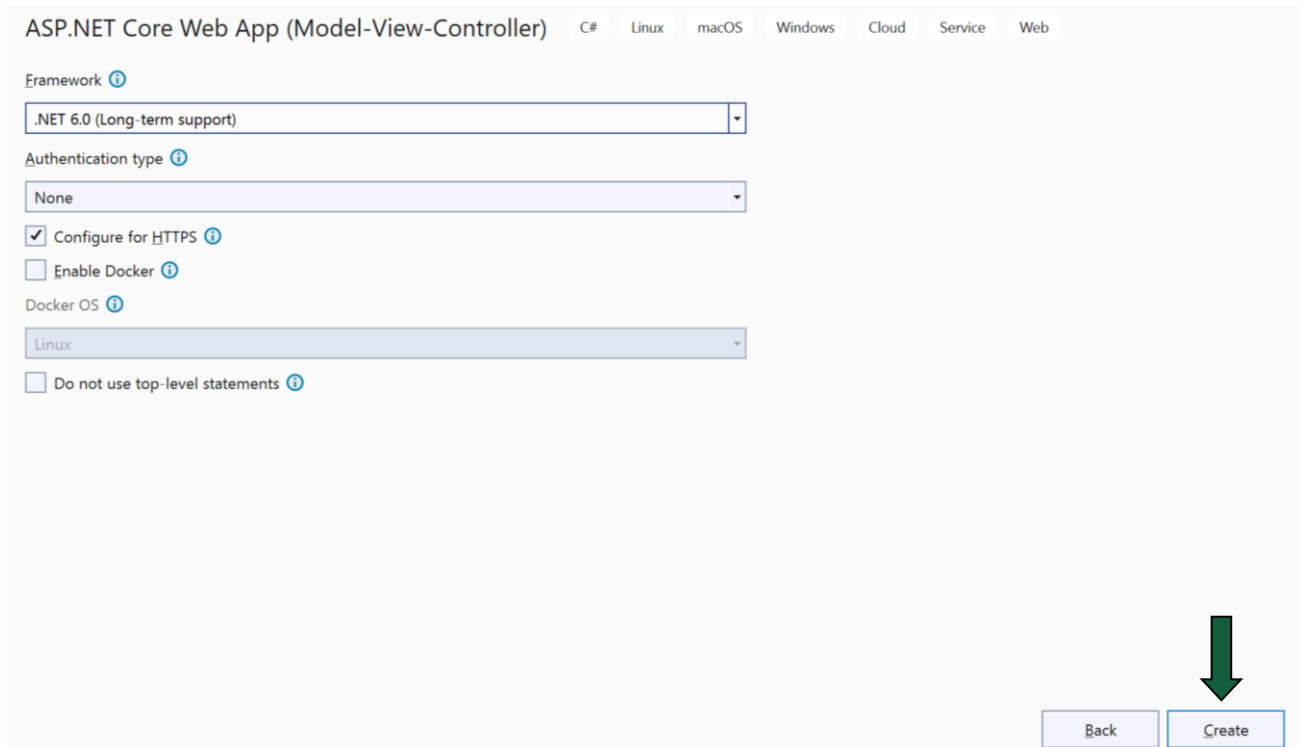
Solution name ⓘ

Curs3

☐ Place solution and project in the same directory

Back Next

PASUL 4:



ASP.NET Core Web App (Model-View-Controller) C# Linux macOS Windows Cloud Service Web

Framework ⓘ
[.NET 6.0 (Long-term support)]

Authentication type ⓘ
[None]

☒ Configure for HTTPS ⓘ
☐ Enable Docker ⓘ

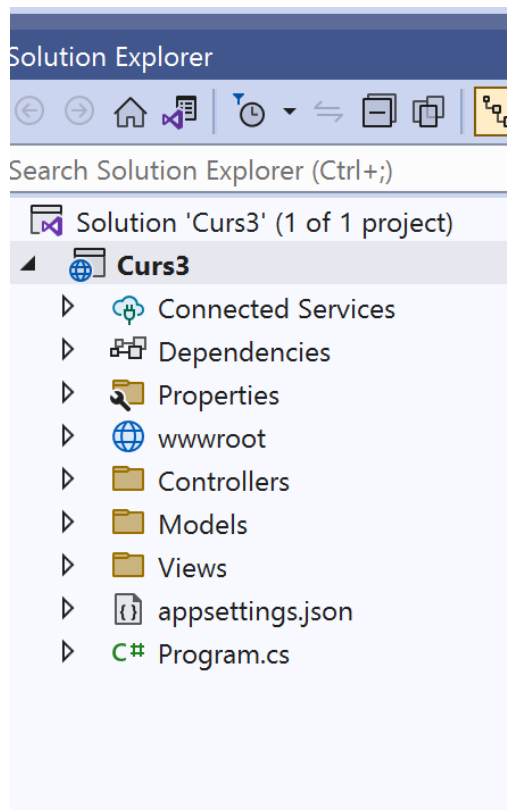
Docker OS ⓘ
[Linux]

☐ Do not use top-level statements ⓘ

Back Create

Structura unui proiect MVC – Sistemul de fisiere

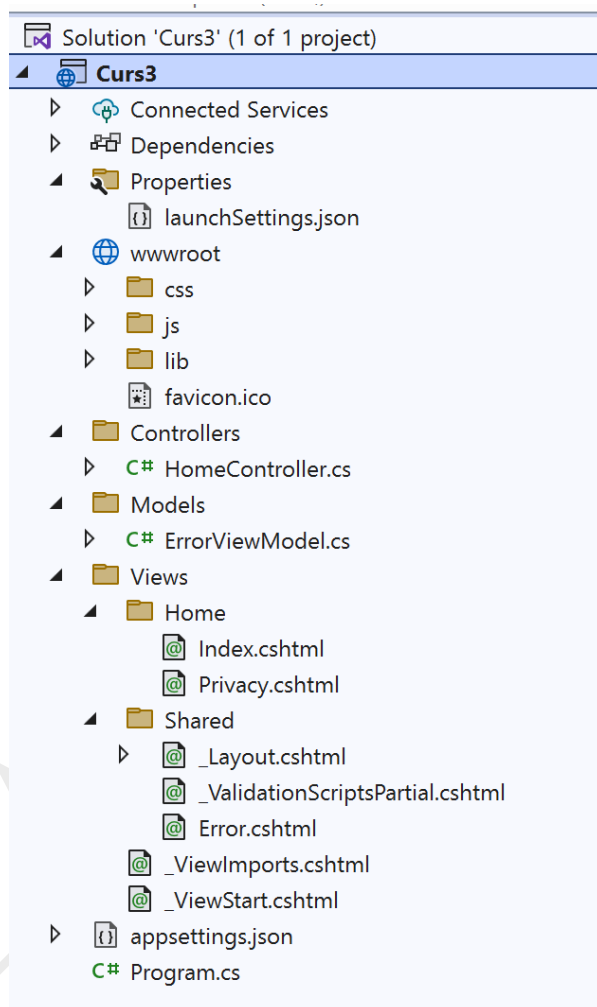
În imaginea următoare este prezentată structura unui proiect MVC, realizat în ASP.NET Core 6.0. Structura proiectului este reprezentată de sistemul de fișiere și foldere pe care le conține aplicația.



- **wwwroot** – Contine fisierele statice precum librarii (ex: Bootstrap), imagini, scripturi (css, js);
- **Controllers** – include toate fisierele de tip Controller. Acestea sunt fisiere care contin cod **c#** si au extensia **.cs**. MVC impune ca numele tuturor controller-elor sa contina la final cuvantul **Controller**;
- **Models** – folderul contine modelele aplicatiei;
- **Views** – folderul contine fisierele de tip View (interfata cu utilizatorul) ale aplicatiei;
- **appsettings.json** – in acest fisier se afla setarile pentru configuratia aplicatiei. De exemplu: se utilizeaza pentru stocarea detaliilor de conectare la baza de date;
- **Program.cs** – este punctul de pornire al aplicatiei care se acceseaza imediat dupa ce aplicatia este rulata. De asemenea, in acest fisier se configureaza modulele aplicatiei: domeniul

aplicatiei (host), serverul web (IIS, Nginx, etc), modulul de autentificare, etc;

În imaginea următoare se pot observa toate folderele și fișierele existente într-un proiect nou creat.



Tot sistemul de fișiere o să fie studiat pe rând în cursurile următoare.

Sistemul de rutare

ASP.NET a introdus termenul de **Routing** (rutarea) pentru a elimina necesitatea maparii fiecarui URL cu un fisier fizic, asa cum era necesar in versiunea anterioara de ASP.NET Web Forms, unde fiecare URL trebuia sa coincida cu un fisier .aspx.

Rutele reprezinta diferite URL-uri din aplicatie care sunt procesate de un anumit Controller si de o anumita metoda pentru generarea unui raspuns catre client. Framework-ul ASP.NET MVC invoca diverse clase de tip Controller, impreuna cu diferite metode ale acestora, in functie de URL-ul cerut de client.

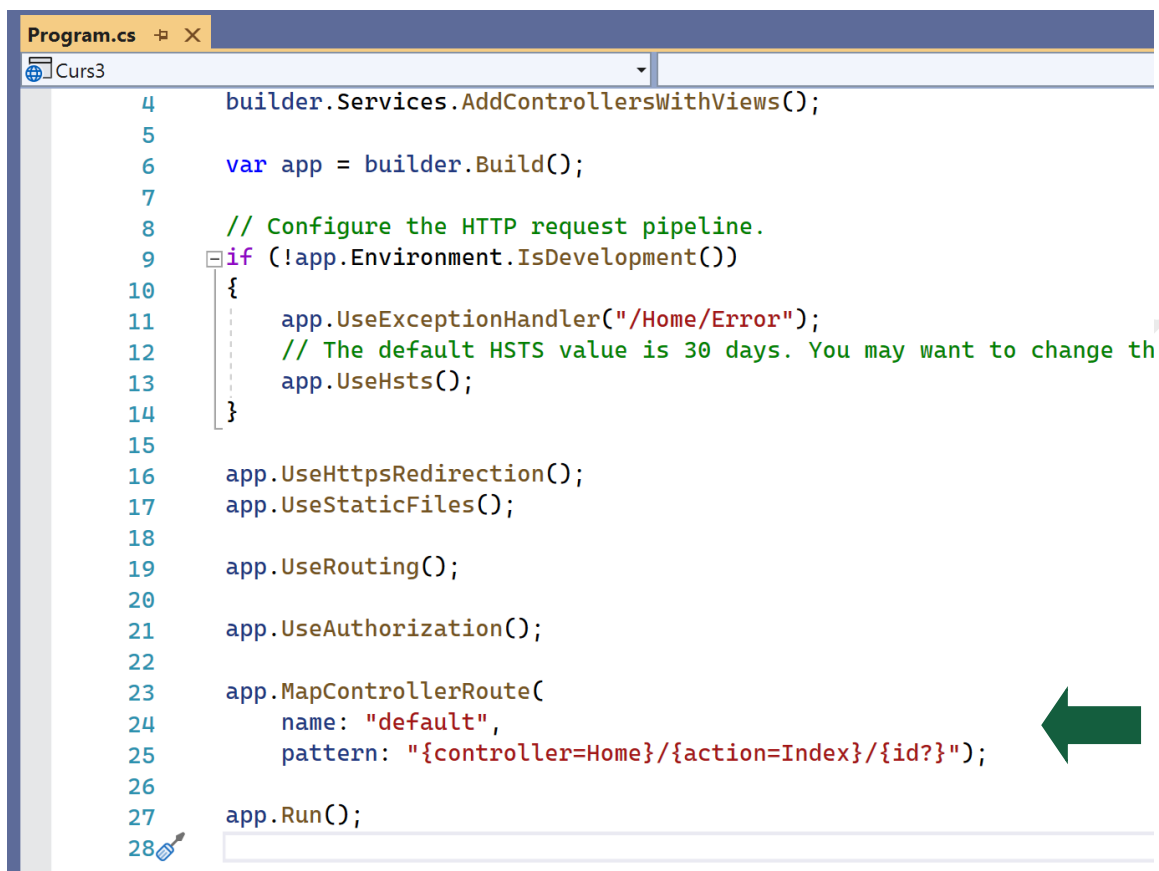
Astfel, pentru a accesa o anumita pagina, este necesar ca pentru aceasta sa existe o ruta definita, cat si un Controller care are o metoda (Action) care sa raspunda acestei resurse.

Formatul de baza al rutelor in ASP.NET este urmatorul:

`/[{NumeController}]/[{NumeActiune}]/[{Parametri}]`

Rutele se definesc in clasa **Program.cs**

In **Program.cs** se poate observa ruta default:



```

4     builder.Services.AddControllersWithViews();
5
6     var app = builder.Build();
7
8     // Configure the HTTP request pipeline.
9     if (!app.Environment.IsDevelopment())
10    {
11        app.UseExceptionHandler("/Home/Error");
12        // The default HSTS value is 30 days. You may want to change th
13        app.UseHsts();
14    }
15
16    app.UseHttpsRedirection();
17    app.UseStaticFiles();
18
19    app.UseRouting();
20
21    app.UseAuthorization();
22
23    app.MapControllerRoute(
24        name: "default",
25        pattern: "{controller=Home}/{action=Index}/{id?}");
26
27    app.Run();
28

```

Metoda **UseRouting()** se ocupa de configurarea rutelor in aplicatiei.

Definirea unei rute are urmatorul format:

```

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

```

Se observa ca variabila **app**, care este de tipul clasei **WebApplication** (clasa care se ocupa de configurarea HTTP pipeline si a rutelor), ofera mai multe metode necesare definirii rutelor.

Metoda **MapControllerRoute** adauga ruta default si primeste ca argumente 2 parametri:

- **name**: care reprezinta numele rutei (ex: name: "default")
- **pattern**: care reprezinta modelul URL-ului sau segmentele acestuia (ex: pattern: {controller=Home}/{action=Index}/{id?}).

In plus sunt definite detaliile dupa cum urmeaza:

- **controller** – primeste ca valoare numele Controller-ului care sa raspunda la aceasta ruta
- **action** - primeste ca valoare numele metodei din Controller care sa raspunda la aceasta ruta
- pentru fiecare parametru adaugat in ruta, defineste **tipul parametrilor**, daca acestia sunt **necesari** sau **optionali** sau se pot seta valorile implicite

In exemplul anterior se defineste ruta **/Home/Index/{id}** care este procesata de Controller-ul **HomeController** prin metoda **Index**. Parametrul **id** este optional si poate fi omis din URL. Fiind urmat de "?" -> **{id?}**, inseamna ca este un parametru optional.

In acest sistem de rutare, pot fi mai multi parametri delimitati prin caracterul "/".

Asadar, in acest caz, metoda **app.MapControllerRoute()** mapeaza endpoint-uri cu pattern-ul:

{controller=Home}/{action=Index}/{id?}

Endpoint-ul din ASP.NET Core este reprezentat de Controller, fiind acea unitate responsabila cu procesarea request-urilor.

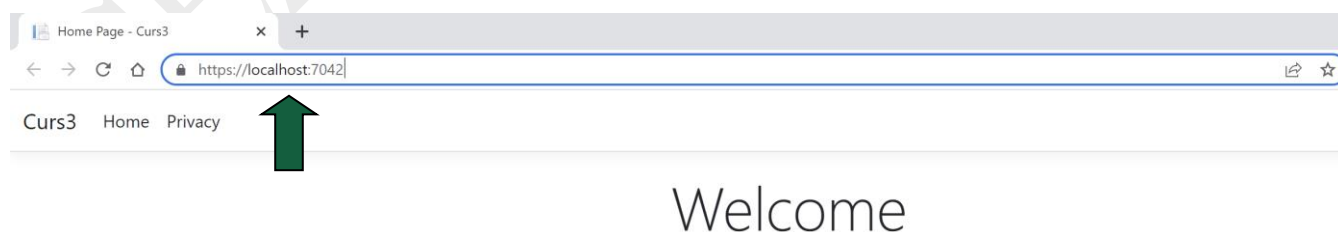
Privind exemplul anterior putem spune ca de fiecare data cand aplicatia primeste un URL care sa se potriveasca cu pattern-ul, atunci o sa se acceseze Controller-ul numit Home si Action-ul (metoda) Index. Acest lucru se intampla doar in cazul in care nu este specificat un alt Controller sau o alta metoda. In cazul in care sunt specificate explicit alte endpoints, atunci acelea o sa fie accesate.

In cazul in care aplicatia este accesata fara segmentele necesare in cadrul URL-ului, adica se cere pagina principala a aplicatiei “/”, framework-ul ASP.NET MVC va raspunde in mod implicit cu metoda “**Index**” din Controllerul “**Home**”.

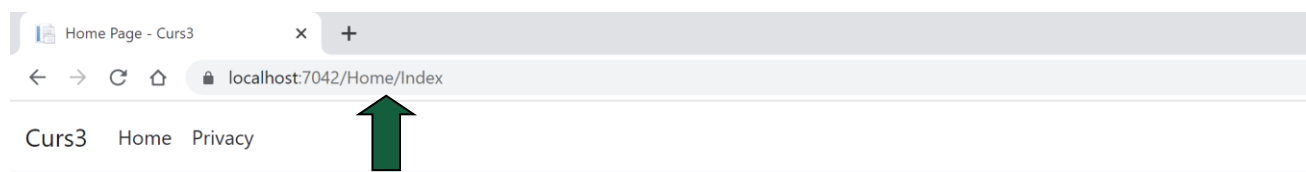
In concluzie, pattern-ul `{controller=Home}/{action=Index}/{id?}` se potriveste cu toate URL-urile de forma: /, /**Home**, /**Home/Index**, /**Students/Index**, /**Students/Index/5**, /**Students/Afisare**, etc.

In exemplul urmator se poate observa existenta Controller-ului **HomeController**. In cadrul clasei HomeController exista mai multe metode (Actions). Se poate observa prezenta metodei **Index()**. In momentul in care exista o metoda, trebuie sa existe si un View asociat. View-ul trebuie sa se numeasca identic cu metoda, deci **Index.cshtml**. Privind configuratia default a rutei, putem observa ca aceasta este ruta default, adica ruta care se acceseaza prima si afiseaza catre utilizatorul final interfata care se afla in Index.cshtml.

Dupa rulare se acceseaza in browser ruta default:

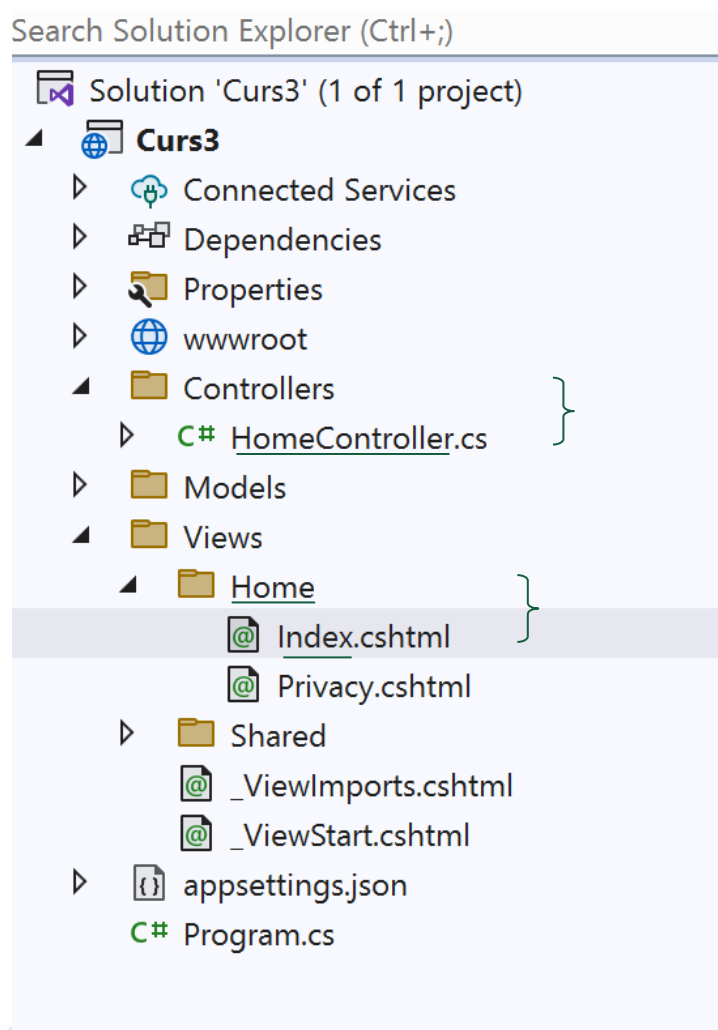


În cazul în care se accesează pagina prin scrierea întregului URL, atunci ruta o să aibă următorul format:

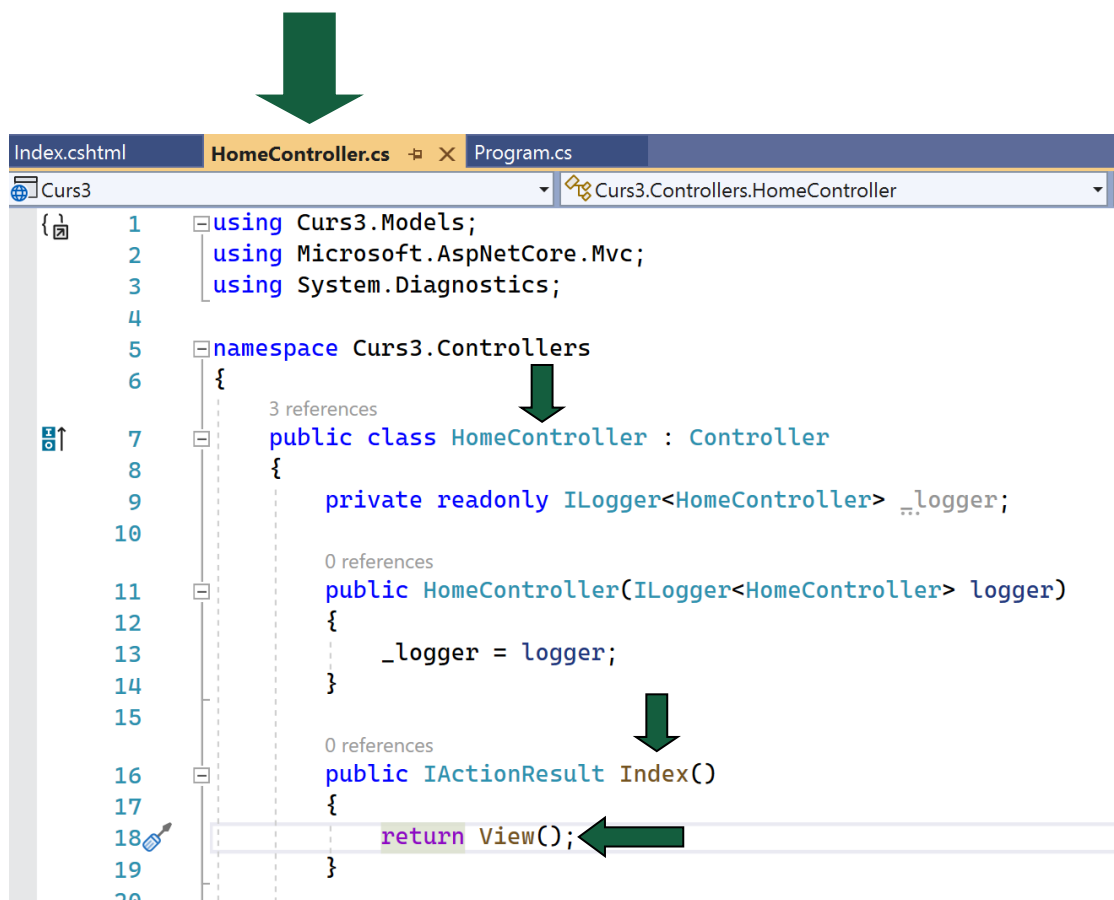


Welcome

In **Solution Explorer** se observa:



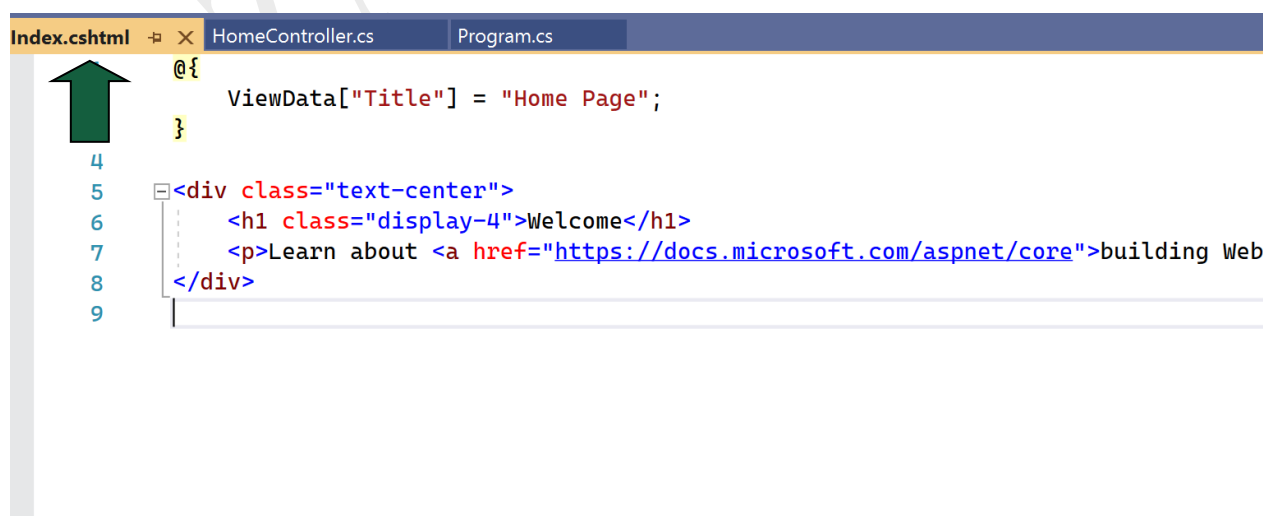
In **HomeController** exista metoda Index:



```

1  using Curs3.Models;
2  using Microsoft.AspNetCore.Mvc;
3  using System.Diagnostics;
4
5  namespace Curs3.Controllers
6  {
7      public class HomeController : Controller
8      {
9          private readonly ILogger<HomeController> _logger;
10
11         public HomeController(ILogger<HomeController> logger)
12         {
13             _logger = logger;
14         }
15
16         public IActionResult Index()
17         {
18             return View();
19         }
20     }
  
```

In **View** -> Folderul Home (asociat Controller-ului HomeController) -> Index.cshtml (pagina pentru codul html asociata metodei Index din Controller)



```

1  @{
2      ViewData["Title"] = "Home Page";
3  }
4
5  <div class="text-center">
6      <h1 class="display-4">Welcome</h1>
7      <p>Learn about <a href="https://docs.microsoft.com/aspnet/core">building Web
8  </div>
9
  
```

În cazul în care definiția rutei default nu conține și valorile implicite, atunci nu se pot accesa paginile aplicației.

De ex: se elimină ruta deja definită și se adaugă următoarea configurație:

```
/*
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
*/

app.MapControllerRoute(
    name: "default",
    pattern: "{controller}/{action}");
```

În momentul rularii apare mesajul **HTTP ERROR 404** deoarece nu găsește pagina. Acest lucru se întâmplă din cauza faptului că sistemul de rutare nu poate asocia ruta cu niciun Controller.



This localhost page can't be found

No webpage was found for the web address: <https://localhost:7042/>

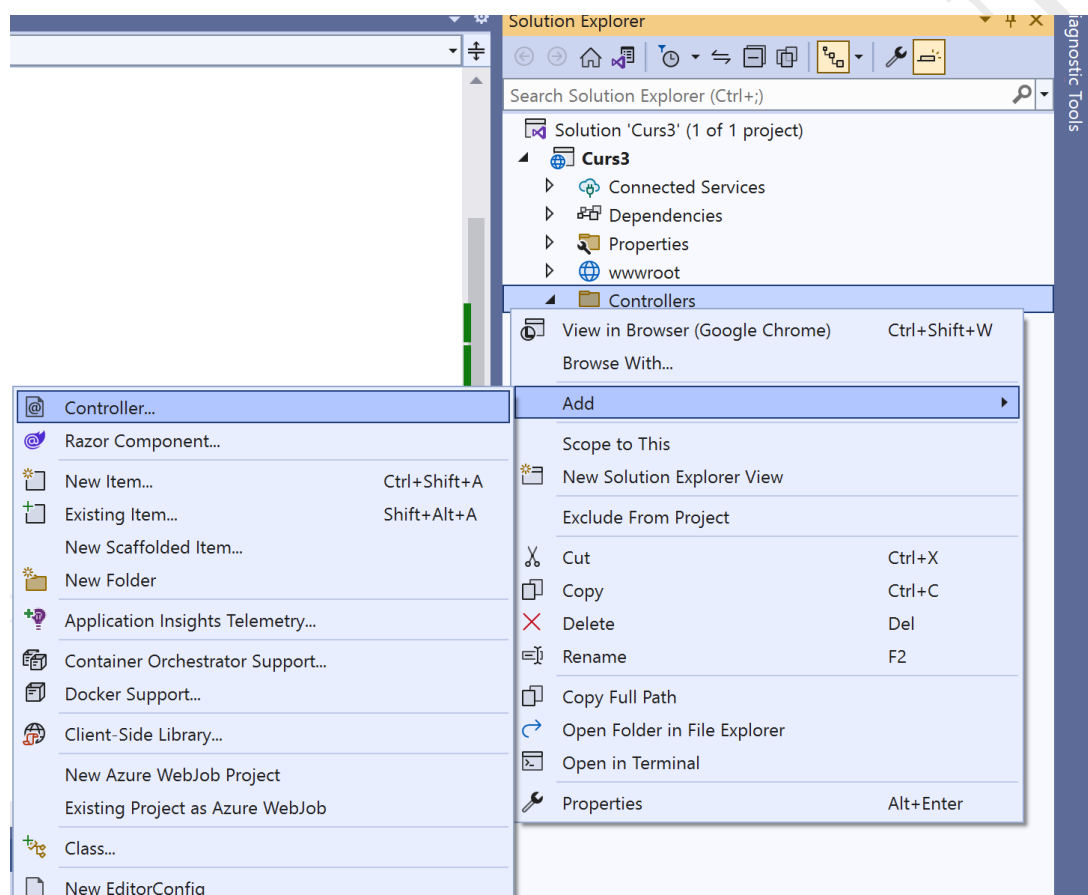
HTTP ERROR 404

Reload

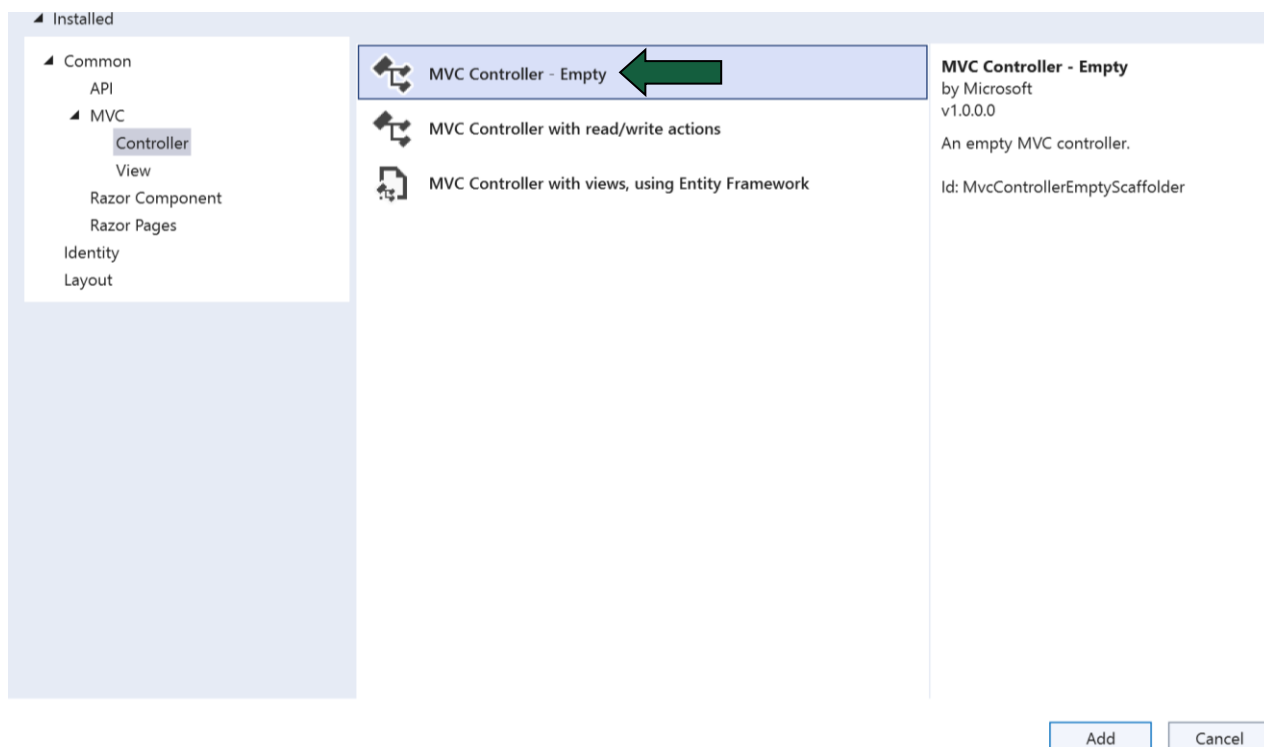
Exemple de implementare a rutelor:

Configurarea rutelor:

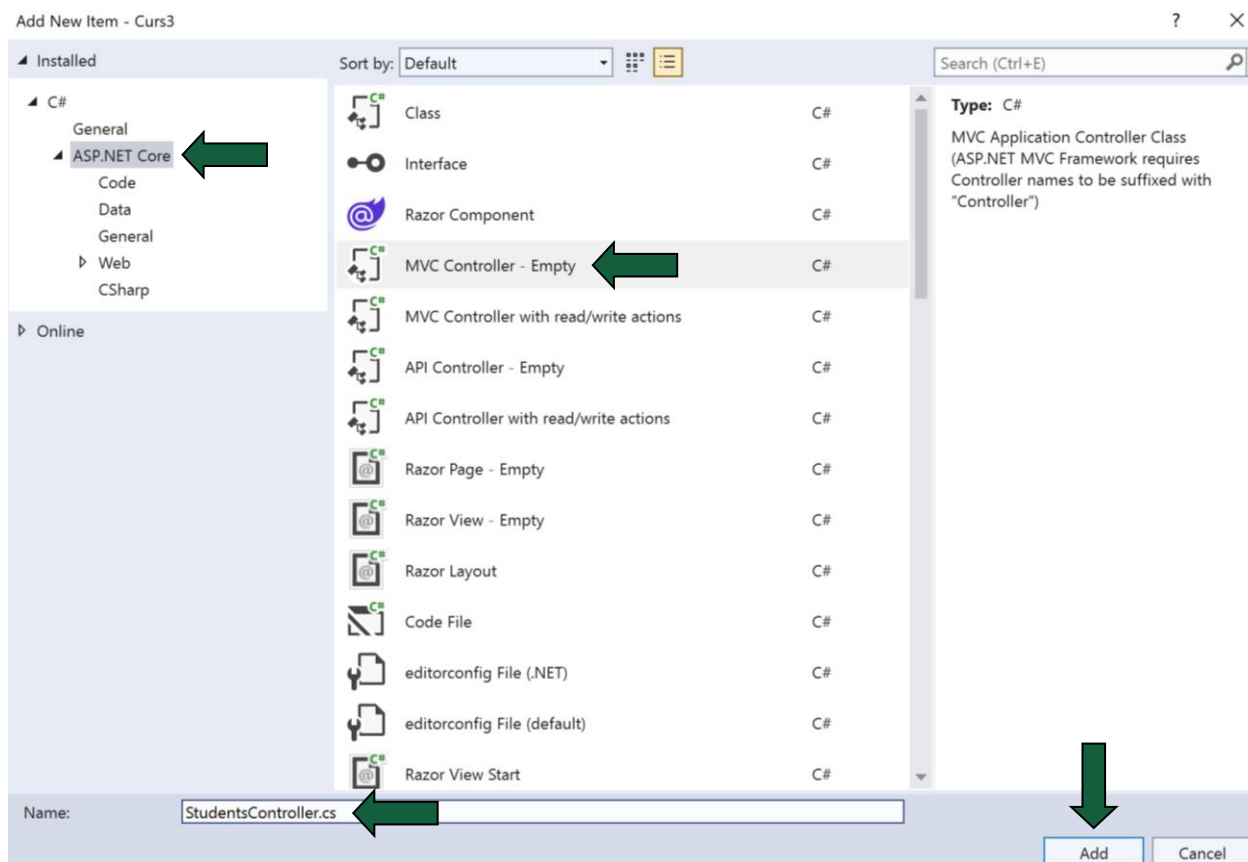
Pentru exemplele urmatoare se va crea un nou Controller, numit **StudentsController**. Click dreapta pe folderul Controller -> Add -> Controller.



Se selecteaza MVC Controller – Empty:



Se modifica numele noului Controller:



Pentru afisarea unui text, o sa se utilizeze metoda Index din cadrul Controller-ului **StudentsController**. In acest moment nu se va utiliza niciun View asociat.

```
public class StudentsController : Controller
{
    public string Index()
    {
        string response = "Hello World";
        return response;
        //return View();
    }
}
```

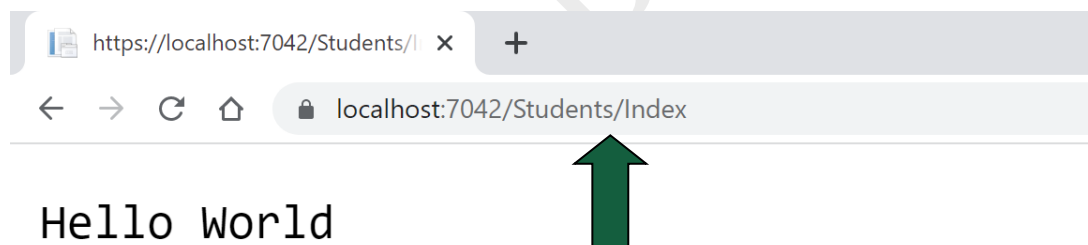
Tipul returnat de metoda Index a fost schimbat in **string** pentru a putea afisa in browser un simplu text. Astfel, valoarea de return devine valoarea variabilei **response**.

Dupa rulare, mesajul o sa fie afisat in browser, accesand ruta **/Students/Index (/NumeController/NumeActiune)**. In momentul accesarii URL-ului, request-ul se trimite aplicatiei, dupa care se incearca maparea URL-ului cu o configuratie de ruta prezenta in Program.cs.

Astfel, pattern-ul o sa primeasca noile valori:

- controller = Students
- action = Index
- id = este optional si poate sa lipseasca

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```



In continuare se defineste o ruta dupa cum urmeaza:

- ruta o sa contina doi parametri – name si id
- parametrul name o sa aiba o valoare implicita “World!”;
- parametrul id o sa fie optional

Metoda Index din Controller o sa afiseze mesajul “Hello World!”, folosind valoarea parametrului **name**, provenita din cadrul rutei.

Definirea rutei:

```
app.MapControllerRoute(
    name: "HelloWorld",
    pattern: "{controller=Students}/{action=Index}/{name=World!}/{id?}");
```

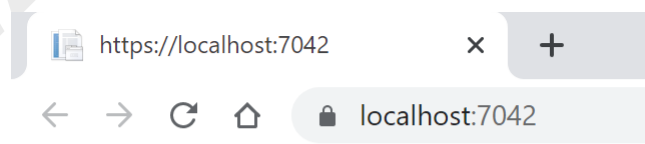
Implementarea metodei in Controller-ul **StudentsController**, metoda **Index**.

```
public string Index(string name, int? id)
{
    string response = "Hello " + name + " ";
    if (id != null)
    {
        response = response + "id = " + id;
    }

    return response;

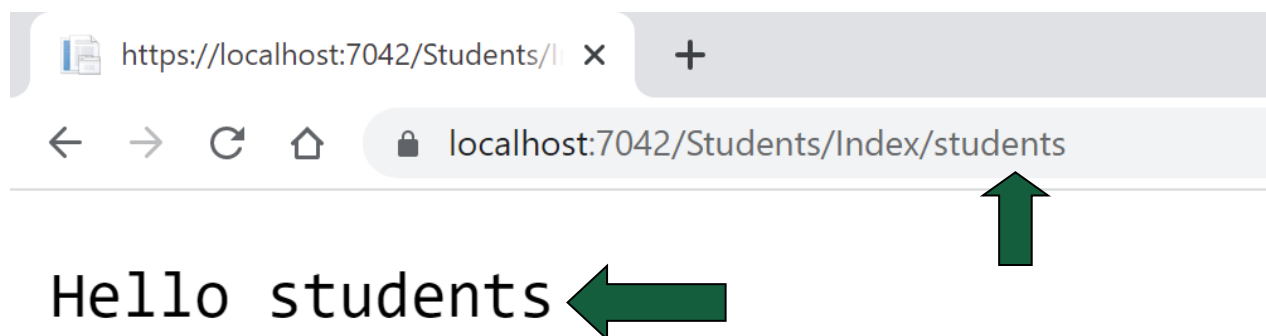
    //return View();
}
```

In momentul in care se ruleaza aplicatia, fara a introduce segmentele URL-ului, se observa ca pentru variabila **name** s-a transmis valoarea acesteia implicita: **World!**, afisandu-se mesajul Hello World!

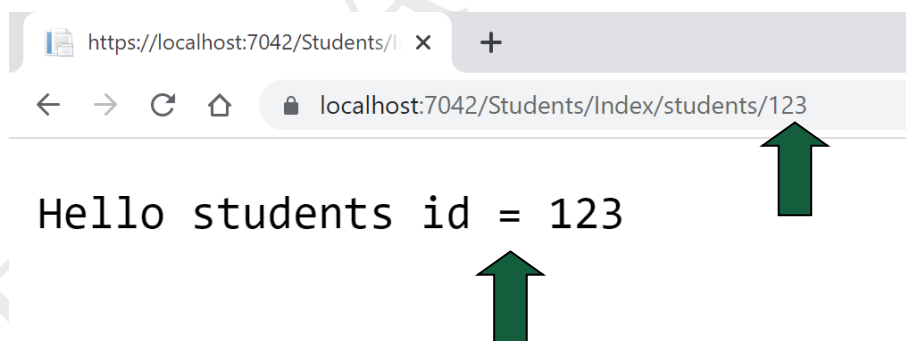


Hello World!

La adaugarea valorii pentru variabila **name** in URL, se observa cum aceasta a fost transmisa catre Controller si a fost afisata in pagina.



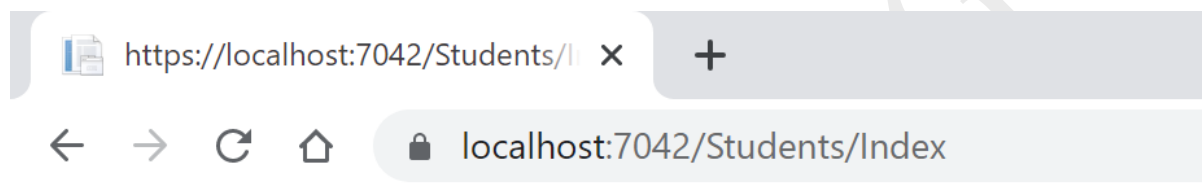
Cand parametrul optional **id** are valoare, secventa de cod specifica acestuia este executata si valoarea sa apare in raspunsul primit de la Controller:



OBSERVATIE:

/! Ruta trebuie definita inaintea rutei default, deja existenta in fisierul Program.cs, deoarece rutele sunt interpretate in mod cascada (de sus in jos). Framework-ul utilizeaza prima configuratie din fisier care contine acelasi numar de parametri ca ruta accesata din browser.

De exemplu, daca ruta default este definita inaintea rutei creata in exemplul anterior, iar URL-ul de accesare este **/Students/Index**, atunci configuratia rutei default o sa se potriveasca si vom avea un rezultat ca cel de mai jos. Nu se afiseaza si valoarea implicita a parametrului **name**.



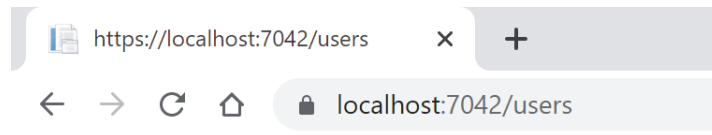
Hello

Definirea rutelor custom

Pentru fiecare **Controller** si **Action** in parte se pot defini si rute custom. De exemplu: daca se doreste accesarea Controller-ului Students si a metodei Index printr-un URL de forma: **/users** se poate implementa urmatoarea ruta:

```
app.MapControllerRoute(
    name: "Users",
    pattern: "users/{controller=Students}/{action=Index}/{name=world!!!}");
```

In varianta anterioara se poate accesa /Students/Index/name doar prin intermediul URL-ului: **/users** deoarece restul parametrilor vor prelua valorile implicite. Daca trebuie introdusa o alta valoare pentru parametrul **name**, atunci URL-ul trebuie sa fie: **users/Students/Index/abc**.



Hello world!!!

Astfel, ruta **/users** a accesat Controller-ul **StudentsController**, metoda **Index** cu paramentru implicit **name=world!!!**

In acelasi mod se poate proceda si pentru mai multe elemente in ruta:

```
app.MapControllerRoute(
    name: "HomePage",
    pattern: "Home/Page/{controller=Home}/{action=Index}");
```



Definitia anterioara a rutei functioneaza si pentru URL-uri de tipul:

```
/Home/Page/Students/Index
/Home/Page/Users/Read
```

Adica functioneaza pentru orice alt nume de Controller si Actiune.

Daca se doreste limitarea unei rute la o singura actiune, dintr-un singur Controller, atunci se foloseste urmatoarea varianta:

```
app.MapControllerRoute(
    name: "HomePage",
    pattern: "Home/Page",
    defaults: new { controller = "Home", action = "Index" });
```

Acest lucru se intampla deoarece pattern-ul nu are niciun parametru configurabil, iar ruta o sa mapeze doar Controller-ul Home si metoda Index.

De asemenea, varianta anterioara se foloseste si pentru cazul in care se doreste accesarea rutei doar printr-un URL custom. De ex: **/users**

```
app.MapControllerRoute(
    name: "Users2",
    pattern: "users/{name?}/{id?}",
    defaults: new { controller = "Students", action = "Index" });
```

OBSERVATIE:

⚠ In momentul scrierii rutelor, dezvoltatorul trebuie sa se asigure ca nu exista ambiguitate intre definitiile acestora.

Constrangerile parametrilor

Pentru a asigura un anumit tip de date sau un anumit format pentru parametrii transmisi catre Controller este necesara declararea unor constrangeri.

Exista mai multe tipuri de constrangeri: de tip, length, max, min, range, regex.

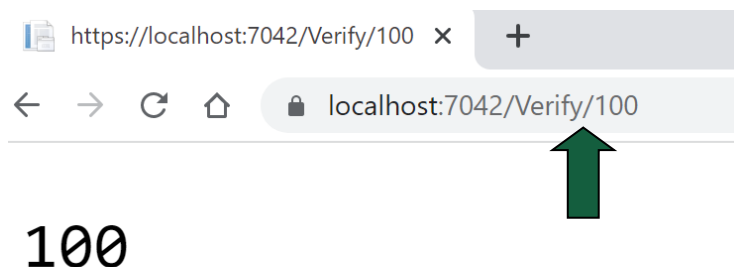
Un exemplu de constrangere este:

```
app.MapControllerRoute(
    name: "HomePage",
    pattern: "Verify/{id:range(10,100)}",
    defaults: new { controller = "Home", action = "Verify" });
```

Pentru un URL de tipul **/Verify/50** se cauta pattern-ul potrivit, dupa care se acceseaza parametrii specifici din **defaults** -> Controller-ul **Home** si metoda **Verify**. Daca parametrul **id** se afla in intervalul inchis 10, 100, atunci o sa acceseze ruta, iar in caz contrar o sa se afiseze 404 Not Found.

Pentru verificare se poate implementa in Controller-ul **Home**, metoda **Verify** care afiseaza id-ul in pagina.

```
public int Verify(int id)
{
    return id;
}
```

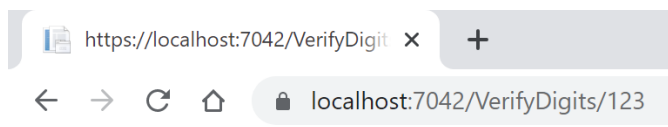


In cazul **expresiilor regulate** se configureaza ruta astfel:

```
app.MapControllerRoute(
    name: "HomePage",
    pattern: "VerifyDigits/{id:regex(\\d+)}",
    defaults: new { controller = "Home", action = "VerifyDigits" });
```

Ruta se acceseaza folosind URL-ul: **/VerifyDigits/id**, id-ul are o constrangere folosind o expresie regulata – se verifica daca este numar accesand apoi metoda **VerifyDigits** din Controller-ul **Home**.

```
public string VerifyDigits(int id)
{
    return "VerifyDigits " + id;
}
```



VerifyDigits 123

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 4

Cuprins

Controller	2
Ce este Controller-ul	2
Crearea unui proiect	3
Adaugarea unui nou Controller	4
Controller-ul implicit – HomeController	7
Actions	9
Structura unei metode	9
Raspunsul actiunilor – ActionResult	10
Parametrii unei actiuni	13
Selectorii	13
ActionName	14
NonAction	15
ActionVerbs	15
Exemplu definire actiuni	17
Redirect in cadrul metodelor	19
Redirect	19
RedirectToRoute	19
RedirectToAction	20
RedirectPermanent/ RedirectToRoutePermanent/ RedirectToActionPermanent	21
Returnare HTTP Status Code	21

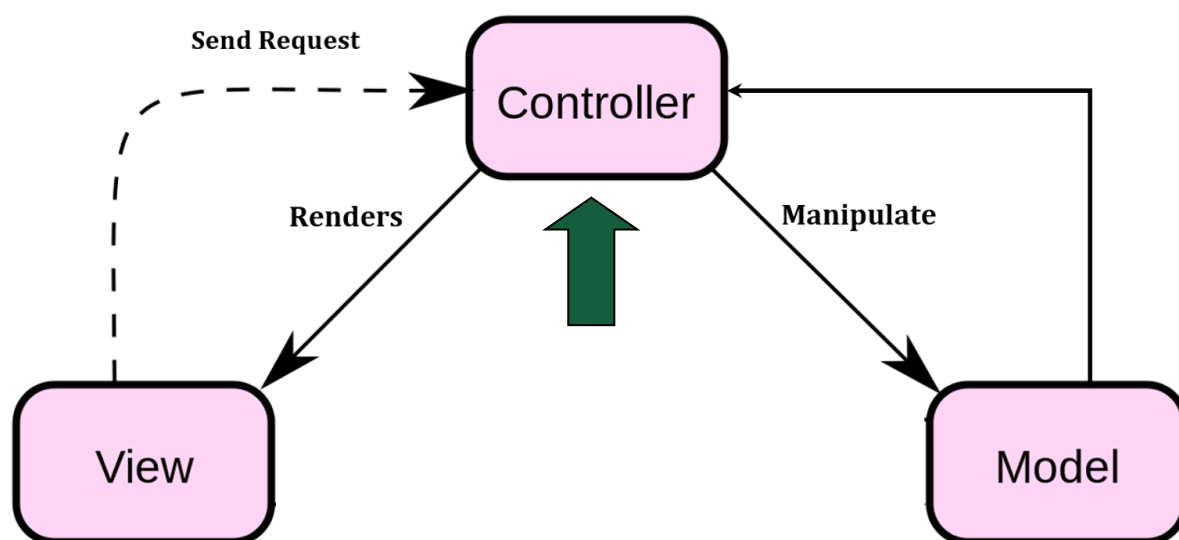
Controller

Ce este Controller-ul

În arhitectura MVC **Controller-ul** este componenta care procesează toate URL-urile aplicației. Controller-ul este o clasă, derivată din clasa de bază `Microsoft.AspNetCore.Mvc`. Această clasă conține metode publice numite **Acțiuni**. Metodele din Controller sunt responsabile pentru a procesa request-urile venite de la browser, pentru apelarea modelelor și procesarea datelor, cât și pentru a trimite răspunsul final către utilizator prin intermediul browser-ului.

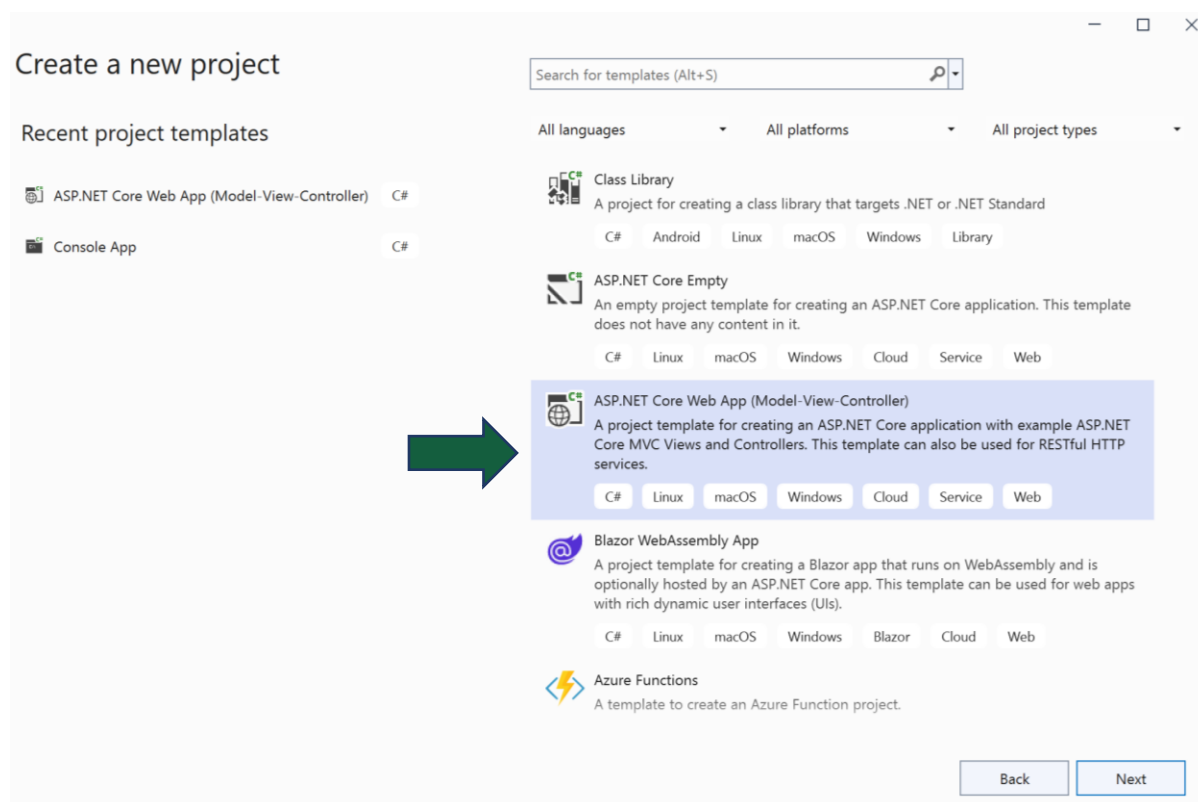
În ASP.NET MVC fiecare Controller este reprezentat de o clasă. Numele Controller-ului trebuie să se termine în cuvântul **Controller**. De exemplu, Controller-ul pentru pagina Home se poate numi **HomeController**.

Controller-ele trebuie să fie adăugate în cadrul folderului **Controllers** din proiectul ASP.NET.



Crearea unui proiect

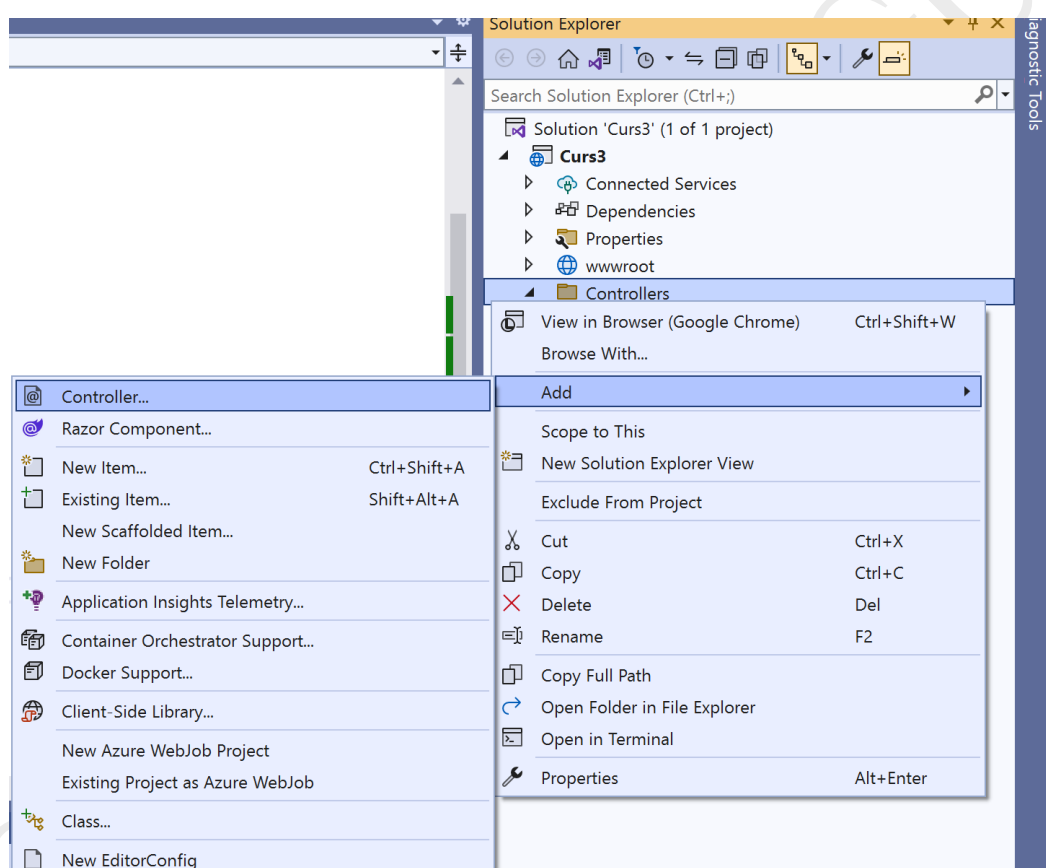
Pentru crearea unui nou proiect se utilizeaza optiunea ASP.NET Core Wep App (Model-View-Controller).



Adaugarea unui nou Controller

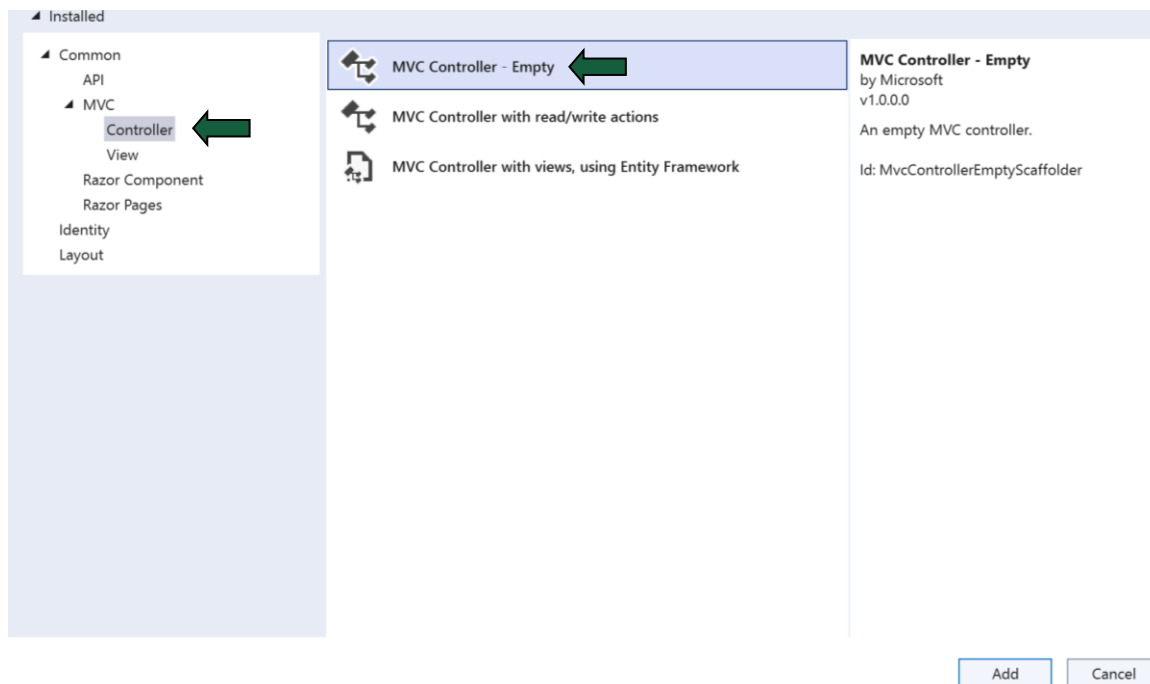
Pas 1:

Pentru a adauga un Controller se procedeaza astfel: click dreapta pe folderul **Controllers** si din meniul **Add** se selecteaza **Controller**.



Pas 2:

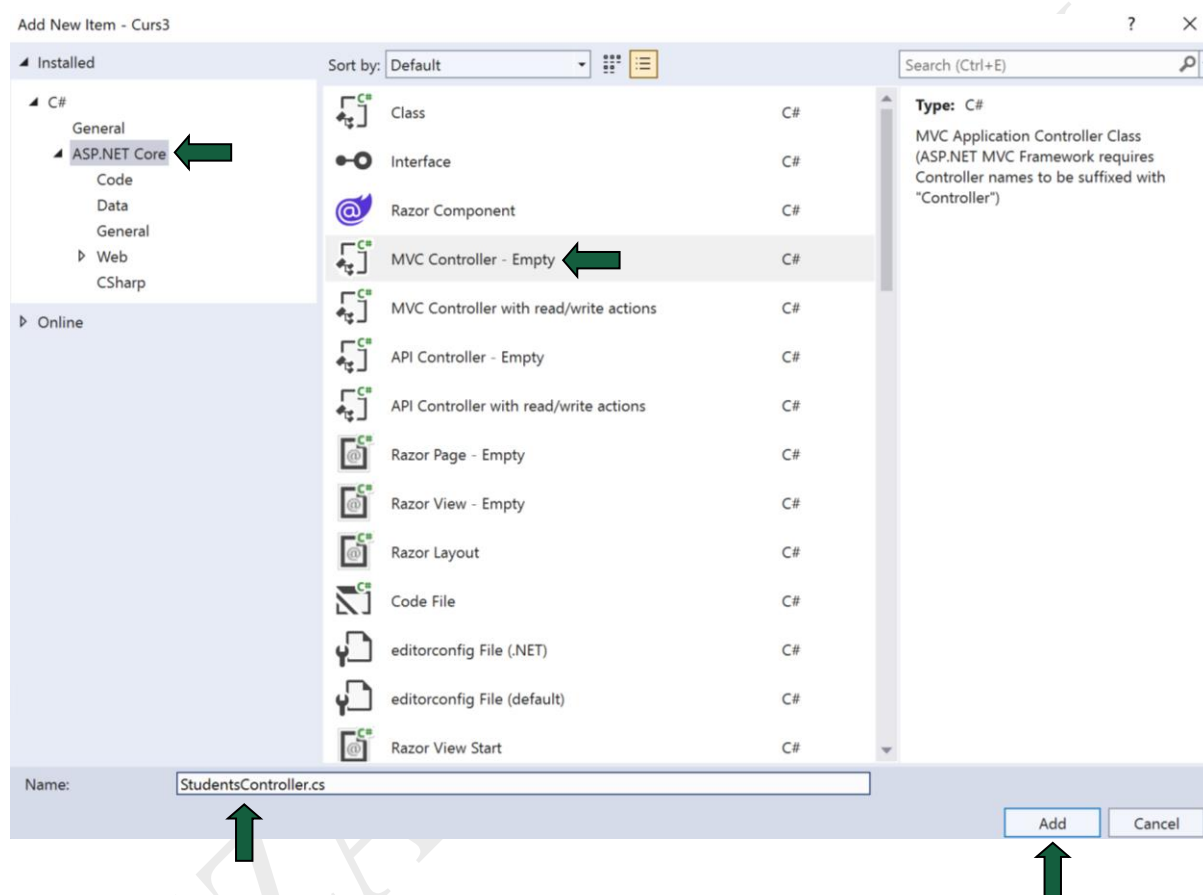
În fereastra aparuta se selecteaza **MVC Controller – Empty**:



Pas 3:

Se selecteaza din meniul aflat in partea stanga **ASP.NET Core**, dupa care optiunea **MVC Controller – Empty**.

Se adauga numele Controller-ului, nume care trebuie sa aiba sufixul Controller. De exemplu: **StudentsController**.



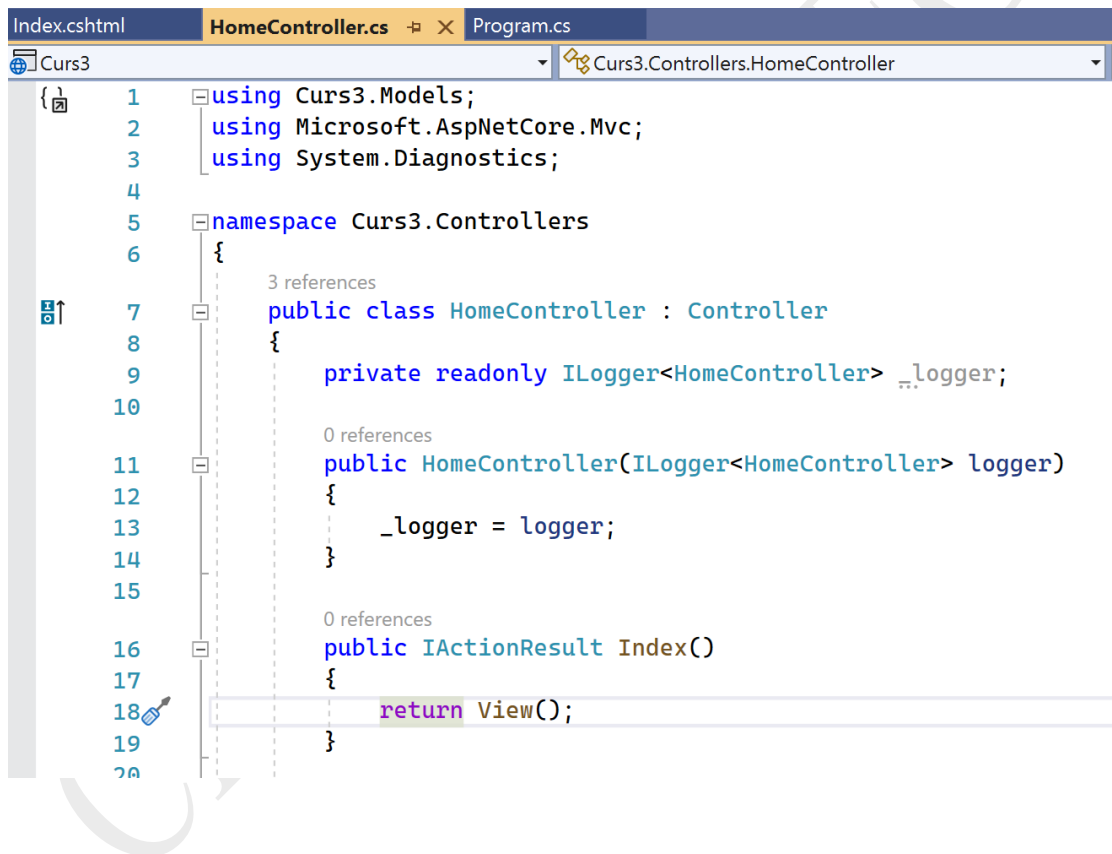
Controller-ul implicit – HomeController

În Folderul Controller se observa existenta unui Controller implicit. Acesta este creat automat în momentul în care se creează un nou proiect.

!! Observatie

Numele unui Controller trebuie să conțină sufixul Controller (ex: **HomeController**).

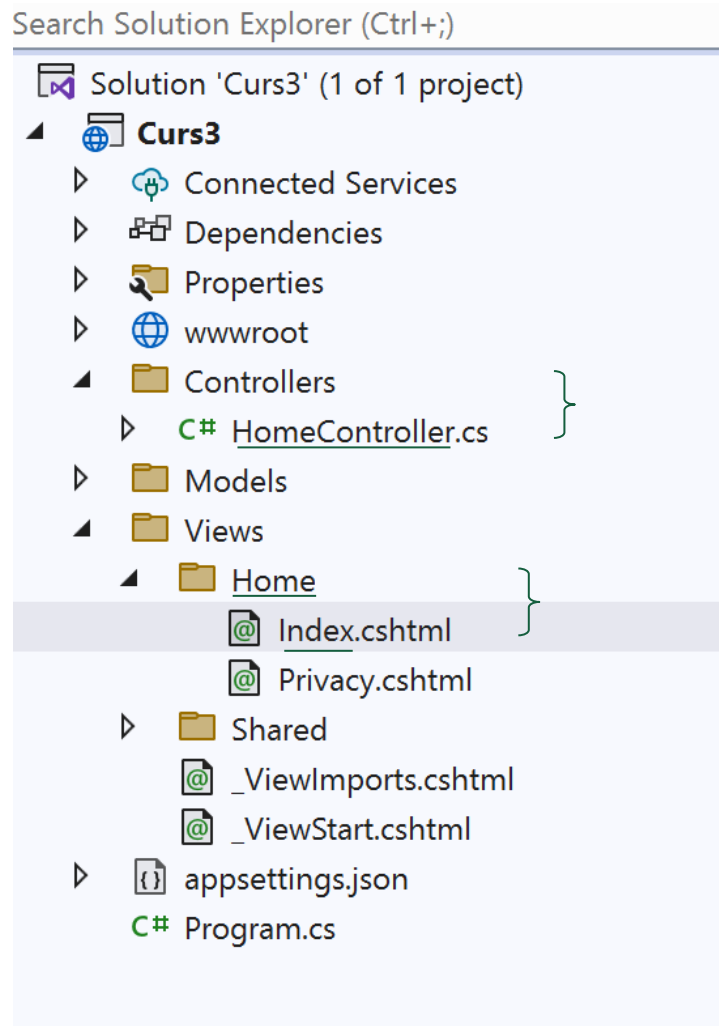
Controller-ul numit HomeController este de fapt o clasă care conține mai multe metode publice (Actions).



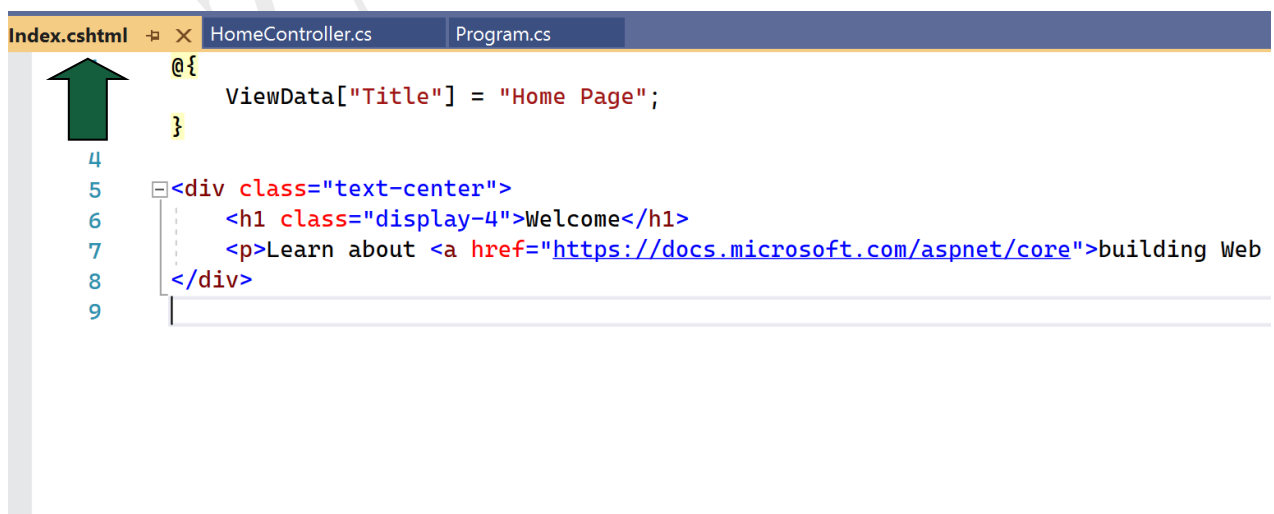
```

1  using Curs3.Models;
2  using Microsoft.AspNetCore.Mvc;
3  using System.Diagnostics;
4
5  namespace Curs3.Controllers
6  {
7      3 references
8      public class HomeController : Controller
9      {
10         private readonly ILogger<HomeController> _logger;
11
12         0 references
13         public HomeController(ILogger<HomeController> logger)
14         {
15             _logger = logger;
16
17         0 references
18         public IActionResult Index()
19         {
20             return View();
21         }
22     }
  
```

Se poate observa metoda **Index()** care returnează un View. Așadar, metoda Index o să aibă propriul View, numit exact ca metoda → **Index.cshtml**, aflat în folderul View → Folderul Home (deoarece acesta este numele Controller-ului) **(VEZI imaginea de mai jos)**



Index.cshtml



View-ul anterior, numit **Index.cshtml**, contine doua tipuri de cod:

- O expresie Razor care contine cod C# – un obiect de tip dictionar **ViewData** (vom studia in cursul destinat View-urilor).
Codul C# este inclus folosind simbolul @
- Elemente de html pentru interfata (UI)

Actions

Structura unei metode

```
public class StudentsController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

In Controller-ul **StudentsController** se observa metoda Index() de tipul **IActionResult**. Aceasta este definita ca fiind **public** pentru a putea fi accesata de framework.

De asemenea, metodele **trebuie sa fie publice**, deoarece ele vor fi apelate in urma request-urilor primite din browser (request-uri HTTP). Pot exista si metode **private**, dar acestea se utilizeaza doar intern, nefiind accesibile prin intermediul URL-urilor.

Actiunile **pot fi supraincarcate** (adica sa existe metode cu acelasi nume, in aceeasi clasa) doar daca se utilizeaza cu parametri de intrare diferiti (tip de date, numar).

Tipul returnat **IActionResult** este o interfata, reprezentand raspunsul actiunii trimis de catre Controller la browser, fiind cel mai frecvent tip utilizat deoarece intotdeauna se va apela un View pentru a afisa informatiile catre utilizatorul final.

Metoda **View()** din interiorul actiunii este definita in clasa abstracta de baza **Controller** si este de tipul **ViewResult : ActionResult**.

Deoarece clasa ActionResult implementeaza interfata IActionResult este suficient ca tipul de return al metodei sa fie **IActionResult**.

Raspunsul actiunilor – ActionResult

Framework-ul MVC include diverse tipuri de rezultat care pot fi returnate prin intermediul **ActionResult**. Aceste clase de rezultat pot fi tipuri de date diferite: html, fisiere, string-uri, json, obiecte, etc.

Posibilele tipuri de date returnate pentru ActionResult sunt:

- **ViewResult** – aceasta clasa returneaza continut HTML
- **EmptyResult** – aceasta clasa returneaza un raspuns gol – pagina returnata nu are niciun continut (ex: returneaza status code 200 -> adica request-ul a fost executat corect, dar raspunsul este gol)
- **ContentResult** - poate fi folosit pentru a returna text
- **FileContentResult / FileStreamResult** - reprezinta continutul unui fisier (folosit pentru descarcarea fisierelor)
- **JsonResult** – reprezinta un JSON care poate fi cerut prin AJAX sau alte metode
- **RedirectResult** – reprezinta redirectionarea catre un nou URL

- **RedirectToRouteResult** – reprezinta redirectionarea catre o alta actiune in acelasi Controller sau in alt Controller
- **PartialViewResult** – returneaza HTML-ul dintr-un partial
- **StatusCodeResult** – returneaza un raspuns de tip: BadRequest (400), Unauthorized (401), Forbidden (403), NotFound (404).

ActionResult este clasa de baza a tuturor claselor enumerate mai sus. ActionResult implementeaza interfata IActionResult. Deci, indiferent de raspunsul folosit, actiunea poate sa aiba tipul de raspuns **IActionResult**.

Pentru a returna tipurile de date mentionate mai sus clasa de baza Controller are urmatoarele metode implementate:

- ViewResult -> View()
- ContentResult -> Content() – primeste ca parametru un string care va fi afisat in browser
- FileContentResult/FilePathResult/FileStreamResult -> File()
- JsonResult -> Json() – primeste ca parametru orice tip de date si va returna un raspuns sub forma JSON (se va serializa parametrul primit sub forma unui string JSON)
- RedirectResult -> Redirect() – primeste ca parametru un URL (in format string) si va redirectiona browser-ul catre acel URL
- RedirectToRouteResult -> RedirectToRoute() – primeste ca parametru un **nume de ruta** (care este definita in fisierul Program.cs) si va redirectiona browser-ul catre acea ruta
- PartialViewResult -> PartialView() – returneaza continutul unui partial

Avantajul utilizarii ca tip de date de return **IActionResult** este acela ca se poate utiliza oricare dintre tipurile de raspuns enumerate mai sus fara a schimba tipul de date al actiunii.

De exemplu, pentru a descarca un fisier prin intermediul unei rute putem folosi urmatoarea secventa de cod:

```
public IActionResult Download()
{
    byte[] fileBytes =
        System.IO.File.ReadAllBytes(@"c:\folder\myfile.ext");
    string downloadName = "myfile.ext";
    return File(fileBytes,
        System.Net.Mime.MediaTypeNames.Application.Octet, downloadName);
}
```

Pasul 1: se citește fisierul ca secvență de bytes de la o cale cunoscută

Pasul 2: setăm un nume de fisier pentru fisierul care va fi descărcat – downloadName

Pasul 3: returnăm metoda File care primește 3 parametri:

- Array-ul de bytes care stochează conținutul fisierului
- MIMEType-ul (MediaType) fisierului descărcat (ex: pentru fișiere **.mp3** -> **audio/mpeg**; pentru **imagini de tip JPEG** -> **image/jpeg**; pentru **imagini de tip PNG** -> **image/png**) pentru encodarea corectă a fisierului salvat
- Numele fisierului care va fi salvat pe client

Parametrii unei actiuni

Fiecare actiune poate sa primeasca parametrii unei rute in cadrul semnaturii acesteia. Parametrii rutei pot fi de orice tip (string, int, float sau chiar de tipul clasei unui Model).

```
public IActionResult Show(Student student)
{
    // obiectul student de tipul Modelului Student va
    // contine informatiile unui student din baza de date
}
```

In exemplul anterior, pentru ruta “/Students/Show/2” obiectul student va fi instantiat in mod automat cu valorile studentului cu ID-ul 2 din baza de date, prin intermediul modelului. Acest lucru va fi detaliat in cursul referitor la Model.

!OBSERVATIE:

Numele parametrilor definiti in semnatura Actiunii **trebuie sa fie acelasi** cu numele parametrilor definiti in **Program.cs**. Diferenta dintre numele parametrilor va conduce catre nerezolvarea acestora (vor avea valoarea null).

Selectorii

Framework-ul ASP.NET Core MVC ofera posibilitatea adaugarii unor attribute actiunilor, pentru a ajuta sistemul de rutare sa aleaga actiunea corecta in momentul procesarii unui request. Aceste attribute se numesc **Selectorii** si sunt:

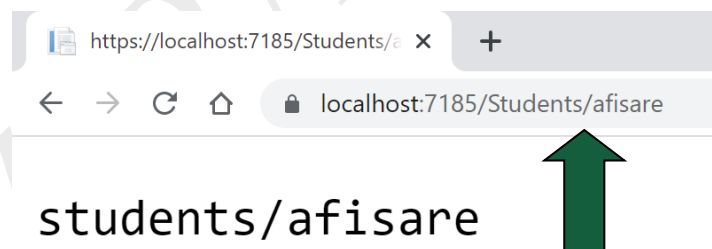
- ActionName
- NonAction
- ActionVerbs

ActionName

Atributul **ActionName** ofera posibilitatea de a alocu un nume unei actiuni, care este diferit de numele acesteia. De exemplu, daca avem o metoda numita **Index** in Controller-ul **StudentsController** putem redenumi aceasta actiune prin intermediul atributului ActionName astfel:

```
[ActionName("afisare")]
public IActionResult Index()
{
    return Content("students/afisare");
}
```

Inainte de adaugarea atributului ActionName pagina se putea accesa prin URL-ul: **/Students/Index**. Dupa adaugarea atributului ActionName cu valoarea "afisare" pagina poate fi accesata prin intermediul URL-ului **/Students/afisare**. Astfel, acest atribut ne ofera posibilitatea rescrierii numelui actiunii. Acest lucru se intampla fara a aduce modificari fisierului Program.cs.



NonAction

Atributul **NonAction** indica faptul ca o metoda a unui Controller nu este o actiune. Acest atribut se foloseste in momentul in care dorim ca o metoda publica a unui Controller sa nu poata fi accesata prin intermediul unei rute.

Exemplu:

```
[NonAction]
public Student GetStudent(int id)
{
    return ...;
}
```

Aceasta metoda nu poate fi accesata prin intermediul unei rute, desi este publica. In schimb, ea poate fi accesata din celelalte metode ale aceluiasi Controller sau ale unui alt Controller.

ActionVerbs

Atributul **ActionVerbs** este folosit in momentul in care se doreste accesarea unei actiuni in functie de **verbul HTTP**. De exemplu, se pot defini doua actiuni cu acelasi nume, inasa care raspund la un verb HTTP diferit si au parametri diferiti.

Verbele HTTP acceptate sunt urmatoarele: **GET, POST, PUT, PATCH, HEAD, OPTIONS** si **DELETE**.

!OBSERVATIE:

In cazul in care atributul ActionVerbs este omis, verbul default folosit este GET.

Aceste verbe sunt folosite in urmatoarele contexte:

- **GET**: este folosit in accesarea unei resurse (cererea unei pagini de la server)
- **POST** – este folosit in crearea unei resurse sau trimiterea datelor la server prin intermediul unui formular
- **PUT/PATCH** – verbul este folosit pentru modificarea (totala sau partiala) a unei resurse. De exemplu: cand se editeaza o intrare deja existenta in baza de date, se foloseste unul dintre aceste verbe
- **DELETE** – verb folosit pentru stergerea unei resurse
- **HEAD** – este identic cu GET, dar returneaza doar antetele pentru raspuns, nu si continutul raspunsului. De obicei se foloseste pentru a verifica daca exista o resursa sau daca poate fi accesata
- **OPTIONS** – returneaza metodele HTTP acceptate de server pentru o adresa URL specificata

!/OBSERVATIE:

In ASP.NET Core MVC pentru editare (PUT) si stergere (DELETE) se foloseste tot POST → [HttpPost].

Exemplu definire actiuni

Exemplu de definire a actiunilor (metodelor) folosind **verbele HTTP** corespunzatoare:

```
public class StudentsController : Controller
{
    // GET: lista tuturor studentilor
    public IActionResult Index()
    {
        return View();
    }

    // GET: vizualizarea unui student
    public IActionResult Show(int id)
    {
        return View();
    }
}
```

Aceste doua actiuni, **Index()** si **Show()** afiseaza informatii despre studenti. **Index** va afisa **lista tuturor studentilor**, iar **Show** va afisa **informatii despre un singur student** in functie de ID-ul primit ca parametru. Deoarece aceste pagini afiseaza informatii si nu trimit nimic la server, vom folosi verbul **GET**. Paginile care au verbul GET se pot accesa direct prin intermediul URL-ului aferent acestora.

```
// GET: se afiseaza formularul de creare a unui student
public IActionResult New()
{
    return View();
}
```

Metoda **New()** care are verbul HTTP GET va afisa prin intermediul view-ului un formular prin care introducem datele aferente unui student. Pentru a trimite datele catre server, formularul trebuie sa defineasca metoda prin care trimite datele (adica verbul HTTP → `<form action="/students/new" method="post">`)

```

[HttpPost]
public IActionResult New(Student student)
{
    // cod creare student
    // dupa crearea studentului, se preia ID-ul nou inserat din
    baza de date
    // se redirectioneaza browser-ul catre studentul nou creat

    return Redirect("/students/" + id);
}

```

Datele introduse in formular vor fi trimise catre server prin intermediul metodei POST. Astfel, putem sa definim o ruta cu acelasi nume, dar cu verbul POST [**HttpPost**]. Aceasta ruta necesita un parametru prin care o sa primeasca datele din formular.

Deoarece metoda are acelasi nume atat in momentul afisarii formularului, cat si in momentul trimiterii datelor catre server, este necesar un tip de date diferit pentru parametrii acestuia, dar si un verb Http diferit.

```

// GET: se doreste editarea unui student
public IActionResult Edit(int ID)
{
    return View();
}

// POST: se trimit modificarile la server si se stocheaza
[HttpPost]
public IActionResult Edit(Student ID)
{
    // cod modificare date student
    // se redirectioneaza browser-ul catre studentul editat
    return Redirect("/Students/Edit" + ID);
    //return RedirectToRoute("students_show", new { id = ID
});
}

```

Aceasta metoda modifica datele studentului si primeste datele prin intermediul verbului HTTP POST. Metodele care creeaza, modifica sau sterg date nu au de obicei un View. Dupa finalizarea procesarii datelor, acestea redirectioneaza utilizatorul la o pagina aferenta actiunii. **De exemplu:** in actiunea de mai sus redirectionam la pagina de afisare a datelor studentului pentru a vedea modificarile efectuate.

```
[HttpPost]
public IActionResult Delete(int id)
{
    // cod stergere student din baza de date
    // redirectionare browser la pagina index a studentilor
    return RedirectToRoute("students_index");
}
} // se inchide clasa StudentsController
```

Redirect in cadrul metodelor

Redirect

Metoda **Redirect** se foloseste in momentul in care se doreste realizarea unui redirect temporar (HTTP 302). Primeste ca argument un string, reprezentand URL-ul pe care trebuie sa il acceseze.

```
Ex: Redirect("/Students/Edit" + ID);
    Redirect("/Home/Index");
```

RedirectToRoute

Metoda **RedirectToRoute** realizeaza un redirect temporar. Primeste ca argument denumirea rutei, denumire existenta in fisierul Program.cs in momentul configurarii sistemului de rutare.

```
Ex: return RedirectToRoute("Nume_Ruta");
```

De asemenea, metoda **RedirectToRoute** mai poate fi utilizata astfel:

```
return RedirectToRoute(new { controller = "Home", action = "Index" });
return RedirectToRoute("students_show", new { id = ID });
```

students_show -> numele rutei din Program.cs

id -> parametrul din Program.cs

ID -> parametrul primit ca parametru de intrare in metoda

RedirectToAction

Metoda **RedirectToAction** se utilizeaza in momentul in care se doreste redirect temporar catre o metoda din acelasi Controller sau dintr-un alt Controller.

Ex: `return RedirectToAction("Edit");` // metoda din acelasi Controller

Se poate redirectiona si catre o metoda dintr-un alt Controller astfel:

```
return RedirectToAction("Nume_Actiune", "Nume_Controller"); →
→ return RedirectToAction("Index", "Students");
// redirect catre metoda Index din Controller-ul Students
```

Si in acest caz se pot trimite parametrii din ruta:

```
return RedirectToAction("Nume_Actiune", "Nume_Controller", new {
    paramDinRuta })
```

RedirectPermanent/ RedirectToRoutePermanent/ RedirectToActionPermanent

Aceste metode se utilizeaza la fel ca in exemplele anterioare, singura diferenta fiind starea redirect-ului. In acest caz se realizeaza un redirect permanent (HTTP 301). Raspunsul o sa fie stocat in memoria cache a browser-ului, iar serverul nu o sa mai fie interogat pentru accesari ulterioare.

Returnare HTTP Status Code

Pentru a returna un status al request-ului HTTP, se poate proceda astfel:

```
public StatusCodeResult BadRequest()
{
    return StatusCode(StatusCodes.Status400BadRequest);
}

public StatusCodeResult Unauthorized()
{
    return StatusCode(StatusCodes.Status401Unauthorized);
}

public IActionResult Forbidden()
{
    return StatusCode(StatusCodes.Status403Forbidden);
}

public IActionResult NotFound()
{
    return StatusCode(StatusCodes.Status404NotFound);
    //sau return NotFound();
}
```

Ca tip de returnare, la fel ca in cazul celorlalte tipuri, se poate utiliza direct IActionResult.

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 5 – Baza de Date - MAC OS

Crearea unui proiect utilizand EF si sistemul de migratii

PASUL 1 – Instalare .NET Core

Pe langa instalarea Visual Studio 2022, mai este necesara si instalarea .NET Core:

<https://learn.microsoft.com/enus/dotnet/core/install/macos?tabs=netcore2x#dependencies>

The latest version of .NET is 6.0.

Download .NET Core



[.NET 6.0](#) (latest)

Long Term Support ⓘ



Build apps - SDK ⓘ

SDK 6.0.402

OS	Installers	Binaries
Linux	Package manager instructions	Arm32 Arm32 Alpine Arm64 Arm64 Alpine x64 x64 Alpine
macOS	Arm64 x64	Arm64 x64
Windows	Arm64 x64 x86 winget instructions	Arm64 x64 x86
All	dotnet-install scripts	

Se selecteaza **x64** pe **MAC OS** cu **INTEL** si **ARM64** pe **MAC OS** cu **procesor M**.

Se ruleaza in linia de comanda:

ln -s /usr/local/share/dotnet/dotnet /usr/local/bin/

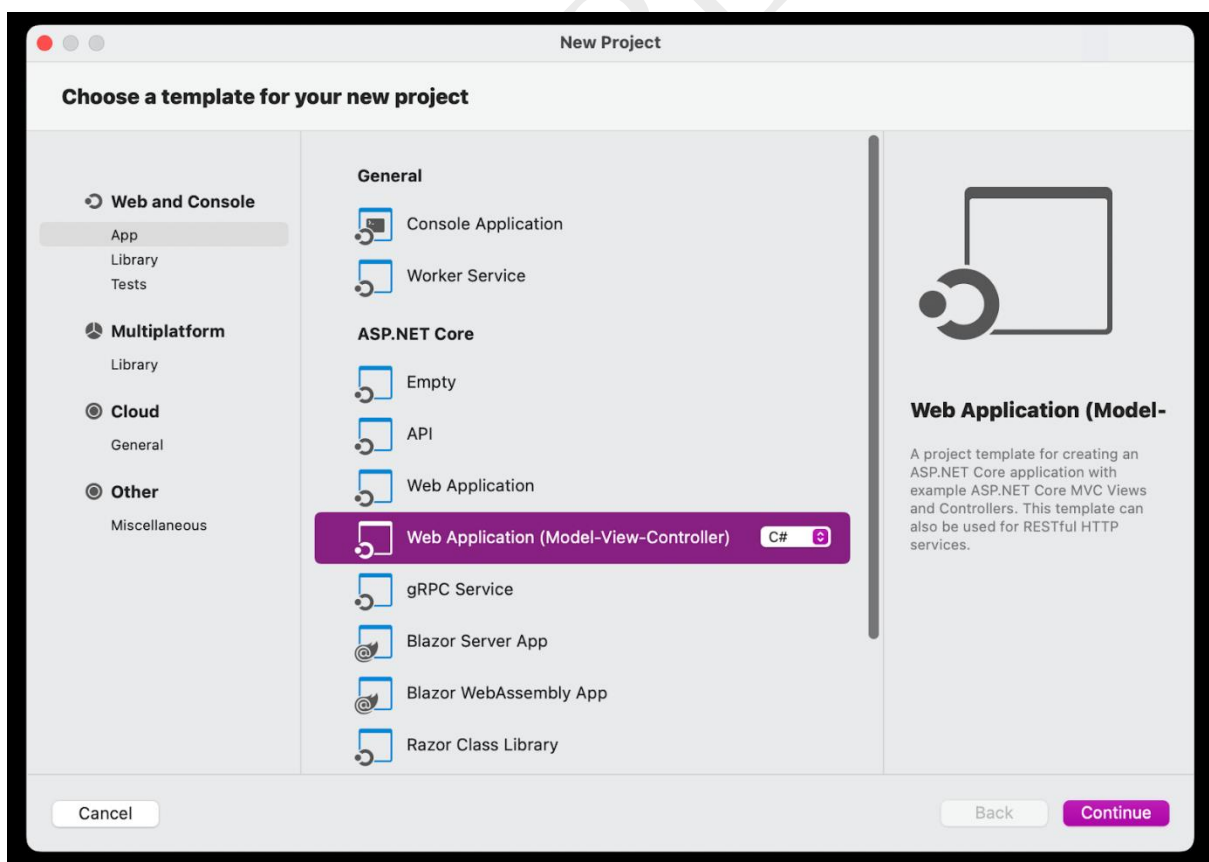
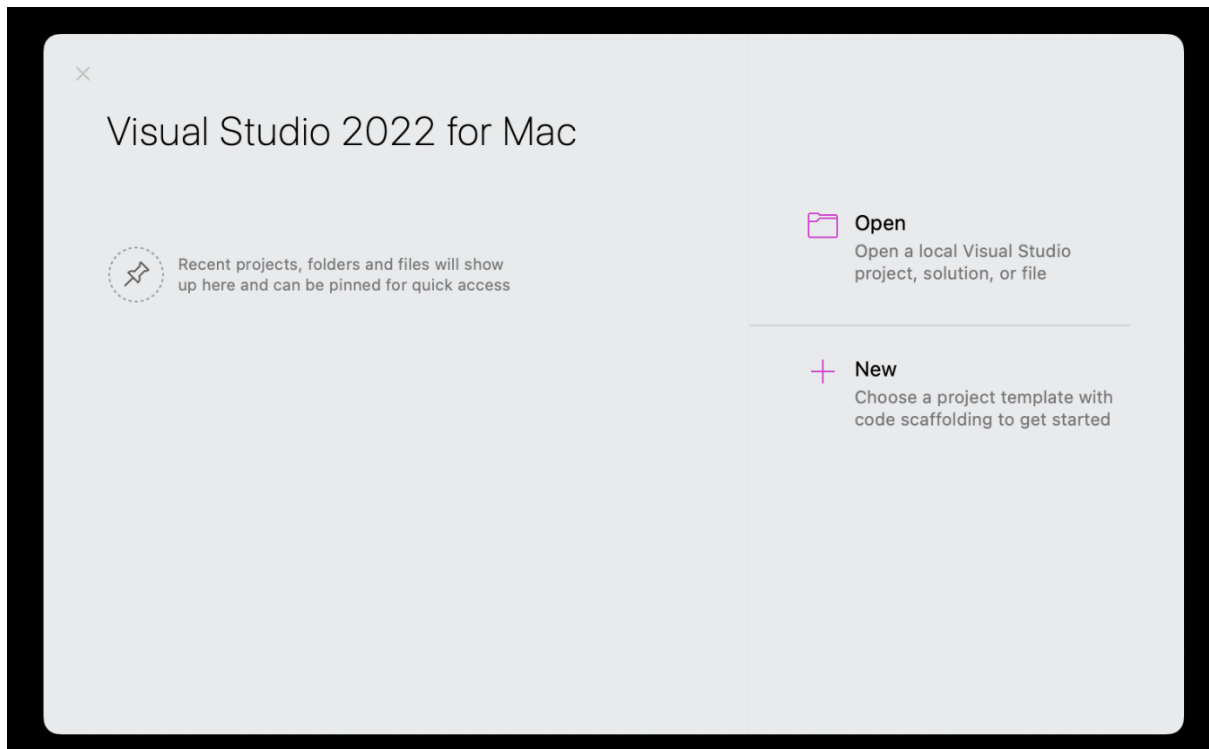
PASUL 2 – Instalare server MySQL

Se instaleaza serverul MySQL urmarind pasii din urmatorul tutorial, pana la comanda **mysql_secure_installation** inclusive:

<https://flaviocopes.com/mysql-how-to-install/>

PASUL 3 – Crearea proiectului

Se creeaza un nou proiect, procedand la fel ca in cursurile anterioare. Proiectul o sa se numeasca **Lab5**.



New Project

Configure your new Web Application (Model-View-Controller)

Target framework: .NET 6.0

Authentication: No Authentication

Advanced: ☒ Configure for HTTPS
☐ Do not use top-level statements

Web Application (Model-View-Controller)

A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.

Cancel Back Continue

New Project

Configure your new Web Application (Model-View-Controller)

Project name: Lab5

Solution Name: Lab5

Location: /Users/.../Projects Browse...

☒ Put project file in a subfolder

Version control: ☐ Use git for version control
☒ Add .gitignore file for inessential files

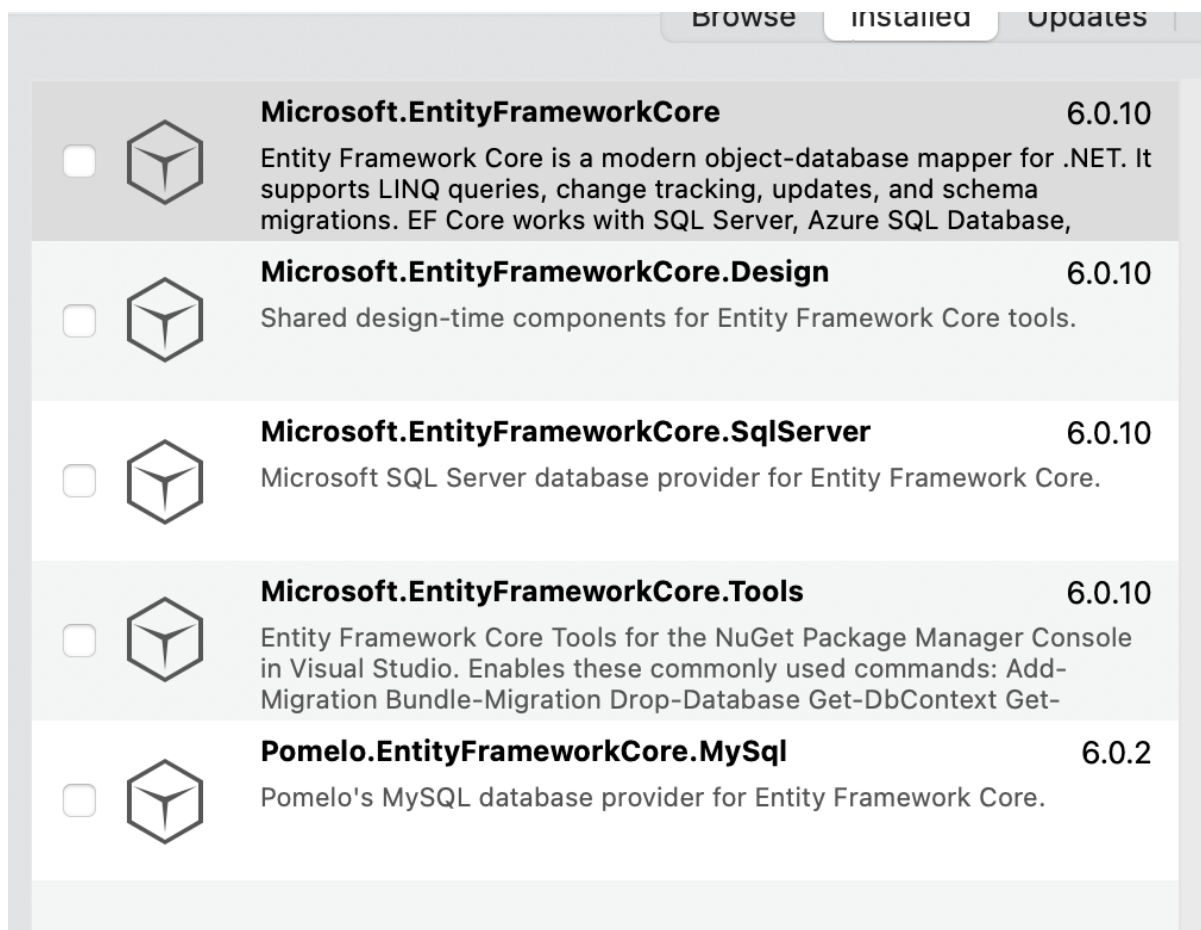
Preview

- /Users/.../Projects
 - Lab5
 - Lab5.sln
 - Lab5
 - Lab5.csproj

Cancel Back Create

PASUL 4 – Adaugare Entity Framework Core

În cadrul noului proiect se adaugă EF selectându-se următoarele pachete:



PASUL 5 – Conexiunea cu Baza de Date

În continuare se creează baza de date împreună cu un user care trebuie să aibă drepturi asupra bazei de date. Acest lucru se face pentru fiecare aplicație nouă.

// se creează baza de date

```
mysql> create database lab5;
```

Query OK, 1 row affected (0.01 sec)

// se creeaza un user cu username daw_example si parola Password1!

```
mysql> CREATE USER 'daw_example'@'localhost' IDENTIFIED WITH
mysql_native_password BY 'Password1!';
```

Query OK, 0 rows affected (0.01 sec)

// userul primeste drepturi asupra bazei de date numita lab5

```
mysql> grant all privileges on lab5.* to 'daw_example'@'localhost';
```

Query OK, 0 rows affected (0.00 sec)

// se inchide sesiunea cu serverul MySQL

```
mysql> exit
Bye
```

PASUL 6 – Configurarea Stringului de conexiune

Se creeaza in Model clasa AppDbContext pentru adaugarea conexiunii la baza de date.

```
public class AppDbContext : DbContext
{
    public AppDbContext() : base ()
    {
    }

    protected override void OnConfiguring
        (DbContextOptionsBuilder options)
    {
        var connectionString =
            "server=localhost;database=lab5;uid=daw_example;password=Password1!";

        var serverVersion = new MySqlServerVersion(new
            Version(8, 0, 31));

        options.UseMySQL(connectionString, serverVersion);
    }
}
```

```
} public DbSet<Student> Students { get; set; }
```

Valorile pe care trebuie sa le configuram in codul de mai sus sunt:

- **database** – numele bazei de date creata in pasul anterior
- **uid** – numele de utilizator creat in pasul anterior
- **password** – parola utilizatorului creat in pasul anterior

Versiunea serverului de baze de date se poate afla conectandu-ne din terminal la baza de date cu utilizatorul creat la pasul anterior.

```
~/Projects/Lab5/Lab5 » mysql -uroot -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 8.0.31 Homebrew

Copyright (c) 2000, 2022, Oracle and/or its affiliates.
```

PASUL 7 – Adaugarea migratiilor in baza de date

In folderul proiectului, in linia de comanda, se ruleaza comenzile prin care se creeaza migratia si se realizeaza update-ul bazei de date.

```
~/Projects/Lab5/Lab5 » dotnet ef migrations add 
InitialMigration
```

```
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'
```

```
~/Projects/Lab5/Lab5 » dotnet ef database 
update
```

```
Build started...
Build succeeded.
Applying migration '20221106130348_InitialMigration'.
Done.
```

Pasul 8 – Managementul bazei de date

Managementul bazei de date se poate realiza in doua moduri:

1. Din terminal

```
mysql> use lab5;
Reading table information for completion of table and column
names
You can turn off this feature to get a quicker startup with -A
Database changed
```

```
mysql> show tables;
+-----+
| Tables_in_lab5 |
+-----+
| __EFMigrationsHistory |
| Articles |
```

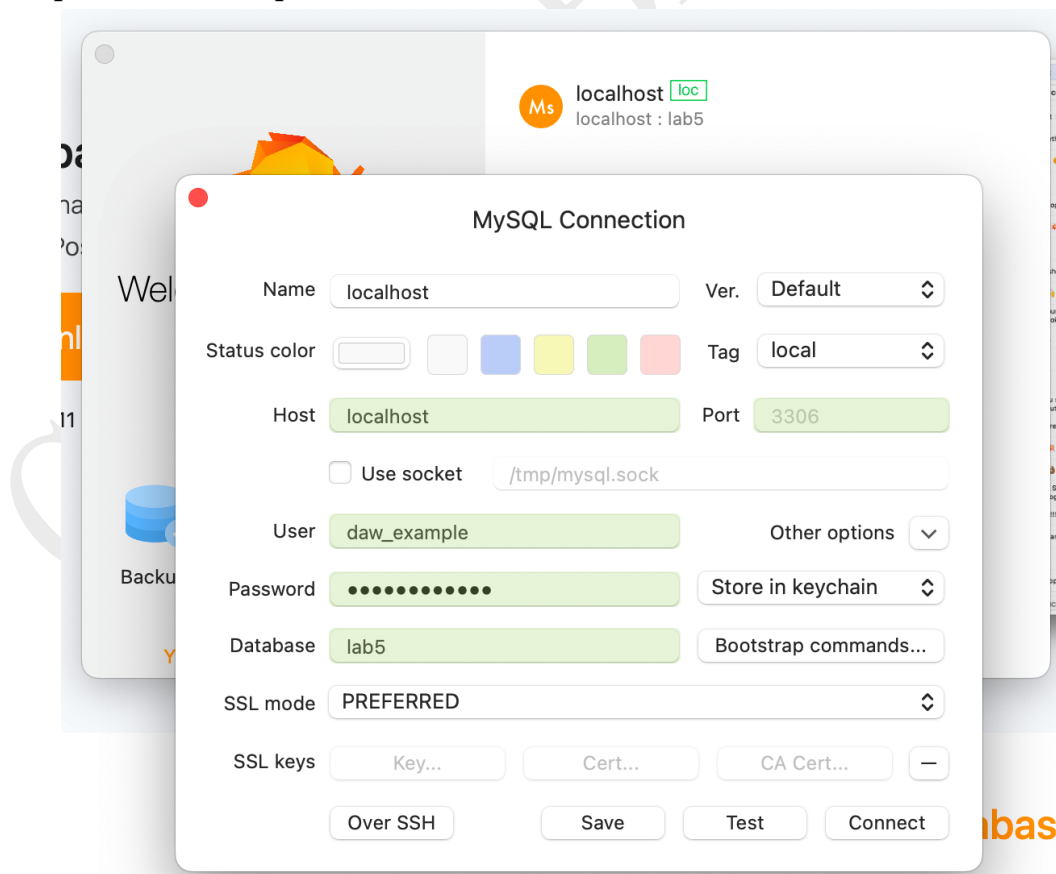
```
mysql> desc Articles;
```

```

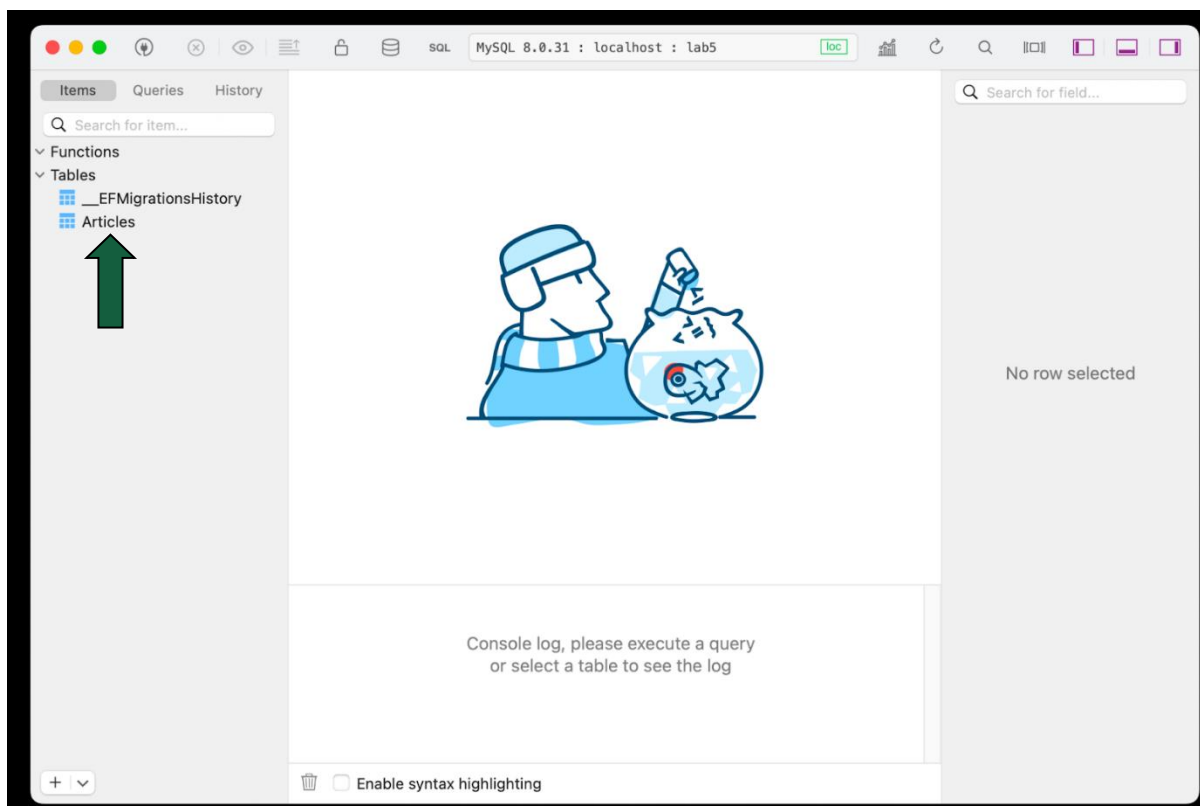
+-----+-----+-----+-----+-----+
+
| Field      | Type          | Null | Key | Default |
Extra      |
+-----+-----+-----+-----+-----+
+
| ArticleId  | int           | NO   | PRI | NULL     | auto_increment
|
| Title      | longtext      | NO   |     |          |
NULL      |
| Content    | longtext      | NO   |     |          |
NULL      |
| Date       | datetime(6)   | NO   |     |          |
NULL      |
+-----+-----+-----+-----+-----+
+
4 rows in set (0.01 sec)
```

2. Utilizand un utilitar extern cum este TablePlus (<https://tableplus.com/>)

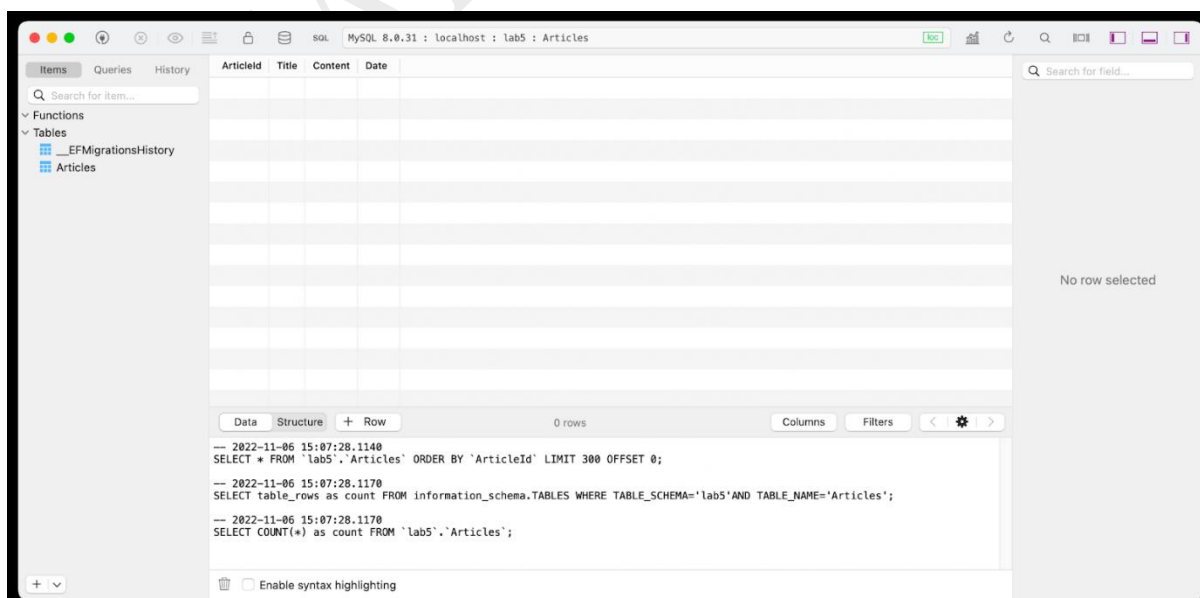
Dupa instalare se porneste si se creeaza o conexiune:



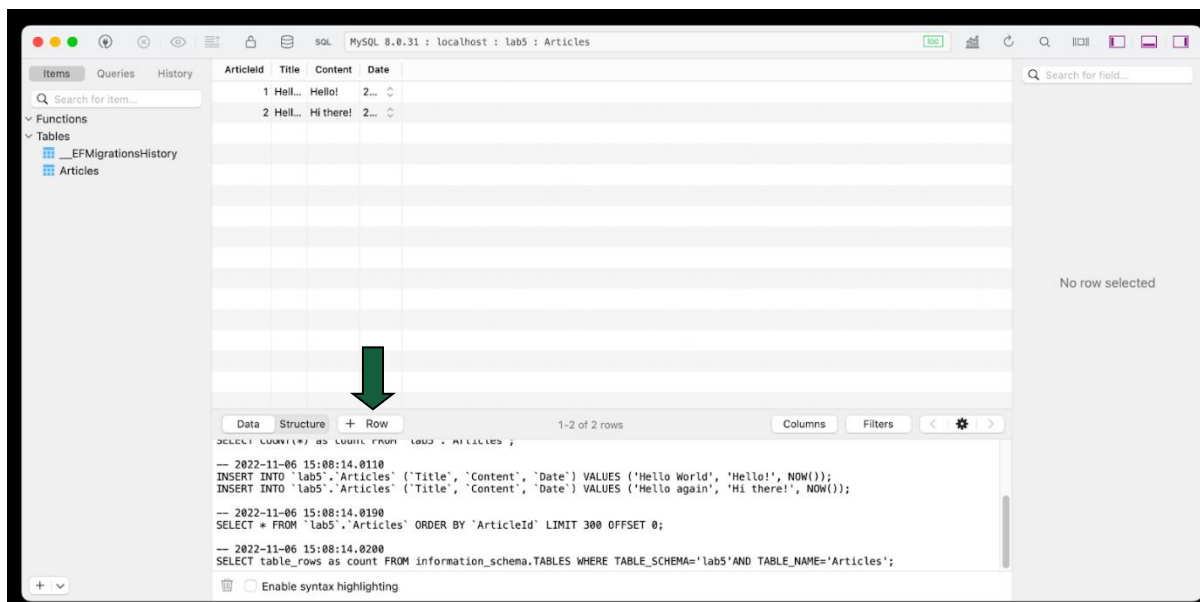
In acest moment avem acces la tabele



Se face click pe tabele pentru a vizualiza/adauga



Se adauga o intrare in baza de date apasand + Row
Pentru salvare se folosesc comenzile CMD + S



C.R.U.D. utilizand Entity Framework

In urmatoarea parte a cursului vom implementa operatiile de tip CRUD asupra entitatii Student, utilizand Entity Framework.

Index

```
private AppDbContext db = new AppDbContext();

public IActionResult Index()
{
    var students = from student in db.Students
                   orderby student.Name
                   select student;

    ViewBag.Students = students;

    return View();
}
```

Preluam toti studentii
din baza de date,
ordonati dupa nume prin
intermediul db.Students

Index.cshtml

```
<h2>Afisare studenti</h2>
<br />

@foreach (var student in ViewBag.Students)
{
    <p>@student.Name</p>
    <p>@student.Email</p>
    <p>@student.CNP</p>

    <br />
    <a href="/Students/Show/@student.StudentID">Afisare
student</a>
    <br />
    <a href="/Students/Edit/@student.StudentID">Editare
student</a>
    <hr />
}

<a href="/Students/New">Adaugare student</a>
```



Show

```
public ActionResult Show(int id)
{
    Student student = db.Students.Find(id);
    ViewBag.Student = student;
    return View();
}
```

Metoda Find() primește ca parametru o valoare pentru coloana care este cheia primară

Show.cshtml

```
<h2>Afisare student</h2>

<br />

<p>@ViewBag.Student.Name</p>
<p>@ViewBag.Student.Email</p>
<p>@ViewBag.Student.CNP</p>

<br />

<a href="/Students/Index">Afisare studenti</a>
```

New

```
public IActionResult New()
{
    return View();
}

[HttpPost]
public IActionResult New(Student s)
{
    try
    {
        db.Students.Add(s);
        db.SaveChanges();
        return RedirectToAction("Index");
    }
    catch (Exception)
    {
        return View();
    }
}
```

Students.Add primește ca parametru un obiect de tip Student iar SaveChanges va face commit în baza de date

New.cshtml

```
<h2>Formular adaugare student</h2>

<form method="post" action="/Students/New">
  <label>Nume</label>
  <br />
  <input type="text" name="Name" />
  <br /><br />
  <label>Adresa e-mail</label>
  <br />
  <input type="text" name="Email" />
  <br /><br />
  <label>CNP</label>
  <br />
  <input type="text" name="CNP" />
  <br />
  <br />
  <button type="submit">Adauga student</button>
</form>
```

Formular adaugare student

Nume

Adresa e-mail

CNP

Adauga student

Model Binding

În ASP.NET MVC **model binding** ne permite să facem legătura între request-urile de tip HTTP și un Model. Model binding este procesul de creare a obiectelor folosind datele trimise de browser printr-un request HTTP (prin intermediul formularelor din View).

Model binding este o legătură între request-urile HTTP și metodele unui Controller (Acțiuni). Deoarece datele trimise prin POST sau GET ajung întotdeauna la Controller, acest mecanism de binding leagă în mod automat variabilele de request cu atributele publice ale modelului. Aceasta mapare se va face după **numele atributelor modelului**.

```
<label>Nume</label>
```

```
<input type="text" name="Name" />
```

```
<label>Adresa e-mail</label>
```

```
<input type="text" name="Email" />
```

```
<label>CNP</label>
```

```
<input type="text" name="CNP" />
```

Parametrii care se vor
trimite prin request la
controller

!! OBSERVATIE

Este necesar ca numele câmpurilor din View să coincidă cu numele atributelor pentru ca binding-ul să funcționeze.

Edit

```
public IActionResult Edit(int id)
{
    Student student = db.Students.Find(id);
    ViewBag.Student = student;
    return View();
}

[HttpPost]
public ActionResult Edit(int id, Student requestStudent)
{
    Student student = db.Students.Find(id);

    try
    {
        student.Name = requestStudent.Name;
        student.Email = requestStudent.Email;
        student.CNP = requestStudent.CNP;
        db.SaveChanges();

        return RedirectToAction("Index");
    }
    catch (Exception)
    {
        return RedirectToAction("Edit", student.StudentID);
    }
}
```

Edit.cshtml

```
<h2>Editare student</h2>

<br />

<form method="post"
action="/Students/Edit/@ViewBag.Student.StudentID">

    <label>Nume</label>
    <br />
    <input type="text" name="Name" value="@ViewBag.Student.Name" />
    <br /><br />
    <label>Adresa e-mail</label>
    <br />
    <input type="text" name="Email" value="@ViewBag.Student.Email" />
    <br /><br />
    <label>CNP</label>
    <br />
```

```



```

Delete

```

[HttpPost]
public ActionResult Delete(int id)
{
    Student student = db.Students.Find(id);
    db.Students.Remove(student);
    db.SaveChanges();
    return RedirectToAction("Index");
}

```

Remove primește ca parametru un obiect de tip Student. SaveChanges salvează modificările

Show.cshtml (se va utiliza view-ul show)

```

<form method="post"
action="/Students/Delete/@ViewBag.Student.StudentID">

    <button type="submit">Sterge studentul</button>

</form>

```

!! OBSERVATIE

În momentul în care sunt necesare în baza de date, fie adăugări sau ștergi de tabele, fie adăugări sau ștergi de coloane sau proprietăți, este nevoie de o nouă migrație în baza de date.

De exemplu: dacă se dorește adăugarea atributului **Address** în clasa Student → `public string Address { get; set; }`

Se adaugă proprietatea, după care se execută o nouă migrație

→ Add-Migration AddAddressToStudent

→ Update-Database

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 5

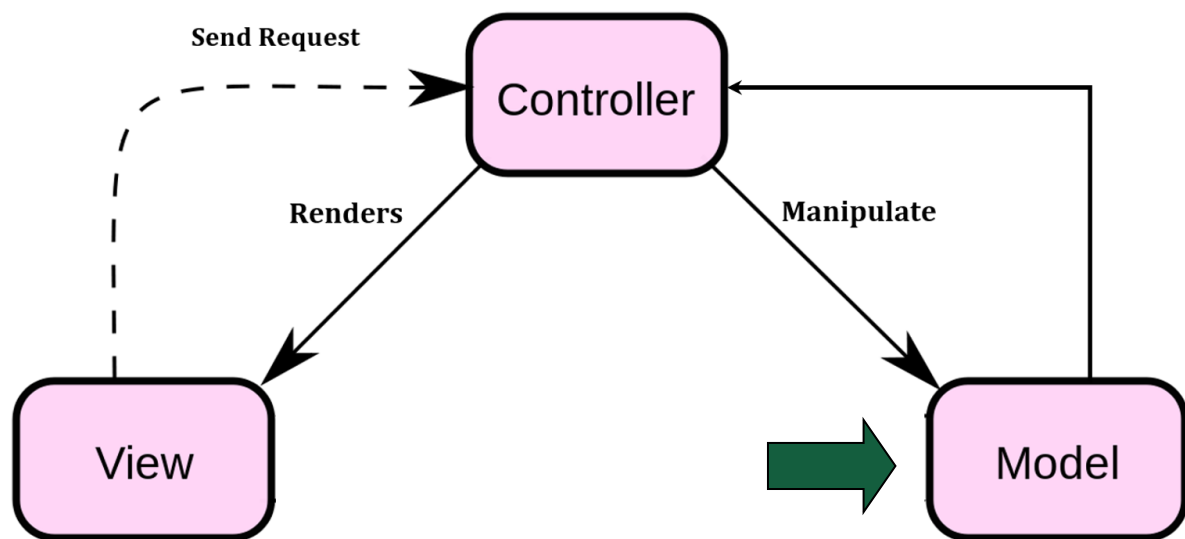
Cuprins

Model (Stratul business – prelucrarea datelor)	2
Ce este Modelul	2
Entity Framework Core.....	3
Entity Framework	3
Entity Framework Core.....	4
Instalare Entity Framework Core.....	4
Entity Framework Core – Migratii	11
Ce sunt migratiile?	11
Crearea unui proiect utilizand EF si sistemul de migratii.....	12
Crearea proiectului.....	12
Adaugare Entity Framework Core	12
Adaugarea Modelului.....	12
Conexiunea cu Baza de Date	15
Adaugarea unei baze de date SQL Server	19
Crearea migratiilor in baza de date	24
C.R.U.D. utilizand Entity Framework	26
Index.....	26
Show.....	27
New	28
Model Binding	29
Edit	30
Delete	31

Model (Stratul business – prelucrarea datelor)

Ce este Modelul

Modelul este responsabil cu gestionarea datelor din aplicatie si manipularea acestora. Acesta raspunde cererilor care vin din View prin intermediul Controller-ului, modelul comunicand doar cu Controller-ul. Este cel mai de jos nivel care se ocupa cu **procesarea** si **manipularea** datelor, reprezentand nucleul aplicatiei, fiind cel care realizeaza legatura cu baza de date. **Modelul** ofera accesul la date prin intermediul atributelor publice ale claselor.



Entity Framework Core

Entity Framework

Pentru stocarea datelor in ASP.NET MVC se utilizeaza o tehnologie open source numita **Entity Framework (EF)**.

Entity Framework este un **ORM (Object Relational Mapper)** pentru .NET, si anume este o colectie de librarii care coreleaza fiecare clasa dintr-un model cu o baza de date. Scopul utilizarii EF este acela de a permite dezvoltatorilor sa se focuseze pe dezvoltarea propriu-zisa a aplicatiei si nu pe baza de date.

Procesarea datelor se poate realiza si prin metode clasice, de exemplu utilizand ADO.NET, dar EF ofera posibilitatea implementarii eficiente a operatiilor de tip CRUD (Create, Read, Update, Delete).

De asemenea, in cadrul EF se poate utiliza **LINQ (Language Integrated Query)** ajutand la integrarea oricarui **RDBMS (Relational Database Management System)** → Oracle Database, SQL Server, etc. Un RDBMS stocheaza date in tabele, pe care ulterior le acceseaza si prelucreaza cu ajutorul unui limbaj **SQL (Structured Query Language)**. Un RDBMS asigura securitatea, integritatea si consistenta datelor.

LINQ permite integrarea query-urilor SQL in cadrul codului C#.

Entity Framework Core

Entity Framework Core este versiunea mai light a EF, cross-platform (Linux, Windows) care functioneaza foarte bine impreuna cu ASP.NET Core. Entity Framework Core suporta, la fel ca EF, atat tehnica **database-first** cat si **code-first**.

EF Core contine atat posibilitatea integrarii unui RDBMS (Oracle Database, Microsoft SQL Server, MySQL), cat si integrarea unor baze de date non-relationale (MongoDB, Redis, CassandraDB).

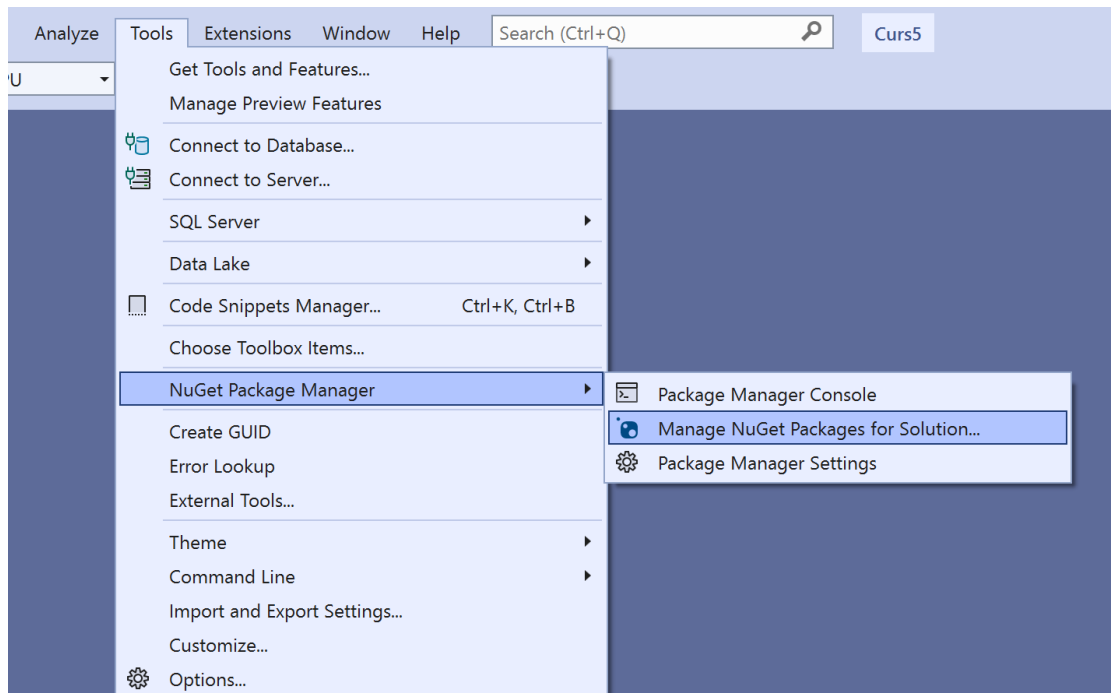
Instalare Entity Framework Core

Entity Framework este un pachet care poate fi instalat folosind **NuGet** si care suporta tehnica **code-first**. Tehnica code-first ofera posibilitatea dezvoltatorilor de a scrie clase prin intermediul carora baza de date va fi generata automat. Acest lucru duce la o dezvoltare curata si rapida a aplicatiilor cu baze de date.

Pentru instalarea Entity Framework (EF) se procedeaza astfel:

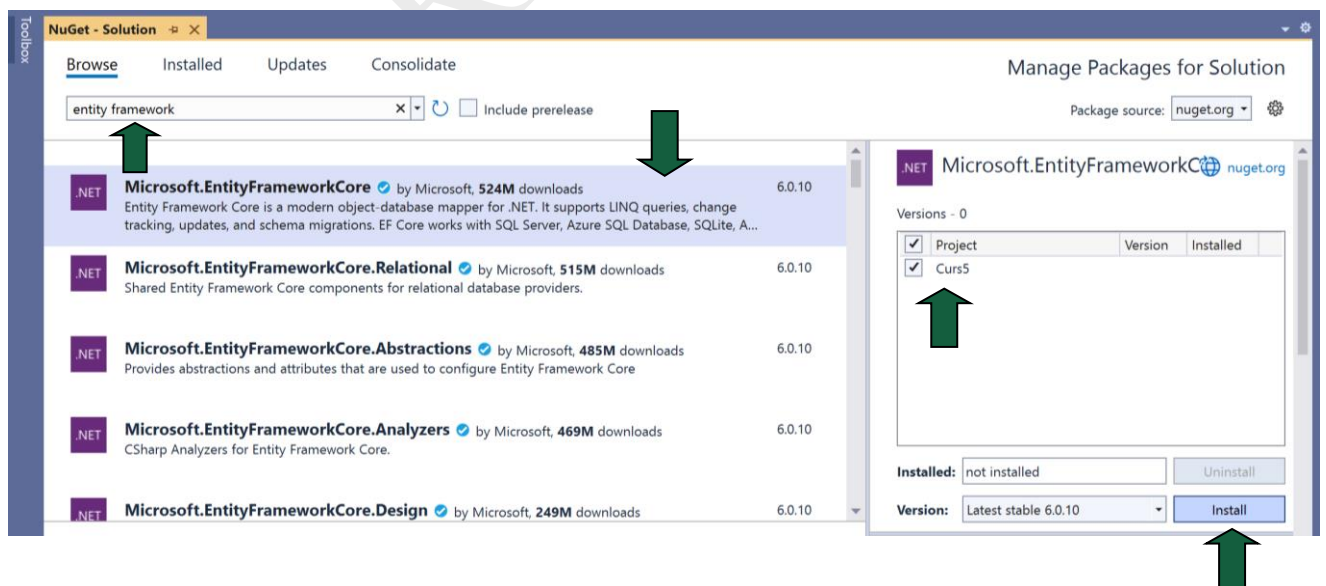
PASUL 1:

Tools → NuGet Package Manager → Manage NuGet Packages for Solution...

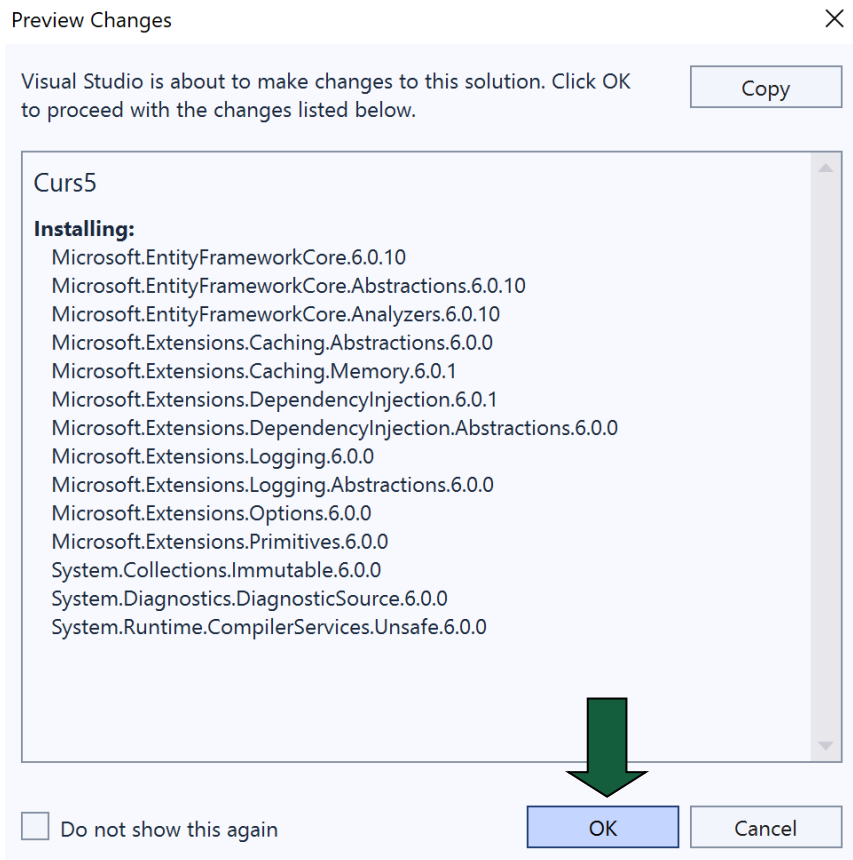


PASUL 2:

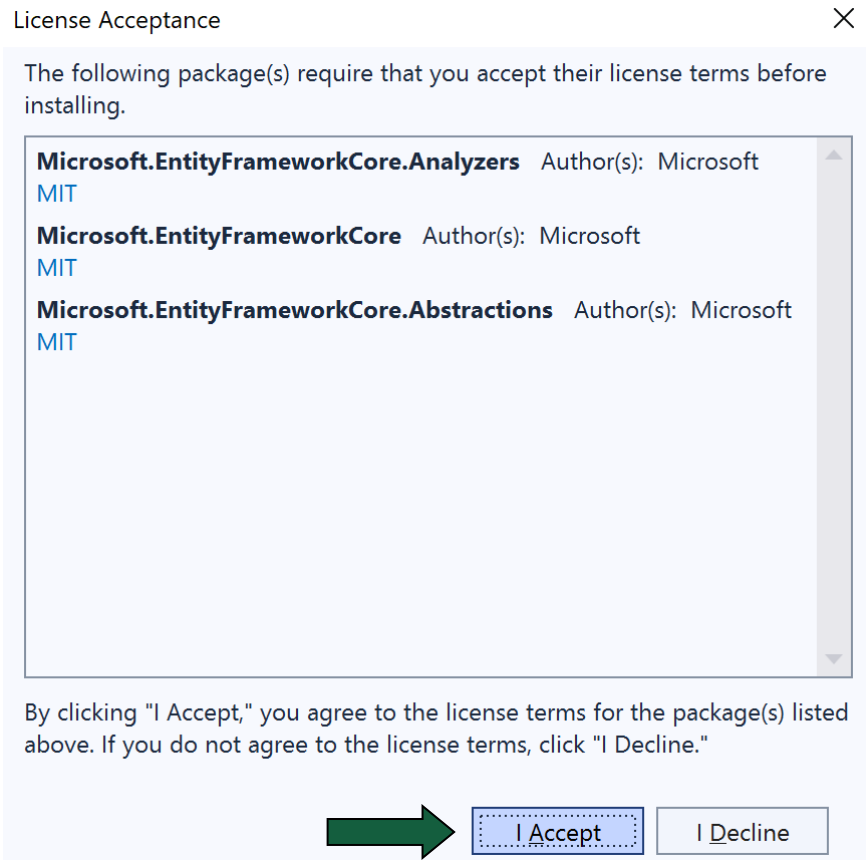
Se acceseaza optiunea **Browse** si se cauta **Microsoft.EntityFrameworkCore** dupa cum urmeaza:



PASUL 3:

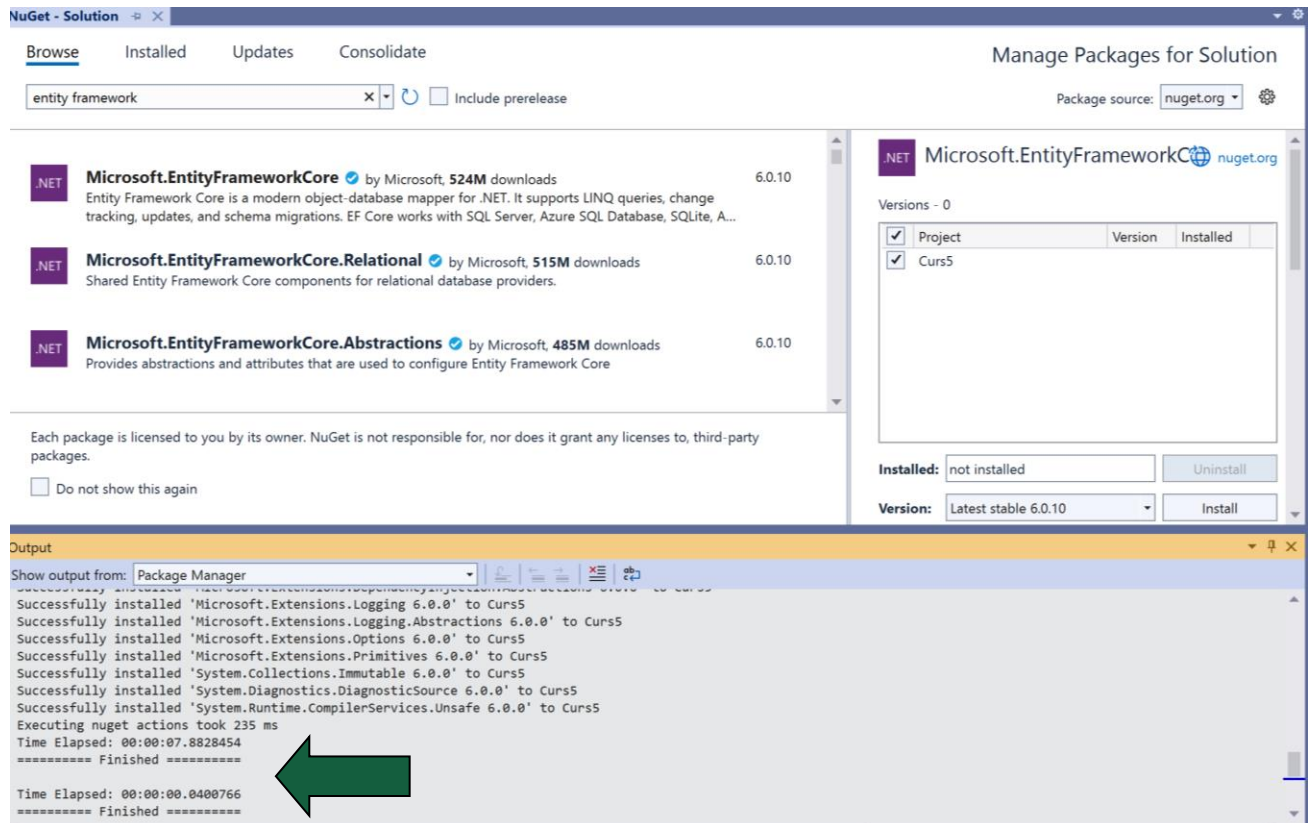


PASUL 4:

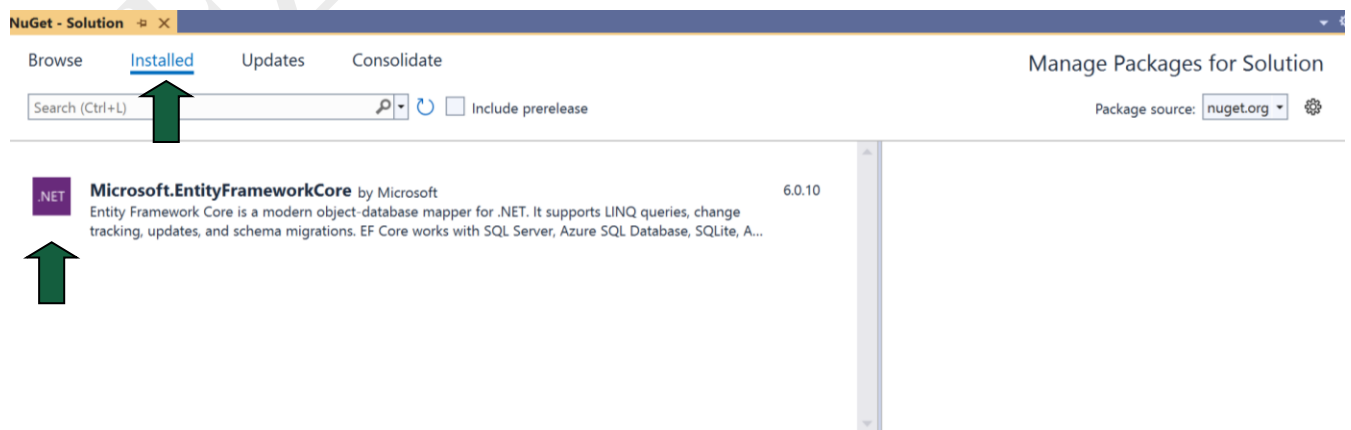


PASUL 5:

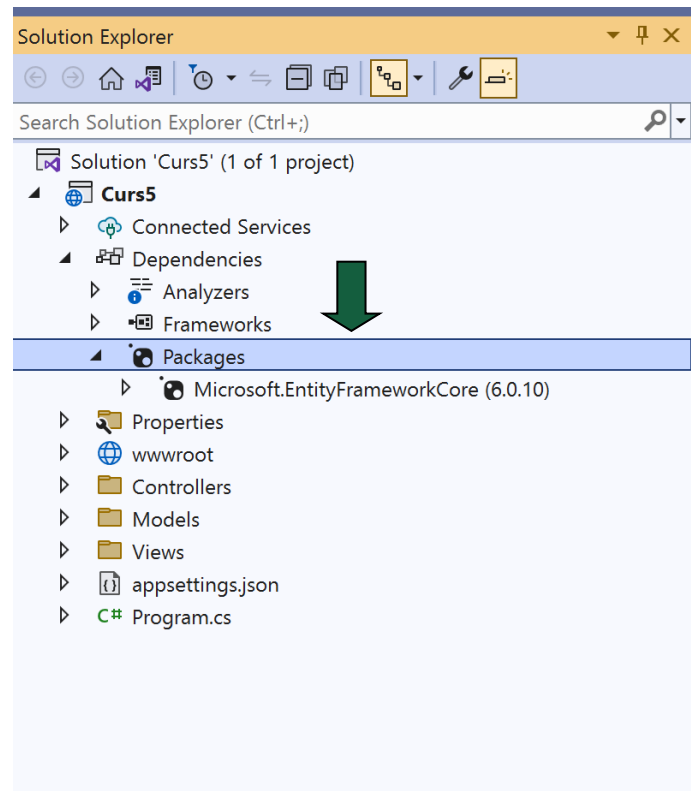
Dupa instalare se poate verifica in consola daca EF a fost adaugat cu succes in cadrul proiectului.



De asemenea, se poate verifica si in sectiunea **Installed**.



Sau chiar in Solution Explorer → Dependencies → Packages



In continuare este necesara includerea pachetului **SqlServer** (pentru baza de date) si **Design** (pachet care contine tool-urile necesare pentru rularea comenzilor de migrare).

Se instaleaza si pachetul **Tools** care include comenzi precum: Add-Migration, Drop-Database, Get-DbContext, Remove-Migration, etc.

- ➔ Microsoft.EntityFrameworkCore.SqlServer
- ➔ Microsoft.EntityFrameworkCore.Design
- ➔ Microsoft.EntityFrameworkCore.Tools

Browse Installed Updates Consolidate

entity framework x Include prerelease

Package source: nuget.org

Microsoft.EntityFrameworkCore by Microsoft, 524M downloads 6.0.10
Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF Core works with SQL Server, Azure SQL Database, SQLite,...

Microsoft.EntityFrameworkCore.Relational by Microsoft, 515M downloads 6.0.10
Shared Entity Framework Core components for relational database providers.

Microsoft.EntityFrameworkCore.Abstractions by Microsoft, 485M downloads 6.0.10
Provides abstractions and attributes that are used to configure Entity Framework Core

Microsoft.EntityFrameworkCore.Analyzers by Microsoft, 469M downloads 6.0.10
CSharp Analyzers for Entity Framework Core.

Microsoft.EntityFrameworkCore.Design by Microsoft, 249M downloads 6.0.10
Shared design-time components for Entity Framework Core tools.

Microsoft.EntityFrameworkCore.SqlServer by Microsoft, 258M downloads 6.0.10
Microsoft SQL Server database provider for Entity Framework Core.

Microsoft.EntityFrameworkCore.Tools by Microsoft, 176M downloads 6.0.10
Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

Microsoft.EntityFrameworkCore by Microsoft, 524M downloads 6.0.10

Versions - 0

Project	Version	Installed
☒ Project		
☒ Curs5		

Installed: not installed Uninstall

Version: Latest stable 6.0.10 Install

Options

Description

Microsoft SQL Server database provider for Entity Framework Core.

Version: 6.0.10

Author(s): Microsoft

Browse Installed Updates 1 Consolidate

entity framework x Include prerelease

Package source: nuget.org

Microsoft.EntityFrameworkCore.Relational by Microsoft, 515M downloads 6.0.10
Shared Entity Framework Core components for relational database providers.

Microsoft.EntityFrameworkCore.Abstractions by Microsoft, 485M downloads 6.0.10
Provides abstractions and attributes that are used to configure Entity Framework Core

Microsoft.EntityFrameworkCore.Analyzers by Microsoft, 469M downloads 6.0.10
CSharp Analyzers for Entity Framework Core.

Microsoft.EntityFrameworkCore.Design by Microsoft, 249M downloads 6.0.10
Shared design-time components for Entity Framework Core tools.

Microsoft.EntityFrameworkCore.SqlServer by Microsoft, 258M downloads 6.0.10
Microsoft SQL Server database provider for Entity Framework Core.

Microsoft.EntityFrameworkCore.Tools by Microsoft, 176M downloads 6.0.5 6.0.10
Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio.

EntityFramework by Microsoft, 184M downloads 6.4.4
Entity Framework 6 (EF6) is a tried and tested object-relational mapper for .NET with many years of feature development and stabilization.

Microsoft.EntityFrameworkCore by Microsoft, 524M downloads 6.0.10

Versions - 0

Project	Version	Installed
☒ Project		
☒ Curs5		

Installed: not installed Uninstall

Version: Latest stable 6.0.10 Install

Options

Description

Shared design-time components for Entity Framework Core tools.

Version: 6.0.10

Author(s): Microsoft

The screenshot shows the Visual Studio Package Manager interface. The search bar contains 'entity framework'. The results list several packages, with 'Microsoft.EntityFrameworkCore.Tools' highlighted. A green arrow points to this package. On the right, the details for 'Microsoft.EntityFrameworkCore.Tools' are shown. A table lists the versions installed in the project and the current version. A green arrow points to the 'Install' button.

Project	Version	Installed
Curs5	6.0.5	6.0.5

Installed: 6.0.5 Uninstall

Version: Latest stable 6.0.10 Install

Entity Framework Core – Migratii

Ce sunt migratiile?

În timpul dezvoltării unei aplicații bază de date se modifică constant, fiind necesare entități noi, proprietăți noi sau chiar eliminarea unor proprietăți existente. Pentru a sincroniza aceste modificări cu baza de date existentă, sunt necesare **migratii**.

În momentul în care apare o modificare în baza de date, Entity Framework Core, prin sistemul de migratii, compară modelul curent cu cel anterior pentru a detecta diferențele dintre cele două versiuni. Ulterior este generat un fișier, continuând codul asociat migrației.

Crearea unui proiect utilizand EF si sistemul de migratii

Crearea proiectului

Se creeaza un nou proiect, procedand la fel ca in cursurile anterioare. Proiectul o sa se numeasca **Curs5**.

Adaugare Entity Framework Core

In cadrul noului proiect se adauga EF (**VEZI** Sectiunea – Instalare Entity Framework Core – din cadrul cursului curent).

Adaugarea Modelului

Pentru adaugarea unui model se porneste de la crearea in cadrul acestuia a tuturor entitatilor. Prin **entitate** ne referim la un tabel din baza de date.

Entitate = un loc, o actiune, o persoana, etc.

Exemplu de entitati dintr-o baza de date care gestioneaza o Universitate → Studenti, Cursuri, Note, MergeLa – tabel asociativ intre Studenti si Cursuri

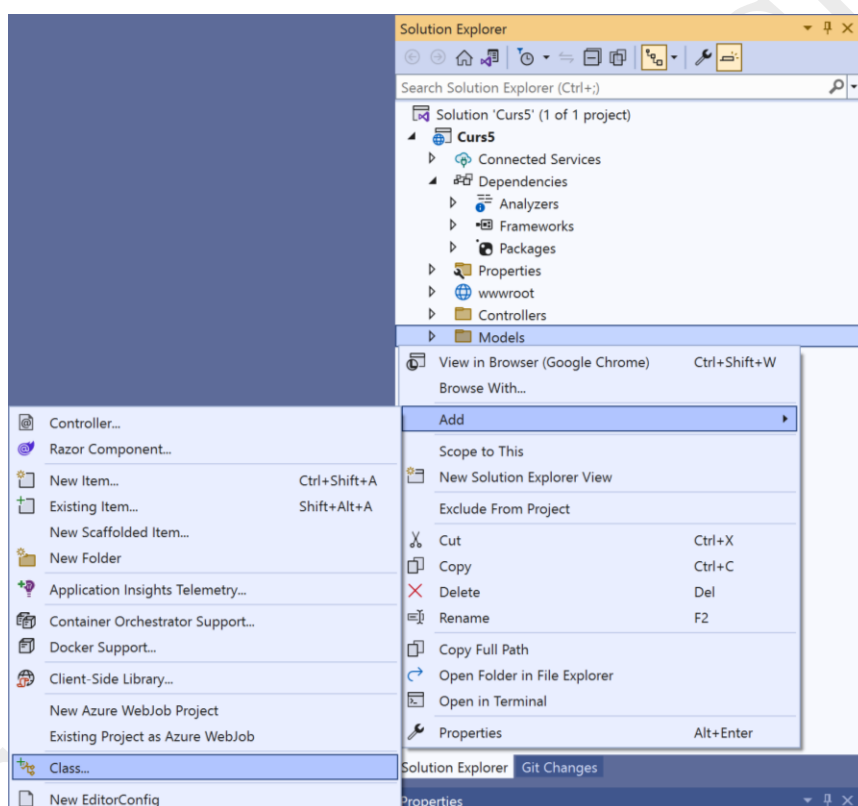
In continuare vom adauga clasa **Student** cu urmatoarele atribute:

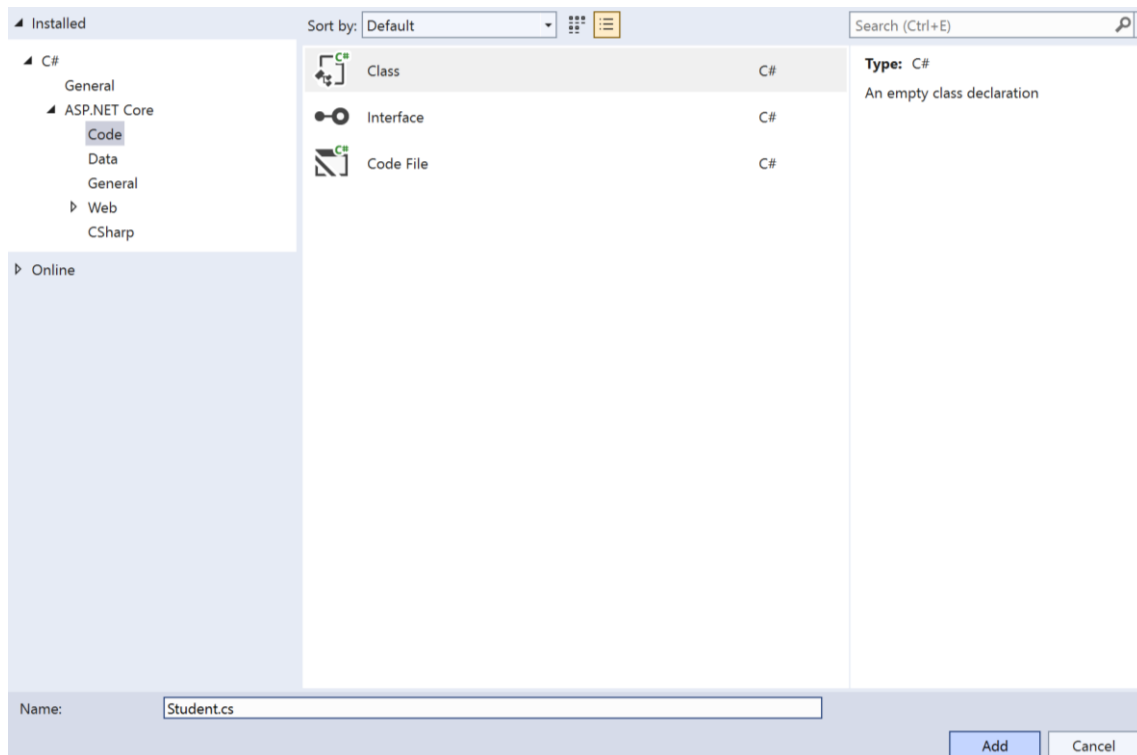
- **StudentID** – de tip int care reprezinta ID-ul studentului
- **Name** – de tip string care reprezinta numele studentului
- **Email** – de tip string care reprezinta o adresa de e-mail a studentului
- **CNP** – de tip string care reprezinta CNP-ul studentului. Aceasta proprietate a fost definita de tip string deoarece spatiul alocat

pentru int nu suporta valori de 13 caractere. De asemenea, definit ca string se poate procesa caracter cu caracter pentru calcule ulterioare (ex: extragere data de nastere).

Pentru adaugarea clasei **Student** se parcurg urmatoorii pasi:

Click dreapta Model → Add → Class → ASP.NET Core → Class → se completeaza numele clasei Student.cs





Clasa **Student** din fisierul Student.cs:

```
public class Student
{
    public int StudentID { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string CNP { get; set; }
}
```

Aceasta clasa reprezinta tabelul **Student** din baza de date, pe care in varianta clasica il adaugam prin intermediul comenzii **CREATE TABLE**.

Atributele / proprietatile clasei sunt: StudentID, Name, Email, CNP. Acestea reprezinta coloanele din tabelul Student, coloane care sunt create in momentul in care se creeaza si tabelul → utilizand comanda **CREATE TABLE**

Conexiunea cu Baza de Date

Conexiunea cu baza de date se poate realiza in doua moduri:

- fara dependency injection
- cu dependency injection.

Varianta 1 – fara Dependency Injection

Pentru a putea adauga layer-ul de conexiune cu baza de date in cadrul unui model, este necesara adaugarea unei noi clase.

Se adauga o clasa, numita sugestiv **AppDbContext**.

```
public class AppDbContext : DbContext
{
    public AppDbContext() : base ()
    {
    }

    protected override void OnConfiguring(
        DbContextOptionsBuilder options)
    {
        options.UseSqlServer(
            @"Stringul de Conexiune");
    }

    public DbSet<Student> Students { get; set; }
}
```

Clasa **AppDbContext** mosteneste clasa de baza **DbContext** din Entity Framework. Clasa de baza realizeaza in mod automat conexiunea cu baza de date, crearea tabelului daca acesta nu exista si contine o proprietate **DbSet**, care trebuie sa primeasca tipul modelului (Student in cazul curent) si numele pluralizat al modelului.

DbSet <Student> Students { get; set; } – prin intermediul acestei secvente de cod vom avea acces la intrarile din baza de date; se pot interoga si stoca instante de tip Student.

Stringul de conexiune la baza de date este preluat de parametrul **options** din metoda **OnConfiguring** prin intermediul urmatoarei secvente de cod:

```
options.UseSqlServer(@"Stringul de Conexiune");
```

UseSqlServer() → metoda prin intermediul careia se configureaza contextul pentru conectarea la baza de date. Primeste ca argument stringul de conectare la baza de date. **Stringul de conectare la baza de date se obtine urmand sectiunea urmatoare din curs.**

In final se adauga in **Program.cs** urmatoarea secventa de cod pentru initializarea bazei de date.

```
builder.Services.AddDbContext<AppDbContext>();
```

In cadrul fiecarui Controller trebuie sa se instantieze contextul pentru realizarea conexiunii la baza de date.

```
private AppDbContext db = new AppDbContext();
```

Varianta 2 – cu Dependency Injection

Se adauga aceeaasi clasa, numita sugestiv **AppDbContext** care in acest caz o sa aiba doar constructorul, astfel:

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext>
options)
        : base(options)
    {
    }
```

```

        public DbSet<Student> Students { get; set; }
    }

```

Conexiunea cu baza de date se va realiza in acest caz exclusiv in **Program.cs**.

```

var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

```

In acest caz, stringul de conexiune o sa fie in **Solution Explorer** → **appsetting.json** adaugand secventa de cod marcata cu BOLD si acolada.

```

{
  "ConnectionStrings": {
    "DefaultConnection":
    "server=(localdb)\\mssqllocaldb;Database=aspnet-53bc9b9d-9d6a-
    id4-8429-
    12761773502;Trusted_Connection=True;MultipleActiveResultSets=tr
    ue";
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

În final, în cadrul fiecărui Controller se realizează conexiunea cu baza de date astfel:

```
public class ArticlesController : Controller
{
    private readonly AppDbContext db;

    public ArticlesController(AppDbContext context)
    {
        db = context;
    }

    ...}
}
```

Pentru mentenanța mai facilă a codului sursă framework-ul ASP.NET Core folosește **dependency injection** pentru a insera automat la runtime instanțe ale unor obiecte necesare. Acest lucru se întâmplă prin intermediul constructorilor care se află în Controllere. Astfel, un Controller poate cere o instanță a unui obiect din cadrul framework-ului doar prin specificarea unui parametru și tipul acestuia.

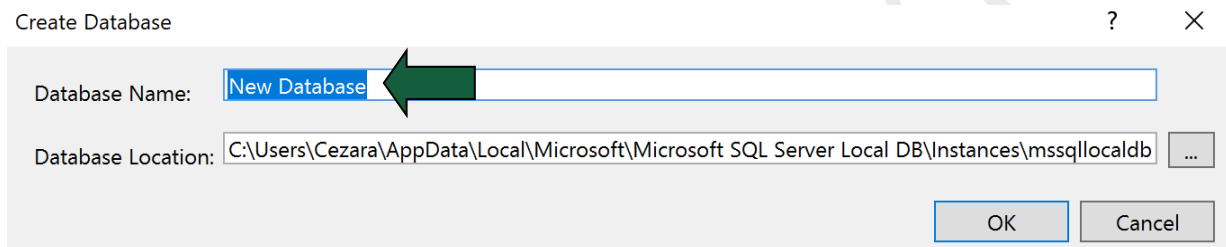
Un astfel de exemplu este conexiunea la baza de date care se cere prin intermediul constructorului specificând un parametru de tip AppDbContext (în cazul nostru particular din curs unde am numit clasa în acest mod). În acest caz, framework-ul știe în mod automat să returneze o instanță a AppDbContext din serviciul aferent bazei de date, configurat în Program.cs.

Adaugarea unei baze de date SQL Server

Pentru adaugarea bazei de date se parcurg urmatoorii pasi:

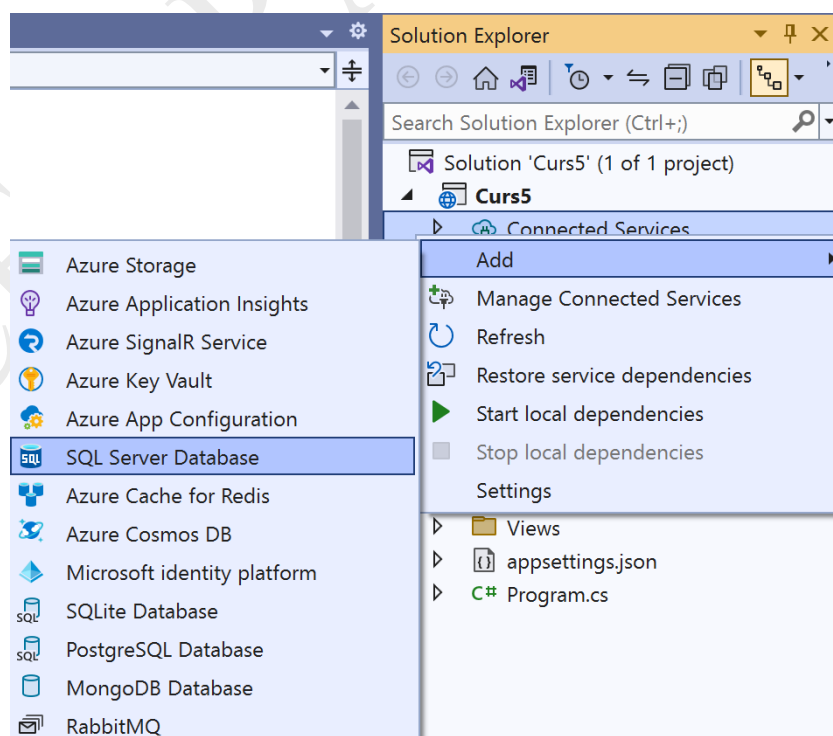
PASUL 1:

In SQL Server Object Explorer (se afla in meniul View) → se deschide sectiunea (localdb)\MSSQLLOCALDB → in folderul Databases → click dreapta → Add new database → se completeaza numele bazei de date (se alege un nume pe care o sa il folosim in cadrul urmatoarelor pasi) → OK



PASUL 2:

In Solution Explorer → click dreapta pe sectiunea Connected Services → Add → SQL Server Database

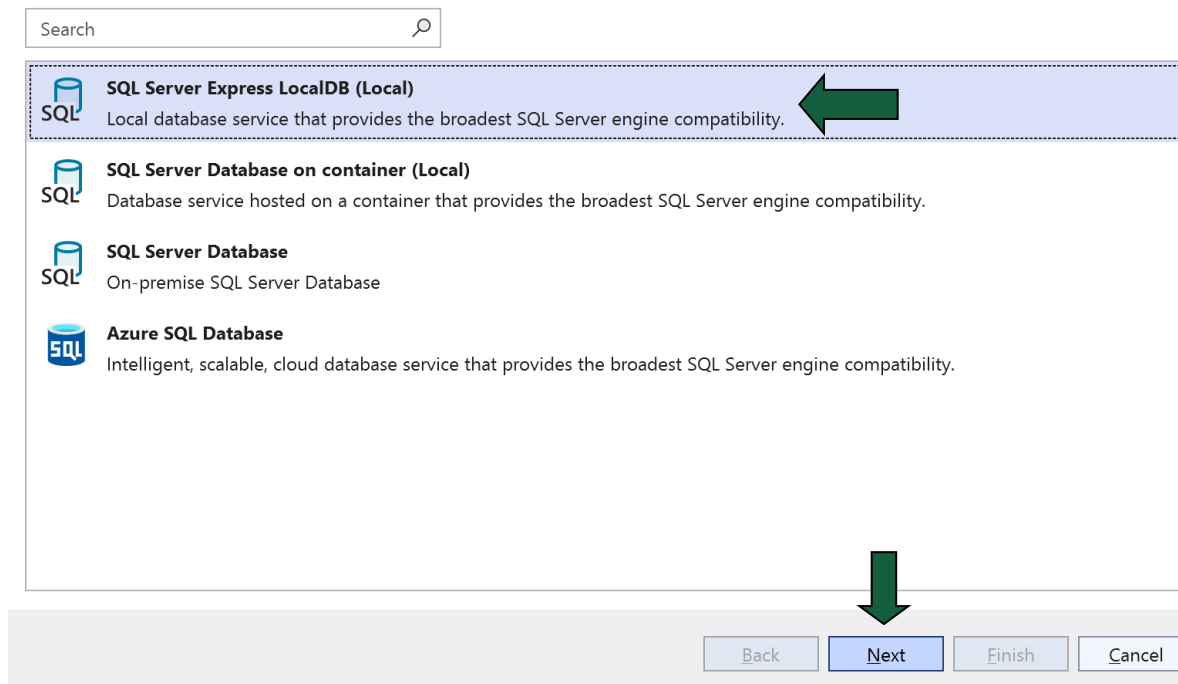


PASUL 3:

Se selecteaza optiunea SQL Server Express LocalDB (Local)

Connect to dependency

Select a service dependency to add to your application.



PASUL 4:

Se adauga un nume conexiunii la baza de date (ex: StudentsDB).

Se bifeaza optiunea **None**.

Se modifica string-ul de conexiune apasand cele 3 puncte (unde se afla cercul rosu) din dreptul casutei Connection String Value.

Se apasa butonul Next.

Connect to SQL Server Express LocalDB (Local)

Provide connection string and specify how to save it

Connection string name

ConnectionStrings:StudentsDB

Connection string value

Server=(localdb)\mssqllocaldb;Database=aspnet-53bc9b9d-9d6a-45d4-8429-2a2761773502;Trusted_Connection=True;MultipleActive...

Save connection string value in [Learn more](#)

- ☐ Local user secrets file
- ☐ Azure Key Vault
- ☒ None

Back

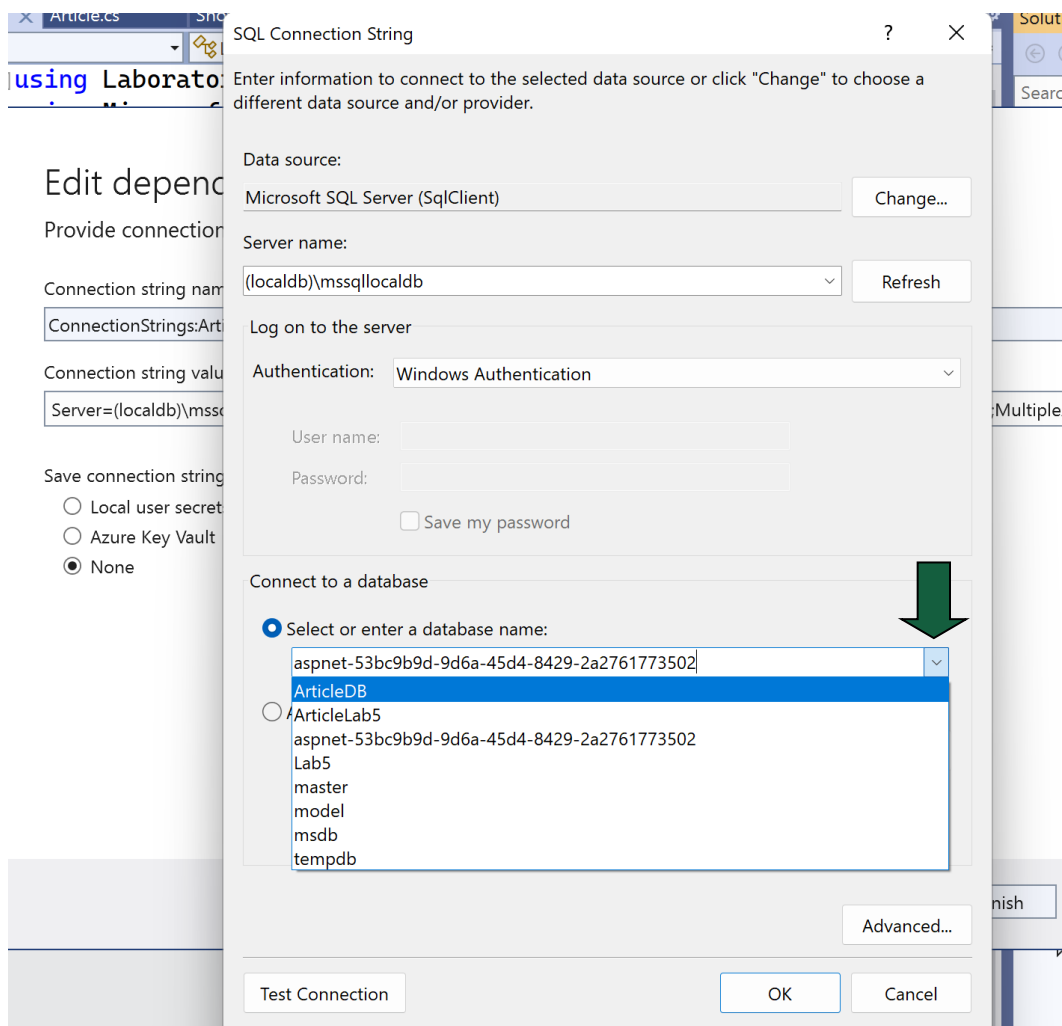
Next

Finish

Cancel

PASUL 5:

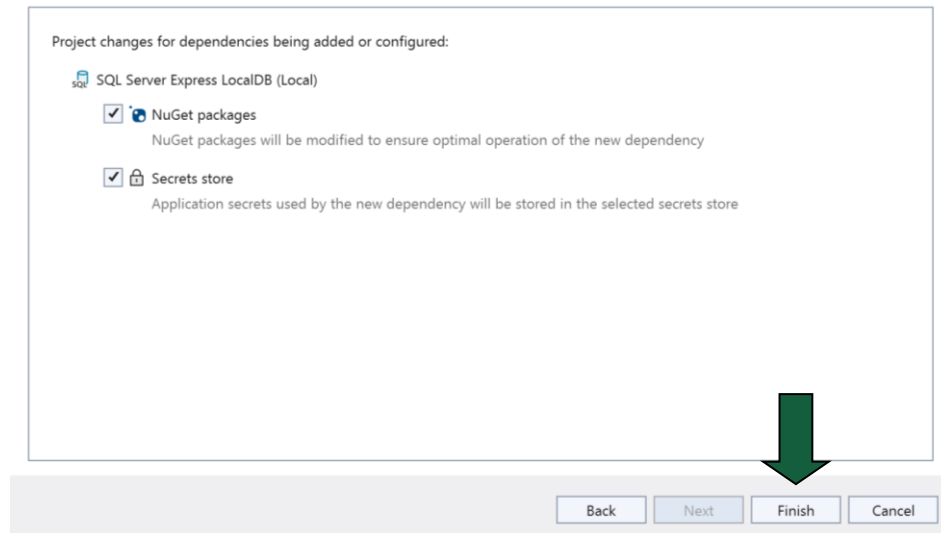
Se selecteaza baza de date creata la Pasul 1.



Se apasa OK → dupa care se copiaza stringul de conexiune (se utilizeaza la PASUL 7).

PASUL 6:

Summary of changes



PASUL 7:

Se adauga stringul de conectare la baza de date ca parametru al metodei **UseSqlServer()** in **OnConfiguring**

```
options.UseSqlServer(@"Server=(localdb)\mssqllocaldb;Database=aspnet-53bc9b9d-9d6a-45d4-8429-2a2761773502;Trusted_Connection=True;MultipleActiveResultSets=true");
```

Daca se utilizeaza varianta 2 – cea cu Dependency Injection – se adauga stringul de conexiune in **Solution Explorer** → **appsetting.json** adaugand secventa de cod marcata cu BOLD si acolada.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
    "Server=(localdb)\mssqllocaldb;Database=aspnet-53bc9b9d-9d6a-45d4-8429-2a2761773502;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
}
```

```

"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
},
"AllowedHosts": "*"
}

```

!/ OBSERVATIE

Stringul dat ca parametru in metoda de mai sus UseSqlServer este unic in cadrul fiecarei baze de date. Asadar, nu trebuie sa utilizati stringul din acest exemplu, ci trebuie sa preluati stringul de conexiune la baza voastra de date urmand pasii anteriori.

In acest moment proiectul contine Entity Framework si are creata o baza de date si o conexiune cu aceasta. **Urmeaza sa integram sistemul de migratii.**

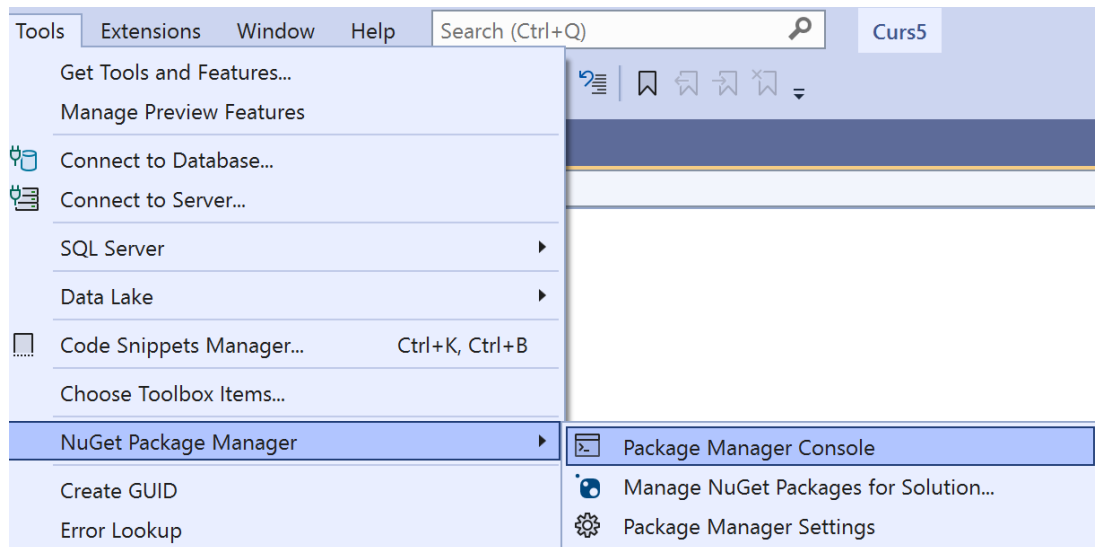
Crearea migratiilor in baza de date

Pentru integrarea sistemului de migratii se parcurg pasii urmatoari:

PASUL 1:

Pentru adaugarea migratiilor o sa se utilizeze consola.

Tools → NuGet Package Manager → Package Manager Console



PASUL 2:

Se adauga migratia utilizand comanda **Add-Migration** urmata de o denumire pe care o dam acestei migratii.

```
Each package is licensed to you by its owner. NuGet is not responsible for, nor does it grant any licenses to, third-party packages. Some packages may include dependencies which are governed by additional licenses. Follow the package source (feed) URL to determine any dependencies.

Package Manager Console Host Version 6.3.0.131

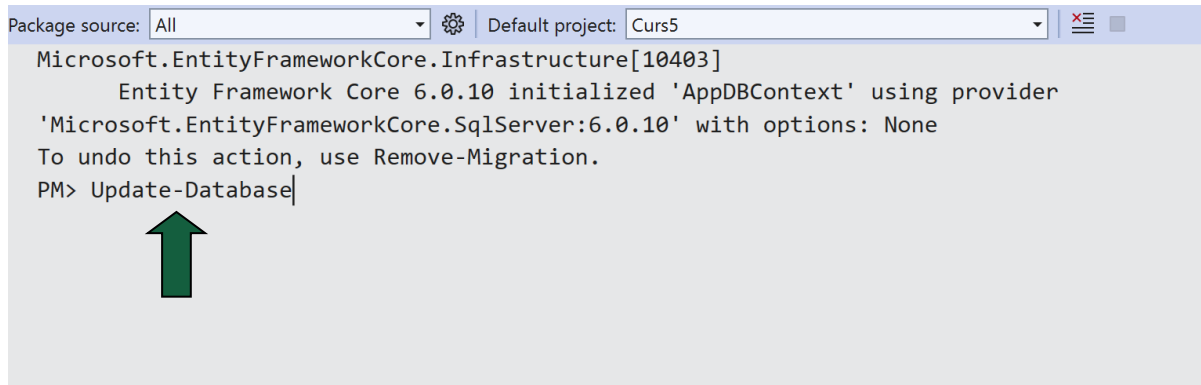
Type 'get-help NuGet' to see all available NuGet commands.

PM> Add-Migration CreateStudent
```

Dupa executarea migratiei, in Solution Explorer o sa se creeze un folder numit **Migrations** unde o sa se genereze fisierele specifice.

PASUL 3:

Se executa comanda **Update-Database** care modifica baza de date, aducand-o la versiunea finala.



```

Package source: All Default project: Curs5
Microsoft.EntityFrameworkCore.Infrastructure[10403]
    Entity Framework Core 6.0.10 initialized 'AppDbContext' using provider
    'Microsoft.EntityFrameworkCore.SqlServer:6.0.10' with options: None
To undo this action, use Remove-Migration.
PM> Update-Database
  
```

C.R.U.D. utilizand Entity Framework

In urmatoarea parte a cursului vom implementa operatiile de tip CRUD asupra entitatii Student, utilizand Entity Framework.

Index

```

private AppDbContext db = new AppDbContext();

public IActionResult Index()
{
    var students = from student in db.Students
                   orderby student.Name
                   select student;

    ViewBag.Students = students;

    return View();
}
  
```

**Preluam toti studentii
din baza de date,
ordonati dupa nume prin
intermediul db.Students**

Index.cshtml

```

<h2>Afisare studenti</h2>
<br />

@foreach (var student in ViewBag.Students)
{
    <p>@student.Name</p>
    <p>@student.Email</p>
    <p>@student.CNP</p>

    <br />
}
  
```

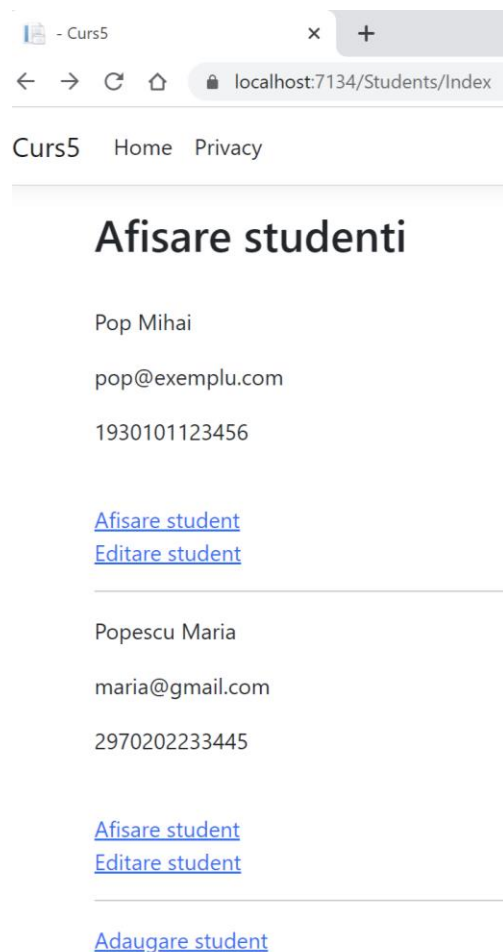
```

        <a href="/Students/Show/@student.StudentID">Afisare
student</a>
        <br />
        <a href="/Students/Edit/@student.StudentID">Editare
student</a>
        <hr />

    }

    <a href="/Students/New">Adaugare student</a>

```



Show

```

public ActionResult Show(int id)
{
    Student student = db.Students.Find(id);
    ViewBag.Student = student;
    return View();
}

```

Metoda Find() primeste ca parametru o valoare pentru coloana care este cheia primara

Show.cshtml

```
<h2>Afisare student</h2>

<br />

<p>@ViewBag.Student.Name</p>
<p>@ViewBag.Student.Email</p>
<p>@ViewBag.Student.CNP</p>

<br />
<a href="/Students/Index">Afisare studenti</a>
```

New

```
public IActionResult New()
{
    return View();
}

[HttpPost]
public IActionResult New(Student s)
{
    try
    {
        db.Students.Add(s);
        db.SaveChanges();
        return RedirectToAction("Index");
    }
    catch (Exception)
    {
        return View();
    }
}
```

Students.Add primește ca parametru un obiect de tip Student iar SaveChanges va face commit în baza de date

New.cshtml

```
<h2>Formular adaugare student</h2>

<form method="post" action="/Students/New">
    <label>Nume</label>
    <br />
    <input type="text" name="Name" />
    <br /><br />
    <label>Adresa e-mail</label>
    <br />
    <input type="text" name="Email" />
```

```

<br /><br />
<label>CNP</label>
<br />
<input type="text" name="CNP" />
<br />
<br />
<button type="submit">Adauga student</button>
</form>

```

Formular adaugare student

Nume

Adresa e-mail

CNP

Adauga student

Model Binding

In ASP.NET MVC **model binding** ne permite sa facem legatura intre request-urile de tip HTTP si un Model. Model binding este procesul de creare a obiectelor folosind datele trimise de browser printr-un request HTTP (prin intermediul formularelor din View).

Model binding este o legatura intre request-urile HTTP si metodele unui Controller (Actiuni). Deoarece datele trimise prin POST sau GET ajung intotdeauna la Controller, acest mecanism de binding leaga in mod automat variabilele de request cu attributele publice ale modelului. Aceasta mapare se va face dupa **numele atributelor modelului**.

```
<label>Nume</label>
```

```
<input type="text" name="Name" />
```

```
<label>Adresa e-mail</label>
```

```
<input type="text" name="Email" />
```

```
<label>CNP</label>
```

```
<input type="text" name="CNP" />
```

Parametrii care se vor trimite prin request la controller

!! OBSERVATIE

Este necesar ca numele campurilor din View sa coincida cu numele atributelor din Model pentru ca binding-ul sa functioneze.

Edit

```
public IActionResult Edit(int id)
{
    Student student = db.Students.Find(id);
    ViewBag.Student = student;
    return View();
}

[HttpPost]
public ActionResult Edit(int id, Student requestStudent)
{
    Student student = db.Students.Find(id);

    try
    {
        student.Name = requestStudent.Name;
        student.Email = requestStudent.Email;
        student.CNP = requestStudent.CNP;
        db.SaveChanges();

        return RedirectToAction("Index");
    }
    catch (Exception)
    {
        return RedirectToAction("Edit", student.StudentID);
    }
}
```

Edit.cshtml

```
<h2>Editare student</h2>

<br />

<form method="post"
action="/Students/Edit/@ViewBag.Student.StudentID">

    <label>Nume</label>
    <br />
    <input type="text" name="Name" value="@ViewBag.Student.Name" />
    <br /><br />
    <label>Adresa e-mail</label>
    <br />
    <input type="text" name="Email" value="@ViewBag.Student.Email" />
    <br /><br />
    <label>CNP</label>
    <br />
    <input type="text" name="CNP" value="@ViewBag.Student.CNP" />
    <br />
    <button type="submit">Modifica student</button>

</form>
```

Delete

```
[HttpPost]
public ActionResult Delete(int id)
{
    Student student = db.Students.Find(id);
    db.Students.Remove(student);
    db.SaveChanges();
    return RedirectToAction("Index");
}
```

Remove primește ca parametru un obiect de tip **Student**. **SaveChanges** salvează modificările

Show.cshtml (se va utiliza view-ul show)

```
<form method="post"
action="/Students/Delete/@ViewBag.Student.StudentID">

    <button type="submit">Sterge studentul</button>

</form>
```

/\ OBSERVATIE

In momentul in care sunt necesare in baza de date, fie adaugari sau stergeri de tabele, fie adaugari sau stergeri de coloane sau proprietati, este nevoie de o noua migratie in baza de date.

De exemplu: daca se doreste adaugarea atributului **Address** in clasa Student → `public string Address { get; set; }`

Se adauga proprietatea, dupa care se executa o noua migratie

→ Add-Migration AddAddressToStudent

→ Update-Database

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 6

Cuprins

Baze de Date – Notiuni generale	3
Ce este o baza de date	3
Ce este un SGBD/DBMS.....	3
Cerinte minimale ale bazelor de date.....	3
Ce este cheia primara	4
Ce este cheia externa.....	4
Ce este o cheie primara compusa.....	4
Ce este o entitate	4
Ce este o relatie	5
Ce este un atribut.....	5
Exemple – Chei, Entitati, Relatii	5
Diagrama Entitate/Relatie	6
Reguli de proiectare a diagramei E/R	6
Exemplu practic pentru proiectarea Diagramei E/R.....	6
Diagrama Conceptuala.....	10
Reguli de transformare a diagramei E/R in Diagrama Conceptuala	10
Exemplu pentru proiectarea Diagramei Conceptuale	10
Adaugarea sistemului de autentificare	12
Implementare cereri folosind Entity Framework Core si LINQ.....	15
Implementarea relatiilor din baza de date	27
Relatia many-to-many – Varianta 1 – utilizand proprietatile	27
Relatia many-to-many – Varianta 2.....	27

Relatia one-to-many – Varianta 1 – utilizand proprietatile	30
Relatia one-to-many – Varianta 2	30
Relatia one-to-one – Varianta 1 - utilizand proprietatile	32
Relatia one-to-one – Varianta 2	33

Baze de Date – Notiuni generale

Ce este o baza de date

O **baza de date** este un ansamblu structurat de date coerente, fara redundanta, astfel incat datele pot fi prelucrate eficient de mai multi utilizatori intr-un mod concurent.

Ce este un SGBD/DBMS

Un SGBD este un sistem de gestiunea a bazelor de date. Denumirea de SGBD este echivalenta cu cea de DBMS (**Database Management System**) reprezentand un produs software care asigura interactiunea cu o baza de date. Un **DBMS** stocheaza date in tabele, pe care ulterior le acceseaza si prelucreaza cu ajutorul unui limbaj **SQL (Structured Query Language)**. Un DBMS asigura securitatea, integritatea si consistenta datelor.

Exemple de DBMS: Oracle Database, SQL Server, MySQL.

Cerinte minimale ale bazelor de date

O baza de date trebuie sa indeplineasca urmatoarele cerinte minimale:

- Redundanta minima
- Sincronizarea datelor – utilizarea simultana a datelor de catre mai multi utilizatori
- Securitatea datelor
- Integritatea datelor – date corecte, posibilitatea recuperarii lor
- Furnizare rapida a datelor – cereri eficiente
- Flexibilitatea datelor – adaptarea rapida la cerinte noi

Ce este cheia primara

Cheia primara este un identificator **UNIC** in cadrul entitatilor. Ea trebuie sa fie cunoscuta in orice moment – ceea ce inseamna ca **nu poate fi null**

Cu ajutorul cheii primare se identifica unic intrarile dintr-un tabel al bazei de date.

Ce este cheia externa

Cheia externa poate fi ori null in intregime, ori trebuie sa refere cheia primara din tabelul de legatura (adica sa corespunda unei valori a cheii primare asociate).

Ce este o cheie primara compusa

Cheia primara compusa se creeaza in momentul in care se combina doua sau mai multe coloane in cadrul unui tabel, formand un tuplu, care mai apoi este utilizat ca identificator unic in cadrul tabelului respectiv. Tuplul o sa identifice unic fiecare intrare din tabelul in care se afla.

Ce este o entitate

Entitate = un loc, o actiune, o persoana, etc.

Exemplu de entitati dintr-o baza de date care gestioneaza o Universitate
→ Studenti, Cursuri, Note, etc

In modelul Entitate/Relatie, entitatile sunt substantive. In modelele relationale entitatile devin tabele.

Nu pot exista in aceeași diagramă două entități cu același nume sau aceeași entitate având un nume diferit.

Ce este o relație

O **relație** este o asocieră dintre două sau mai multe entități. În modelul Entitate/Relație acestea sunt verbe. În modelul relational relațiile devin fie tabele speciale, fie coloane care referă chei primare.

Relațiile au asociată și o cardinalitate, existând trei tipuri de cardinalități:

- **one-to-many (1:m)**
- **one-to-one (1:1)**
- **many-to-many (m:m)**

Ce este un atribut

Un **atribut** este o proprietate care descrie o entitate. Fiecare atribut trebuie să aibă un tip de date, un nume și constrângeri.

Exemple – Chei, Entități, Relații

Un exemplu detaliat se află în documentul numit **Chei_Entități_Relatii** aflat pe site în secțiunea **Săptămână 6**.

Diagrama Entitate/Relatie

Diagrama Entitate/Relatie este un mod de reprezentare a unui sistem din lumea reala, fiind un model neformalizat. Este compus din entitati si relatiile dintre acestea.

Reguli de proiectare a diagramei E/R

1. Identificarea entitatilor care fac parte din aplicatia pe care dorim sa o implementam
2. Identificarea relatiilor dintre entitati si scrierea acestora in cadrul diagramei
3. Identificarea cardinalitatilor minime si maxime pentru fiecare relatie in parte
4. Identificarea atributelor asociate fiecarei entitati
5. Stabilirea cheilor primare (atributele care identifica in mod unic fiecare entitate)

Exemplu practic pentru proiectarea Diagramei E/R

Sa se proiecteze o baza de date care modeleaza o aplicatie de tip **Engine de stiri** avand urmatoarele reguli de proiectare:

- sa existe cel putin 4 tipuri de utilizatori: vizitator neinregistrat, utilizator inregistrat, editor si administrator;
- orice utilizator poate vizualiza stirile aparute pe site. Pe pagina principala vor aparea stirile cele mai recente;
- stirile vor fi impartite pe categorii (create dinamic): stiinta, tehnologie, sport, etc, existand posibilitatea de adaugare a noi categorii (administratorul poate face CRUD pe categorii);
- stirile dintr-o anumita categorie sunt afisate intr-o pagina separata unde pot fi sortate dupa diferite criterii: data aparitiei si alfabetic;

- editorii se ocupa de publicarea stirilor noi si pot vizualiza, edita, sterge propriile stiri;
- utilizatorii pot adauga comentarii la stirile aparute, isi pot sterge si edita propriile comentarii;
- stirile pot fi cautate prin intermediul unui motor de cautare propriu;
- administratorii se ocupa de buna functionare a intregii aplicatii (ex: pot face CRUD pe stiri, pe categorii, etc.) si pot activa sau revoca drepturile utilizatorilor si editorilor;

PASUL 1 – Identificarea entitatilor

- sa existe cel putin 4 tipuri de utilizatori → **USER**
- vizitator neinregistrat, utilizator inregistrat, editor si administrator → **ROLES**
- orice utilizator poate vizualiza stirile → **ARTICLES**
- stirile vor fi impartite pe categorii → **CATEGORIES**
- utilizatorii pot adauga comentarii la stirile aparute → **COMMENTS**
- restul functionalitatilor se implementeaza folosind logica de backend (cod), nefiind nevoie de alte entitati/tabele.

!/ OBSERVATIE

In **Entity Framework** clasele se denumesc la singular deoarece sistemul converteste automat fiecare clasa intr-un tabel, pluralizand numele. Se defineste clasa **Article.cs** in Models, clasa care o sa devina automat tabelul **Articles**.

PASUL 2 – Identificarea relatiilor si a cardinalitatilor

Avem urmatoarele entitati identificate: **USER, ROLES, ARTICLES, COMMENTS, CATEGORIES**

- sa existe cel putin 4 tipuri de utilizatori: vizitator neinregistrat, utilizator inregistrat, editor si administrator → **un utilizator poate avea un singur rol (!\ OBS** – avand in vedere ca Entity Framework utilizeaza in acest caz o relatie de tip many-to-many vom implementa si noi o astfel de relatie, dar o vom utiliza ca fiind one-to-many facand asocierea UNUI utilizator cu UN singur rol – ca logica in aplicatie)
- editorii se ocupa de publicarea stirilor noi si pot vizualiza, edita, sterge propriile stiri (un user de tip editor poate sa faca CRUD, iar un user inregistrat poate vizualiza articole) → **un editor vizualizeaza/redacteaza/editeaza/sterge/ mai multe articole, iar un articol este vizualizat/creat/editat/sters/ de un singur editor → relatia (cardinalitatea maxima) este one-to-many**
Pentru vizualizare se foloseste logica scrisa in cod.
- stirile vor fi impartite pe categorii → **un articol face parte dintr-o singura categorie, iar o categorie contine mai multe articole → cardinalitatea maxima este one-to-many**
- utilizatorii pot adauga comentarii la stirile aparute, isi pot sterge si edita propriile comentarii → **un articol contine mai multe comentarii, iar un comentariu apartine unui singur articol → cardinalitate one-to-many**
→ **un user posteaza mai multe comentarii, iar un comentariu este postat de un singur user → one-to-many**

PASUL 3- Proiectarea diagramei E/R

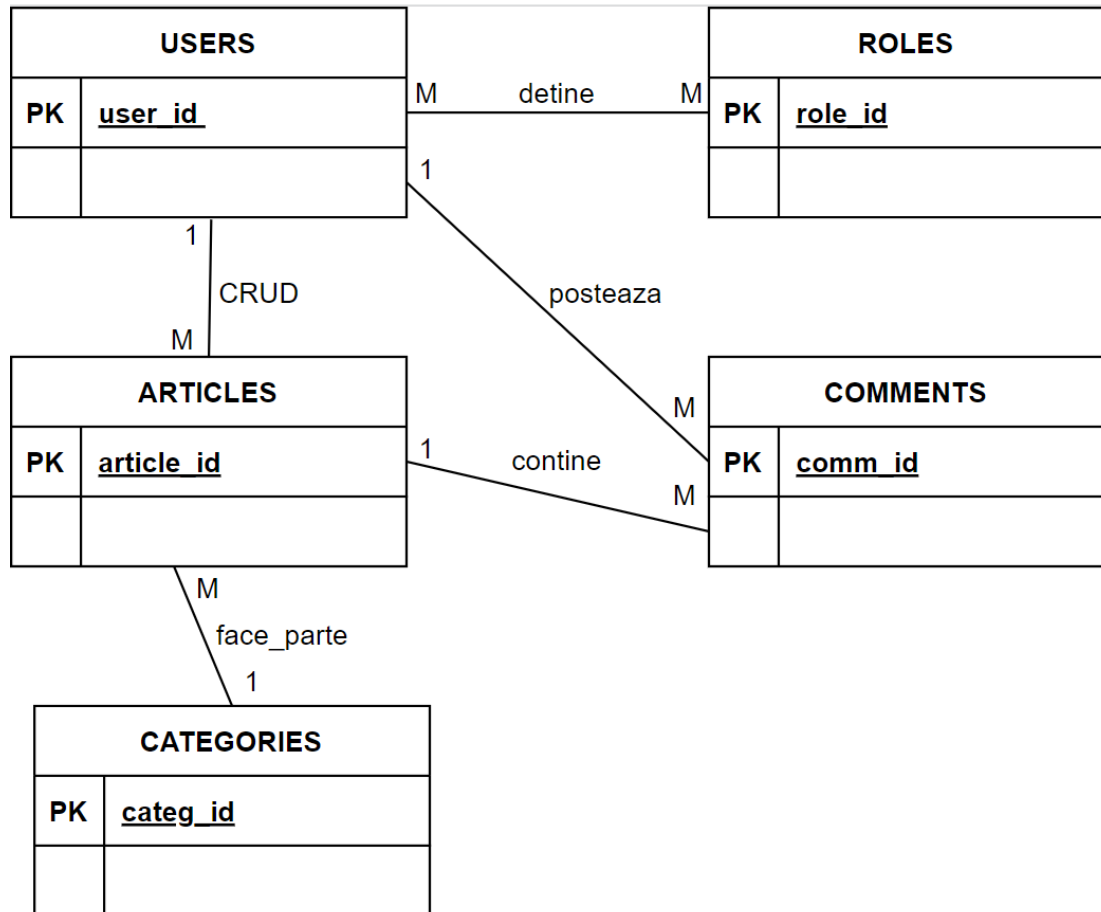


Diagrama Conceptuala

Diagrama Conceptuala este un model formal de organizare a datelor, continand legatura dintre acestea sub forma de tabele.

Reguli de transformare a diagramei E/R in Diagrama Conceptuala

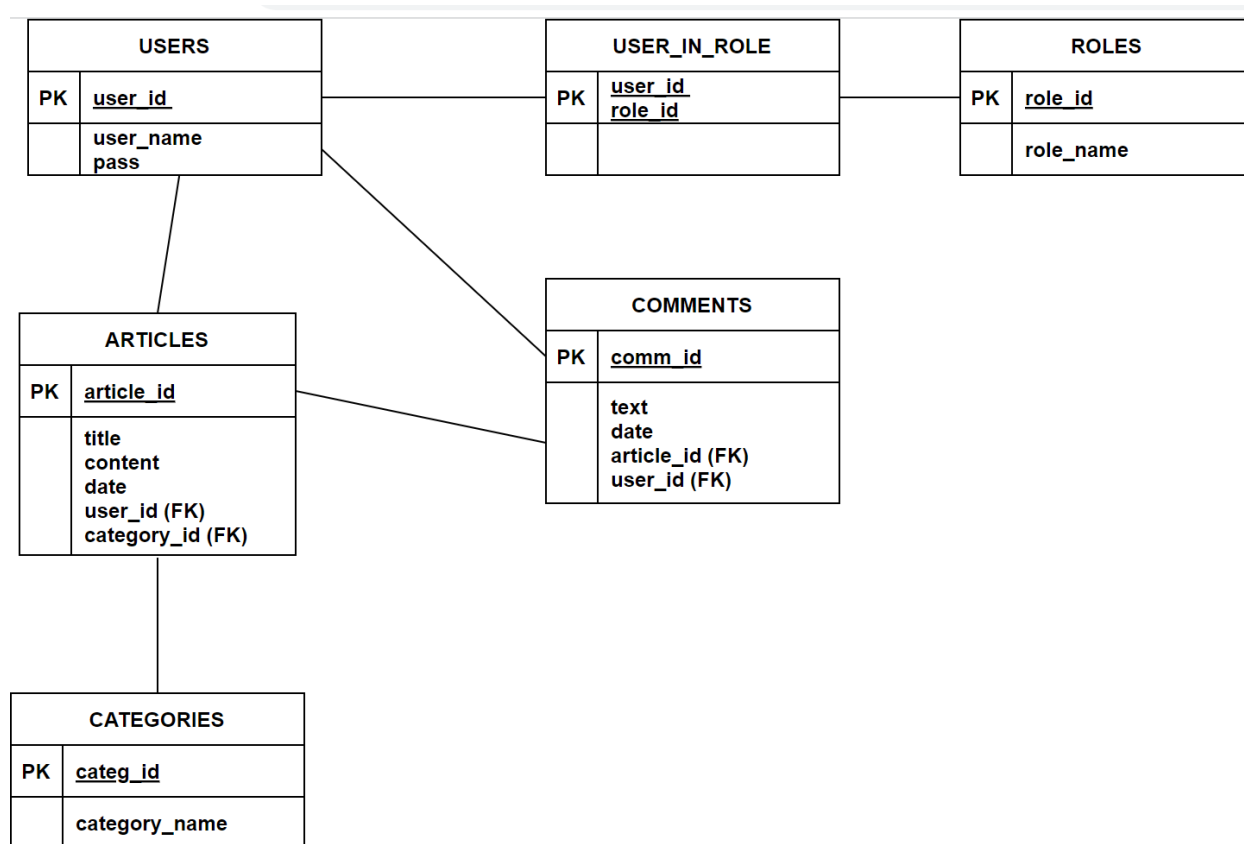
Pentru transformarea Diagramei Entitate/Relatie in Diagrama Conceptuala se urmeaza urmatoarele reguli:

- Entitatile devin tabele;
- Relatiile one-to-one si one-to-many devin chei externe
 - In cazul relatiilor **one-to-many** cheia externa se plaseaza in tabelul in dreptul caruia se afla cardinalitatea **many**
 - In cazul relatiilor **one-to-one** cheia externa se plaseaza in tabelul care contine mai putine intrari in baza de date (din motive de performanta/eficienta)
- Relatiile many-to-many devin tabel asociativ

Exemplu pentru proiectarea Diagramei Conceptuale

Urmand regulile anterioare transformam Diagrama Entitate/Relatie din sectiunea **Exemplu practic pentru proiectarea Diagramei E/R – PASUL 3** in Diagrama Conceptuala.

Diagrama Conceptuala:

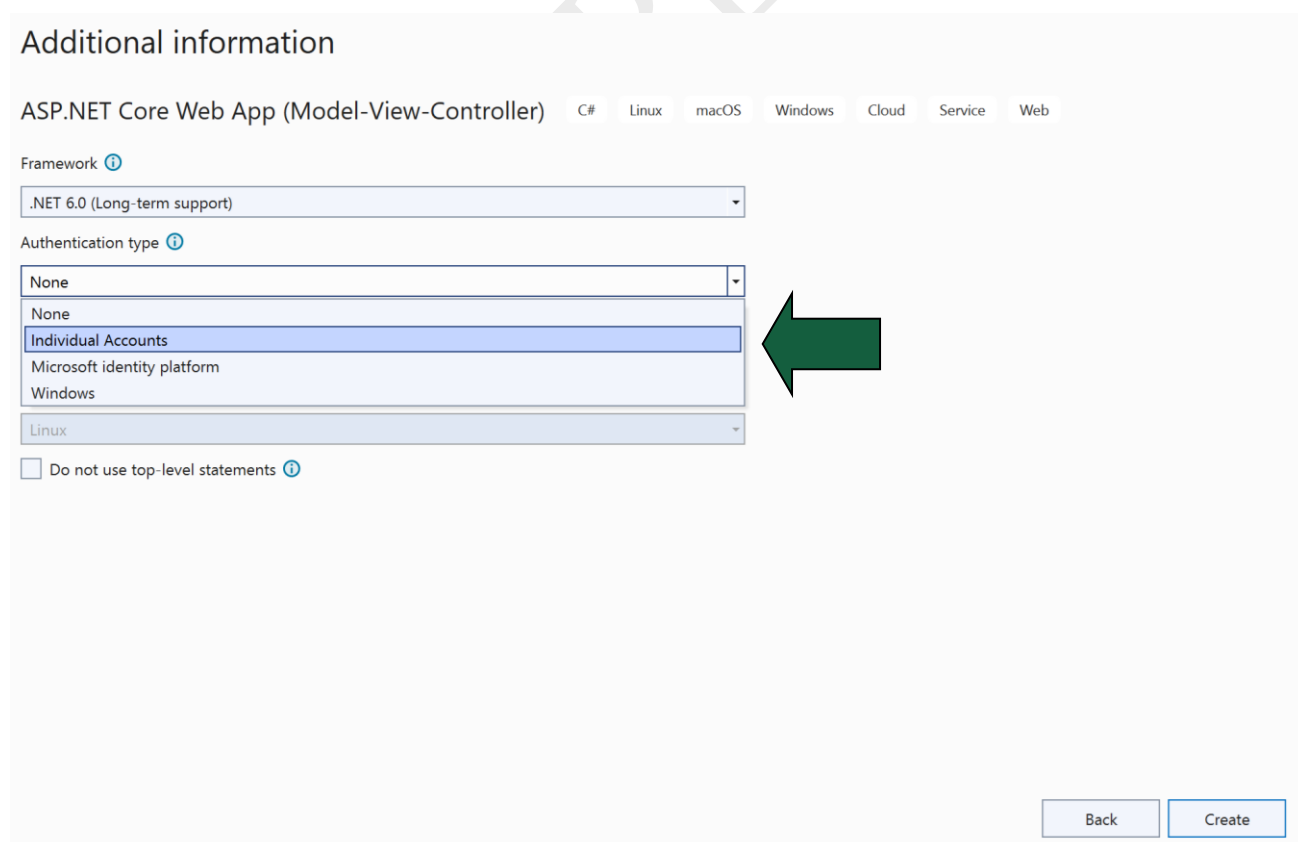


Adaugarea sistemului de autentificare

Framework-ul ASP.NET Core ofera posibilitatea integrarii unui sistem de autentificare folosind **Identity**.

Identity este compus dintr-o suita de clase si secvente de cod care faciliteaza implementarea rapida a unui sistem de autentificare complex. Acest sistem ofera posibilitatea autentificarii folosind user si parola, alocarea de roluri pentru utilizatori, autentificare folosind conturi 3rd party (autentificare prin retele de socializare – Google, Facebook, Twitter, etc).

Pentru a genera un proiect care include componenta **Identity** pentru autentificare, trebuie sa alegem la crearea proiectului forma de autentificare: **Individual Accounts**.



Additional information

ASP.NET Core Web App (Model-View-Controller) C# Linux macOS Windows Cloud Service Web

Framework ⓘ

.NET 6.0 (Long-term support)

Authentication type ⓘ

None

None

Individual Accounts

Microsoft identity platform

Windows

Linux

☐ Do not use top-level statements ⓘ

Back Create

Proiectul nou creat contine **Identity Framework** si toate mecanismele aferente autentificarii.

Inainte de rularea proiectului trebuie executata in consola (Tools → NuGet Package Manager → Package Manager Console) comanda **Update-Database**

Dupa rularea comenzii, ne putem inregistra cu un cont.

Laborator6 Home Privacy Register Login

Register

Create a new account.

Email

Password

Confirm password

Register

Use another service to register.

There are no external authentication services configured. See this [article about setting up this ASP.NET application to support logging in via external services](#).

Dupa inregistrare se apasa [Click here to confirm your account](#) deoarece nu exista inclus serviciul de trimitere de e-mailuri.

Laborator6 Home Privacy Register Login

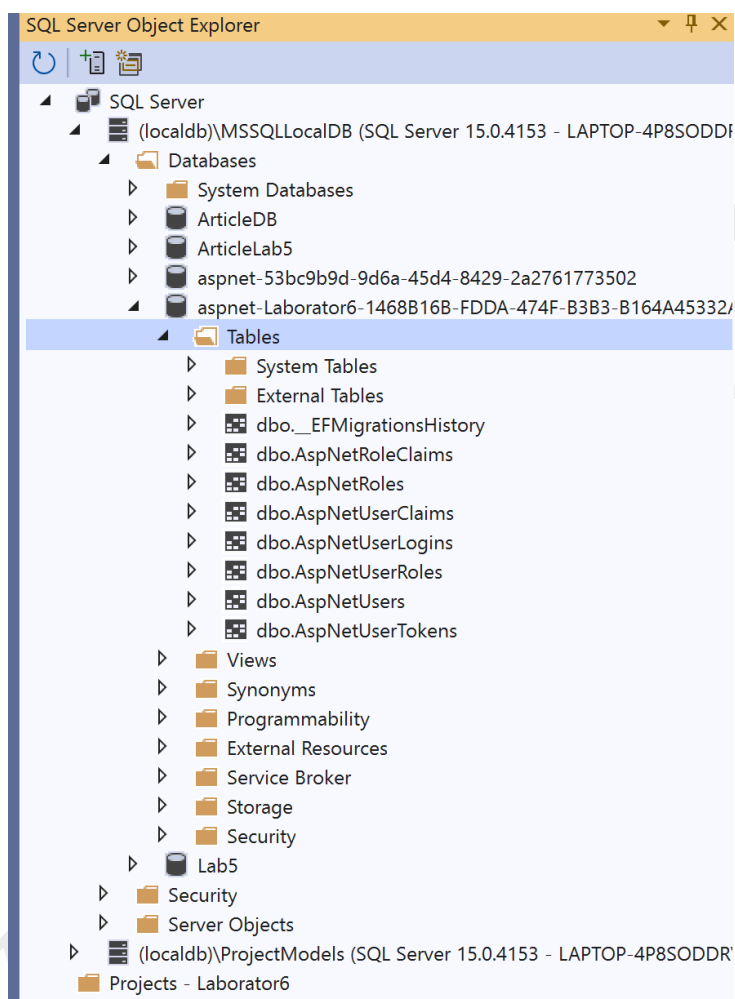
Register confirmation

This app does not currently have a real email sender registered, see [these docs](#) for how to configure a real email sender. Normally this would be emailed: [Click here to confirm your account](#)

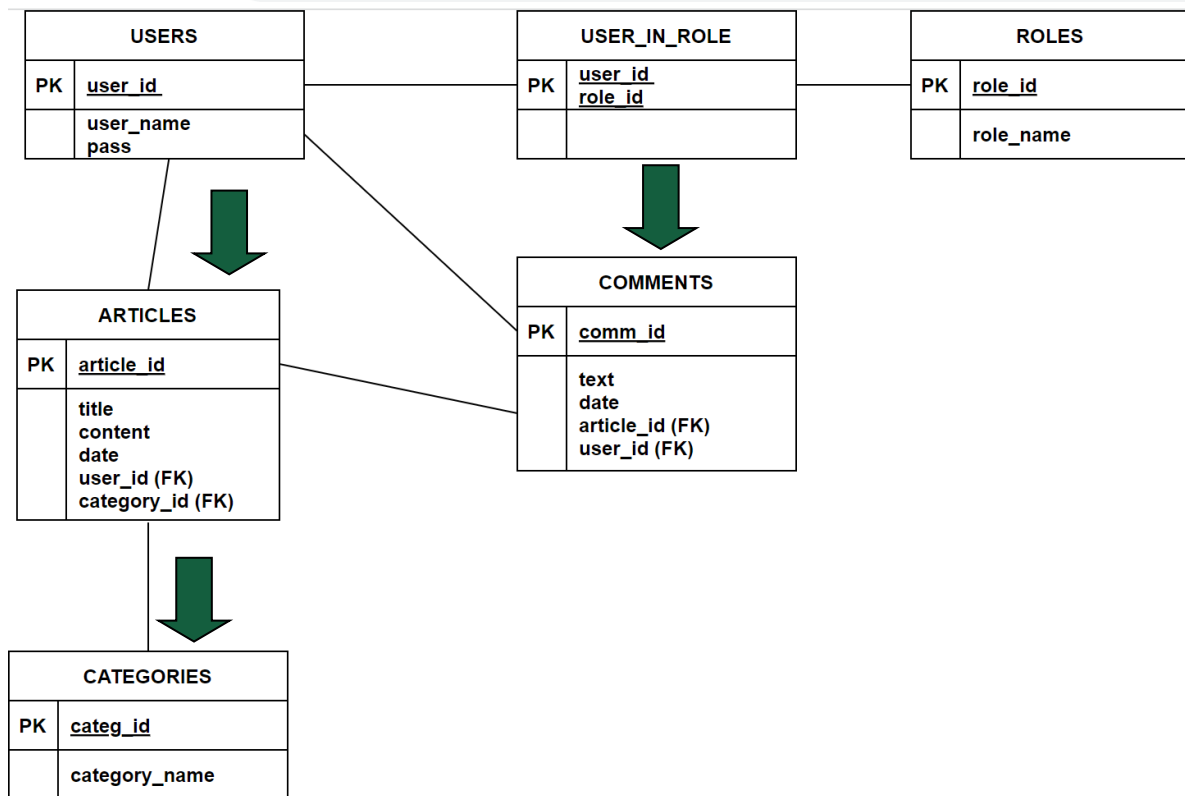
Confirm email

Thank you for confirming your email.

In proiect → SQL Server Object Explorer → se pot vizualiza tabelele create pentru sistemul de autentificare si pentru rolurile pe care le pot avea utilizatorii



Implementare cereri folosind Entity Framework Core si LINQ



Se considera entitatile **Article**, **Category** si **Comment** cu urmatoarele proprietati:

Article

- Id (int – primary key)
- Title (string – titlul este obligatoriu)
- Content (string – continutul este obligatoriu)
- Date (DateTime)
- CategoryId (int – cheie externa – categoria din care face parte articolul)

Category:

- Id (int – primary key)
- CategoryName (string – numele este obligatoriu)

Comment:

- Id (int – primary key)
- Content (string – continutul comentariului este obligatoriu)
- Date (DateTime – data la care a fost postat comentariul)
- ArticleId (int – cheie externa – articolul caruia ii apartine comentariul)

Implementarea claselor folosind Entity Framework:

```

public class Article
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Titlul este obligatoriu")]
    public string Title { get; set; }

    [Required(ErrorMessage = "Continutul articolului este obligatoriu")]
    public string Content { get; set; }

    public DateTime Date { get; set; }

    [Required(ErrorMessage = "Categoria este obligatorie")]
    public int CategoryId { get; set; }

    public virtual Category Category { get; set; }

    public virtual ICollection<Comment> Comments { get; set; }
}

```

```

public class Category
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Numele categoriei este obligatoriu")]
    public string CategoryName { get; set; }

    public virtual ICollection<Article> Articles { get; set; }
}

public class Comment
{
    [Key]
    public int CommentId { get; set; }

    [Required(ErrorMessage = "Continutul este obligatoriu")]
    public string Content { get; set; }

    public DateTime Date { get; set; }

    public int ArticleId { get; set; }

    public virtual Article Article { get; set; }
}

```

!/ OBSERVATIE

Dupa implementarea claselor se executa migratiile:

- Add-Migration NumeMigratie
- Update-Database

JOIN

1. Sa se afiseze articolele impreuna cu categoria din care fac parte

```
var query = db.Articles.Include("Category");
```

unde parametrul **Category** este proprietatea din clasa Article

```
→ public virtual Category Category { get; set; }
```



Codul asociat generat in consola:

```
SELECT [a].[Id], [a].[CategoryId], [a].[Content], [a].[Date],
[a].[Title], [c].[Id], [c].[CategoryName]
FROM [Articles] AS [a] INNER JOIN [Categories] AS [c]
ON [a].[CategoryId] = [c].[Id]
```

```
▼ [
  ▼ {
    "id": 1,
    "title": "Articol1",
    "content": "Continut1",
    "date": "2022-07-11T00:00:00",
    "categoryId": 1,
    ▼ "category": {
      "id": 1,
      "categoryName": "Categ1",
      ▼ "articles": [
        null
      ]
    },
    "comments": null
  },
  ▼ {
    "id": 2,
    "title": "Articol2",
    "content": "Continut2",
    "date": "2022-10-10T00:00:00",
    "categoryId": 2,
    ▼ "category": {
      "id": 2,
      "categoryName": "Categ2",
      ▼ "articles": [
        null
      ]
    },
    "comments": null
  }
]
```

2. Sa se afiseze articolele, categoria din care fac parte si comentariile asociate. Sa se afiseze titlul articolului, numele categoriei, comentariul si data la care a fost postat comentariul respectiv.

```
var query = from item in
db.Articles.Include("Category").Include("Comments")

select new
{
    ArticleTitle = item.Title,
    CategoryTitle = item.Category.CategoryName,
    Comments = (
        from comm in item.Comments
        select new {
            CommentTitle = comm.Content,
            CommentDate = comm.Date
        }
    )
};
```

→ unde **Category** si **Comments** sunt proprietatile din clasa Article

```
public virtual Category Category { get; set; }
public virtual ICollection<Comment> Comments { get; set; }
```

Codul asociat generat in consola:

```
SELECT [a].[Title], [c].[CategoryName], [a].[Id], [c].[Id],
[c0].[Content], [c0].[Date], [c0].[CommentId]
FROM [Articles] AS [a]
INNER JOIN [Categories] AS [c] ON [a].[CategoryId] = [c].[Id]
LEFT JOIN [Comments] AS [c0] ON [a].[Id] = [c0].[ArticleId]
ORDER BY [a].[Id], [c].[Id]
```

```

▼ [
  ▼ {
    "articleTitle": "Articol1",
    "categoryTitle": "Categ1",
    ▼ "comments": [
      ▼ {
        "commentTitle": "Comm1",
        "commentDate": "2022-07-11T00:00:00"
      },
      ▼ {
        "commentTitle": "Comm2",
        "commentDate": "2022-07-02T00:00:00"
      }
    ]
  },
  ▼ {
    "articleTitle": "Articol2",
    "categoryTitle": "Categ2",
    "comments": []
  }
]

```

Aceste proprietati sunt **virtuale** pentru a se executa **lazy loading**. Lazy loading este un design pattern folosit in dezvoltarea web care intarzie incarcarea anumitor resurse pentru a imbunatati performanta.

In cazul nostru, lazy loading se realizeaza prin declararea proprietatilor de tip **virtual** si duce la incarcarea claselor doar in momentul in care sunt folosite proprietatile respective. Un exemplu in acest sens ar fi utilizarea joinului folosind **Include**, dar nefolosirea proprietatii Category. In acest caz vom observa ca indiferent daca exista in join, nefiind utilizata pentru afisare in query, nu o sa fie incarcata proprietatea Category din cadrul clasei Article.

```

var query = from item in
db.Articles.Include("Category").Include("Comments")

select new
{
    ArticleTitle = item.Title,
    //CategoryTitle = item.Category.CategoryName,
    Comments = (
        from comm in item.Comments
        select new {
            CommentTitle = comm.Content,
            CommentDate = comm.Date
        }
    )
};

```

Codul asociat generat in consola:

```

SELECT [a].[Title], [a].[Id], [c].[Content], [c].[Date],
[c].[CommentId]
FROM [Articles] AS [a]
LEFT JOIN [Comments] AS [c] ON [a].[Id] = [c].[ArticleId]
ORDER BY [a].[Id]

```

```

[
  {
    "articleTitle": "Articol1",
    "comments": [
      {
        "commentTitle": "Comm1",
        "commentDate": "2022-07-11T00:00:00"
      },
      {
        "commentTitle": "Comm2",
        "commentDate": "2022-07-02T00:00:00"
      }
    ]
  },
  {
    "articleTitle": "Articol2",
    "comments": []
  }
]

```

GROUP BY si functii grup (de exemplu COUNT)

Pentru fiecare categorie sa se afiseze numarul de articole care fac parte din categoria respectiva. O sa se afiseze id-ul si numele categoriei, impreuna cu numarul de articole care fac parte din categoria respectiva.

```
var query = from category in db.Categories
            join article in db.Articles
            on category.Id equals article.CategoryId
            group category by category.Id into groupedCategories

            select new
            {
                CategoryId = groupedCategories.Key,
                ArticlesCount = groupedCategories.Count(),
                CategorySpecificSelection = from _item in
groupedCategories
                                         select new
                                         {
                                             CategoryName = _item.CategoryName,
                                         }
            };
```

Codul generat in consola:

```
SELECT [t].[Id], [t].[c], [t0].[CategoryName], [t0].[Id],
[t0].[Id0]
FROM (
    SELECT [c].[Id], COUNT(*) AS [c]
    FROM [Categories] AS [c]
    INNER JOIN [Articles] AS [a] ON [c].[Id] =
[a].[CategoryId]
    GROUP BY [c].[Id]
) AS [t]
LEFT JOIN (
    SELECT [c0].[CategoryName], [c0].[Id], [a0].[Id] AS
[Id0]
    FROM [Categories] AS [c0]
    INNER JOIN [Articles] AS [a0] ON [c0].[Id] =
[a0].[CategoryId]
) AS [t0] ON [t].[Id] = [t0].[Id]
ORDER BY [t].[Id], [t0].[Id]
```

```

[
  {
    "categoryId": 1,
    "articlesCount": 1,
    "categorySpecificSelection": [
      {
        "categoryName": "Categ1"
      }
    ]
  },
  {
    "categoryId": 2,
    "articlesCount": 2,
    "categorySpecificSelection": [
      {
        "categoryName": "Categ2"
      },
      {
        "categoryName": "Categ2"
      }
    ]
  }
]

```

In acelasi mod se pot selecta in continuare informatiile despre articolele asociate, adaugand urmatoarea linie de cod in secventa anterioara:

```

var query = from category in db.Categories
            join article in db.Articles
            on category.Id equals article.CategoryId
            group category by category.Id into groupedCategories
            select new
            {
                CategoryId = groupedCategories.Key,
                ArticlesCount = groupedCategories.Count(),
                CategorySpecificSelection = from _item in
groupedCategories
                                         select new
                                         {
                                             CategoryName = _item.CategoryName,
                                             CategoryArticles = _item.Articles
                                         }
            };

```

Codul generat in consola:

```

SELECT [t].[Id], [t].[c], [t0].[CategoryName], [t0].[Id],
[t0].[Id0], [t0].[Id1], [t0].[CategoryId], [t0].[Content],
[t0].[Date], [t0].[Title]
FROM (
    SELECT [c].[Id], COUNT(*) AS [c]
    FROM [Categories] AS [c]
    INNER JOIN [Articles] AS [a] ON [c].[Id] =
[a].[CategoryId]
    GROUP BY [c].[Id]
) AS [t]
LEFT JOIN (
    SELECT [c0].[CategoryName], [c0].[Id], [a0].[Id] AS
[Id0], [a1].[Id] AS [Id1], [a1].[CategoryId], [a1].[Content],
[a1].[Date], [a1].[Title]
    FROM [Categories] AS [c0]
    INNER JOIN [Articles] AS [a0] ON [c0].[Id] =
[a0].[CategoryId]
    LEFT JOIN [Articles] AS [a1] ON [c0].[Id] =
[a1].[CategoryId]
) AS [t0] ON [t].[Id] = [t0].[Id]
ORDER BY [t].[Id], [t0].[Id], [t0].[Id0]

```

```

{
  "categoryId": 1,
  "articlesCount": 1,
  "categorySpecificSelection": [
    {
      "categoryName": "Cate1",
      "categoryArticles": [
        {
          "id": 1,
          "title": "Articol1",
          "content": "Continut1",
          "date": "2022-07-11T00:00:00",
          "categoryId": 1,
          "category": null,
          "comments": null
        }
      ]
    }
  ]
}

```

```
▼ {
  "categoryId": 2,
  "articlesCount": 2,
  ▼ "categorySpecificSelection": [
    ▼ {
      "categoryName": "Categ2",
      ▼ "categoryArticles": [
        ▼ {
          "id": 2,
          "title": "Articol2",
          "content": "Continut2",
          "date": "2022-10-10T00:00:00",
          "categoryId": 2,
          "category": null,
          "comments": null
        },
        ▼ {
          "id": 5,
          "title": "Articol3",
          "content": "Continut3",
          "date": "2022-10-11T00:00:00",
          "categoryId": 2,
          "category": null,
          "comments": null
        }
      ]
    }
  ],
},
```

Implementarea relatiilor din baza de date

Relatia many-to-many – Varianta 1 – utilizand proprietatile

In cazul in care se doreste implementarea unei **relatii de tip many-to-many**, utilizand conventiile din framework, si anume utilizarea proprietatilor in cadrul claselor, se procedeaza ca in exemplul urmator. In acest caz framework-ul **genereaza automat** tabelul asociativ.

De exemplu: daca avem clasele **Student** si **Course**, impreuna cu relatiile:

- un student participa la mai multe cursuri
- in cadrul unui curs participa mai multi student

Este suficient ca:

- in clasa **Student** sa existe o proprietate
 - **public virtual ICollection <Course> Courses { get; set; }**
- iar in clasa **Course** sa existe o proprietate:
 - **public virtual ICollection <Student> Students { get; set; }**

Relatia many-to-many – Varianta 2

In acest caz, relatia many-to-many se implementeaza de la zero, prin suprascrierea conventiilor existente. Aceasta metoda ne ofera mai multe optiuni de configurare, avand acces facil la attributele modelelor. In exemplul urmator este utilizat Entity Framework Core – Fluent API.

Pentru implementarea claselor se procedeaza astfel:

Article.cs

```
public class Article
{
    [Key]
    public int Id { get; set; }

    public string Title { get; set; }

    public virtual ICollection<ArticleCategory>? ArticleCategories
{ get; set; }
}
```

Category.cs

```
public class Category
{
    [Key]
    public int Id { get; set; }

    public string CategoryName { get; set; }

    public virtual ICollection<ArticleCategory>? ArticleCategories
{ get; set; }
}
```

ArticleCategory.cs

```
public class ArticleCategory
{
    public int? ArticleId { get; set; }

    public int? CategoryId { get; set; }

    public virtual Article? Article { get; set; }

    public virtual Category? Category { get; set; }
}
```

ApplicationDbContext.cs

```

public class ApplicationDbContext : DbContext
{
    public
    ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Article> Articles { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<ArticleCategory> ArticleCategories { get; set; }

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // definire primary key compus
        modelBuilder.Entity<ArticleCategory>()
            .HasKey(ac => new { ac.ArticleId, ac.CategoryId });

        // definire relatii cu modelele Category si Article (FK)
        modelBuilder.Entity<ArticleCategory>()
            .HasOne(ac => ac.Article)
            .WithMany(ac => ac.ArticleCategories)
            .HasForeignKey(ac => ac.ArticleId);

        modelBuilder.Entity<ArticleCategory>()
            .HasOne(ac => ac.Category)
            .WithMany(ac => ac.ArticleCategories)
            .HasForeignKey(ac => ac.CategoryId);
    }
}

```

Relatia one-to-many – Varianta 1 – utilizand proprietatile

Exista si in acest caz doua variante de implementare. Cel mai des este utilizata varianta prezenta in sectiunea **Implementare cereri folosind Entity Framework Core si LINQ** din cursul curent, si anume cea in care se utilizeaza conventiile din cadrul framework-ului. Aceste conventii sunt proprietatile adaugate in cadrul fiecarei clase.

Relatia one-to-many – Varianta 2

De asemenea, daca varianta suprascrierii conventiilor este mai potrivita, atunci se procedeaza ca in exemplul urmator. Varianta potrivita se alege in functie de ceea ce dorim sa modelam.

Article.cs

```
public class Article
{
    [Key]
    public int Id { get; set; }

    public string Title { get; set; }

    public int CategoryId { get; set; }

    public virtual Category Category { get; set; }
}
```

Category.cs

```
public class Category
{
    [Key]
    public int Id { get; set; }

    public string CategoryName { get; set; }

    public virtual ICollection<Article> Articles { get; set; }
}
```

ApplicationDbContext.cs

```
public class ApplicationDbContext : DbContext
{
    public
    ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Article> Articles { get; set; }
    public DbSet<Category> Categories { get; set; }

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // definire relatii cu modelele Category si Article (FK)

        modelBuilder.Entity<Article>()
            .HasOne<Category>(a => a.Category)
            .WithMany(c => c.Articles)
            .HasForeignKey(a => a.CategoryId);
    }
}
```

Relatia one-to-one – Varianta 1 - utilizand proprietatile

Article.cs

```
public class Article
{
    [Key]
    public int Id { get; set; }

    public string Title { get; set; }

    public int CategoryId { get; set; }

    public virtual Category Category { get; set; }
}
```

Category.cs

```
public class Category
{
    [Key]
    public int Id { get; set; }

    public string CategoryName { get; set; }

    public virtual Article Article { get; set; }
}
```

Relatia one-to-one – Varianta 2

Article.cs

```
public class Article
{
    [Key]
    public int Id { get; set; }

    public string Title { get; set; }

    public int CategoryId { get; set; }

    public virtual Category Category { get; set; }
}
```

Category.cs

```
public class Category
{
    [Key]
    public int Id { get; set; }

    public string CategoryName { get; set; }

    public virtual Article Article { get; set; }
}
```

ApplicationDbContext.cs

```
public class ApplicationDbContext : DbContext
{
    public
    ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
}
```

```
public DbSet<Article> Articles { get; set; }

public DbSet<Category> Categories { get; set; }

protected override void OnModelCreating(ModelBuilder
modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // definire relatii cu modelele Category si Article (FK)
    modelBuilder.Entity<Article>()
        .HasOne<Category>(a => a.Category)
        .WithOne(c => c.Article)
        .HasForeignKey<Article>(a => a.CategoryId);
}
}
```

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 7

Cuprins

View (interfata cu utilizatorul).....	2
Ce este View-ul.....	2
Razor in cadrul unui proiect ASP.NET Core MVC	4
Crearea unui View in cadrul unui proiect.....	7
Trimiterea datelor catre View	9
Model	9
ViewBag.....	10
ViewData	11
TempData.....	12
Helpere pentru View	17
Implementarea alternativa a Helperelor.....	23

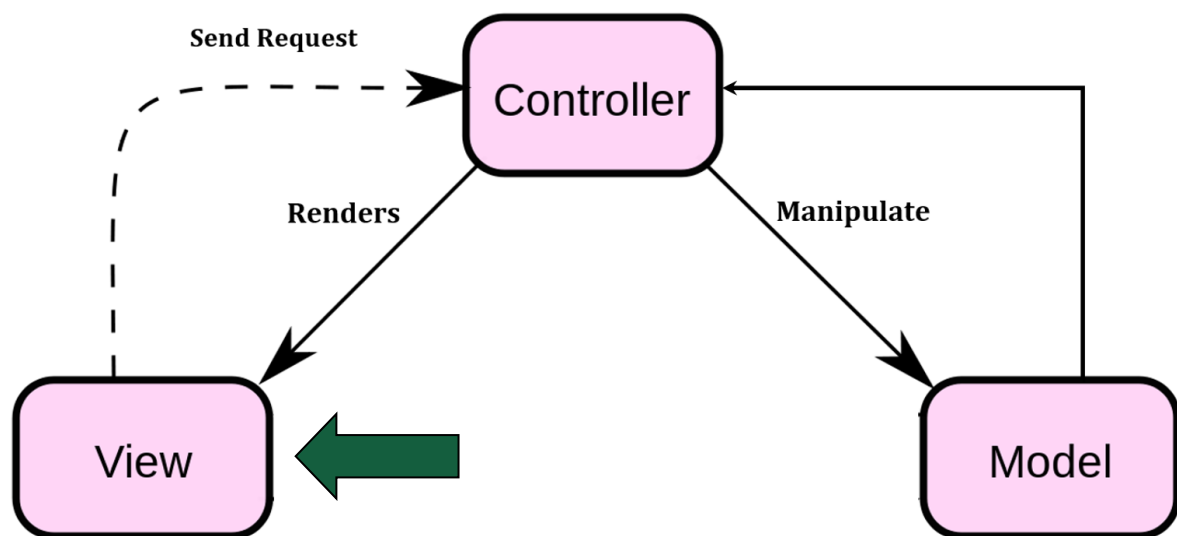
View (interfata cu utilizatorul)

Ce este View-ul

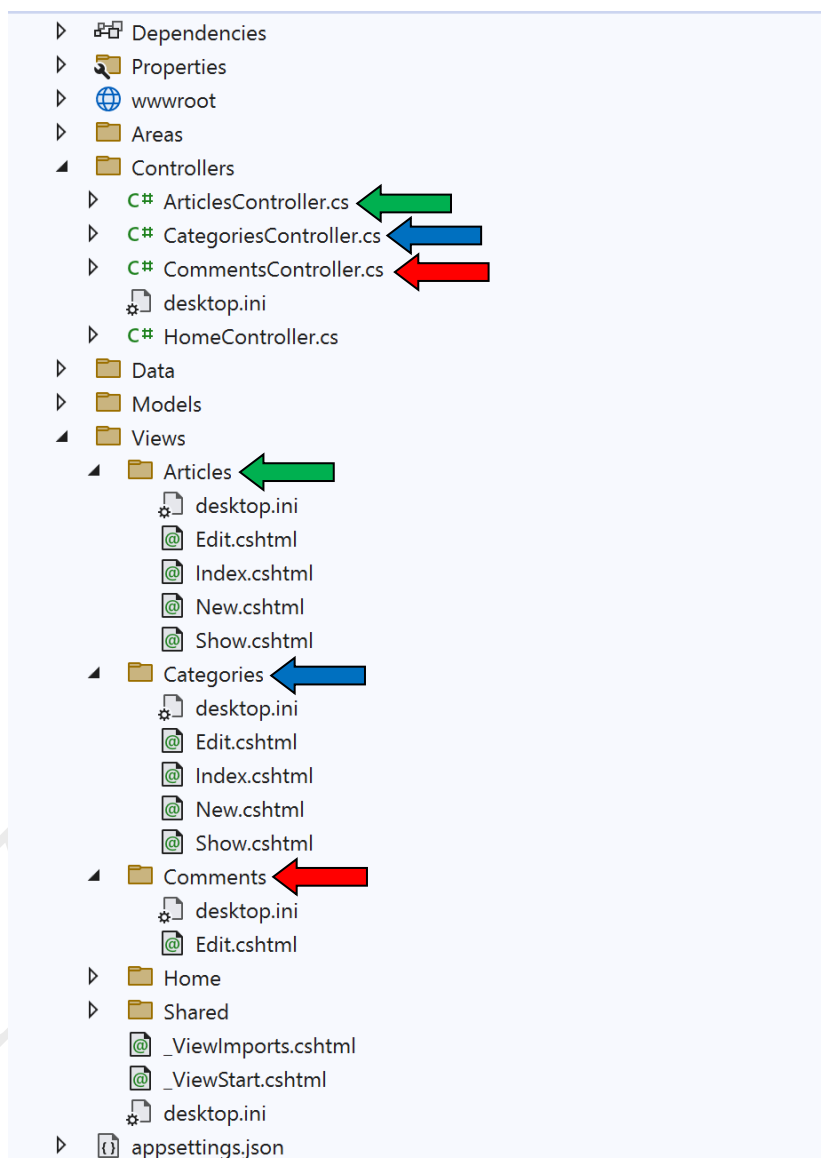
View-ul reprezinta interfata cu utilizatorul, fiind componenta arhitecturii MVC cu care utilizatorii interactioneaza prin intermediul browser-ului.

In View se afiseaza datele, adica inregistrarile din baza de date si informatiile generate de aplicatie.

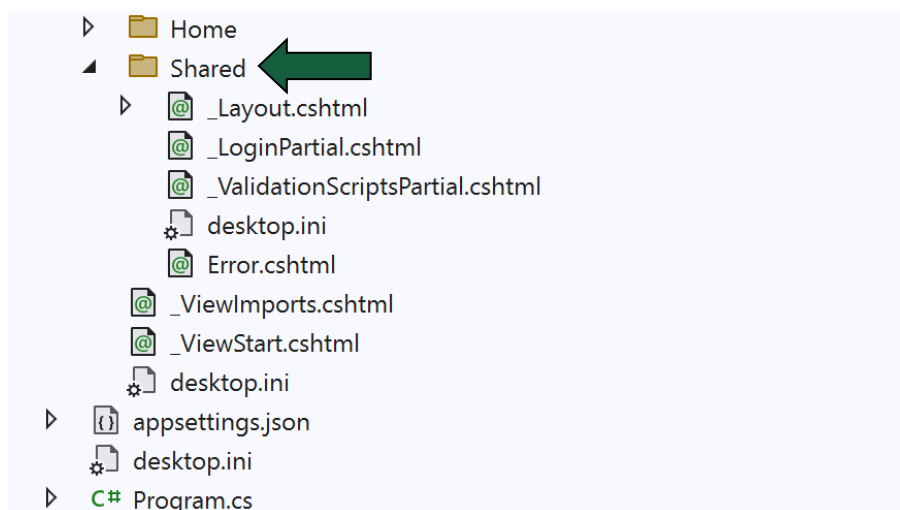
View-ul poate contine HTML static sau chiar HTML trimis din Controller (HTML dinamic). In cadrul arhitecturii MVC, View-ul comunica doar cu Controller-ul, iar cu Modelul comunica indirect tot prin intermediul Controller-ului.



View-urile in ASP.NET Core MVC se afla in folderul **Views**. Diferitele actiuni (metode) implementate intr-un Controller randeaza diferite View-uri, ceea ce inseamna ca folderul Views contine cate un folder separat pentru fiecare Controller, cu acelasi nume ca si Controller-ul, dupa cum se observa in imaginea urmatoare:



Folderul **Shared** contine View-urile (layout-uri, view-uri parțiale) care sunt partajate (folosite) in cadrul mai multor View-uri existente in aplicatie.



Razor in cadrul unui proiect ASP.NET Core MVC

Încă din ASP.NET MVC 3 motorul (engine-ul) de afișare a View-urilor în acest framework este **Razor**. Acest motor ne oferă posibilitatea de a mixa tag-uri HTML cu cod C#. În Razor se utilizează caracterul **@** pentru a începe o secvență de cod **server-side**.

Sintaxa Razor este simplă și a fost construită cu scopul de a minimiza lungimea codului scris. Aceasta este foarte compactă, ușor de învățat și este suportată de editorul Visual Studio.

Exemple sintaxa Razor:

- Se folosește **@** pentru executarea codului server-side pe o singură linie (de exemplu: pentru afișarea valorii unei variabile).

Exemplu – se afiseaza valoarea variabilei **ViewBag.Article** trimisa din Controller in View, preluandu-se atributul Title din modelul Article (conform exemplului implementat in cadrul laboratorului)

```
<h3>@ViewBag.Article.Title</h3>
```

- Parcurgerea colectiei **ViewBag.Articles** pentru afisarea articolelor

```
@foreach (var article in ViewBag.Articles)
```

- Pentru a executa un bloc intreg de cod (mai multe linii de cod) sau pentru a da o valoare unei variabile este necesar sa folosim acolade, astfel: `@{ /* cod */ }`

Exemplu – variabila ViewBag.Title primeste valoarea “Index”

```
@{
    ViewBag.Title = "Index";
}
```

Cu alte cuvinte, atunci cand se integreaza cod C# in HTML, se utilizeaza simbolul `@` pentru o singura linie de cod si `@{` pentru o secventa de cod.

- In cadrul unei secvente de cod se poate include cod HTML si text utilizand simbolul `@`:

Exemplul 1 – pentru includerea textului:

```
@if(1 > 0)
{
    @:este adevarat
}
```

În acest exemplu, putem vedea cum se poate afișa o secvență de text în cadrul unui bloc de cod. Secvența de mai sus va afișa pe ecran mesajul “este adevărat”.

Exemplul 2 – pentru includerea codului HTML (exemplu preluat din Laborator 6):

```
<select name="CategoryId">
    @foreach (var item in ViewBag.Categories)
    {
        <option value="@item.Id">@item.CategoryName</option>
    }
</select>
```

➤ Condiția **if** începe cu: **@if{ ... }**

Exemplu:

```
@if(1 > 10)
{
    @:este fals
}
```

➤ Loop-ul are următoarea sintaxă: **@for{ ... }**

Exemplu:

```
@for (int i = 0; i < 10; i++)
{
    @i.ToString()
}
```

- **@model** ofera posibilitatea afisarii valorilor modelului oriunde in View

Exemplu:

@model Curs7.Models.Student – includerea se realizeaza in View-ul asociat metodei

```
. . .
<h1>@Model.Name</h1>
<p>@Model.Email</p>
<p>@Model.CNP</p>
```

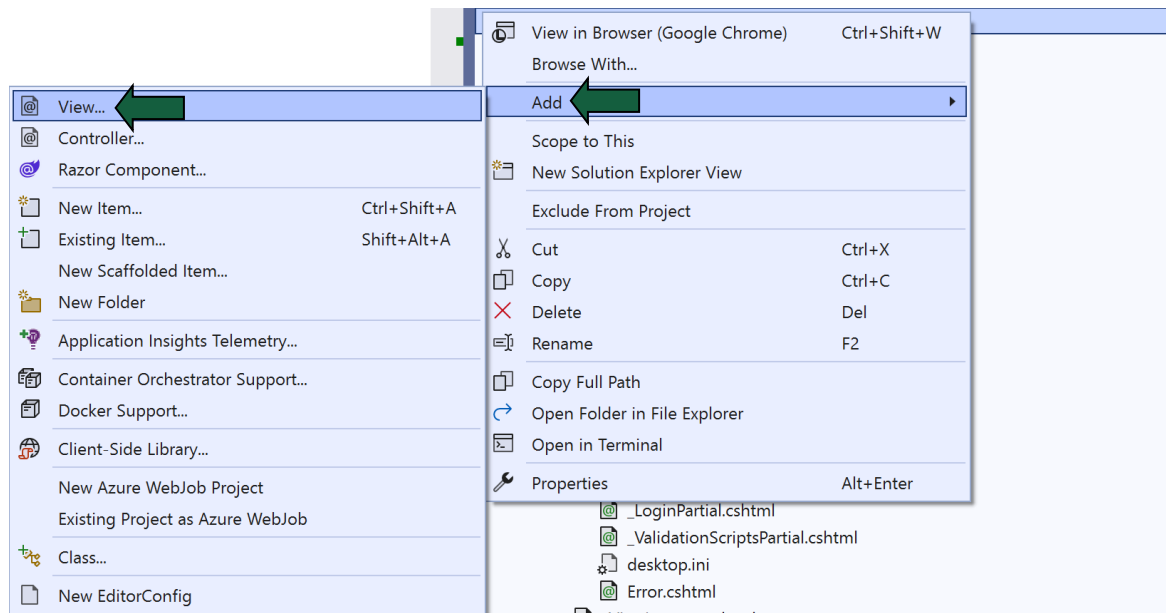
Crearea unui View in cadrul unui proiect

Se creeaza un View dupa cum urmeaza:

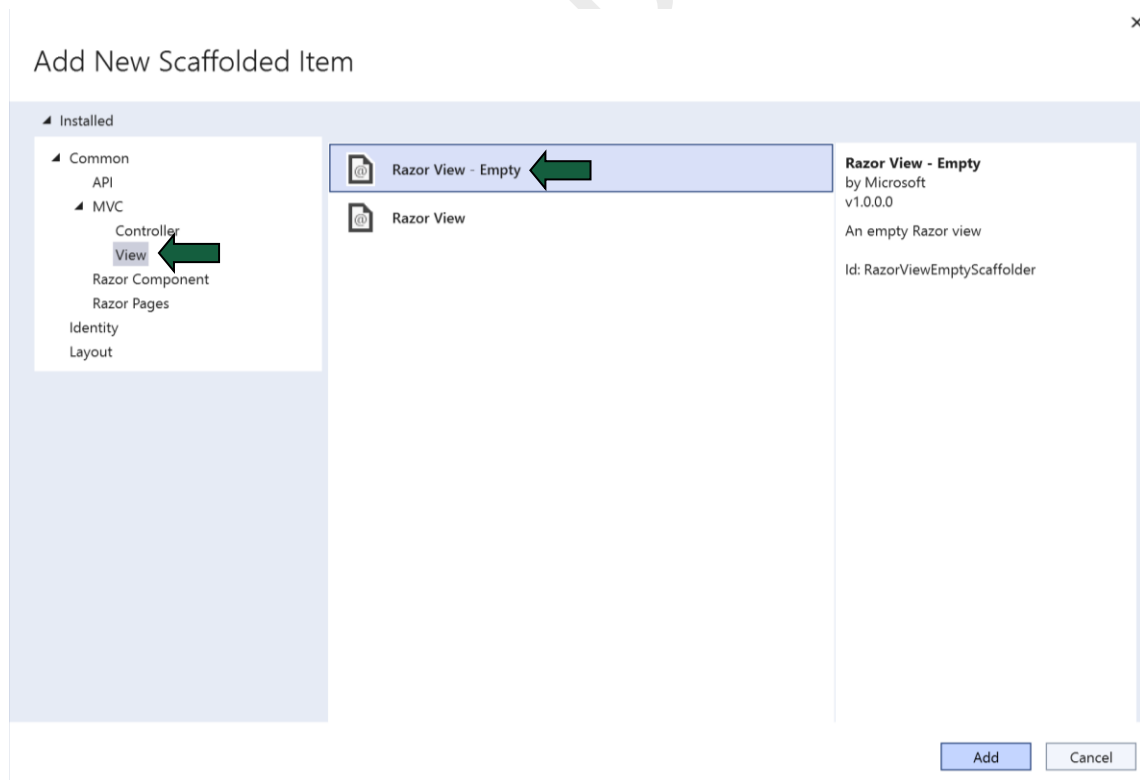
PASUL 1:

Click dreapta pe folderul corespunzator din folderul View → **Add** → **View**.

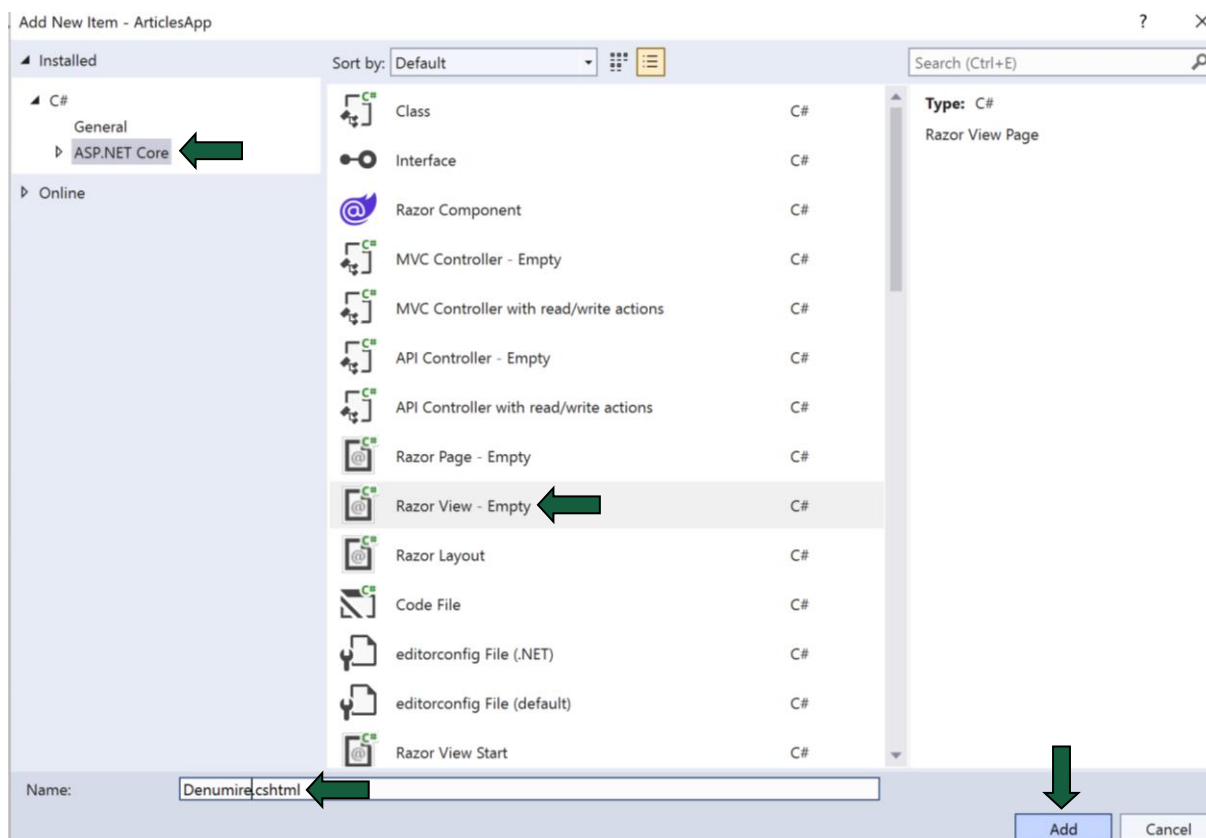
De exemplu: pentru prelucrarea articolelor din baza de date (**VEZI Laborator 6**) o sa avem Controller-ul ArticlesController, iar in folderul Views trebuie creat folderul Articles asociat. In acesta o sa existe View-urile asociate cu metodele din Controller.



PASUL 2:



PASUL 3:



Trimiterea datelor catre View

De foarte multe ori este necesar sa trimitem diferite date catre View pentru afisarea acestora in browser. Pe langa informatiile primite din baza de date (prin intermediul modelului) exista cazuri in care vom trimite si alte tipuri de date.

Model

Trimiterea datelor din baza de date se face prin intermediul helper-ului **@model**. Astfel, pentru a afisa informatiile stocate in proprietatile unui model, se utilizeaza urmatoarea secventa de cod:

```
@model Curs7.Models.Student
```

Includerea modelului necesar pentru afisare

```
@{
    ViewBag.Title = "Afisare student";
}
```

```
<h1>@Model.Name</h1>
<p>@Model.Email</p>
<p>@Model.CNP</p>
```

Se utilizeaza @Model pentru preluarea si afisarea datelor

Pentru a putea trimite Modelul catre View si pentru a putea fi folosit este nevoie de urmatoarea secventa de cod in Controller:

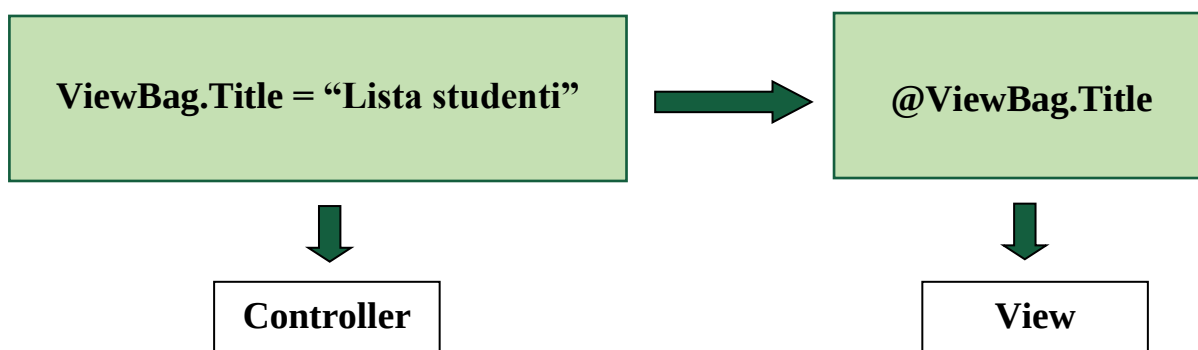
```
public IActionResult Show(int id)
{
    Student student = db.Students.Find(id);
    return View(student);
}
```

Aceasta secventa de cod paseaza obiectul de tip **Model** (in exemplul acesta, un obiect de tipul clasei **Student**) catre View.

ViewBag

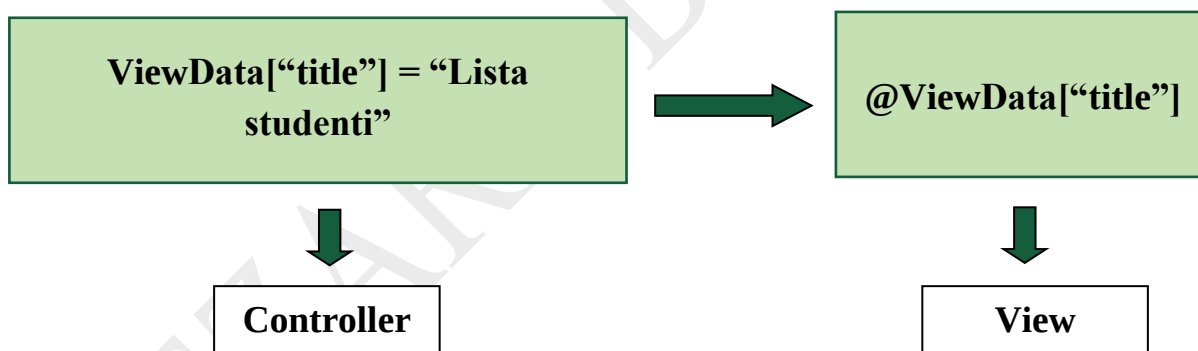
Helperul ViewBag – ofera posibilitatea transferului de date intre Controller si View. Aceste date sunt cele care nu se regasesc in Model.

Array-urile care contin obiecte de tipul unui Model, se pot trimite catre View tot prin ViewBag.



ViewData

Helperul ViewData – este similar cu ViewBag, singura diferenta dintre acestea fiind ca **ViewData** este reprezentat de un Dictionar si nu de un obiect, similar cu un array cheie-valoare. Fiecare cheie trebuie sa fie de tip string.



! OBSERVATIE

ViewBag insereaza datele alocate proprietatilor in dictionarul asociat variabilei **ViewData**. Acest lucru necesita ca numele cheilor (pentru ViewData) respectiv numele proprietatilor (pentru ViewBag) sa fie **diferite**.

Exemplu:

```
ViewBag.Title = "Titlu";
ViewData["Title"] = "Titlu 2";
```

Acest lucru conduce la suprascrierea valorii atributului "Title" din ViewBag (adica Titlu), cu ultima valoare alocata in cheia din ViewData (si anume "Titlu 2")

Astfel, in View, atat **@ViewBag.Title**, cat si **@ViewData["Title"]** vor afisa "Titlu 2".

TempData

Helperul TempData – poate seta o valoare care va fi disponibila intr-un request subsecvent. Astfel, daca valoarea a fost setata in Actiunea1, iar aceasta actiune va face un redirect catre Actiunea2, valoarea setata in TempData va fi disponibila in Actiunea2 (**ATENTIE! Doar la prima accesare**).

Exemplu: Sa presupunem ca avem C.R.U.D. pentru obiectul Student. Controller-ul are metoda Delete care va sterge studentul din baza de date, prin verbul HTTP Post, neafisand in acest caz un View. Aceasta metoda executa codul aferent stingerii din baza de date si redirectioneaza catre metoda Index.

Pentru stergerea unui student, se vor executa 2 request-uri HTTP dupa cum urmeaza:

- **View-ul Show** (care afiseaza datele unui student si butonul de stergere) – aceasta este pagina unde are loc request-ul, prin apasarea butonului de stergere. La apasarea butonului se va face un Request catre metoda Delete din Controller.
- **[Request 1]:** Se va accesa metoda Delete si se va executa codul aferent stingerii din baza de date. Dupa stergere, metoda redirectioneaza catre Index.

- **[Request 2]:** Browser-ul va ajunge in metoda Index si va prelua lista studentilor, impreuna cu mesajul primit din cadrul primului request, prin intermediul variabilei TempData. Aceste valori sunt apoi trimise in View-ul Index pentru afisarea catre utilizatorul final.

Pentru o mai buna experienta de utilizare a aplicatiei (**UX – User Experience**) este necesara afisarea catre utilizatorul final a unui mesaj. Mesajul are ca scop instiintarea utilizatorului cu privire la actiunea pe care tocmai a executat-o (resursa a fost stearsa cu succes). Acest lucru trebuie sa se intample in pagina Index si se poate realiza prin intermediul helper-ului **TempData** (deoarece avem doua request-uri subsecvente).

Implementare:

View-ul Show – din acest View o sa se execute request-ul (stergerea unui student din baza de date, in functie de id-ul acestuia)

```
@model Curs7.Models.Student
```

```
@{
    ViewBag.Title = "Afisare student";
}
```

```
<h1>@Model.Name</h1>
<p>@Model.Email</p>
<p>@Model.CNP</p>
```



```
<form method="post" action="/Students/Delete/@Model.Id">

    <button class="btn btn-danger" type="submit">Stergere
student</button>

</form>
```

In StudentsController → metoda Delete – in aceasta metoda se executa codul aferent **request-ului 1**, si anume se va sterge un student din baza de date. Dupa stergere, metoda redirectioneaza catre Index. In acest request variabila TempData["message"] o sa stocheze stringul in cheia message, dupa care o sa trimita valoarea catre request-ul 2 → Index.

```
[HttpPost]
public ActionResult Delete(int id)
{
    Student student = db.Students.Find(id);
    db.Students.Remove(student);
    db.SaveChanges();

    TempData["message"] = "Studentul cu numele " +
student.Name + " a fost sters din baza de date";

    return RedirectToAction("Index");
}
```

In StudentsController → metoda Index – browser-ul va ajunge in metoda Index si va prelua lista studentilor, impreuna cu mesajul primit din cadrul primului request, prin intermediul variabilei TempData. Aceste valori sunt apoi trimise in View-ul Index pentru afisarea catre utilizatorul final.

```
public ActionResult Index()
{
    var students = from student in db.Students
select student;

    ViewBag.Students = students;

    if (TempData.ContainsKey("message"))
    {
        ViewBag.Msg = TempData["message"].ToString();
    }
    return View();
}
```

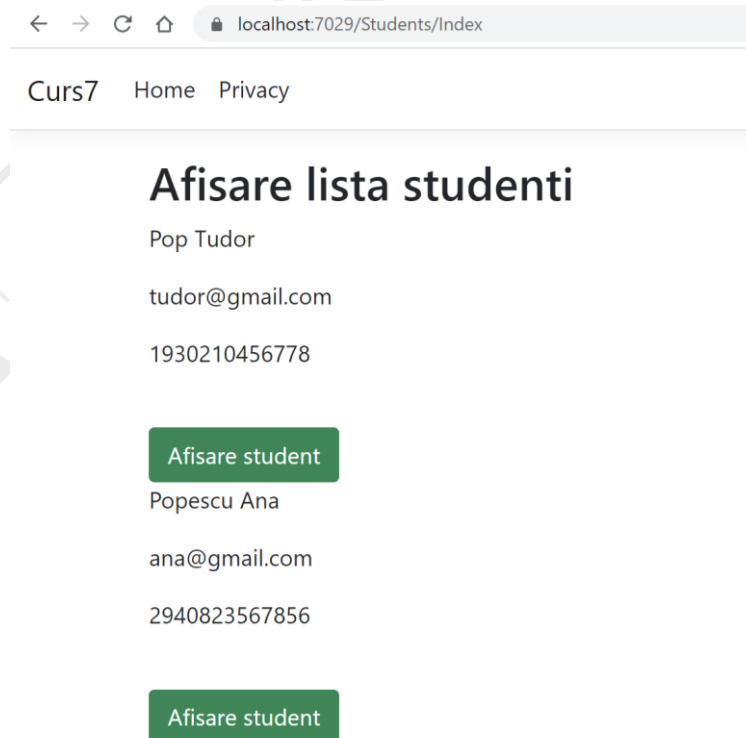
In View-ul Index – sunt afisati toti studentii din baza de date, dar si mesajul provenit din TempData.

```
<h2>Afisare lista studenti</h2>
<br />

@if(ViewBag.Msg != null){
    <h2 class="alert-success p-3 rounded-3 text-center mb-5">@ViewBag.Message</h2>
}

@foreach (var student in ViewBag.Students)
{
    <p>@student.Name</p>
    <p>@student.Email</p>
    <p>@student.CNP</p>
    <br />
    <a class="btn btn-success"
href="/Students/Show/@student.Id">Afisare student</a>
    <br />
}
```

/Students/Index



/Students/Show/id

← → ↻ 🏠 localhost:7029/Students/Show/4

Curs7 Home Privacy

Pop Tudor
tudor@gmail.com
1930210456778

Stergere student

/Students/Index

← → ↻ 🏠 localhost:7029/Students

Curs7 Home Privacy

Afisare lista studenti
Studentul cu numele Pop Tudor a fost sters din baza de date
Popescu Ana
ana@gmail.com
2940823567856

Afisare student

Helpere pentru View

ASP.NET MVC Core, prin intermediul motorului Razor, ofera o lista de **Helpere** care pot genera elemente de tip HTML. Aceste Helpere sunt folosite in combinatie cu un Model pentru a usura munca dezvoltatorului in generarea formularelor de procesare a datelor acestuia.

Aceste **Helpere utilizeaza sistemul de binding** pentru a afisa valorile modelului in elementele de tip HTML (**de exemplu:** in cazul editarii datelor unui model) cat si pentru trimiterea acestora catre Controller. Toate Helperele se acceseaza prin intermediul obiectului **@Html** disponibil in View.

Printre cele mai importante **Helpere** se enumara:

- **Html.ActionLink** – genereaza un URL
- **Html.TextBox** – genereaza un element de tipul TextBox
- **Html.TextArea** – genereaza un element de tipul TextArea
- **Html.CheckBox** – genereaza un element de tipul Check-box, util pentru valorile de tip Boolean
- **Html.RadioButton** – genereaza un element de tipul Radio button
- **Html.DropDownList** – genereaza un element de tipul Dropdown, util pentru valorile de tip Enum
- **Html.ListBox** – genereaza un element de tipul Dropdown cu selectie multipla
- **Html.Hidden** – genereaza un input field ascuns
- **Html.Password** – genereaza un camp pentru introducerea parolelor (textul introdus in camp este ascuns)
- **Html.Display** – este util pentru afisarea textelor
- **Html.Label** – genereaza un label pentru un element mentionat
- **Html.Editor** – acest helper genereaza unul din elementele de mai sus in functie de tipul proprietatii modelului. Astfel, daca editorul este alocat unui camp de tip **int** va genera un input de tip numeric; daca editorul este alocat unui camp de tip string va genera un textbox, etc.

Folosirea acestor Helpere este similara cu scrierea manuala a codului HTML aferent formularelor, inasa, Helperele **ofera si posibilitatea de binding a datelor in mod automat.**

Exemplu: sa consideram formularul urmator pentru **adaugarea unui student in baza de date.**

```
<form method="post" action="/Students/New">
  <label>Nume</label>
  <br />
  <input type="text" name="Name" />
  <br /><br />
  <label>Adresa e-mail</label>
  <br />
  <input type="text" name="Email" />
  <br /><br />
  <label>CNP</label>
  <br />
  <input type="text" name="CNP" />
  <br />
  <button type="submit">Adauga student</button>
</form>
```

Acesta se va rescrie, folosind **Helperle Html** dupa cum urmeaza:

```
<form method="post" action="/Students/New">
  @Html.Label("Name", "Nume Student")
  <br />
  @Html.TextBox("Name", null, new { @class = "form-control" })
  <br /><br />
  @Html.Label("Email", "Adresa de e-mail")
  <br />
  @Html.TextBox("Email", null, new { @class = "form-control"
  })
  <br /><br />
  @Html.Label("CNP", "CNP Student")
  <br />
  @Html.TextBox("CNP", null, new { @class = "form-control" })
  <br />
  <button type="submit">Adauga student</button>
</form>
```

Generarea unui label pentru atributul "Name"

Generarea unui input field pentru proprietatea Email

Folosind aceste Helpere, codul HTML generat de View este urmatorul:

```
<form method="post" action="/Students/New">
  <label for="Name">Nume Student</label>
  <br />
  <input class="form-control" id="Name" name="Name" type="text"
value="" />
  <br /><br />
  <label for="Email">Adresa de e-mail</label>
  <br />
  <input class="form-control" id="Email" name="Email"
type="text" value="" />
  <br /><br />
  <label for="CNP">CNP Student</label>
  <br />
  <input class="form-control" id="CNP" name="CNP" type="text"
value="" />
  <br />
  <button type="submit">Adauga student</button>
</form>
```

Pentru **label** se genereaza urmatoarele elemente:

```
@Html.Label("Name", "Nume Student")
      ↓      ↓
<label for="Name">Nume Student</label>
```

Pentru **TextBox** se genereaza urmatoarele elemente:

```
@Html.TextBox("Name", null, new { @class = "form-control" })
      ↓      ↓      ↓      ↓      ↓
<input id="Name" name="Name" type="text" value="" class="form-
control" />
```

Continutul HTML generat de Helpere este similar cu cel scris manual. Insa, in cazul **editarii**, unde este necesar sa preluam valorile existente in **Model** pentru modificarea ulterioara a acestora, putem apela la helperul **Html.Editor** pentru a genera in mod automat campurile de editare si pentru a prelua automat aceste valori.

Formularul initial pentru **editarea unui student**, implementat manual cu preluarea manuala a valorilor si alocarea acestora inputurilor HTML prin intermediul atributului **value** are urmatorul continut:

```
<form method="post" action="/Students/Edit/@ViewBag.Student.Id">

    <label>Nume</label>

    <input type="text" name="Name" value="@ViewBag.Student.Name" />

    <label>Adresa e-mail</label>

    <input type="text" name="Email" value="@ViewBag.Student.Email" />

    <label>CNP</label>
    <input type="text" name="CNP" value="@ViewBag.Student.CNP" />

    <button type="submit">Modifica student</button>

</form>
```

Acesta poate fi rescris prin intermediul helperelor astfel:

```
@model Curs7.Models.Student
```

```
<form method="post" action="/Students/Edit/@Model.Id">
```

```
    @Html.Label("Name", "Nume Student")
```

```
    <br />
```

```
    @Html.Editor("Name")
```

```
    <br /><br />
```

```
    @Html.Label("Email", "Adresa de e-mail")
```

```
    <br />
```

```
    @Html.Editor("Email")
```

```
    <br /><br />
```

Acest helper genereaza in mod automat campul necesar

```
    @Html.Label("CNP", "CNP Student")
```

```
    <br />
```

```
    @Html.Editor("CNP")
```

```
    <br /><br />
```

```
    <button type="submit">Modifica student</button>
```

```
</form>
```

Edit

Nume Student

Adresa de e-mail

CNP Student

Codul generat este urmatorul:

```
<form method="post" action="/Students/Edit/1">

    <label for="Name">Nume Student</label>

    <br />

    <input class="text-box single-line" id="Name" name="Name"
type="text" value="Student 2" />

    <br /><br />

    <label for="Email">Adresa de e-mail</label>

    <br />

    <input class="text-box single-line" data-val="true" data-
val-required="The Email field is required." id="Email"
name="Email" type="text" value="user@test.com" />

    <br /><br />

    <label for="CNP">CNP Student</label>

    <br />

    <input class="text-box single-line" data-val="true" data-
val-maxlength="The field CNP must be a string or array type with
a maximum length of 13;" data-val-maxlength-max="13"
data-val-minlength="The field CNP must be a string or array type
with a minimum length of 13;" data-val-minlength-
min="13" id="CNP" name="CNP" type="text" value="1121123123123"
/>

    <br /><br />

    <button type="submit">Modifica student</button>

</form>
```

Putem observa ca **Helper-ul Html.Editor** a generat toate campurile necesare pentru formularul de editare a studentului, conform definitiei modelului Student. De asemenea, in formular se observa ca toate attributele **value** asociate elementelor formularului au primit valorile modelului in mod automat (**prin Binding**).

Implementarea alternativa a Helperelor

Html.TextBox – genereaza un element de tipul TextBox → Devine input in noua versiune de utilizare a helperelor. In continuare sunt prezentate ambele variante:

➤ Varianta 1 → Helper @Html

```
@Html.TextBoxFor(m => m.Title, null, new { @class = "form-control" })
```

➤ Varianta 2 → Helper in-line

```
<input asp-for="Title" class="form-control" />
```

Html.DropDownList – genereaza un element de tipul Dropdown
→ Devine select

➤ Varianta 1 → Helper @Html

```
@Html.DropDownListFor(m => m.CategoryId, new  
SelectList(Model.Categories, "Value", "Text"), "Selectati  
categoria", new { @class = "form-control" })
```

➤ Varianta 2 → Helper in-line

```
<select class="form-control" asp-for="CategoryId" asp-
items="Model.Categories">
    <option disabled selected value="">Selectati
Categoria</option>
</select>
```

Html.Label → Devine Label

➤ Varianta 1 → Helper @Html

```
@Html.LabelFor(m => m.Title, "Titlu Articol")
```

➤ Varianta 2 → Helper in-line

```
<label asp-for="Title"></label>
```

Fiecare Helper are un atribut numit **asp-for** folosit pentru a indica proprietatea pe care modelul o utilizeaza pentru procesul de model binding.

Helperele pot avea si alte atribute care incep cu **asp-** si care sunt folosite pentru diverse procese, cum ar fi:

- popularea unui select cu o lista de optiuni (**asp-items**)
- setarea unei rute pentru un form **asp-controller** si **asp-action**
- setarea unui url pentru un un tag <a>: **asp-controller** si **asp-action**

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 8

Cuprins

View (interfata cu utilizatorul).....	2
Validarea cu ajutorul atributelor	2
Atribute de validare la nivel de Model	2
Error Message	3
Exemple de implementare la nivel de Model	3
Preluarea validarilor in View	9
Exemplu de implementare la nivel de View	10
Helper-ul @Html.BeginForm	11
@Html.ValidationMessage si @Html.ValidationMessageFor	12
Helper-ul @Html.ValidationSummary	12
Vizualizarea mesajelor de validare in browser	13
Implementarea validarilor la nivel de Controller.....	16
View-uri partajate	16
Layout View.....	17
Adaugarea unui nou Layout.....	19
Partial View.....	26

View (interfata cu utilizatorul)

Validarea cu ajutorul atributelor

In ASP.NET Core MVC validarea se poate realiza prin intermediul adaugarii atributelor necesare in Model. Atributele de validare ofera posibilitatea integrarii regulilor de validare pentru fiecare atribut/proprietate a modelului.

Atribute de validare la nivel de Model

Cele mai utilizate atribute de validare sunt urmatoarele:

- **Required** – verifica daca inputul este diferit de null;
- **StringLength** – verifica daca lungimea unui string este mai mica sau egala decat limita impusa;
- **Range** – verifica daca valoarea inputului se afla intr-un anumit interval;
- **RegularExpression** – verifica daca valoarea respecta expresia regulata;
- **CreditCard** – verifica daca valoarea are un format specific unui cod bancar;
- **CustomValidation** – reprezinta o validare custom (creata de dezvoltator pentru a valida un anumit atribut);
- **EmailAddress** – verifica daca valoarea inputului are un format specific unei adrese de e-mail;
- **FileExtension** – verifica daca valoarea corespunde unei denumiri de extensie;

- **MaxLength** – specifica valoarea maxima pe care o poate avea un array sau un string;
- **MinLength** – specifica valoarea minima pe care o poate avea un array sau un string;
- **Phone** – verifica daca valoarea reprezinta un numar de telefon valid;
- **DataType** – verifica tipul de date al inputului;

Error Message

Atributele de validare permit utilizarea parametrului **ErrorMessage** pentru afisarea catre utilizatorul final a unui mesaj de eroare specific, in functie de validarea utilizata pentru respectivul atribut.

Exemplu:

```
[Required(ErrorMessage = "Continutul articolului este obligatoriu")]
```

Exemple de implementare la nivel de Model

Exemplul 1 – validari asupra modelului Student

Se adauga asupra modelului Student urmatoarele validari:

- **Numele** este obligatoriu
- **Emailul** este obligatoriu si trebuie sa aiba un format specific unei adrese de e-mail
- **CNP-ul** este obligatoriu si trebuie sa aiba exact 13 caractere
- **Adresa** este obligatorie si nu poate avea mai mult de 30 de caractere

```

public class Student
{
    [Key]
    public int StudentId { get; set; }

    [Required(ErrorMessage = "Numele studentului este obligatoriu")]
    public string Name { get; set; }

    [Required(ErrorMessage = "Campul e-mail este obligatoriu")]
    [EmailAddress(ErrorMessage = "Adresa de e-mail nu este valida")]
    public string Email { get; set; }

    [MinLength(13, ErrorMessage = "Lungimea minima trebuie sa fie de 13 caractere")]
    [MaxLength (13, ErrorMessage = "Lungimea maxima trebuie sa fie de 13 caractere")]
    [Required(ErrorMessage = "CNP-ul este obligatoriu")]
    public string CNP { get; set; }

    [StringLength(50, ErrorMessage = "Adresa nu poate avea mai mult de 50 de caractere")]
    [Required(ErrorMessage = "Adresa este obligatorie")]
    public string Address { get; set; }
}

```

Exemplul 2 – validari asupra modelului Article (exemplul din cadrul laboratorului)

Se adauga asupra modelului Article urmatoarele validari:

- **Titlul** articolului este obligatoriu, poate avea o lungime maxima de 100 de caractere si nu poate avea mai putin de 5 caractere
- **Continutul** articolului este obligatoriu
- **Categoria** din care face parte articolul este obligatorie

```

public class Article
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Titlul este obligatoriu")]
    [StringLength(100, ErrorMessage = "Titlul nu poate avea
mai mult de 100 de caractere")]
    [MinLength(5, ErrorMessage = "Titlul trebuie sa aiba mai
mult de 5 caractere")]
    public string Title { get; set; }

    [Required(ErrorMessage = "Continutul articolului este
obligatoriu")]
    public string Content { get; set; }

    public DateTime Date { get; set; }

    [Required(ErrorMessage = "Categoria este obligatorie")]
    public int CategoryId { get; set; }

    public virtual Category Category { get; set; }

    public virtual ICollection<Comment> Comments {get; set;}

    [NotMapped]
    public IEnumerable<SelectListItem> Categ { get; set; }
}

```

!/OBSERVATII

Mesajele de eroare aflate in Model, ca valoare a parametrului ErrorMessage, o sa fie preluate in View pentru afisare.

Pentru preluarea mesajelor de validare in View **VEZI Sectiunea urmatoare din curs – Preluarea Validarilor in View**. Dupa adaugarea validarilor in Model, urmeaza preluarea mesajelor de validare in View.

În cazul în care în formularul de adăugare a unui nou articol nu se completează toate inputurile, o să apară mesajele de eroare asociate, deoarece **Title**, **Content** și **CategoryId** sunt câmpuri obligatorii. În cazul în care titlul are o lungime mai mică de 5 caractere, atunci o să apară mesajul de eroare asociat.

De asemenea, se poate observa următoarea problemă în cazul în care se încearcă adăugarea unui articol fără completarea câmpurilor:

Adăugare articol

Titlu Articol

Titlul este obligatoriu

Continut Articol

Continutul articolului este obligatoriu

Selectați categoria

The value " " is invalid.

Adăuga articol

Pentru **Titlu** și **Continut** mesajele de eroare se afișează corect. În cazul ultimului input, dropdown-ul în care se selectează categoria, mesajul de eroare nu este cel scris în Modelul Article.

Se întâmplă acest lucru deoarece **CategoryId** este cheie externă. Cheia externă poate fi și null, acest lucru ducând la prezenta acelui mesaj de validare, mesaj implementat by default. Chiar dacă acest atribut este obligatoriu, în implementarea lui internă se consideră că poate fi și null. În momentul în care este null, se afișează automat mesajul de validare intern “The value is invalid”.

În acest caz, este nevoie ca acel câmp să fie opțional la nivel de Model, astfel încât în momentul în care inputul nu are o valoare selectată să preia corect mesajul configurat la nivel de Model și să paseze acel mesaj pentru afișare în View.

În același mod trebuie implementate și proprietățile aflate în Modelul Article deoarece proprietatea Category, de exemplu, nu primește o valoare exact în momentul în care pentru un articol se selectează categoria din care face parte. Acest pas are loc după asocierea unui articol cu o categorie. Asadar, și în cazul proprietăților este nevoie ca acestea să fie declarate *optional* la nivel de Model.

Codul devine:

```
...
[Required(ErrorMessage = "Categoria este obligatorie")]
public int? CategoryId { get; set; }

public virtual Category? Category { get; set; }

public virtual ICollection<Comment>? Comments {get; set;}

[NotMapped]
public IEnumerable<SelectListItem>? Categ { get; set; }
...
```

Adaugare articol

Titlu Articol

Titlul este obligatoriu

Continut Articol

Continutul articolului este obligatoriu

Selectati categoria

Categoria este obligatorie

Adauga articol

In cazul in care Titlul este completat, dar are mai putin de 5 caractere, o sa se afiseze mesajul urmator de eroare, conform validarii.

Adaugare articol

Titlu Articol

Titlul trebuie sa aiba mai mult de 5 caractere



Continut Articol

Continutul articolului este obligatoriu

Selectati categoria

Categoria este obligatorie

Adauga articol

Preluarea validarilor in View

Pentru preluarea validarilor si afisarea mesajelor asociate in View, se utilizeaza Helper-ul `@Html.ValidationMessageFor` astfel:

- Primul parametru este o lambda expresie care **selecteaza atributul modelului** pentru care se va afisa mesajul de validare
- Al doilea parametru este un string si reprezinta mesajul de validare afisat pe ecran

In cazul in care acesta este gol sau null se va afisa **mesajul de validare aflat in Model** ca parametru al variabilei ErrorMessage.

In cazul in care acest parametru are o valoare, atunci o sa fie suprascris mesajul din Model cu cel primit ca parametru in cadrul Helper-ului.

In cazul in care in Model nu exista un mesaj configurat pentru o anumita validare, iar in View parametrul este null, atunci se va afisa un mesaj default, generat de framework.

- Al treilea parametru este optional si reprezinta o lista de attribute care poate fi adaugata mesajului afisat

Exemplu de implementare la nivel de View

```
@using (Html.BeginForm(actionName: "New", controllerName:
"Articles"))
{
    @Html.Label("Title", "Titlu Articol")
    <br />
    @Html.TextBox("Title", null, new { @class = "form-control"
})

    @Html.ValidationMessageFor(m => m.Title, null, new {
@class = "text-danger" })
    <br /><br />

    @Html.Label("Content", "Continut Articol")
    <br />
    @Html.TextArea("Content", null, new { @class = "form-
control" })

    @Html.ValidationMessage("Content", null, new { @class =
"text-danger" })
    <br /><br />

    <label>Selectati categoria</label>
    @Html.DropDownListFor(m => m.CategoryId, new
SelectList(Model.Categ, "Value", "Text"),
"Selectati categoria", new { @class = "form-control" })

    @Html.ValidationMessageFor(m => m.CategoryId, null, new {
@class = "text-danger" })
    <br /><br />

    <button class="btn btn-success" type="submit">Adauga
articol</button>
}
```

Helper-ul @Html.BeginForm

Se poate observa utilizarea unui nou Helper → `Html.BeginForm`

Acest Helper inlocuieste tagul **<form>** utilizat pana in acest moment in cazul formularelor din View.

Implementarea unui formular utilizand Helper-ul:

```
@using (Html.BeginForm (actionName: "NumeMetoda",
    controllerName: "NumeController",
    method: FormMethod.Post,
    routeValues: new { id = Model.NumeId })))
```

In cazul in care se utilizeaza Helper-ul si este nevoie de trimiterea unui id catre Controller (de exemplu in cazul editarii) exista urmatoarele variante de implementare:

1. Trimiterea id-ului prin intermediul Helper-ului `Html.BeginForm` cu ajutorul parametrului `routeValues`, unde Model este Modelul primit ca parametru din Controller si inclus in View prin intermediul Helper-ului `@model` → `@model NumeProiect.Models.NumeClasa` (EX din laborator: `@model ArticlesApp.Models.Article`)

```
@using (Html.BeginForm(actionName: "Edit", controllerName:
    "Articles", routeValues: new { id = Model.Id })))
```

2. Trimiterea id-ului in interiorul formularului prin intermediul Helper-ului `@Html.HiddenFor`

`@Html.HiddenFor(m => m.Id)` -> unde m este un parametru de tipul Modelului primit ca parametru din Controller si inclus in View la fel ca in exemplul anterior

@Html.ValidationMessage si @Html.ValidationMessageFor

!/OBSERVATIE

Implementarile urmatoare sunt echivalente, acest lucru fiind valabil in cazul tuturor Helperelor:

```
@Html.ValidationMessageFor(m => m.Title, null, new { @class = "text-danger" })
```

si

```
@Html.ValidationMessage("Title", null, new { @class = "text-danger" })
```

Helper-ul @Html.ValidationSummary

Helperul @Html.ValidationSummary ofera posibilitatea afisarii unui **sumar cu toate erorile aparute in timpul validarii.**

Acesta se adauga in formularul din cadrul View-ului:

```
@Html.ValidationSummary(false, "", new { @class = "text-danger" })
```

Pentru afisarea corecta a mesajelor de eroare, doar in momentul in care validarea datelor nu este corecta, este necesar sa adaugam urmatoarele linii de cod in fisierul **site.css** aflat in wwwroot → folderul css:

```
.field-validation-valid {
    display: none;
}
.validation-summary-valid {
    display: none;
}
```

Este necesar sa procedam asa doar in momentul in care mesajele de validare se afla doar in View, in cadrul parametrului doi.

```
@Html.ValidationMessageFor(m => m.Title, "Titlul este obligatoriu", new { @class = "text-danger" })
```

Vizualizarea mesajelor de validare in browser

Se pot observa mesajele de validare preluate din cadrul Modelului Article, atat la nivelul fiecarui input prin intermediul Helper-ului `@Html.ValidationMessageFor`, dar si mesajele preluate prin intermediul Helper-ului `@Html.ValidationSummary`

Adaugare articol

- Titlul este obligatoriu
- Continutul articolului este obligatoriu
- Categoria este obligatorie



Titlu Articol

@Html.ValidationSummary

Titlul este obligatoriu

Continut Articol

Continutul articolului este obligatoriu

Selectati categoria

Selectati categoria

Categoria este obligatorie

Adauga articol

In cazul in care exista un mesaj de validare in View, acesta o sa suprascrie mesajul configurat la nivel de Model, dupa cum urmeaza:

Mesajul de validare din View-ul New:

```

@Html.TextBox("Title", null, new { @class = "form-control"
})
@Html.ValidationMessageFor(m => m.Title, "OBLIGATORIU", new
{ @class = "text-danger" })

```

Mesajul de validare din Modelul Article:

```

[Required(ErrorMessage = "Titlul este obligatoriu")]
public string Title { get; set; }

```

Se poate observa faptul ca mesajul de validare din View il suprascrie pe cel din Model doar la nivelul inputului. In cazul in care se realizeaza sumarul cu toate mesajele de validare, sunt preluate tot mesajele de validare existente la nivel de Model.

De aceea este recomandata utilizarea mesajelor de validare in Model.

In cazul in care sunt utilizate validari, fara scrierea mesajelor specifice, framework-ul o sa afiseze mesaje default.

De exemplu: “The Title field is required.”

“The field Title must be a string or array type with a minimum length of '5'.”

Adaugare articol

- Titlul este obligatoriu
- Continutul articolului este obligatoriu
- Categoria este obligatorie

Titlu Articol

OBLIGATORIU

Continut Articol

Continutul articolului este obligatoriu

Selectati categoria

Categoria este obligatorie

Adauga articol

Implementarea validărilor la nivel de Controller

Pentru functionarea corectă a validărilor, cât și pentru identificarea corectă a datelor în partea de server (server-side) este necesar să adăugăm în Controller-ul care modifică datele, verificarea stării modelului. Astfel, prin intermediul variabilei **ModelState** putem să aflăm dacă toate validările au trecut cu succes.

```
[HttpPost]
public IActionResult New(Article article)
{
    article.Date = DateTime.Now;
    article.Categ = GetAllCategories();

    if (ModelState.IsValid)
    {
        db.Articles.Add(article);
        db.SaveChanges();
        TempData["message"] = "Articolul a fost
adaugat";
        return RedirectToAction("Index");
    }
    else
    {
        return View(article);
    }
}
```

View-uri partajate

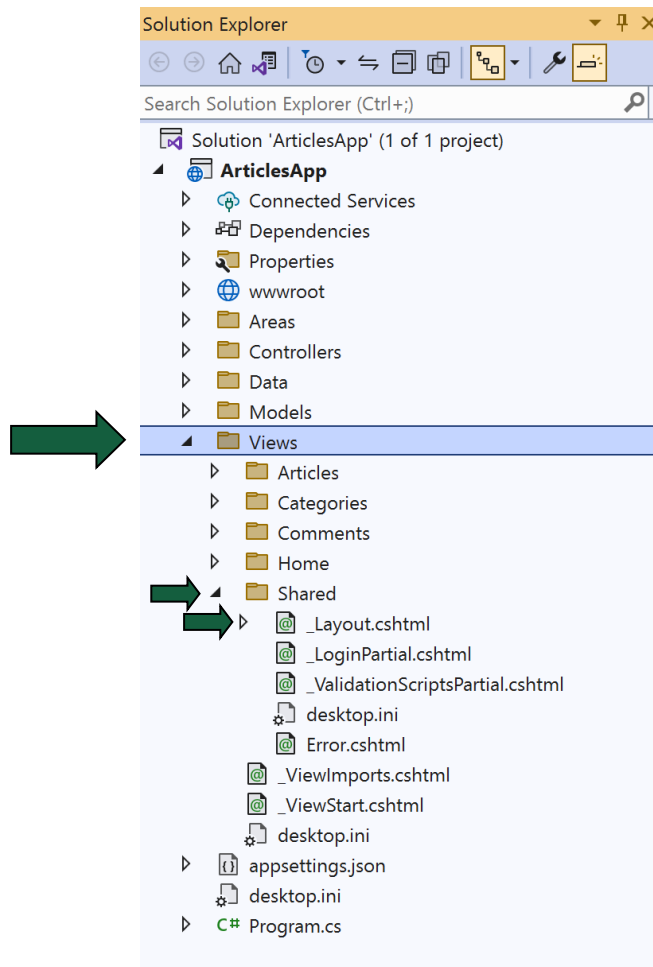
Interfața unei aplicații, indiferent de tehnologia cu care este realizată, întotdeauna o să conțină foarte multe componente comune tuturor paginilor: Header, Footer, Meniuri, etc. Aceste componente nu se modifică de la o pagină la alta, iar repetarea scrierii aceluiași cod devine redundantă. Pentru a facilita implementarea se pot utiliza View-uri globale.

Layout View

Layout View – permite scrierea unui cod comun pentru toate paginile, cat si un Placeholder in care se va include continutul celorlalte pagini. Acest placeholder este definit prin intermediul variabilei **@RenderBody()**. Locul in care este plasata aceasta variabila in Layout, va fi locul in care se va afisa continutul View-urilor aferente.

De exemplu, in momentul in care cream un nou proiect, acesta genereaza in mod automat un layout care include toate resursele necesare: Head, Stiluri CSS, JavaScript, Header, Footer, etc. In acest layout se afla metoda **RenderBody()** prin care toate View-urile create sunt incluse.



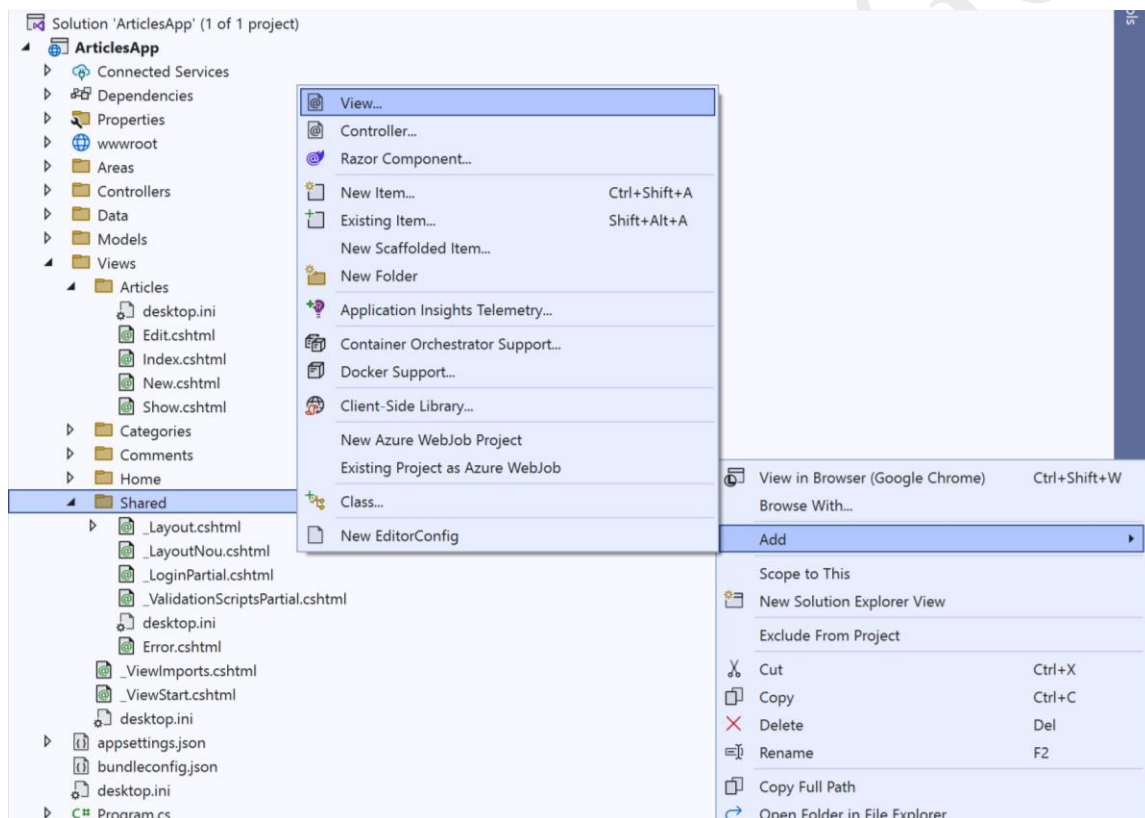


Adaugarea unui nou Layout

Implicit, proiectul ASP.NET Core are in folderul Shared din View un Layout. Acest Layout este utilizat in cadrul tuturor paginilor existente in proiect. In momentul in care se doreste utilizarea unui alt Layout, se poate crea si utiliza dupa cum urmeaza:

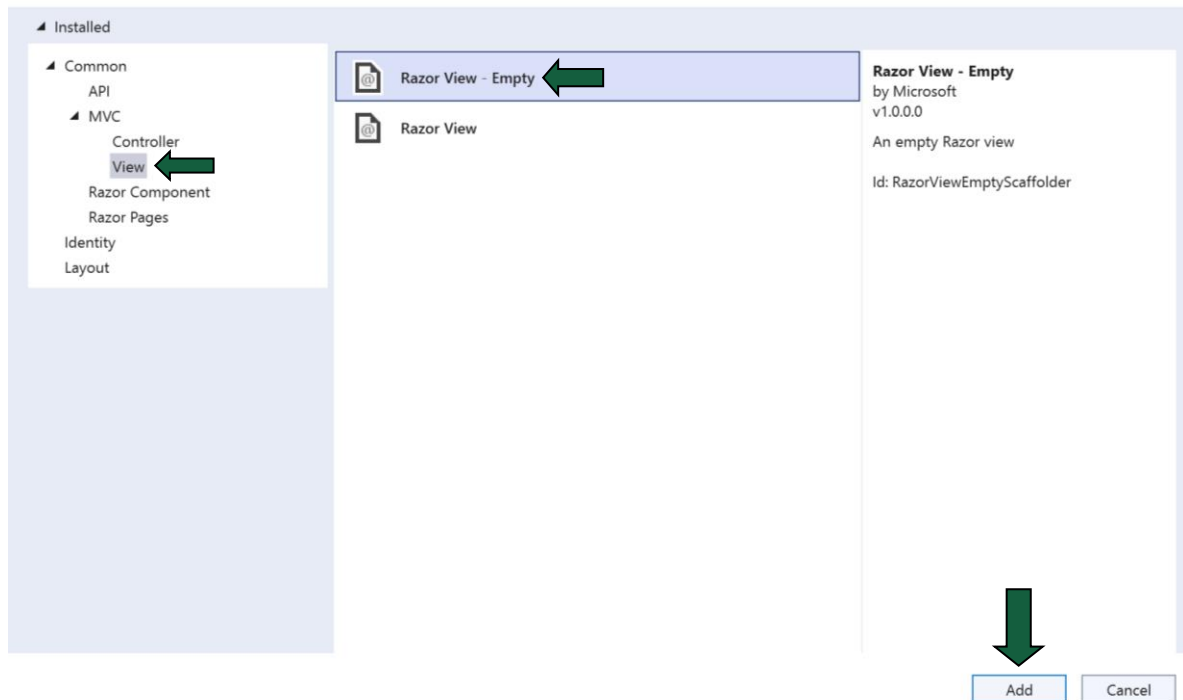
PASUL 1 – crearea noului Layout

Views → Shared → click dreapta → Add → View

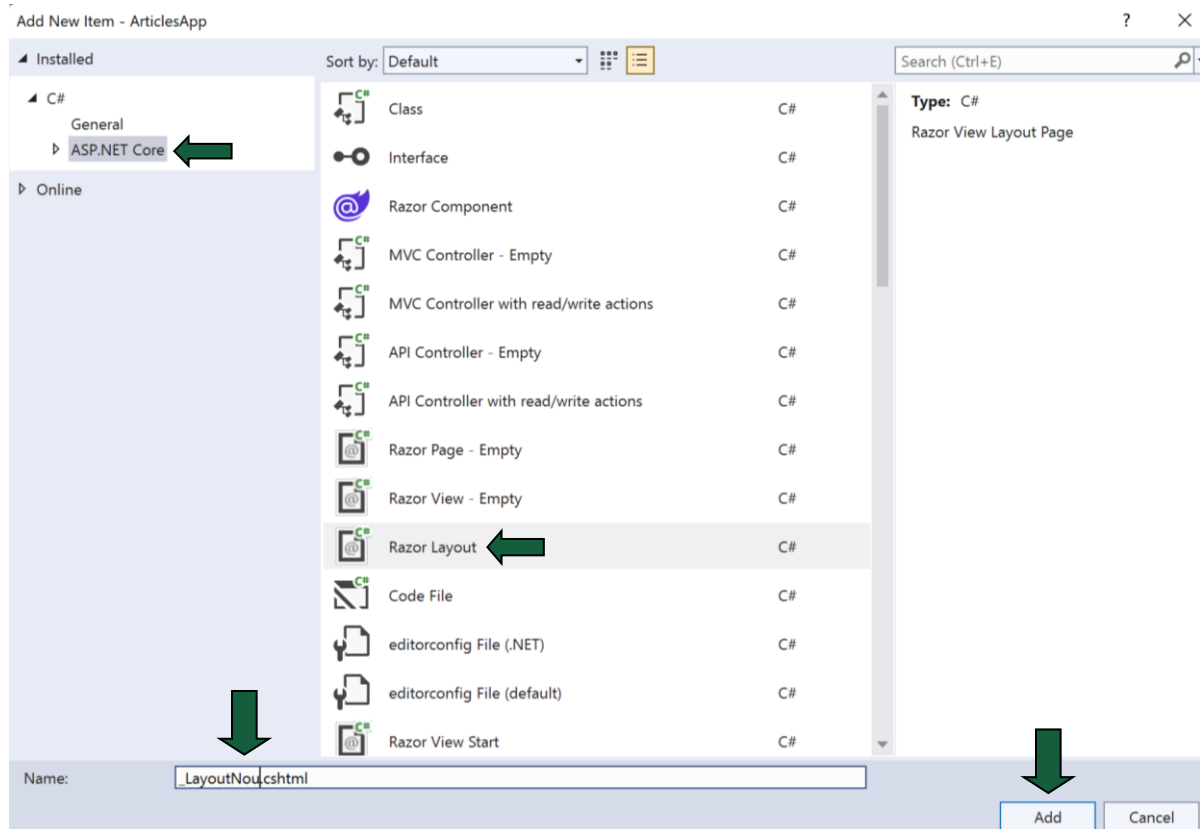


Se selecteaza Razor View – Empty

Add New Scaffolded Item



Se selecteaza ASP.NET Core → Razor Layout
Se redenumeste Layout-ul → Add



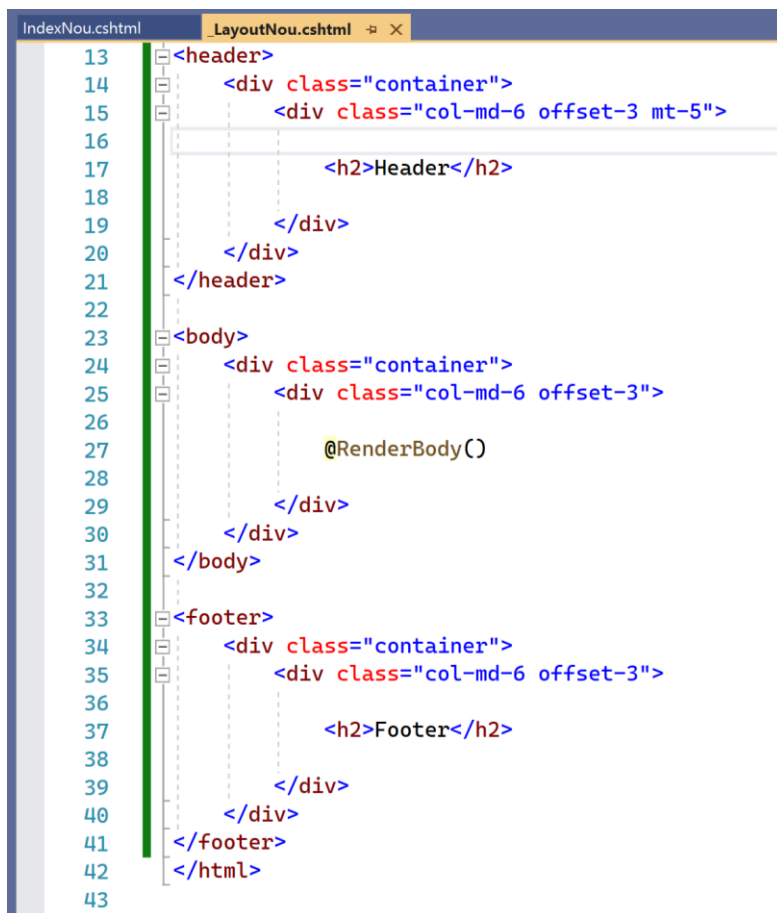
View-ul generat de framework o sa arate astfel:

```

1      <!DOCTYPE html>
2
3      <html>
4      <head>
5          <meta name="viewport" content="width=device-width" />
6          <title>@ViewBag.Title</title>
7      </head>
8      <body>
9          <div>
10             @RenderBody()
11          </div>
12      </body>
13      </html>
14

```

PASUL 2 – modificarea noului Layout, adaugand header si footer

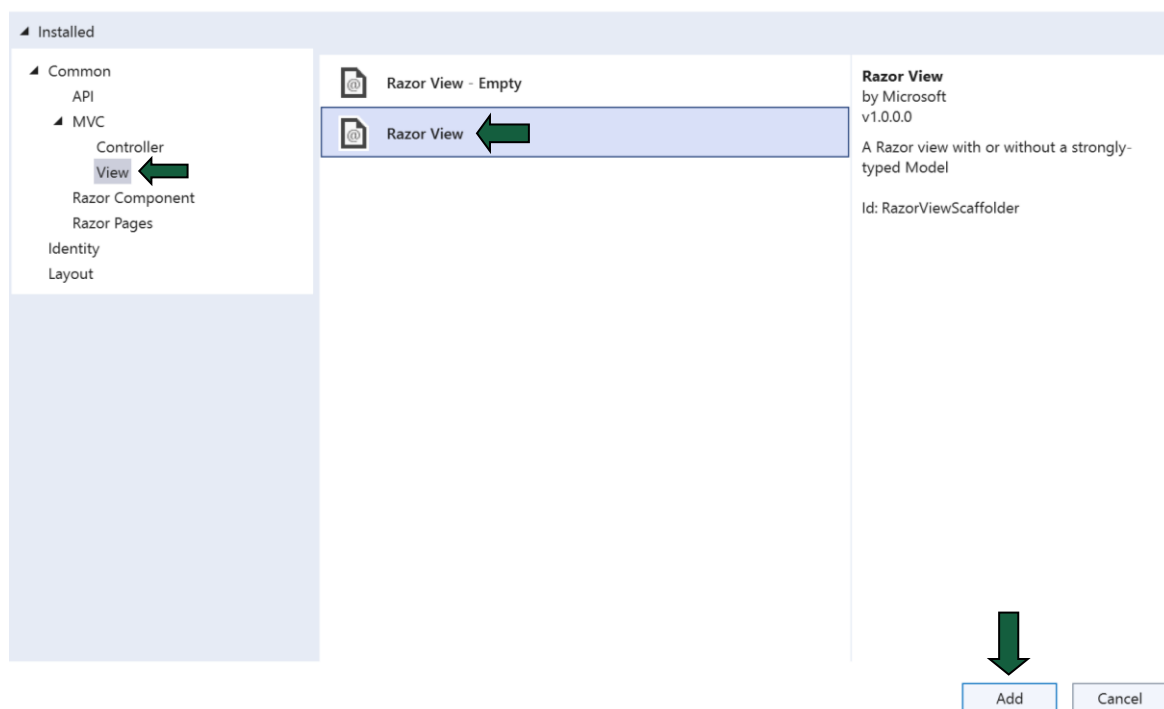


```
13 <header>
14   <div class="container">
15     <div class="col-md-6 offset-3 mt-5">
16       <h2>Header</h2>
17     </div>
18   </div>
19 </header>
20
21 <body>
22   <div class="container">
23     <div class="col-md-6 offset-3">
24       @RenderBody()
25     </div>
26   </div>
27 </body>
28
29 <footer>
30   <div class="container">
31     <div class="col-md-6 offset-3">
32       <h2>Footer</h2>
33     </div>
34   </div>
35 </footer>
36 </html>
```

PASUL 3 – adaugarea unui nou View care o sa aiba ca Layout noul Layout creat:

Se adauga un nou View de tipul Razor View

Add New Scaffolded Item



Se adauga un nume pentru respectivul View, dupa care se alege Layout-ul

Add Razor View

View name: IndexNou

Template: Empty (without model)

Model class:

Data context class:

Options

☐ Create as a partial view

☒ Reference script libraries

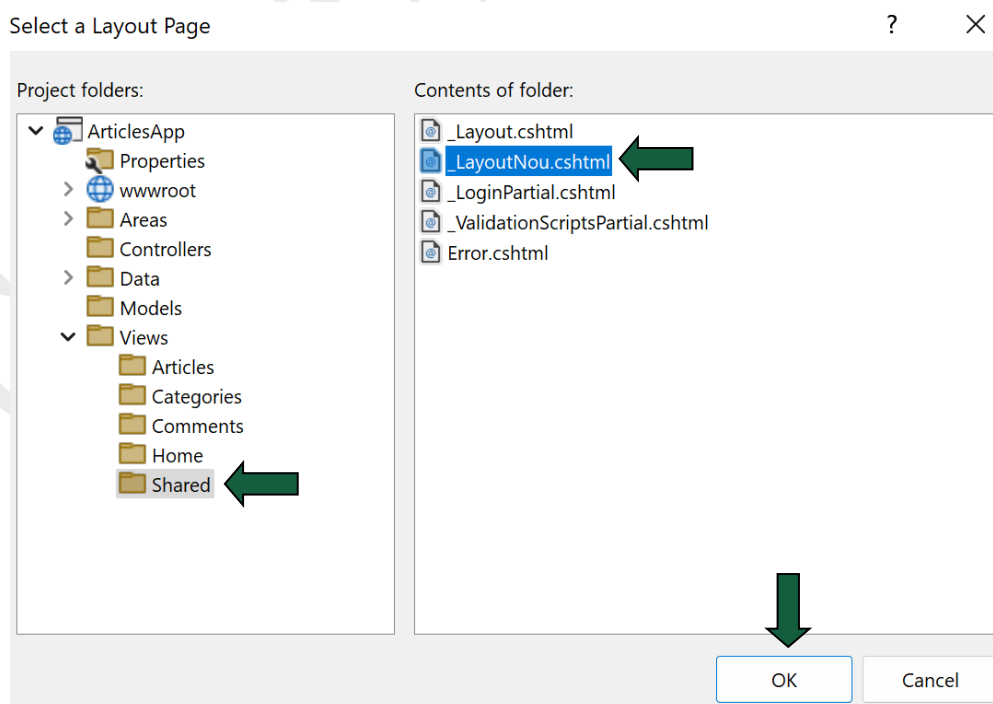
☒ Use a layout page

...

(Leave empty if it is set in a Razor _viewstart file)

Add Cancel

Se selecteaza Layout-ul



In acest pas se observa Layout-ul pe care tocmai l-am selectat

Add Razor View

View name: IndexNou

Template: Empty (without model)

Model class:

Data context class:

Options

- ☐ Create as a partial view
- ☒ Reference script libraries
- ☒ Use a layout page

~/Views/Shared/_LayoutNou.cshtml

(Leave empty if it is set in a Razor _viewstart file)

Add Cancel

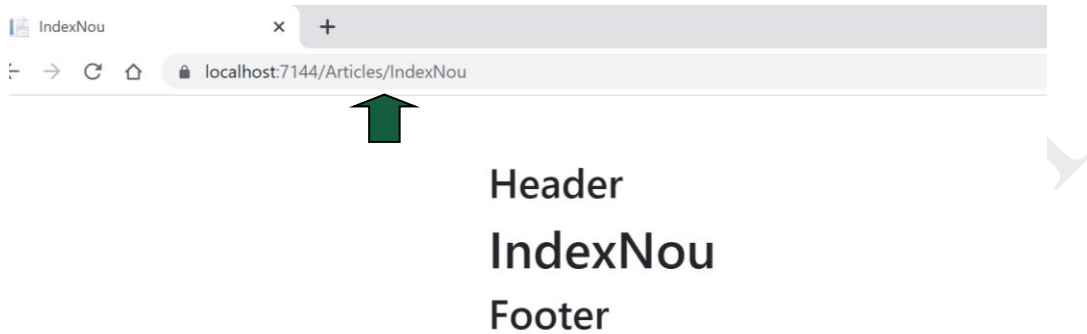
Dupa executarea pasilor anteriori, se poate observa cum in cadrul View-ului numit *IndexNou*, Layout-ul numit *LayoutNou* se include prin intermediul parametrului *Layout* care primeste ca valoare calea din sistemul de fisiere. Mai jos se poate observa secventa de cod generata:

```

IndexNou.cshtml  _Layout.cshtml  _LayoutNou.cshtml
1
2  @{
3      ViewData["Title"] = "IndexNou";
4      Layout = "~/Views/Shared/_LayoutNou.cshtml";
5  }
6
7  <h1>IndexNou</h1>
8
9  |

```

Dupa rulare si accesarea URL-ului → /Articles/IndexNou se poate observa includerea cu succes a noului Layout (noul Layout nu are stilizare, exemplul fiind realizat cu scop demonstrativ).



Partial View

Partialele reprezinta secvente de cod specifice View-urilor care pot fi refolosite in una sau mai multe pagini. In cadrul dezvoltarii aplicatiilor web, codul poate fi reutilizat pentru a optimiza timpul de scriere si pentru a nu include acelasi cod in mod repetitiv.

Secventele de cod care se repeta in cadrul mai multor pagini pot fi incluse intr-un View sau intr-un Layout pentru a fi afisate.

In cadrul aplicatiei dezvoltate in laborator, afisarea unui articol contine acelasi cod, atat in cazul afisarii tuturor articolelor din baza de date → View-ul Index, cat si in cazul afisarii unui singur articol → View-ul Show. In acest caz, pentru a elimina redundanta, si anume scrierea repetitiva a aceluasi cod in cadrul ambelor View-uri, o sa utilizam un **Partial View**, dupa cum urmeaza:

Secventa de cod, cu ajutorul careia se afiseaza informatiile corespunzatoare unui articol, se afla inclusa in ambele View-uri, *Index.cshtml* si *Show.cshtml*.

```
<div class="card-body">

    <h3 class="card-title alert-success py-3 px-3 rounded-2">@Model.Title</h3>

    <div class="card-text">@Model.Content</div>

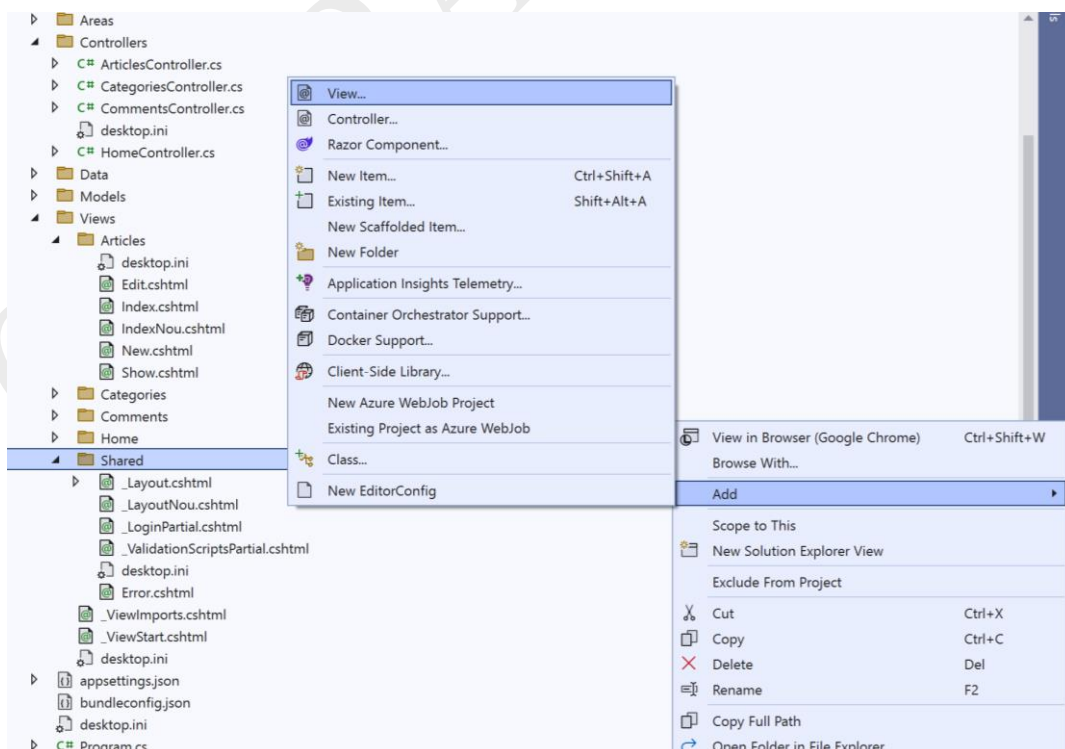
    <div class="d-flex justify-content-between flex-row mt-5">

        <div><i class="bi bi-globe"></i>
@Model.Category.CategoryName</div>

        <span class="alert-success">@Model.Date</span>

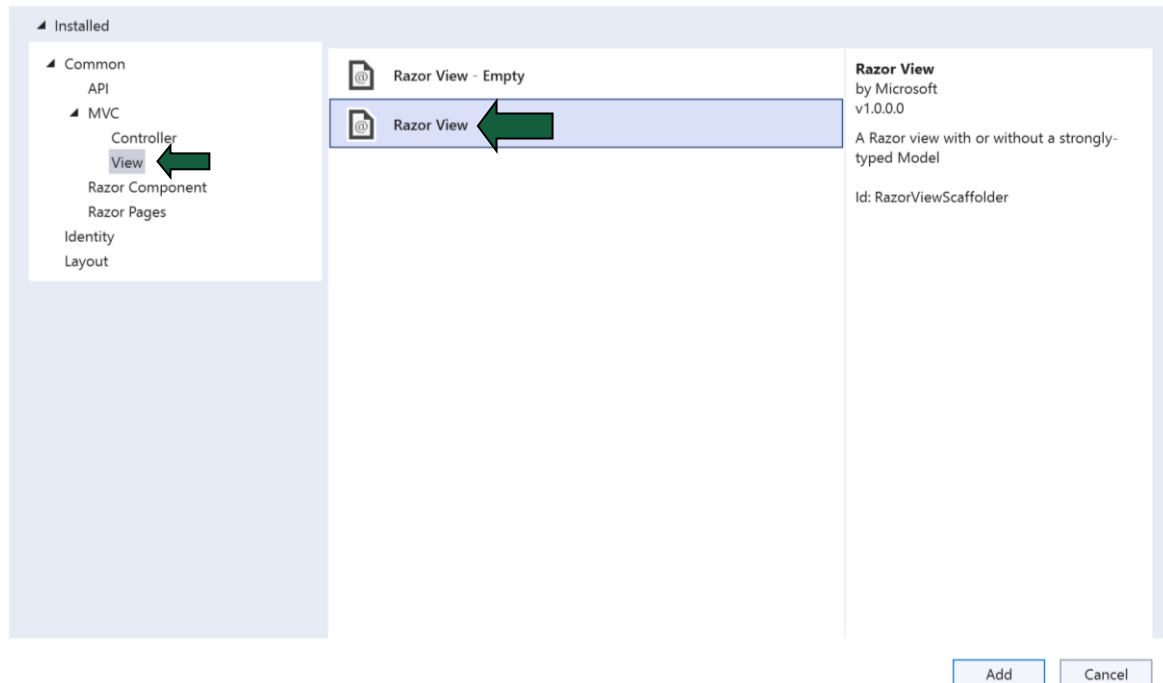
    </div>
</div>
```

Pentru eliminarea redundantei, se adauga un nou View de tip Partial View. Acest View o sa contina secventa de cod care se repeat.



Se selecteaza urmatoarele optiuni:

Add New Scaffolded Item



View-ul o sa fie de tipul Partial View, bifand optiunea
“Create as a partial view”

Add Razor View

View name: ←

Template:

Model class:

Data context class:

Options

☒ Create as a partial view ←

☐ Reference script libraries

☒ Use a layout page

...

(Leave empty if it is set in a Razor _viewstart file)

View-ul numit *“ArticleInfo”* o sa contina secventa de cod care se repeta in ambele View-uri, atat in Index.cshtml, cat si in Show.cshtml.

```

ArticleInfo.cshtml  Index.cshtml  Show.cshtml
1
2
3
4 <div class="card-body">
5
6     <h3 class="card-title alert-success py-3 px-3 rounded-2">@Model.Title</h3>
7
8     <div class="card-text">@Model.Content</div>
9
10    <div class="d-flex justify-content-between flex-row mt-5">
11
12        <div><i class="bi bi-globe"></i> @Model.Category.CategoryName</div>
13
14        <span class="alert-success">@Model.Date</span>
15
16    </div>
17
18 </div>

```

Pentru includerea partialului se utilizeaza Helper-ul specific `@Html.Partial` sau tag-ul `<partial></partial>`

Exemplu utilizand aplicatia dezvoltata in cadrul laboratorului:

Pentru apelarea partialului din **View-ul Show** se utilizeaza pentru primul parametru numele partialului, iar pentru al doilea parametru **Modelul**.

```
@Html.Partial("ArticleInfo", Model)
```

Pentru apelarea partialului din **View-ul Index** se utilizeaza pentru primul parametru numele partialului, iar pentru al doilea parametru obiectul **article** de tipul **Model**. Astfel, in loop trebuie sa declaram tipul modelului si sa pasam acest parametru la partial. Prin intermediul acestui cod, in partial putem sa folosim variabila **@Model** pentru afisarea datelor.

```
@foreach (ArticlesApp.Models.Article article in
 ViewBag.Articles)
{
    ...
    @Html.Partial("ArticleInfo", article)
    ...
}
```

In cazul modificarii partialului, modificarile se reflecta asupra tuturor View-urilor care utilizeaza partialul respectiv, nefiind nevoie de modificari in fiecare View in parte. Acest lucru eficientizeaza atat timpul de scriere a codului, cat si mentenanta ulterioara a acestuia in cazul in care apar modificari in timp.

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 9

Cuprins

Sistemul de autentificare.....	2
Roluri. Asocierea dintre roluri si utilizatori.....	4
Atributul [Authorize]	15
Implementare New, Edit si Delete utilizand roluri.....	15

Sistemul de autentificare

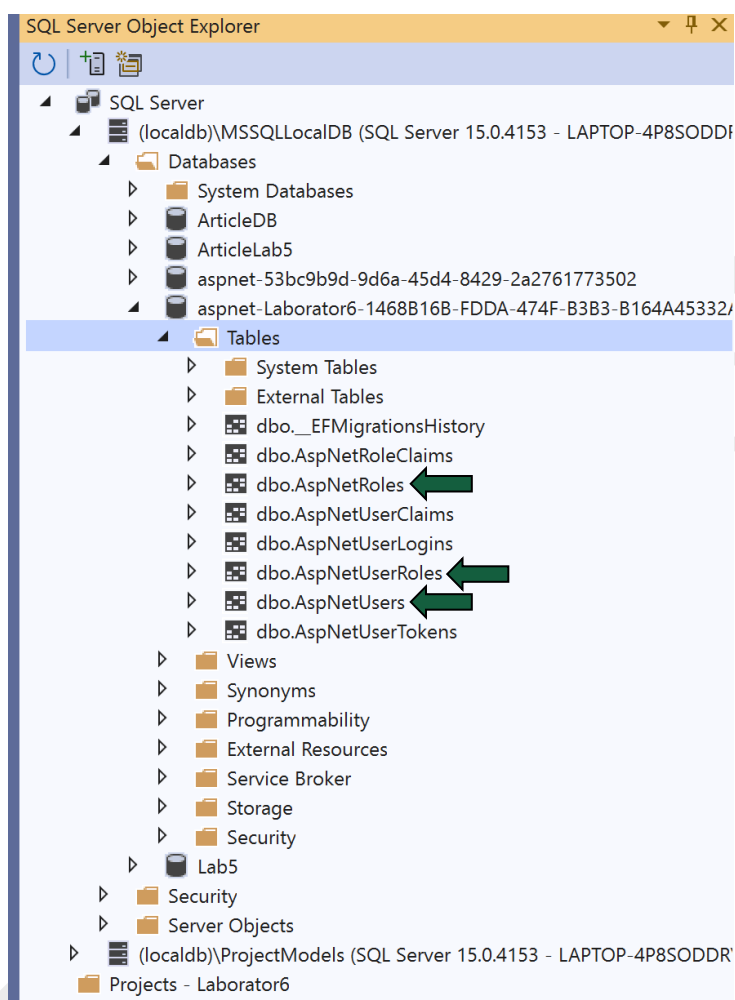
Framework-ul ASP.NET Core ofera posibilitatea integrarii unui sistem de autentificare folosind **Identity**.

Identity este compus dintr-o suita de clase si secvente de cod care faciliteaza implementarea rapida a unui sistem de autentificare complex. Acest sistem ofera posibilitatea autentificarii folosind user si parola, alocarea de roluri pentru utilizatori, autentificare folosind conturi 3rd party (autentificare prin retele de socializare – Google, Facebook, Twitter, etc). De asemenea, Identity include si posibilitatea crearii si manipularii rolurilor pe care le pot avea utilizatorii. ASP.NET Core Identity utilizeaza baza de date SQL Server pentru stocarea utilizatorilor, a rolurilor, dar si pentru asocierea dintre useri si roluri.

Pentru a genera un proiect care include componenta **Identity** pentru autentificare, trebuie sa alegem la crearea proiectului forma de autentificare: **Individual Accounts**.

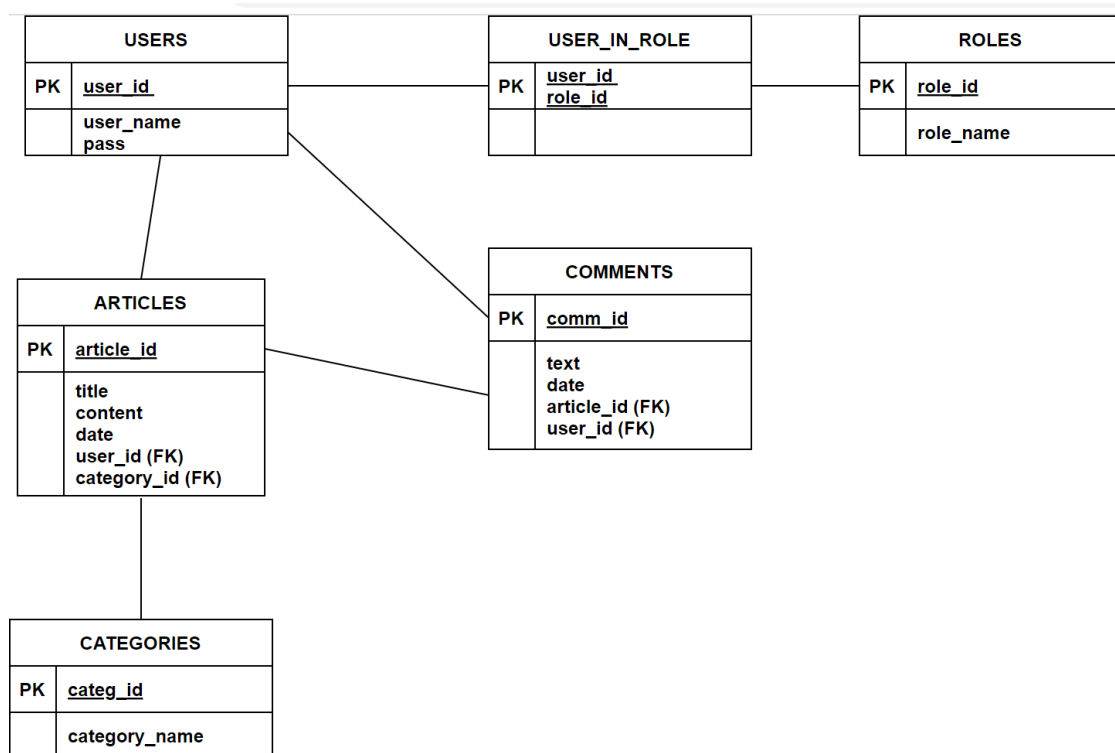
Adaugarea sistemului de autentificare a fost studiata in cadrul **Cursului 6 – Sectiunea Adaugarea sitemului de autentificare**.

Dupa crearea proiectului impreuna cu sistemul de autentificare, se pot vizualiza tabelele:



Roluri. Asocierea dintre roluri si utilizatori

Exemplele urmatoare sunt realizate in cadrul aplicatiei implementata in laborator (Engine de stiri), conform urmatoarei diagrame:



Se considera entitatile **Article**, **Category** si **Comment** cu urmatoarele proprietati:

Article:

- **Id** – int → id-ul articolului (cheie primara)
- **Title** – string → titlul articolului este obligatoriu (Required), poate avea o lungime maxima de 100 de caractere (StringLength) si nu poate avea mai putin de 5 caractere (MinLength)
- **Content** – string → continutul articolului este obligatoriu (Required)
- **Date** – DateTime → data si ora la care este postat articolul
- **CategoryId** – int → cheie externa – categoria din care face parte articolul este obligatorie (Required)
- **UserId** – string → cheie externa – reprezinta utilizatorul care a postat articolul

Category:

- **Id** – int → id-ul categoriei (cheie primara)
- **CategoryName** – string → numele categoriei este obligatoriu (Required)

Comment:

- **Id** – int → id-ul comentariului (cheie primara)
- **Content** – string → continutul comentariului este obligatoriu (Required)
- **Date** – DateTime → data la care a fost postat comentariul
- **ArticleId** – int (cheie externa) → articolul caruia ii apartine comentariul
- **UserId** – string (cheie externa) → utilizatorul care a postat comentariul

In continuare vom modifica anumite configuratii existente si generate de ASP.NET Core Identity, astfel incat sistemul de autentificare sa functioneze si sa putem configura si rolurile din cadrul aplicatiei.

Pentru adaugarea rolurilor si pentru realizarea asocierii dintre utilizatori si roluri trebuie parcursi urmatoarii pasi:

PASUL 1:

Pentru definirea utilizatorilor din aplicatie, Identity include in baza de date un tabel Users, impreuna cu toate attributele necesare (Id, UserName, Email, Password, PhoneNumber, etc).

In cadrul **Pasului 1**, se creeaza o noua clasa in folderul Models, clasa care o sa mosteneasca clasa de baza IdentityUser din pachetul Microsoft.AspNetCore.Identity. In cazul in care dorim sa extindem clasa User, adaugand attribute, putem realiza acest lucru in clasa pe care urmeaza sa o implementam.

Se adauga o clasa pe care o numim **ApplicationUser**

```
namespace ArticlesApp.Models
{
    public class ApplicationUser : IdentityUser
    {
    }
}
```

Clasa pe care o mosteneste, **IdentityUser**, este clasa care descrie Userul in baza de date (cea care contine toate attributele unui utilizator).

PASUL 2:

In cadrul **Pasului 2** se adauga serviciile necesare in fisierul **Program.cs**

```
builder.Services.AddDefaultIdentity<ApplicationUser>(options =>
options.SignIn.RequireConfirmedAccount = true)
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>(
);
```

In fisier exista deja configuratia pentru **AddDefaultIdentity** (serviciul de User Interface, cookies si toate mecanismele necesare pentru functionarea autentificarii) si **AddEntityFrameworkStores** (realizeaza conexiunea cu baza de date, conectandu-se la baza de date a carui string de conexiune se afla in appsettings.json), fiind necesara adaugarea serviciului de management al rolurilor → **AddRoles**.

PASUL 3:

In folderul Data exista contextul bazei de date, fisierul **ApplicationDbContext.cs**, care contine conexiunea cu baza de date si configurarea provider-ului de baze de date (in cazul nostru SQL Server) folosind dependency injection.

În secvența următoare de cod, clasa `ApplicationDbContext` moștenește clasa de bază `IdentityDbContext` – clasa care se ocupă cu managementul utilizatorilor și rolurilor (conține proprietăți și metode cu ajutorul cărora se pot prelucra utilizatorii și rolurile din aplicație).

Clasa este de forma: `IdentityDbContext<TUser>` ceea ce înseamnă că primește tipul clasei creată pentru prelucrarea utilizatorilor din baza de date.

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
        : base(options)
    {
    }
    ...
}
```

PASUL 4:

Se creează o nouă clasă în folderul `Models`, cu ajutorul căreia se vor adăuga în baza de date Rolurile necesare. În același timp se pot asocia și utilizatori cu fiecare rol. Acest lucru se face o singură dată, urmând ca utilizatorii care creează cont să primească un rol în momentul înregistrării (logica la nivel de Controller).

Se creează o clasă, numită sugestiv **SeedData**, în cadrul căreia vom implementa o metodă numită **Initialize** prin intermediul căreia vom popula cele trei tabele: `Users`, `Roles` și `UserRoles`.

Un Service Provider este utilizat pentru a injecta dependențele în aplicație (baza de date, sesiuni, pachete, autentificare, etc). În cazul nostru, cerem serviciul care realizează conexiunea cu baza de date.

```

public static class SeedData
{
    public static void Initialize(IServiceProvider
serviceProvider)
    {
        using (var context = new ApplicationDbContext(
            serviceProvider.GetRequiredService
            <DbContextOptions<ApplicationDbContext>>()))
        {
            // Verificam daca in baza de date exista cel putin un
            rol
            // insemnand ca a fost rulat codul
            // De aceea facem return pentru a nu insera rolurile
            inca o data
            // Acesta metoda trebuie sa se execute o singura data
            if (context.Roles.Any())
            {
                return; // baza de date contine deja roluri
            }

            // CREAREA ROLURILOR IN BD
            // daca nu contine roluri, acestea se vor crea
            context.Roles.AddRange(
                new IdentityRole { Id = "2c5e174e-3b0e-446f-86af-
483d56fd7210", Name = "Admin", NormalizedName = "Admin".ToUpper() },
                new IdentityRole { Id = "2c5e174e-3b0e-446f-86af-
483d56fd7211", Name = "Editor", NormalizedName = "Editor".ToUpper() },
                new IdentityRole { Id = "2c5e174e-3b0e-446f-86af-
483d56fd7212", Name = "User", NormalizedName = "User".ToUpper() }
            );

            // o noua instanta pe care o vom utiliza pentru
            crearea parolelor utilizatorilor
            // parolele sunt de tip hash
            var hasher = new PasswordHasher<ApplicationUser>();

            // CREAREA USERILOR IN BD
            // Se creeaza cate un user pentru fiecare rol
            context.Users.AddRange(
                new ApplicationUser
                {
                    Id = "8e445865-a24d-4543-a6c6-9443d048cdb0",
                    // primary key
                    UserName = "admin@test.com",
                    EmailConfirmed = true,
                    NormalizedEmail = "ADMIN@TEST.COM",
                    Email = "admin@test.com",
                    NormalizedUserName = "ADMIN@TEST.COM",
                    PasswordHash = hasher.HashPassword(null,
"Admin1!")
                },
                new ApplicationUser
                {

```

```

        Id = "8e445865-a24d-4543-a6c6-9443d048cdb1",
// primary key
        UserName = "editor@test.com",
        EmailConfirmed = true,
        NormalizedEmail = "EDITOR@TEST.COM",
        Email = "editor@test.com",
        NormalizedUserName = "EDITOR@TEST.COM",
        PasswordHash = hasher.HashPassword(null,
"Editor1!")
    },
    new ApplicationUser
    {
// primary key
        Id = "8e445865-a24d-4543-a6c6-9443d048cdb2",
        UserName = "user@test.com",
        EmailConfirmed = true,
        NormalizedEmail = "USER@TEST.COM",
        Email = "user@test.com",
        NormalizedUserName = "USER@TEST.COM",
        PasswordHash = hasher.HashPassword(null,
"User1!")
    }
};

// ASOCIEREA USER-ROLE
context.UserRoles.AddRange(
    new IdentityUserRole<string>
    {
        RoleId = "2c5e174e-3b0e-446f-86af-
483d56fd7210",
        UserId = "8e445865-a24d-4543-a6c6-
9443d048cdb0"
    },
    new IdentityUserRole<string>
    {
        RoleId = "2c5e174e-3b0e-446f-86af-
483d56fd7211",
        UserId = "8e445865-a24d-4543-a6c6-
9443d048cdb1"
    },
    new IdentityUserRole<string>
    {
        RoleId = "2c5e174e-3b0e-446f-86af-
483d56fd7212",
        UserId = "8e445865-a24d-4543-a6c6-
9443d048cdb2"
    }
);

context.SaveChanges();
}
}
}

```

Id-urile (Id, RoleId, UserId) au fost generate utilizand:
<https://guidgenerator.com/>

PASUL 5:

Prin instanta curenta a aplicatiei se apeleaza din clasa SeedData, metoda Initialize, ducand la crearea rolurilor si a utilizatorilor in baza de date.

In **Program.cs** se adauga **Pasul 5**:

```
builder.Services.AddControllersWithViews();

var app = builder.Build();

// PASUL 5 - useri si roluri
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    SeedData.Initialize(services);
}
```

PASUL 6:

Se adauga proprietatile in clasele din Model, astfel:

In clasa Article

```
// PASUL 6 - useri si roluri

    public virtual ApplicationUser User { get; set; } → un
    articol apartine unui singur utilizator
```

In clasa Comment

// PASUL 6 – useri si roluri

`public virtual ApplicationUser User { get; set; }` → un comentariu apartine unui singur utilizator

PASUL 7:

In folderul Views → Shared → _LoginPartial.cshtml → se modifica astfel:

```
@inject SignInManager<IdentityUser> SignInManager
@inject UserManager<IdentityUser> UserManager
```



```
@inject SignInManager<ApplicationUser> SignInManager
@inject UserManager<ApplicationUser> UserManager
```

PASUL 8:

Se realizeaza o noua migratie in baza de date.

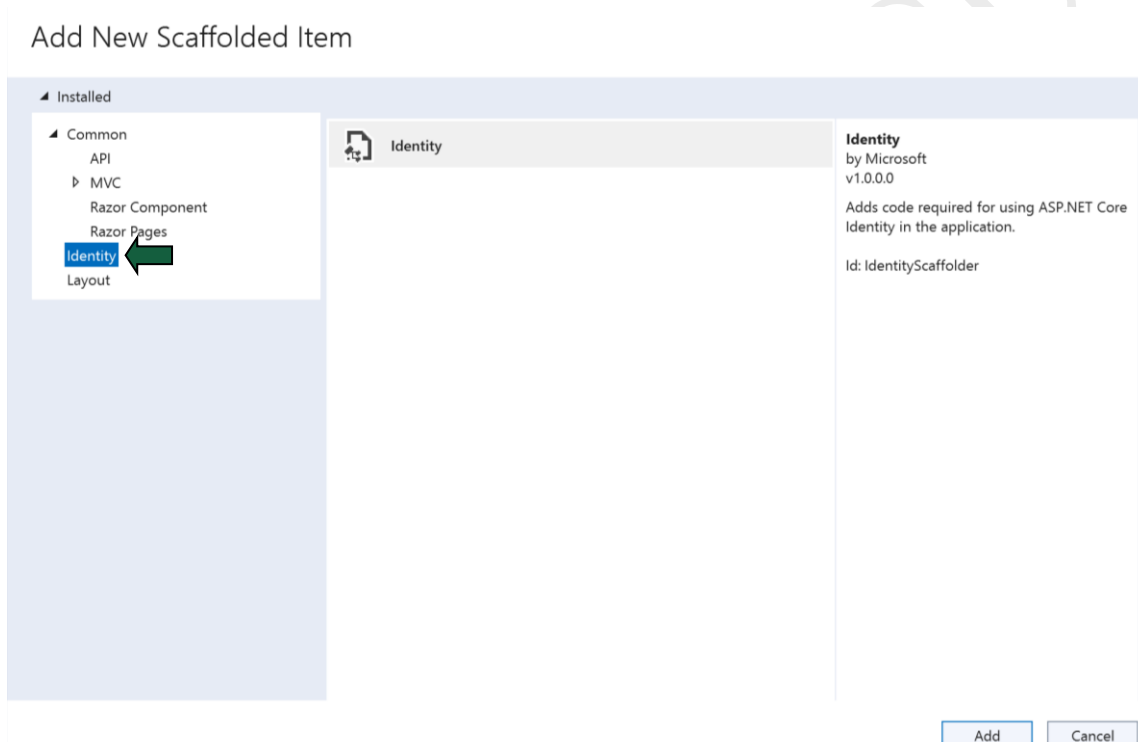
Se ruleaza pe rand:

1. Add-Migration DenumireMigratie
2. Update-Database

PASUL 9:

Pentru modificarea functionalitatilor implementate in framework, se poate utiliza optiunea **Add Scaffolded Item**, prin intermediul careia se poate aduce in folderele de lucru codul sursa.

Click dreapta pe numele proiectului din Solution Explorer → Add → New Scaffolded Item



Pentru exemplul din cadrul cursului avem nevoie de functionalitatea de inregistrare. In momentul in care un utilizator se inregistreaza in aplicatie, acesta trebuie sa primeasca un rol. In cazul aplicatiei Engine de stiri, utilizatorii inregistrati o sa primeasca rolul User. Userii inregistrati pot vedea articolele, pot lasa comentarii, pot edita si sterge propriile comentarii. Utilizatorii devin editori numai in momentul in care Adminul schimba rolul.

Pentru adaugarea rolului de User in momentul inregistrarii, trebuie modificata metoda Register.

Add Identity

Select an existing layout page, or specify a new one:

~/Views/Shared/_LayoutNou.cshtml



(Leave empty if it is set in a Razor _viewstart file)

☐ Override all files

Choose files to override

- | | | |
|--|--|---|
| ☐ Account\StatusMessage | ☐ Account\AccessDenied | ☐ Account\ConfirmEmail |
| ☐ Account\ConfirmEmailChange | ☐ Account\ExternalLogin | ☐ Account\ForgotPassword |
| ☐ Account\ForgotPasswordConfirmation | ☐ Account\Lockout | ☐ Account>Login |
| ☐ Account>LoginWith2fa | ☐ Account>LoginWithRecoveryCode | ☐ Account\Logout |
| ☐ Account\Manage\Layout | ☐ Account\Manage\ManageNav | ☐ Account\Manage\StatusMessage |
| ☐ Account\Manage\ChangePassword | ☐ Account\Manage\DeletePersonalData | ☐ Account\Manage\Disable2fa |
| ☐ Account\Manage\DownloadPersonalData | ☐ Account\Manage\Email | ☐ Account\Manage\EnableAuthenticator |
| ☐ Account\Manage\ExternalLogins | ☐ Account\Manage\GenerateRecoveryCodes | ☐ Account\Manage\Index |
| ☐ Account\Manage\PersonalData | ☐ Account\Manage\ResetAuthenticator | ☐ Account\Manage\SetPassword |
| ☐ Account\Manage\ShowRecoveryCodes | ☒ Account\Manage\TwoFactorAuthentication | ☒ Account\Register |
| ☐ Account\RegisterConfirmation | ☐ Account\ResendEmailConfirmation | ☐ Account\ResetPassword |
| ☐ Account\ResetPasswordConfirmation | | |

Data context class

ApplicationDbContext (ArticlesApp.Data)

User class

Add

Cancel

Codul inclus se poate accesa din Solution Explorer → Areas → Identity → Pages → Account → Register.cshtml → Register.cshtml.cs\

// PASUL 9 – useri si roluri (adaugarea rolului la inregistrare)

```
await _userManager.AddToRoleAsync(user, "User");
```

PASUL 10:

Ultimul pas este reprezentat de configurarea managerului de utilizatori și roluri. În Controller-ul **ArticlesController** se implementează următoarea secvență de cod:

```
public class ArticlesController : Controller
{
    private readonly ApplicationDbContext db;

    private readonly UserManager<ApplicationUser> _userManager;

    private readonly RoleManager<IdentityRole> _roleManager;

    public ArticlesController(
        ApplicationDbContext context,
        UserManager<ApplicationUser> userManager,
        RoleManager<IdentityRole> roleManager
    )
    {
        db = context;
        _userManager = userManager;
        _roleManager = roleManager;
    }

    ...
}
```

Documentație Microsoft – User Manager: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.usermanager-1?view=aspnetcore-6.0>

Documentație Microsoft – Role Manager: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.rolemanager-1?view=aspnetcore-6.0>

Atributul [Authorize]

Pentru a restrictiona accesul utilizatorilor la metodele din cadrul unui Controller, se utilizeaza atributul **[Authorize]** la nivelul Controller-ului respectiv.

Pentru a restrictiona accesul in cadruli fiecarei metode, se utilizeaza acelasi atribut, astfel:

```
[Authorize(Roles = "User,Editor,Admin")]
public IActionResult Index()
{ ... }
```

Doar utilizatorii cu rolul User, Editor sau Admin, pot accesa metoda Index.

Implementare New, Edit si Delete utilizand roluri

In cazul metodelor de adaugare, editare si stergere trebuie sa se tina cont si de id-ul utilizatorului care adauga, editeaza sau sterge articolul. In cazul adaugarii o sa se preia id-ul utilizatorului pentru adaugarea lui in baza de date. In cazul editarii se verifica daca utilizatorul care doreste sa editeze articolul este utilizatorul caruia ii apartine articolul respectiv (adica utilizatorul care a postat articolul). Asemănător se procedează și în cazul stingerii unui articol. Un user poate sterge un articol doar dacă respectivul articol îi aparține. Un utilizator nu poate sterge articole postate în platforma de alți utilizatori.

Exemplu implementare metoda New cu Post din ArticlesController:

```
[Authorize(Roles = "Editor,Admin")]
[HttpPost]
public IActionResult New(Article article)
{
    article.Date = DateTime.Now;

    article.UserId = _userManager.GetUserId(User);
```

```

        if (ModelState.IsValid)
        {
            db.Articles.Add(article);
            db.SaveChanges();
            TempData["message"] = "Articolul a fost adaugat";
            return RedirectToAction("Index");
        }
        else
        {
            article.Categ = GetAllCategories();
            return View(article);
        }
    }
}

```

In exemplul anterior se poate observa cum cu ajutorul managerului de useri este preluat id-ul userului curent, folosind metodei `GetUserId`. Astfel, in momentul inserarii articolului in baza de date, se insereaza si id-ul utilizatorului care a postat articolul respectiv.

```

_userManager.GetUserId(User);

```

In cazul editarii, se verifica daca utilizatorul care doreste sa modifice articolul este utilizatorul care a postat articolul. Preluarea id-ului userului curent se realizeaza tot cu ajutorul metodei `GetUserId`. De asemenea, Administratorul avand drepturi depline asupra aplicatiei, se verifica si rolul. Daca utilizatorul are rolul Admin, atunci el poate edita articolul. Pentru verificarea rolului se utilizeaza metoda `IsInRole("NumeRol")`

Edit - HttpGet

```

...
    if (article.UserId == _userManager.GetUserId(User) ||
        User.IsInRole("Admin"))
    {
        return View(article);
    }

    else
    {
        TempData["message"] = "Nu aveti dreptul sa faceti
        modificari asupra unui articol care nu va apartine";
        return RedirectToAction("Index");
    }
    ...

```

La fel se procedeaza si in cazul editarii cu HttpPost, dar si in cazul stergerii. Se verifica daca utilizatorul care intentioneaza sa editeze sau sa stearga articolul este userul care a creat respectivul articol. In plus, se verifica si rolul utilizatorului care face actiunea. Daca acesta este Admin, atunci poate face C.R.U.D. asupra oricarei entitati din aplicatie.

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 10

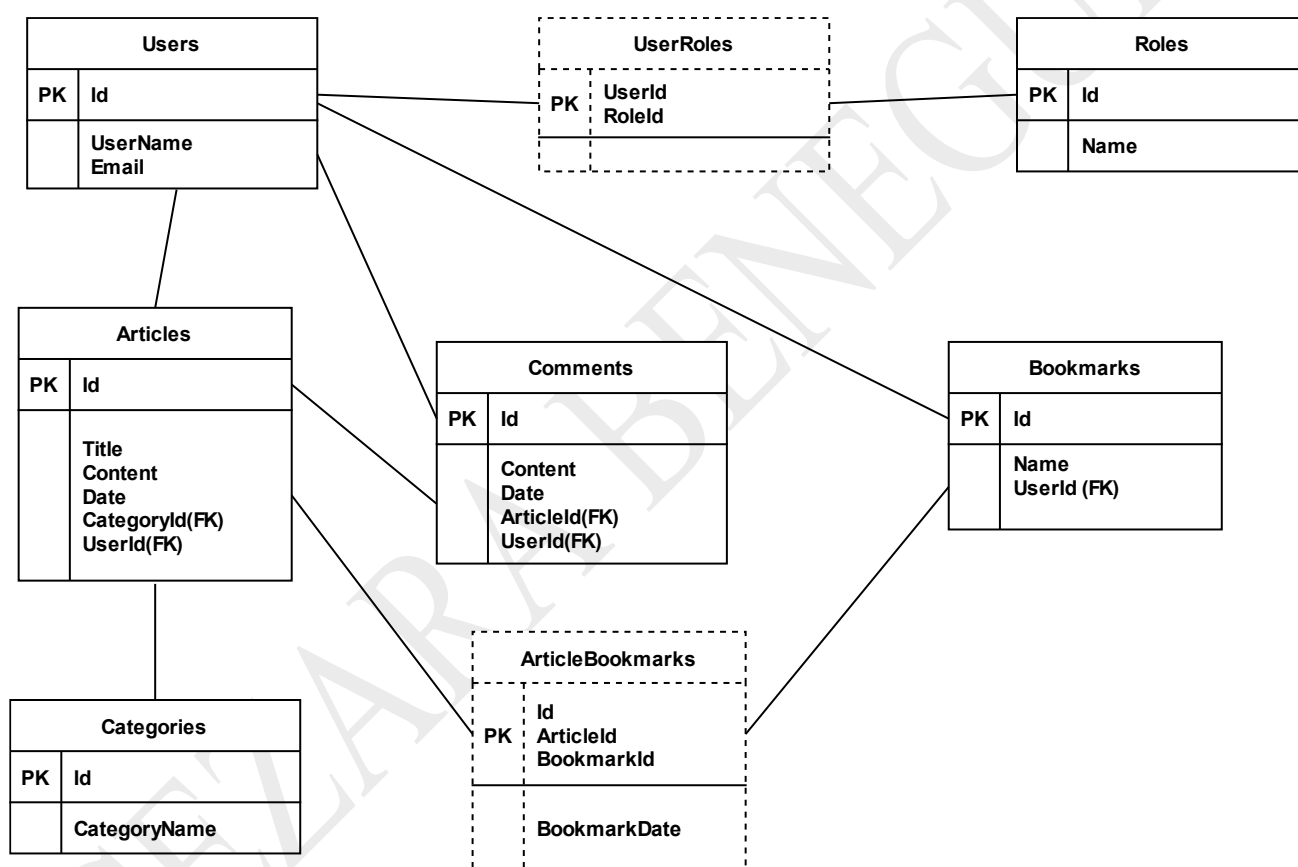
Cuprins

Implementarea relatiei many-to-many	2
Descrierea diagramei.....	2
Implementarea claselor	4
Stocarea fisierelor in baza de date	8

Implementarea relatiei many-to-many

Descrierea diagramei

Pentru implementarea si exemplificarea relatiei many-to-many vom utiliza urmatoarea diagrama conceptuala. Se va extinde diagrama proiectata in cursurile anterioare, prin adaugarea unui nou tabel → **Bookmark**



Relatia many-to-many se afla intre **Article** si **Bookmark**, avand urmatoarele specificatii:

- Un utilizator isi poate crea propriile colectii (Bookmarks);
- In momentul in care un utilizator isi adauga colectii, acesta o sa aiba acces doar la colectiile pe care le-a creat, existand posibilitatea editarii si a stergerii lor;
- Atat utilizatorii cu rolurile User sau Editor, cat si utilizatorii cu rolul Admin, pot crea propriile colectii. Utilizatorii cu rolul User sau Editor o sa aiba acces doar la colectiile create de ei, iar utilizatorii cu rolul Admin o sa aiba acces la toate colectiile existente in platforma;
- Dupa crearea colectiilor, utilizatorii isi pot adauga articole in colectii. Articolele pe care le pot adauga trebuie sa existe in platforma. Ei doar selecteaza un articol si il adauga intr-o colectie. Utilizatorii nu pot vedea colectiile altor utilizatori. Fiecare utilizator are acces doar la colectiile sale;
- Un articol poate face parte din mai multe colectii, iar o colectie poate contine mai multe articole;

Se considera urmatoarele clase: **Bookmark**, **ArticleBookmark** (tabelul asociativ) cu urmatoarele proprietati:

Bookmark:

- **Id** – int → id-ul colectiei (cheie primara)
- **Name** – string → denumirea colectiei, fiind o proprietate obligatorie (Required)
- **UserId** – string → cheie externa – reprezinta utilizatorul care a creat colectia

ArticleBookmark:

- **Id, ArticleId, BookmarkId** – int → cheia primara compusa
- **Id** – valoare unica, avand auto-increment
- **BookmarkDate** – DateTime → Data si ora la care a fost adaugat un articol in cadrul unei colectii

Implementarea claselor

Pentru implementarea claselor se procedeaza astfel:

Bookmark.cs

```
public class Bookmark
{
    [Key]
    public int Id { get; set; }

    [Required(ErrorMessage = "Numele colectiei este obligatoriu")]
    public string Name { get; set; }

    public string? UserId { get; set; }

    public virtual ApplicationUser? User { get; set; }

    public virtual ICollection<ArticleBookmark>? ArticleBookmarks
    { get; set; }
}
```

Article.cs

```
public class Article
{
    ...

    public virtual ICollection<ArticleBookmark>? ArticleBookmarks
    { get; set; }
}
```

ArticleBookmark.cs

```
public class ArticleBookmark
{
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    public int? ArticleId { get; set; }
    public int? BookmarkId { get; set; }

    public virtual Article? Article { get; set; }
    public virtual Bookmark? Bookmark { get; set; }

    public DateTime BookmarkDate { get; set; }
}
```

ApplicationUser.cs

```
public class ApplicationUser : IdentityUser
{
    public virtual ICollection<Comment>? Comments { get; set; }
    public virtual ICollection<Article>? Articles { get; set; }
    public virtual ICollection<Bookmark>? Bookmarks { get; set; }
}
```

ApplicationDbContext.cs

```

public class ApplicationDbContext :
IdentityDbContext<ApplicationUser>
{
    public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<ApplicationUser> ApplicationUsers { get; set; }
    public DbSet<Article> Articles { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<Comment> Comments { get; set; }
    public DbSet<Bookmark> Bookmarks { get; set; }
    public DbSet<ArticleBookmark> ArticleBookmarks { get; set; }

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // definire primary key compus
        modelBuilder.Entity<ArticleBookmark>()
            .HasKey(ab => new { ab.Id, ab.ArticleId, ab.BookmarkId
});

        // definire relatii cu modelele Bookmark si Article (FK)
        modelBuilder.Entity<ArticleBookmark>()
            .HasOne(ab => ab.Article)
            .WithMany (ab => ab.ArticleBookmarks)
            .HasForeignKey(ab => ab.ArticleId);

        modelBuilder.Entity<ArticleBookmark>()
            .HasOne(ab => ab.Bookmark)
            .WithMany(ab => ab.ArticleBookmarks)
            .HasForeignKey(ab => ab.BookmarkId);
    }
}

```

Se pot adauga si proprietati suplimentare in clasa ApplicationUser, extinzand astfel clasa. In exemplul urmator se adauga doua attribute → **FirstName** si **LastName**.

ApplicationUser.cs

```
public class ApplicationUser : IdentityUser
{
    public virtual ICollection<Comment>? Comments { get; set; }
    public virtual ICollection<Article>? Articles { get; set; }
    public virtual ICollection<Bookmark>? Bookmarks { get; set; }

    public string? FirstName { get; set; }
    public string? LastName { get; set; }

    [NotMapped]
    public IEnumerable<SelectListItem>? AllRoles { get; set; }
}
```

!/ OBSERVATIE

Dupa modificarea claselor se executa migratiile.

Add-Migration NumeMigratie

Update-Database

Stocarea fisierelor in baza de date

```
public class Article
{
    [Key]
    public int Id { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
    public DateTime Date { get; set; }

    // Adaugam un string unde vom salva calea imaginii pentru articol
    public string Image { get; set; }
}
```

View-ul asociat:

```
<form enctype="multipart/form-data" asp-controller="Articles" asp-
action="UploadImage">

    <input class="form-control" placeholder="Article Name" type="text"
name="Title" />

    <br />

    <textarea class="form-control" placeholder="Article Content"
style="width: 100%; height: 100px;" name="Content"></textarea>

    <br />

    <input class="form-control" placeholder="Date" type="text"
name="Date" />

    <br />

    <input class="form-control" type="file" name="ArticleImage" />

    <br />

    <input class="btn btn-success" type="submit" value="Upload" />

</form>
```

Controller:

```
// Variabila locala de tip AppDbContext

private AppDbContext _context;
private IWebHostEnvironment _env;

// In constructor, se face dependency injection
public ArticlesController(AppDbContext context, IWebHostEnvironment
env)
{
    // Alocam conexiunea (injectata) cu baza de date unei proprietati
    locale pentru a fi refolosita in metodele controller-ului
    _context = context;
    _env = env;
}

// Afisam view-ul cu form-ul
public IActionResult UploadImage()
{
    return View();
}

// Facem upload la fisier si salvam modelul in baza de date

[HttpPost]

public async Task<IActionResult> UploadImage(Article article,
IFormFile ArticleImage)
{
    // Verificam daca exista imaginea in request (daca a fost
    incarcata o imagine)

    if (ArticleImage.Length > 0)
    {
        // Generam calea de stocare a fisierului
        var storagePath = Path.Combine(
            _env.WebRootPath, // Preluam calea folderului wwwroot
            "images", // Adaugam calea folderului images
            ArticleImage.FileName // Numele fisierului
        );
```

```
// Generam calea de afisare a fisierului care va fi stocata in
baza de date
var databaseFileName = "/images/" + ArticleImage.FileName;

// Uploadam fisierul la calea de storage
using (var fileStream = new FileStream(storagePath,
FileMode.Create))
{
    await ArticleImage.CopyToAsync(fileStream);
}

// Salvam storagePath-ul in baza de date
article.Image = databaseFileName;
_context.Articles.Add(article);
_context.SaveChanges();

return View();
}
```

Dezvoltarea Aplicatiilor Web utilizand ASP.NET Core MVC

Curs 11

Cuprins

Afisarea paginata.....	2
Exemplu de implementare	2
Includerea unui editor de text	5
Exemplu de implementare	6
Functionalitatea de cautare	15
Descrierea si implementarea motorului de cautare.....	15
Design-ul intr-o aplicatie Web.....	19
Reguli de baza in design	19
User Experience (UX).....	22
Alegerea culorilor potrivite.....	26

Afisarea paginata

Afisarea paginata este utila in cazul in care trebuie sa afisam un numar mare de elemente, numar care se modifica constant. De exemplu, trebuie sa afisam foarte multe categorii, articole, produse, etc. In acest caz se utilizeaza afisarea paginata, in cadrul careia dezvoltatorii implementeaza afisarea in functie de o serie de reguli. Regulile trebuie stabilite luand in considerare numarul de elemente pe care dorim sa-l afisam, tinand cont si de faptul ca acest numar o sa isi modifice valoarea in functie de actiunile utilizatorilor in cadrul aplicatiei (de ex: utilizatorii pot adauga/sterge articole sau produse).

Exemplu de implementare

Exemplul urmator este realizat in cadrul aplicatiei Engine de stiri (aplicatia dezvoltata in laborator).

Pentru afisarea paginata o sa se utilizeze din Bootstrap – Pagination → <https://getbootstrap.com/docs/5.2/components/pagination/>

Se vor afisa 3 articole pe pagina, modificandu-se atat metoda Index din ArticlesController, cat si View-ul Index corespunzator.

Includerea si modificarea componentei din Bootstrap:

```
<nav aria-label="Page navigation example">
  <ul class="pagination">
    <li class="page-item">
      <a class="page-link" href="#" aria-label="Previous">
        <span aria-hidden="true">&laquo;</span></a>
    </li>
    <li class="page-item"><a class="page-link" href="#">1</a></li>
    <li class="page-item"><a class="page-link" href="#">2</a></li>
    <li class="page-item">
      <a class="page-link" href="#" aria-label="Next">
        <span aria-hidden="true">&raquo;</span></a>
    </li>
  </ul>
</nav>
```

Secventa de cod anterioara o sa se modifice astfel. Modificarile sunt scrise cu negru si bold.

@* Afisarea paginata a articolelor *@

```
<div>
  <nav aria-label="Page navigation example">
    <ul class="pagination">
      <li class="page-item">
        <a class="page-link"
href="/Articles/Index?page=1" aria-label="Previous">
          <span aria-hidden="true">&laquo;</span>
        </a>
      </li>

      @for (int i = 1; i <= ViewBag.lastPage; i++)
      {
        <li class="page-item"> <a class="page-link"
href="/Articles/Index?page=@i">@(i)</a> </li>
      }

      <li class="page-item">
        <a class="page-link"
href="/Articles/Index?page=@(ViewBag.lastPage)" aria-
label="Next">
          <span aria-hidden="true">&raquo;</span>
        </a>
      </li>
    </ul>
  </nav>
</div>
```

ArticlesController → metoda Index

```
[Authorize(Roles = "User,Editor,Admin")]
public IActionResult Index()
{
    // Alegem sa afisam 3 articole pe pagina
    int _perPage = 3;

    var articles = db.Articles.Include("Category")
.Include("User").OrderBy(a => a.Date);

    if (TempData.ContainsKey("message"))
    {
        ViewBag.message =
TempData["message"].ToString();
        ViewBag.Alert = TempData["messageType"];
    }

    // Fiind un numar variabil de articole, verificam de
fiecare data utilizand
    // metoda Count()

    int totalItems = articles.Count();

    // Se preia pagina curenta din View-ul asociat
    // Numarul paginii este valoarea parametrului page
din ruta
    // /Articles/Index?page=valoare

    var currentPage =
Convert.ToInt32(HttpContext.Request.Query["page"]);

    // Pentru prima pagina offsetul o sa fie zero
    // Pentru pagina 2 o sa fie 3
    // Asadar offsetul este egal cu numarul de articole
care au fost deja afisate pe paginile anterioare
    var offset = 0;

    // Se calculeaza offsetul in functie de numarul
paginii la care suntem
    if (!currentPage.Equals(0))
    {
        offset = (currentPage - 1) * _perPage;
    }
}
```

```

        // Se preiau articolele corespunzatoare pentru
        fiecare pagina la care ne aflam
        // in functie de offset
        var paginatedArticles =
articles.Skip(offset).Take(_perPage);

        // Preluam numarul ultimei pagini

        ViewBag.lastPage = Math.Ceiling((float)totalItems /
(float)_perPage);

        // Trimitem articolele cu ajutorul unui ViewBag
        catre View-ul corespunzator
        ViewBag.Articles = paginatedArticles;

        return View();
    }

```

Includerea unui editor de text

In aceasta sectiune vom integra o componenta externa (componenta 3rd party). Din categoria componentelor 3rd party fac parte: servere web (Apache, Nginx, IIS, etc), framework-uri (.NET, Rails, Spring, etc), librarii.

Componenta pe care o vom integra este un editor de text open-source, bazat pe Bootstrap si jQuery, numita **Summernote**. Este o librarie de JavaScript folosita pentru a implementa un editor de tipul **WYSIWUG - What You See Is What You Get**

Acest tip de editor ofera posibilitatea prelucrarii textului de catre utilizatorii finali, asemanator cu scrierea textului in Word sau folosind limbaj de markup (HTML).

Pagina oficiala → <https://summernote.org/>

Exemplu de implementare

Exemplul urmator este realizat in cadrul aplicatiei Engine de stiri (aplicatia dezvoltata in laborator).

Se doreste utilizarea editorului de text atat in momentul adaugarii unui articol, cat si in momentul editarii acestuia.

Se integreaza Summernote accesand link-ul urmator si urmand pasii:

PASUL 1:

<https://summernote.org/getting-started/#without-bootstrap-lite>

PASUL 2:

In **_Layout.cshtml** se include in Head componenta de css:

```
<link href="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.min.css" rel="stylesheet">
```

```

2  <html lang="en">
3  <head>
4      <meta charset="utf-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>@ViewData["Title"] - ArticlesApp</title>
7      <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
8      <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.10.1/font/bootstrap-icons.css">
9
10     <link rel="stylesheet" href="~/ArticlesApp.styles.css" asp-append-version="true" />
11     <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
12
13     <!-- se include summernote css -->
14     <link href="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.min.css" rel="stylesheet">
15
16 </head>
17 <body>
18     <header>
19         <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow"
20             <div class="container-fluid">
21                 <a class="navbar-brand" asp-area="" asp-controller="Articles" asp-action="Index">ArticlesApp</a>
22

```

PASUL 3:

In `_Layout.cshtml` se include in Body componenta de js:

```
<script src="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.min.js"></script>
```

```

94 <footer class="border-top footer text-muted">
95   <div class="container">
96     &copy; 2022 - ArticlesApp - <a asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
97   </div>
98 </footer>
99
100 <script src="~/lib/jquery/dist/jquery.min.js"></script>
101 <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
102
103 <script src="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.min.js"></script>
104
105 <script src="~/js/site.js" asp-append-version="true"></script>
106
107
108 @await RenderSectionAsync("Scripts", required: false)
109
110
111 </body>
112 </html>

```

PASUL 4:

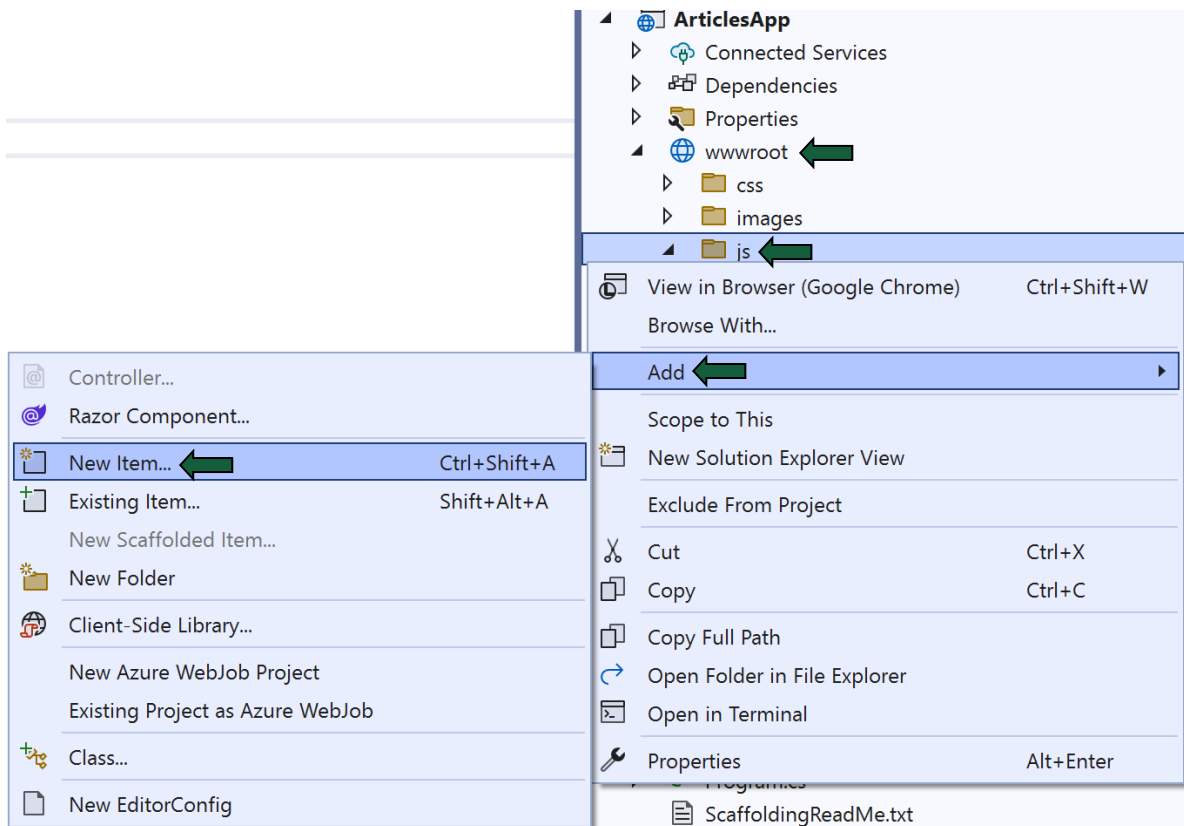
Editorul se initializeaza adaugand urmatoarea secventa de cod intr-un fiser de tip JavaScript (js).

```

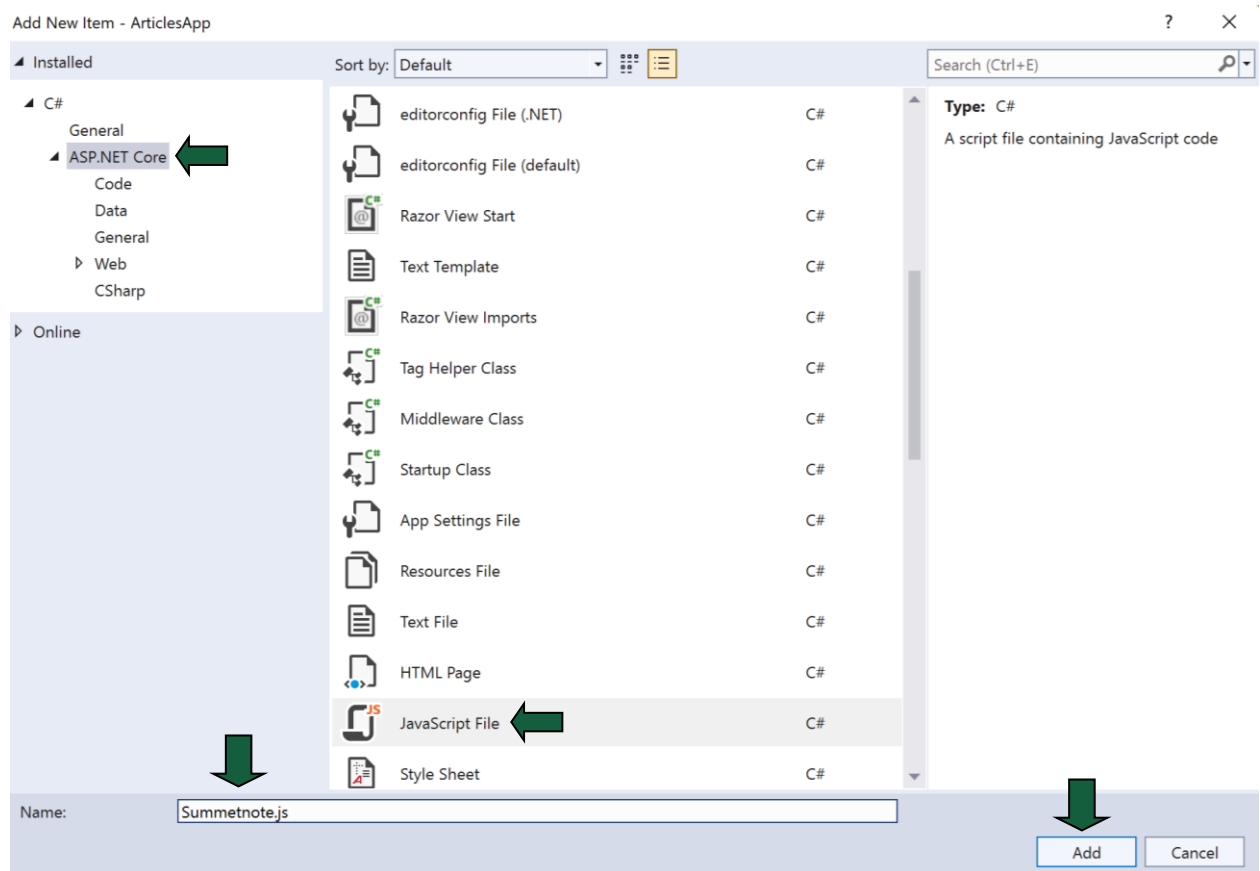
$(document).ready(function () {
  $(''.summernote).summernote({
    height: 300,
    minHeight: 200,
    focus: true,
  });
});

```

PASUL 5:



PASUL 6:



PASUL 7:

Scriptul-ul se include in _Layout in Body:

```

93
94 <footer class="border-top footer text-muted">
95   <div class="container">
96     &copy; 2022 - ArticlesApp - <a asp-area="" asp-controller="Home" asp-action="Privacy">P
97   </div>
98 </footer>
99
100 <script src="~/lib/jquery/dist/jquery.min.js"></script>
101 <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
102
103 <script src="https://cdn.jsdelivr.net/npm/summernote@0.8.18/dist/summernote-lite.min.js"></scri
104
105 <script src="~/js/site.js" asp-append-version="true"></script>
106 <script src="~/js/Summernote.js" asp-append-version="true"></script> ←
107
108 @await RenderSectionAsync("Scripts", required: false)
109
110
111 </body>
112 </html>

```

PASUL 8:

Pentru utilizarea editorului se foloseste clasa definita in fisierul Summernote.js

```
@Html.TextArea("Content", null, new { @class = "summernote" })
```



Dupa integrarea editorului se observa o problema de afisare:

Adaugare articol

Titlu Articol

Continut Articol

The image shows a rich text editor interface. The toolbar includes icons for bold, italic, underline, strikethrough, font color, background color, text color, text size, text alignment, list, link, unlink, image, video, table, code, and help. A dropdown menu for font family is set to 'sans-serif'. Green arrows point to the font family dropdown, the table icon, and the list icon.

Pentru afisarea corecta a dropdown-ului se utilizeaza stilizarea de mai jos (astfel incat sa nu afiseze acele sageti duplicat):

```
.note-editor .dropdown-toggle::after {
    display: none;
}
```

PASUL 9:

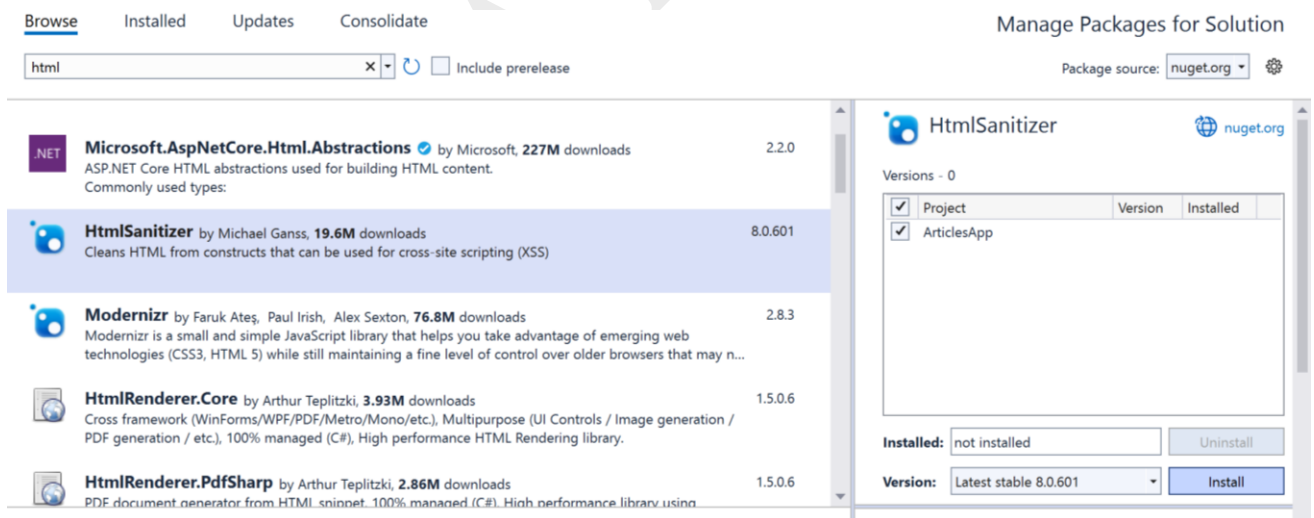
Pentru afisare vom utiliza `@Html.Raw()`

```
<div class="card-text">@Html.Raw(Model.Content)</div>
```

PASUL 10:

Prevenire XSS (Cross Site Scripting). XSS presupune posibilitatea inserarii unui cod, de catre o persoana neautorizata, intr-o pagina web. In cazul exemplului din cadrul cursului, un utilizator neautorizat poate introduce cod JavaScript, in formular, in momentul inserarii unui nou articol in platforma.

Pentru a preveni XSS este nevoie de instalarea unui nou pachet din NuGet package → **HtmlSanitizer**



In cadrul adaugarii si editarii din Controller o sa avem urmatoarele modificari:

ArticlesController → metoda New cu POST

[HttpPost]

```
public IActionResult New(Article article)
{
    var sanitizer = new HtmlSanitizer();

    article.Date = DateTime.Now;
    article.UserId = _userManager.GetUserId(User);

    if (ModelState.IsValid)
    {
        article.Content = sanitizer.Sanitize(article.Content);

        article.Content = (article.Content);

        db.Articles.Add(article);
        db.SaveChanges();
        TempData["message"] = "Articolul a fost adaugat";
        TempData["messageType"] = "alert-success";
        return RedirectToAction("Index");
    }
    else
    {
        article.Categ = GetAllCategories();
        return View(article);
    }
}
```

ArticlesController → metoda Edit cu POST

```
[HttpPost]
[Authorize(Roles = "Editor,Admin")]
public IActionResult Edit(int id, Article requestArticle)
{
    Var sanitizer = new HtmlSanitizer(); ←
    Article article = db.Articles.Find(id);

    if (ModelState.IsValid)
    {
        if (article.UserId == _userManager.GetUserId(User) ||
        User.IsInRole("Admin"))
        {
            article.Title = requestArticle.Title;

            requestArticle.Content = ←
            sanitizer.Sanitize(requestArticle.Content);

            article.Content = requestArticle.Content;

            article.CategoryId =
            requestArticle.CategoryId;
            TempData["message"] = "Articolul a fost
            modificat";
            TempData["messageType"] = "alert-success";
            db.SaveChanges();
            return RedirectToAction("Index");
        }
        else
        {
            TempData["message"] = "Nu aveti dreptul sa
            faceti modificari asupra unui articol care nu va apartine";
            TempData["messageType"] = "alert-danger";
            return RedirectToAction("Index");
        }
    }
    else
    {
        requestArticle.Categ = GetAllCategories();
        return View(requestArticle);
    }
}
```

În cazul editării, pentru preluarea corectă a conținutului articolului din baza de date, trebuie să utilizăm `TextArea`. Astfel, se înlocuiește helperul `@Html.Editor` cu helperul `@Html.TextArea`.

```
@* @Html.Editor("Content", new { htmlAttributes = new { @class =
"form-control summernote" } }) *@

@Html.TextArea("Content", Model.Content, new { @class = "form-
control summernote" })
```

Funcționalitatea de căutare

Funcționalitatea de căutare se află integrată în orice aplicație, indiferent de natura ei (aplicație web, aplicație mobilă, desktop, etc).

Cel mai mare motor de căutare este **GOOGLE**, urmat de Bing și Yahoo. Acestea ocupă în momentul de față primele 3 locuri.

Descrierea și implementarea motorului de căutare

Pentru a exemplifica integrarea motorului de căutare, o să utilizăm aplicația dezvoltată în laborator – Engine de stiri. În cadrul aplicației se integrează funcționalitatea de căutare a articolelor după **titlu**, **conținut**, dar și după **conținutul comentariilor** postate de utilizatori în cadrul articolelor respective.

Se utilizează componenta Bootstrap → Input group

<https://getbootstrap.com/docs/5.0/forms/input-group/>

Implementare:

View-ul Index

```
<form method="GET">

    <div class="input-group mb-3">

        <input type="text" class="form-control"
placeholder="Search topics or keywords" name="search"
value="@ViewBag.SearchString">

        <button class="btn btn-outline-success"
type="submit">Search</button>

    </div>

</form>
```

ArticlesController – Metoda Index

```
[Authorize(Roles = "User,Editor,Admin")]
public IActionResult Index()
{
    var articles = db.Articles.Include("Category")
        .Include("User").OrderBy(a => a.Date);

    var search = "";

    // MOTOR DE CAUTARE

    if (Convert.ToString(HttpContext.Request.Query["search"]) !=
null)
    {
        // eliminam spatiile libere
        search =
Convert.ToString(HttpContext.Request.Query["search"]).Trim();
    }
}
```

```

// Cautare in articol (Title si Content)
List<int> articleIds = db.Articles.Where
(
    at => at.Title.Contains(search)
        || at.Content.Contains(search)
).Select(a => a.Id).ToList();

// Cautare in comentarii (Content)
List<int> articleIdsOfCommentsWithSearchString =
db.Comments.Where
(
    c => c.Content.Contains(search)
).Select(c => (int)c.ArticleId).ToList();

// Se formeaza o singura lista formata din toate id-urile
// selectate anterior
List<int> mergedIds =
articleIds.Union(articleIdsOfCommentsWithSearchString).ToList();

// Lista articolelor care contin cuvantul cautat
// fie in articol -> Title si Content
// fie in comentarii -> Content
articles = db.Articles.Where(article =>
mergedIds.Contains(article.Id))
.Include("Category")
.Include("User")
.OrderBy(a => a.Date);

}

ViewBag.SearchString = search;

// AFISARE PAGINATA
{ ... implementarea se afla in sectiunea anterioara }

if(search != "")
{
    ViewBag.PaginationBaseUrl = "/Articles/Index/?search="
+ search + "&page";
} else
{
    ViewBag.PaginationBaseUrl = "/Articles/Index/?page";
}

return View();
}

```

Dupa cum se observa in implementarea din cadrul metodei Index din ArticlesController, trebuie sa ne asiguram ca afisarea paginata din cadrul sectiunii anterioare functioneaza si in momentul integrarii motorului de cautare.

Astfel, in cazul in care se foloseste motorul de cautare, URL-ul o sa fie de forma: `/Articles/Index/?search&page` ceea ce inseamna ca trebuie sa avem in ruta atat stringul cautat, cat si numarul paginii.

Afisarea paginata din cadrul View-ului Index o sa se modifice astfel:

@* Afisarea paginata a articolelor *@

```
<div>
  <nav aria-label="Page navigation example">
    <ul class="pagination">
      <li class="page-item">
        <a class="page-link"
href="@ViewBag.PaginationBaseUrl=1" aria-label="Previous">
          <span aria-hidden="true">&laquo;</span>
        </a>
      </li>

      @for (int i = 1; i <= ViewBag.lastPage; i++)
      {
        <li class="page-item"> <a class="page-link"
href="@ViewBag.PaginationBaseUrl=@i">@(i)</a> </li>
      }

      <li class="page-item">
        <a class="page-link"
href="@ViewBag.PaginationBaseUrl=@(ViewBag.lastPage)" aria-
label="Next">
          <span aria-hidden="true">&raquo;</span>
        </a>
      </li>
    </ul>
  </nav>
</div>
```

Design-ul intr-o aplicatie Web

Reguli de baza in design

Pentru realizarea aplicatiilor Web, in ceea ce priveste design-ul acestora, se pot respecta anumite reguli:

- **Simplitatea** – prea multa informatie intr-o pagina distrage atentia utilizatorului deoarece acesta trebuie sa citeasca si sa parcurga mult prea multa informatie pana la informatia de care are nevoie. Pastrand paginile simple, aplicatia va fi mult mai usor de folosit.
- **Design-ul este esential** – prima impresie conteaza. Aceasta regula se aplica si in dezvoltarea unei aplicatii web deoarece este primul lucru pe care il observa un utilizator atunci cand acceseaza aplicatia.
- **Culorile sunt foarte importante** – atunci cand alegeti culorile folositi o paleta consistenta de culori. De asemenea, sa va asigurati ca nu exista nuante foarte apropiate care nu pot fi deosebite si mai ales ca exista un contrast puternic intre text si fundal.
- **Navigarea ar trebui sa fie intuitiva** – un utilizator nu trebuie sa caute ceea ce doreste sa acceseze. Paginile trebuie sa fie bine organizate cu un design de tipul top-down, utilizatorii navigand usor printre diferitele sectiuni existente intr-o pagina.
- **Consistentă este extrem de importanta** – utilizatorii nu ar trebui sa aiba sentimentul ca viziteaza un alt site sau o alta aplicatie web de fiecare data cand acceseaza o alta pagina a aplicatiei. Consistentă face navigarea in aplicatie mult mai simpla.
- **Aplicatia trebuie sa fie responsive** – utilizatorii acceseaza aplicatia utilizand o varietate de device-uri, de la smartphone-uri la calculatoare personale, de aceea este esential ca aplicatia sa se incadreze oricarei

rezolutii. CSS media queries sunt ideale pentru realizarea rapida a unei aplicatii web responsive.

➤ **Utilizarea unui continut real in momentul dezvoltarii design-ului.**

Orice aplicatie este bazata pe continut si se dezvolta in jurul continutului. De cele mai multe ori atunci cand se dezvolta design-ul nu se acorda importanta continutului, folosindu-se Lorem Ipsum in locul textului real si placeholder in locul imaginilor. Chiar daca totul arata bine in timpul dezvoltarii design-ului, nu o sa mai fie la fel atunci cand aplicatia va contine datele reale. Scopul in dezvoltarea unei aplicatii web este de a fi cat mai aproape de experienta reala a utilizatorului

➤ **Fontul** – in general fonturile Sans Serif, cum ar fi Arial si Verdana, sunt mai usor de citit intr-o aplicatie web, fiind fonturi fara finisaje decorative. Dimensiunea fontului pentru citirea cu usurinta este de 16px

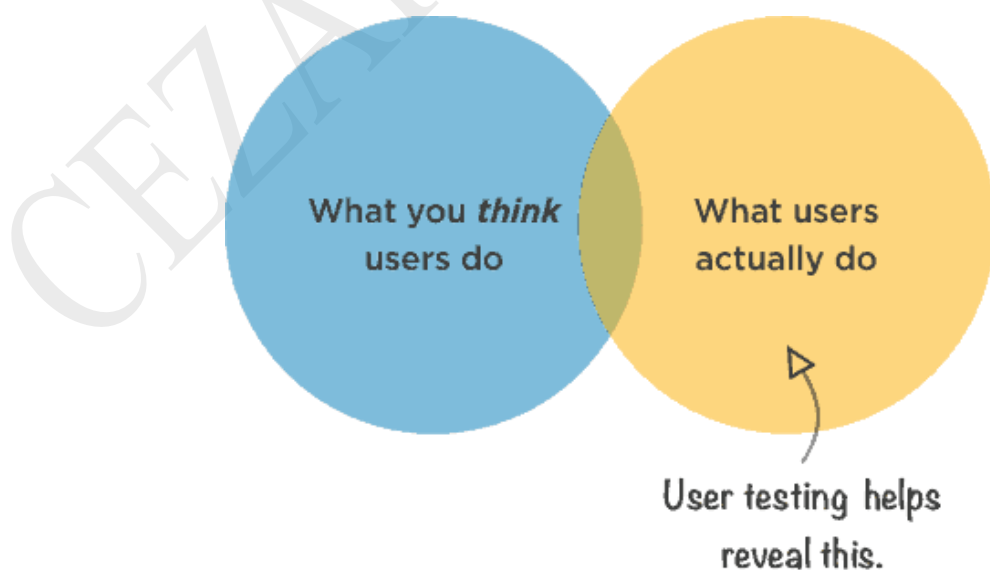


- **Imaginile** – ajuta la o buna interactiune cu utilizatorul
- **“F” Pattern Design** – informatiile sa fie prezentate, in ordinea relevantei, de la stanga la dreapta si de sus in jos, un studiu aratand ca partea de sus-stanga a ecranului este mult mai vizualizata decat cea de jos-dreapta

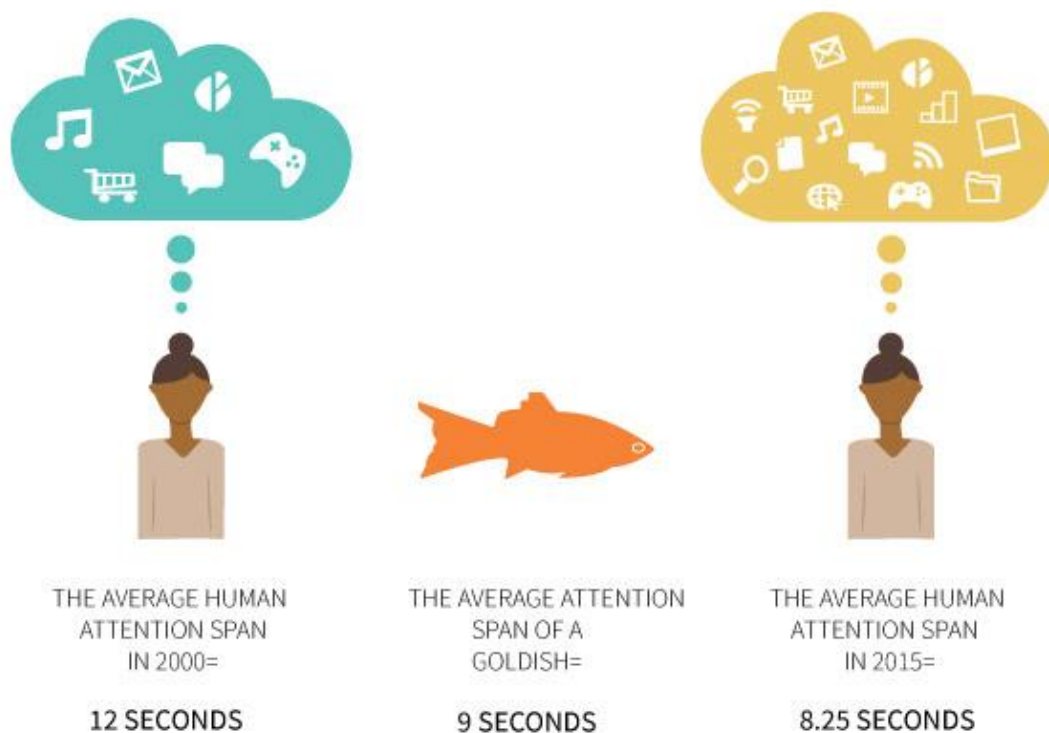
User Experience (UX)

În primul rând, interfața cu utilizatorul (User Interface) este o parte din UX, existând o diferență semnificativă între cele două. Interfața cu utilizatorul este spațiul unde utilizatorul interacționează cu aplicația, cu diferite componente ale aplicației, iar User Experience este rezultatul final pe care îl produce interacțiunea cu aplicația.

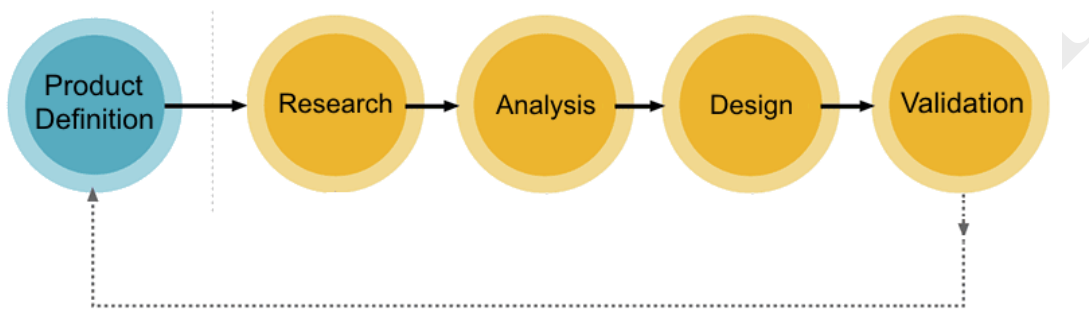
- **Scopul aplicației** – cel mai important lucru este scopul aplicației; Cine o să fie utilizatorii finali? Care sunt nevoile lor? Ce își doresc ei? De aceea, dezvoltarea unei aplicații poate începe cu un “user research”, fiind un proces esențial pentru UX.
- **Tu nu ești utilizatorul final** – este un lucru esențial în dezvoltarea unei aplicații. Dezvoltatorul nu este utilizatorul final, iar cel mai important lucru este testarea de către un utilizator real. Un dezvoltator nu se va comporta niciodată ca un utilizator real deoarece are experiența și va utiliza aplicația exact cum a fost dezvoltată. Utilizatorii care vor folosi aplicația au moduri diferite de gândire, un background diferit, experiența diferită sau chiar scopuri diferite, ceea ce face testarea cu utilizatori reali cea mai bună formă de testare pentru o aplicație web (usability testing)



- **Adaptarea design-ului pentru atentia de scurta durata** – nu incarcati utilizatorii cu foarte multa informatie. O perioada de atentie inseamna acea perioada de timp in care cineva se concentreaza asupra unei sarcini fara sa fie distras. Un studiu realizat de Microsoft in 2015 a aratat faptul ca media atentiei umane a scazut de la 12 secunde la 8 secunde, ceea ce inseamna ca avem o perioada de atentie mai scurta decat cea a unui pestisor auriu. Acest lucru inseamna ca este necesar ca designerii sa afiseze oamenilor informatiile necesara cat mai repede posibil. De aceea, lucrurile care nu sunt necesare pot fi eliminate. Acest lucru nu inseamna ca trebuie sa limitam experientele utilizatorilor intr-o aplicatie web, ci doar ca informatiile trebuie sa fie relevante.



- **UX depinde de fiecare proiect in parte** – nu exista un UX general care poate fi aplicat oricarui proiect. Fiecare proiect este unic, are cerinte unice, ceea ce duce la un UX diferit fiecarei aplicatii. De exemplu, atunci cand dezvoltati un nou proiect este necesara alocarea unui timp suplimentar pentru a cerceta ce tipuri de utilizatori vor accesa aplicatia, dar si care sunt specificatiile aplicatiei.

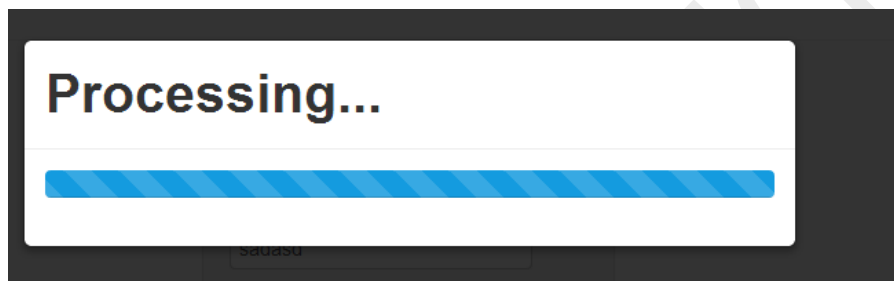


- **Prevenirea erorilor este mai usoara decat rezolvarea lor** – multe erori apar din cauza utilizatorilor (email incorect, parole care nu respecta anumite reguli, etc.). In acest caz aplicatia poate fi dezvoltata oferind sugestii de completare si notificand utilizatorii de fiecare data.

Register Yourself

You have entered an invalid e-mail address. Please try again.

- **Afisarea feedback-ului** – utilizatorii trebuie sa fie mereu informati de tot ceea ce se intampla in cadrul unei aplicatii: mesaje dupa trimiterea formularelor, mesaje in momentul editarii/stergerii datelor, feedback vizual. Acesta din urma este foarte important fiind un mijloc bun de informare.



- **Designul trebuie sa fie simplu si intuitiv**, utilizatorii folosind de fiecare data intuitia. Steve Krug spunea ca principalul motiv pentru care utilizatorii folosesc intuitia atunci cand aceseaza o aplicatie este ca nu le pasa. *“If we find something that works, we stick to it. It doesn’t matter to us if we understand how things work, as long as we can use them”*. S-a constatat ca utilizatorii nu citesc informatia, ci doar o scaneaza, cautand cateva puncte de referinta care sa ii ghideze prin continutul paginii.
- **Spatiile albe sunt importante** – informatia este perceputa mult mai repede deoarece un utilizator, dupa cum am mentionat anterior, scaneaza informatia si o imparte in sectiuni usor de interpretat. Structurile complexe sunt greu de citit, scanat, analizat si lucrat cu ele.

Alegerea culorilor potrivite

Alegerea culorilor in dezvoltarea unei aplicatii web, de cele mai multe ori, nu este atat de simpla. In continuare o sa urmarim cele mai importante reguli in acest sens: (<https://flatuicolors.com/>)

- **Tehnica 60-30-10** – aceasta tehnica se aplica si in decorarea caselor si este una simpla. Pentru a fi in balanta, culorile trebuie combinate in proportie de 60%-30%-10%. Cea mai mare parte reprezinta culoarea dominanta, urmata de culoarea secundara, iar procentul de 10% il reprezinta culoarea care ajuta la realizarea accentelor in aplicatie
- **Contrastul** – prin folosirea acestuia se aduce individualitate fiecarui element de interfata, facandu-le pe fiecare in parte vizibile si accesibile. Contrastul este folosit pentru punerea in evidenta a anumitor sectiuni sau pentru butoane de tip call-to-action
- **Psihologia culorilor** – fiecare culoare influenteaza utilizatorul intr-un anumit fel si livreaza catre acesta un anumit mesaj. De exemplu:
 - **rosu** simbolizeaza atat un sentiment pozitiv (incredere) cat si un sentiment negativ (erori, folosire incorecta a unui element)
 - **verde** simbolizeaza succes, calmitate, natura (un task executat cu succes, folosirea corecta a aplicatiei, etc)
 - **alb** reprezinta puritate si claritate (ofera sentimentul de simplitate in cadrul unei aplicatii. Cum a spus Steve Jobs, inspirat de Leonardo da Vinci: “simplitatea este sofisticarea absoluta”)

Culorile se pot alege folosind tool-uri existente. De exemplu, <https://paletton.com/> este o astfel de aplicatie care asista utilizatorul sa genereze paleta de culori corecte plecand de la o culoare de baza. Plecand de la culoarea de baza **albastru** observam ca aplicatia ne alege culorile complementare **rosu** si **galben** si genereaza si paleta de culori aferenta acestor culori.

